CS388 NLP - Complete Slide Notes

All original slide-note PDFs from the slides-notes folder, unchanged, combined in course order (UT Austin, Prof. Greg Durrett). A divider page before each week adds topics, key points, and exam relevance. Use the PDF bookmarks panel to jump to any week or segment.

Week 1 - Classification 4

- seg-course-preview - Course Preview 5
- seg-0 - What is NLP? Course Outline 7
- seg-1 - Linear Classification 17
- seg-2 - Sentiment Analysis & Feature Extraction 18
- seg-3 - Gradient Descent / ML Basics 21
- seg-4 - Perceptron 22
- seg-5 - Perceptron as Minimizing Loss 23
- seg-6 - Logistic Regression 27
- seg-7 - Sentiment Analysis with Bag-of-Words 30
- seg-8 - Optimization Basics 34

Week 2 - Multiclass & Neural Networks 36

- seg-9 - Multiclass Classification 37
- seg-10 - Multiclass Perceptron & Logistic Regression 39
- seg-11 - Examples of Text Classification 41
- seg-11a - Fairness in Classification 48
- seg-12 - Neural Networks 53
- seg-13 - Neural Networks: Visualization 55
- seg-14 - Vectorization, Softmax & Backpropagation 58
- seg-15 - Computation Graphs & PyTorch 66
- seg-16 - Training Neural Networks (batching, init, dropout, optimizers) 74

Week 3 - Word Embeddings 81

- seg-17 - Word Embeddings 82
- seg-18 - Skip-gram 84
- seg-19 - word2vec Variants, GloVe, fastText 87
- seg-20 - Bias in Word Embeddings 96
- seg-21 - Applying Embeddings; Deep Averaging Networks 99

Week 4 - Language Modeling & Self-Attention 104

- seg-41 - Language Modeling (n-grams) 105
- seg-42 - Smoothing in n-gram LMs 107
- seg-lm3-lm-evaluation - LM Evaluation (Perplexity) 109
- seg-43 - Neural Language Models 110
- seg-lm5-rnns-and-their-shortcomings - RNNs and their Shortcomings 111
- seg-lm6-attention - Attention 115
- seg-lm7-self-attention - Self-Attention 121
- seg-lm8-multihead-self-attention - Multi-head Self-Attention 128
- seg-lm9-position-encoding - Positional Encoding 133

Week 5 - Transformers & Decoding 137

- seg-trans1-transformer-architecture - Transformer Architecture 138
- seg-trans2-using-transformers - Using Transformers 143
- seg-trans3-transformer-language-models - Transformer Language Models 147
- seg-trans4-transformer-extensions - Transformer Extensions 154
- seg-subword-segmentation - Subword Segmentation (BPE, WordPiece) 159
- seg-infer1-beam-search - Decoding: Beam Search 163
- seg-infer2-nucleus-sampling - Decoding: Nucleus Sampling 167

Week 6 - Pre-training & Seq2seq 174

- seg-68 - BERT: Masked Language Modeling 175

seg-69 - BERT: Model and Applications 180
seg-s2s1-seq2seq - Encoder-Decoder (seq2seq) Models 191
seg-s2s2-bart - BART 195
seg-s2s3-t5 - T5 200

Weeks 7-8 - Structured Prediction: Tagging & Parsing 205

seg-22 - Part-of-Speech Tagging 206
seg-23 - Sequence Labeling with Classifiers 208
seg-24 - Hidden Markov Models 211
seg-25 - HMM Parameter Estimation 212
seg-26 - HMM Inference: Viterbi 214
seg-28 - Trigram Taggers & POS Tagging Results 217
seg-34 - Constituency Parsing 224
seg-35 - Probabilistic Context-Free Grammars 231
seg-36 - CKY Algorithm 233
seg-37 - Refining PCFGs: Markovization & Lexicalization 235
seg-38 - Dependency Parsing 243
seg-39 - Transition-based Dependency Parsing 253

Week 9 - ChatGPT / GPT-4 and Large Language Models 264

seg-llm1-gpt3 - GPT-3 265
seg-llm2-zero-shot - Zero-shot Prompting 272
seg-llm3-few-shot - Few-shot Prompting (In-context Learning) 279
seg-llm4-induction-heads - Induction Heads 288
seg-llm5-instruction-tuning - Instruction Tuning 295
seg-llm6-rlhf - RLHF 300
seg-llm7-hallucination - Hallucination & Factuality 307

Week 10 - Explanations 313

seg-80 - Interpretability: Motivation & Types 314
seg-81 - Local Explanations (LIME, Gradients) 318
seg-82 - Text Explanations 326
seg-83 - Probing 332
seg-84 - Dataset Bias & Annotation Artifacts 335
seg-expl6-cot - Chain-of-Thought 341
seg-expl7-cot-analysis - Chain-of-Thought Analysis & Tool Use 351

Week 11 - Question Answering & Dialogue 357

seg-72 - Reading Comprehension 358
seg-73 - Reading Comprehension: Setup & Baselines 359
seg-76 - BERT for Question Answering 361
seg-77 - Problems in QA (adversarial, artifacts) 363
seg-78 - Open-Domain QA 369
seg-79 - Multi-hop QA 374
seg-91 - Chatbots: Turing Test & Early Systems 382
seg-93 - Task-oriented Dialogue 388
seg-dial4-chatbots - Neural Chatbots 396

Week 12 - Machine Translation & Summarization 408

seg-61 - Machine Translation: Introduction 409
seg-62 - MT Framework 410
seg-63 - Word Alignment 412
seg-64 - IBM Model 1 413
seg-65 - Phrase-based Machine Translation 416
seg-mt5-neural-pt - Neural and Pre-trained MT 426
seg-85 - Summarization 436
seg-86 - Extractive Summarization (MMR, ROUGE) 440
seg-90 - Pre-trained Summarization & Factuality 446

Weeks 13-14 - Multilinguality, Language Grounding, Ethics 451

seg-95 - Morphology 452
seg-97 - Cross-lingual Tagging & Parsing 459
seg-98 - Multilingual Embeddings / mBERT 463
seg-ground1-grounding - Language Grounding 469
seg-ground2-langvis - Language and Vision 475
seg-eth1-bias - Ethics: Bias Amplification 485
seg-eth2-exclusion - Ethics: Exclusion 490
seg-eth3-dangers - Ethics: Dangers of Automation & LLMs 494
seg-eth4-unethical - Ethics: Unethical Use & Paths Forward 501

WEEK 1

Classification

Segments in this section

seg-course-preview.pdf - Course Preview p. 5
seg-0.pdf - What is NLP? Course Outline p. 7
seg-1.pdf - Linear Classification p. 17
seg-2.pdf - Sentiment Analysis & Feature Extraction p. 18
seg-3.pdf - Gradient Descent / ML Basics p. 21
seg-4.pdf - Perceptron p. 22
seg-5.pdf - Perceptron as Minimizing Loss p. 23
seg-6.pdf - Logistic Regression p. 27
seg-7.pdf - Sentiment Analysis with Bag-of-Words p. 30
seg-8.pdf - Optimization Basics p. 34

Key points (added notes)

- Features: bag-of-words, n-grams, tf-idf; preprocessing (tokenize, UNK, index).
- Perceptron: update w += y f(x) only on mistakes; converges iff the data are linearly separable.
- Logistic regression: P(+|x) = sigmoid(w.f(x)); minimize NLL (convex); update w += a f(x)(1 - P(+|x)).
- SGD and step sizes; Newton / Adagrad / Adam as adaptive approximations.

EXAM: hand-run perceptron for one epoch (check the tie rule!), true/false on convergence and convexity.

Note: segments 1-6 and 8 are handwritten lecture notes; typed equivalents are in the study guide, Sections 1-3.

GD

How many years after the start of the Revolutionary War did George Washington step down as president?

The American Revolutionary War began in 1775 and George Washington was inaugurated as the first President of the United States on April 30, 1789. He served two terms as President and stepped down at the end of his second term on March 4, 1797.

To calculate the number of years between the start of the Revolutionary War in 1775 and the time George Washington stepped down as President in 1797:

1797 - 1775 = 22 years.

So, George Washington stepped down as President 22 years after the start of the Revolutionary War.

ChatGPT is a large neural network language model:

What's a language model?

What kind of neural network is it?

How is it trained?

What can it be used for?

Course Outline

- Goals:
  - Build from **ML fundamentals** to **ChatGPT**
  - Cover the major methods and applications in modern NLP research
  - Give you the ability to know what tools to use for which problems
- Assignments:
  - Designed to give you hands-on experience
  - Implementation of neural network models in PyTorch
  - Thinking about data will be key!
- See the syllabus and course website for more details

What is the goal of NLP?

- Be able to solve problems that require **deep understanding** of text
- Systems that talk to us: dialogue systems, machine translation, summarization

What is the goal of NLP?

- Be able to solve problems that require **deep understanding** of text
- Systems that talk to us: dialogue systems, machine translation, summarization

The Political Bureau of the CPC Central Committee

July 30

hold a meeting

中共中央政治局7月30日召开会议，会议分析研究当前经济形势，部署下半年经济工作。

People's Daily, August 10, 2020

Translate

The Political Bureau of the CPC Central Committee held a meeting on July 30 to analyze and study the current economic situation and plan economic work in the second half of the year.

What is the goal of NLP?

- Build systems that extract information from text and answer questions

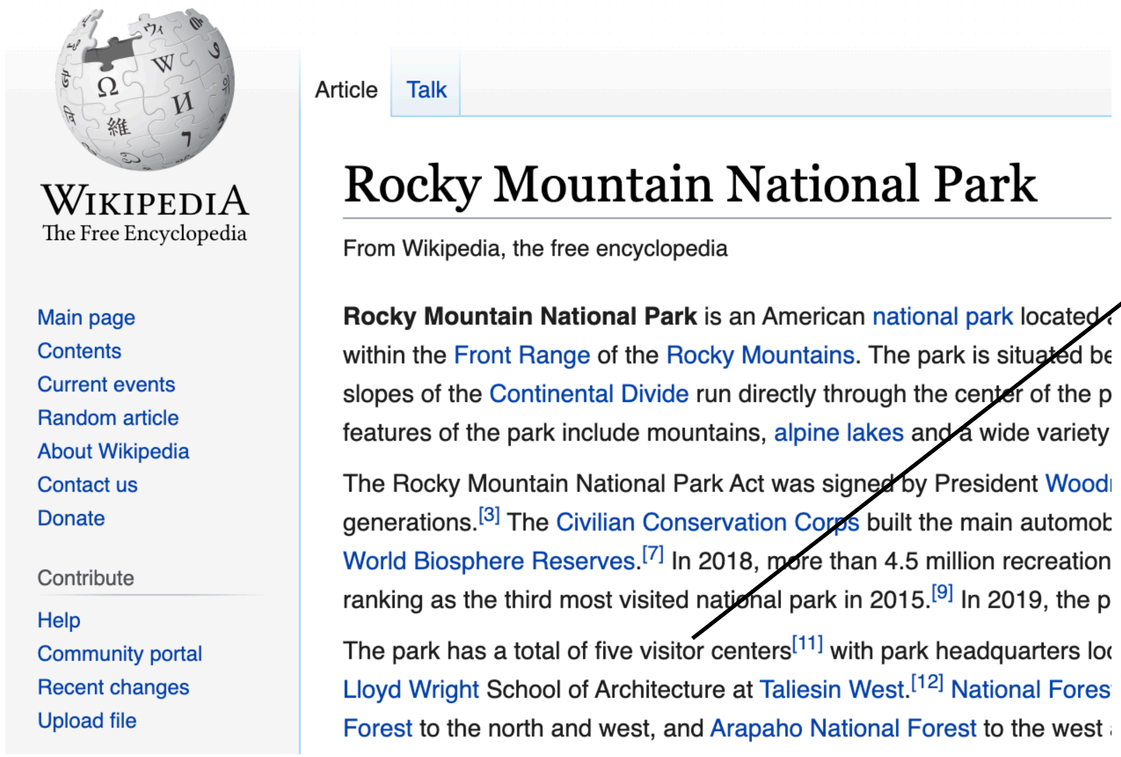
When was Abraham Lincoln born?

map to `Birthday` field

Name	`Birthday`
Lincoln, Abraham	2/12/1809
Washington, George	2/22/1732
Adams, John	10/30/1735

→ **February 12, 1809**

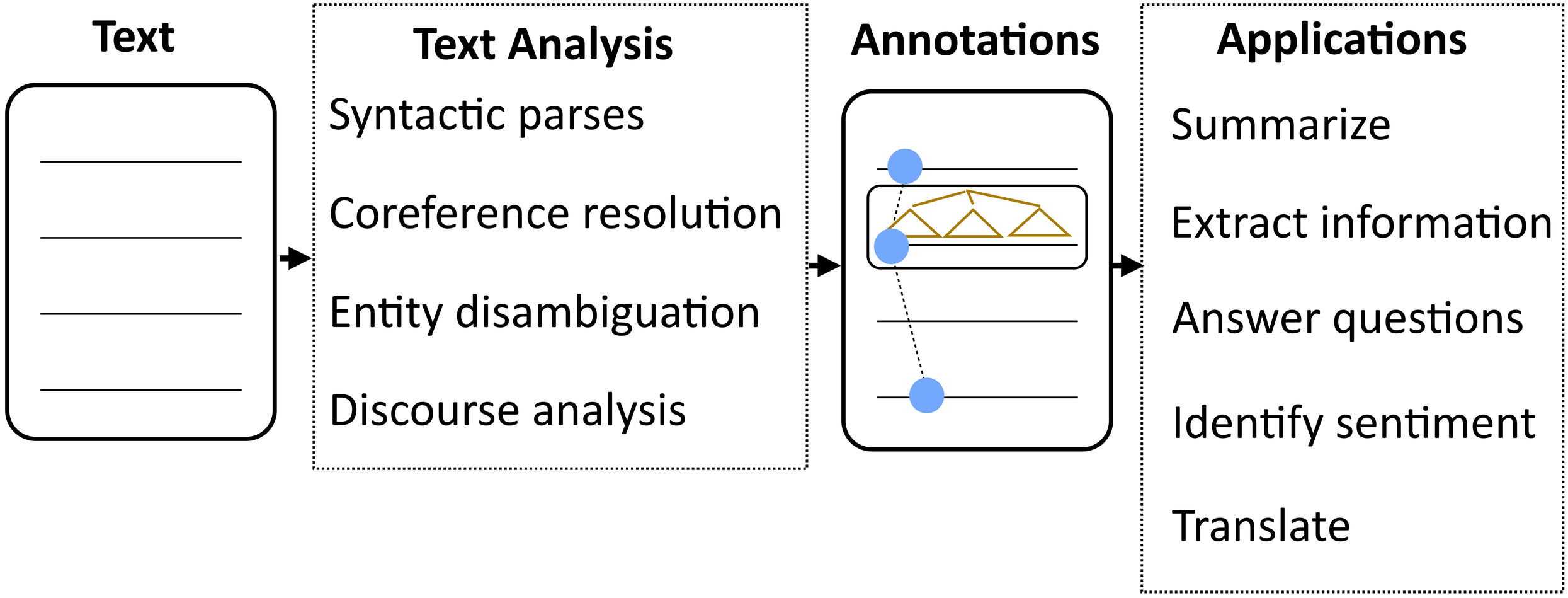
How many visitors centers are there in Rocky Mountain National Park?



The park has a total of five visitor centers

→ **five**

“Standard” NLP Pipeline



- All of these components are modeled with statistical approaches using machine learning

How do we represent language?

Text

Labels

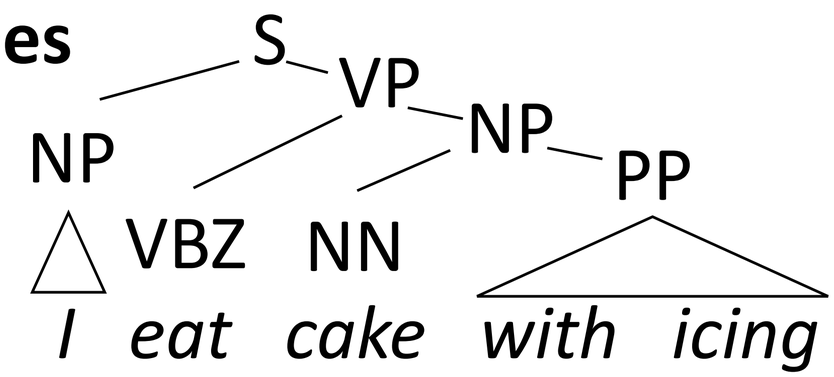
the movie was good +

Beyoncé had one of the best videos of all time **subjective**

Sequences/tags

PERSON *Tom Cruise* *stars in the new* **WORK_OF_ART** *Mission Impossible* *film*

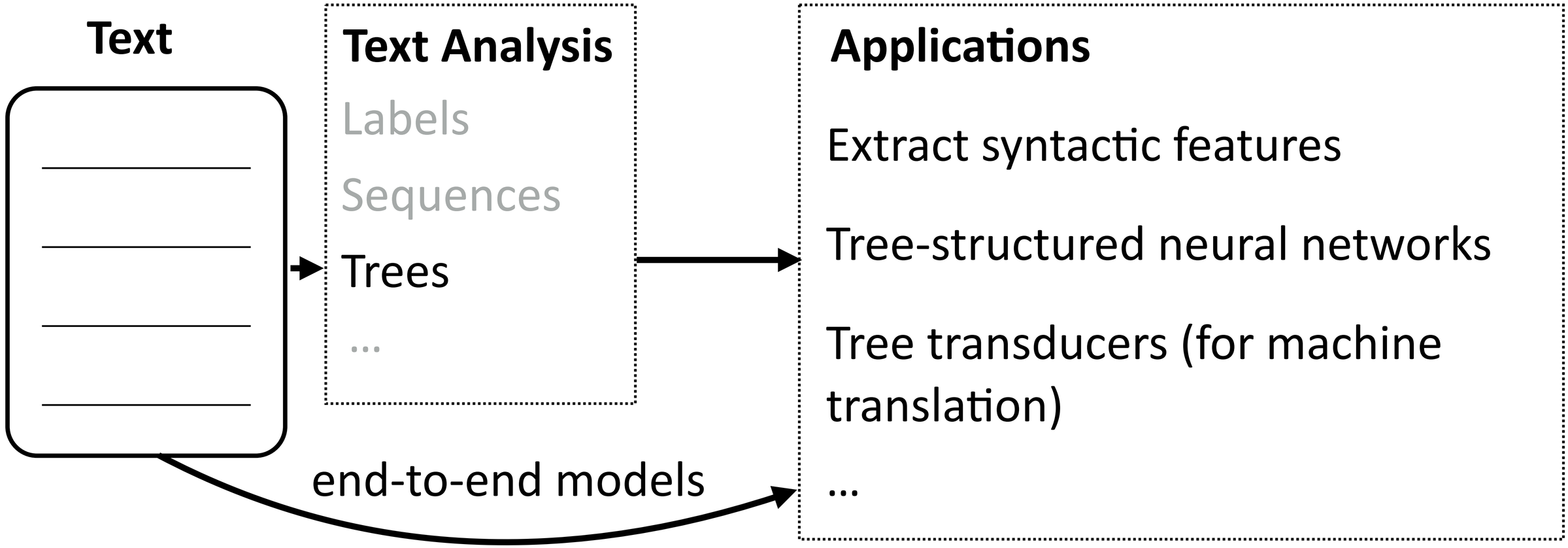
Trees



$\lambda x.\ flight(x) \wedge dest(x)=Miami$

flights to Miami

How do we use these representations?



- Why is this prediction hard? Because language is complex and ambiguous!
- What ambiguities do we need to resolve?

What makes language hard?

Teacher Strikes Idle Kids

Ban on Nude Dancing on Governor’s Desk

Iraqi Head Seeks Arms

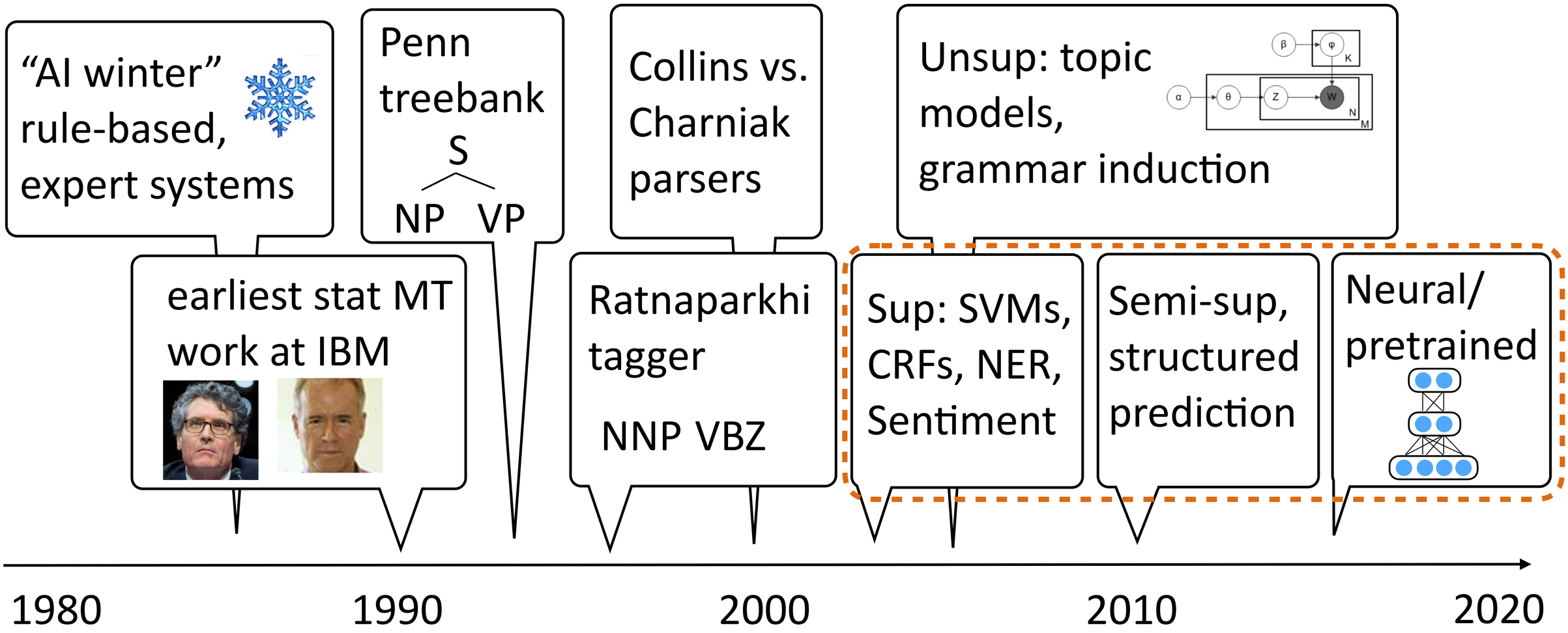
What makes language hard?

- There aren't just one or two possibilities, but many!

il fait vraiment beau ⟶

It is really nice out
It's really nice
The weather is beautiful
It is really beautiful outside
He makes truly beautiful
It fact actually handsome

A brief history of NLP techniques



- This course focuses on **supervised learning**, **semi-supervised methods**, and **neural models (including pre-training)**
- All of these models can handle ambiguity by learning how to map text into linguistic representations using data

Outline of the course

- Classification: linear and neural, word representations
- Text analysis: tagging (HMMs, CRFs) and parsing (PCFGs)
- Language modeling and pre-training
- Question answering and semantics
- Machine translation
- Applications

Linear Binary Classification

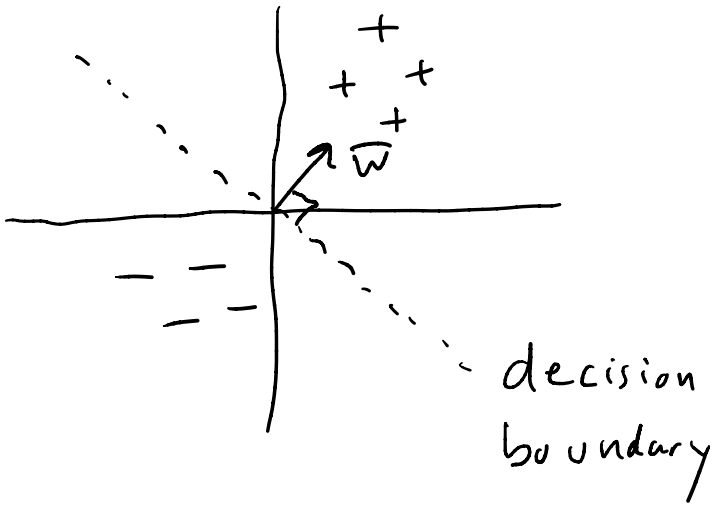
Point $\bar{x} \in \mathbb{R}^d$ $\quad f(\bar{x})$ $\quad f$: feature extractor

$\bar{x}$: string $\rightarrow f(\bar{x}) \in \mathbb{R}^d$

Label $y \in \{-1, +1\}$

Classifier : weight vector $\bar{w}$

(0)



$$\bar{w}^T f(\bar{x}) \stackrel{?}{\geq} 0$$

$\bar{w}^T f(\bar{x}) + b \geq 0$

↑ bias $\quad$ X not use

$f(\bar{x})$ $\quad [3, -1, 2]$

↓

add 1 to every feat vector

$\tilde{f}(\bar{x})$ $\quad [3, -1, 2, 1]$

Sentiment Analysis and Basic Feature Extraction

the movie was great! would watch again!

the film was awful; I'll never watch again!

① text $\bar{x} \Rightarrow f(\bar{x})$ feature extraction

② $\{f(\bar{x}^{(i)}), y^{(i)}\}_{i=1}^{D}$ dataset of D labeled exs,
$\Rightarrow$ train classifier

Feature extraction

the movie was great

Bag-of-words: Assume 10,000 words in vocabulary

[1 0 0 0 1 1 1]

the a of at ... movie ... was ... great ...

y is 9996 0s

Counts (how many "the" are present)

presence/absence (0/1)

Bag-of-ngrams

n-gram: sequence of n consecutive words

2-grams: the movie, movie was, was great

tf-idf

↳ tf x idf

tf: count of the term

idf: inverse document frequency $\log \frac{N}{\{D : w \in D\}}$

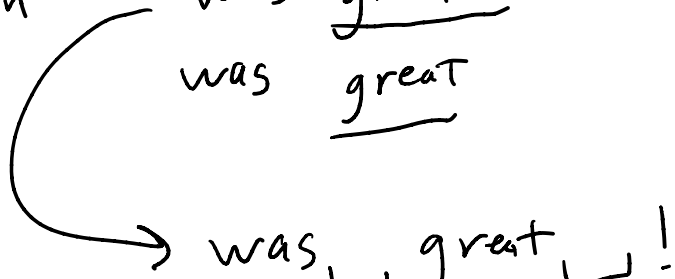
N: total # docs

$\{D : w \in D\}$: # docs with w

Preprocessing

① Tokenization

was great!
was great



was ␣ great ␣ !

[... great ... great! ...]

wasn't → was ␣ n't

② [Sometimes] Stopword removal (the, of, a, ...)

③ [Sometimes] Casing (lowercasing, truecasing)

④ Handling unknown words Durrett ⇒ UNK

⑤ Indexing: map each {word, n-gram} into ℕ

use a map

Basic Machine Learning, Gradient Descent

Parameters $\bar{w}$ to optimize

Labeled data $(\bar{x}^{(i)}, y^{(i)})_{i=1}^{D}$ D points

input, label

Suppose 10,000 feats. $\bar{w} \in \mathbb{R}^{10{,}000}$. Searching for an optimal $\bar{w}$

Optimization problem: formulate <u>training objective</u>, which is (for us) linear over examples, optimize

objective: $\sum_{i=1}^{D} \text{loss}(\bar{x}^{(i)}, y^{(i)}, \bar{w})$

Stochastic gradient descent

for t up to num epochs:

for i up to D:

sample $j \sim [1, D]$

$\bar{w} \leftarrow \bar{w} - \alpha \frac{\partial}{\partial \bar{w}} \text{loss}(\bar{x}^{(j)}, y^{(j)}, \bar{w})$

step size

gradient of the loss w.r.t. $\bar{w}$ points towards $\bar{w}$ that give higher loss

Perceptron

Binary classifier $\bar{w}^T f(\bar{x}) \overset{?}{>} 0$

↳ $y \in \{-1, +1\}$

for t up to epochs:

  for i up to D:

    $y_{pred} \leftarrow \bar{w}^T f(\bar{x}^{(i)}) \overset{?}{>} 0$

    $\bar{w} \leftarrow \bar{w}$ if $y_{pred} = y^{(i)}$

    else $\bar{w} \leftarrow \bar{w} + \alpha f(\bar{x}^{(i)})$ if $y^{(i)} = +1$

    else $\bar{w} \leftarrow \bar{w} - \alpha f(\bar{x}^{(i)})$ if $y^{(i)} = -1$

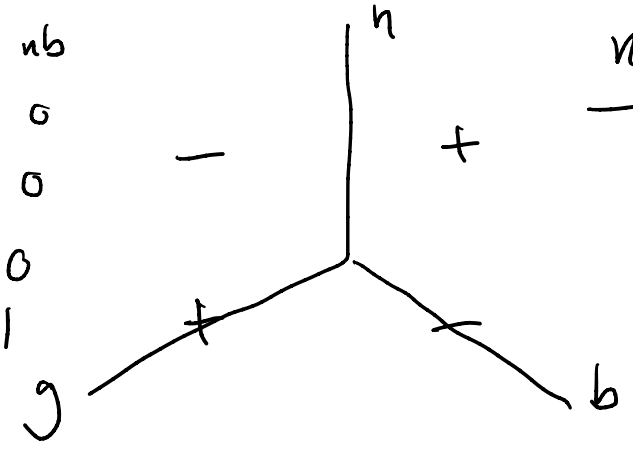
Ex: $\alpha = 1$

		y	f(x̄) mgbn	w̄
				0000
ep1	movie good	+1	1100	1100 $y_{pred} = -1$
	movie bad	-1	1010	0 1 -1 0 $y_{pred} = +1$
	not good	-1	0101	0 0 -1 -1 $y_{pred} = +1$
ep2	movie good	+1	1100	1 1 -1 -1 $y_{pred} = -1$

Ex 2

	y	g	b	n	ng	nb
good	+1	1	0	0	0	0
bad	-1	0	1	0	0	0
not good	-1	1	0	1	1	0
not bad	+1	0	1	1	0	1

↳ Add bigrams

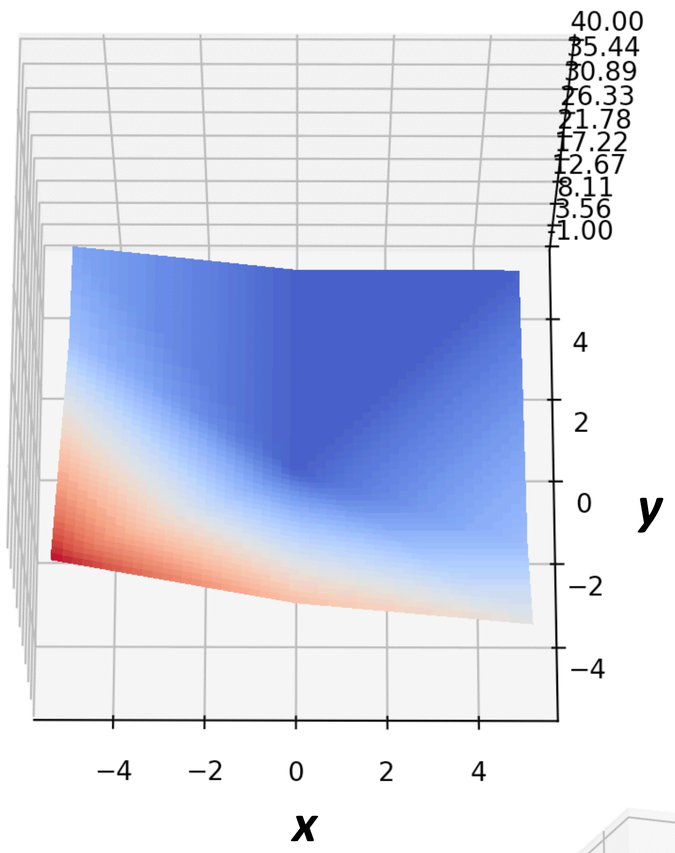


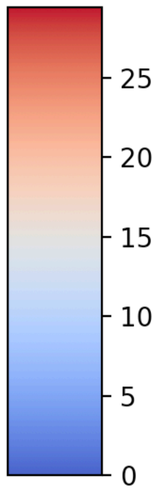
not separable

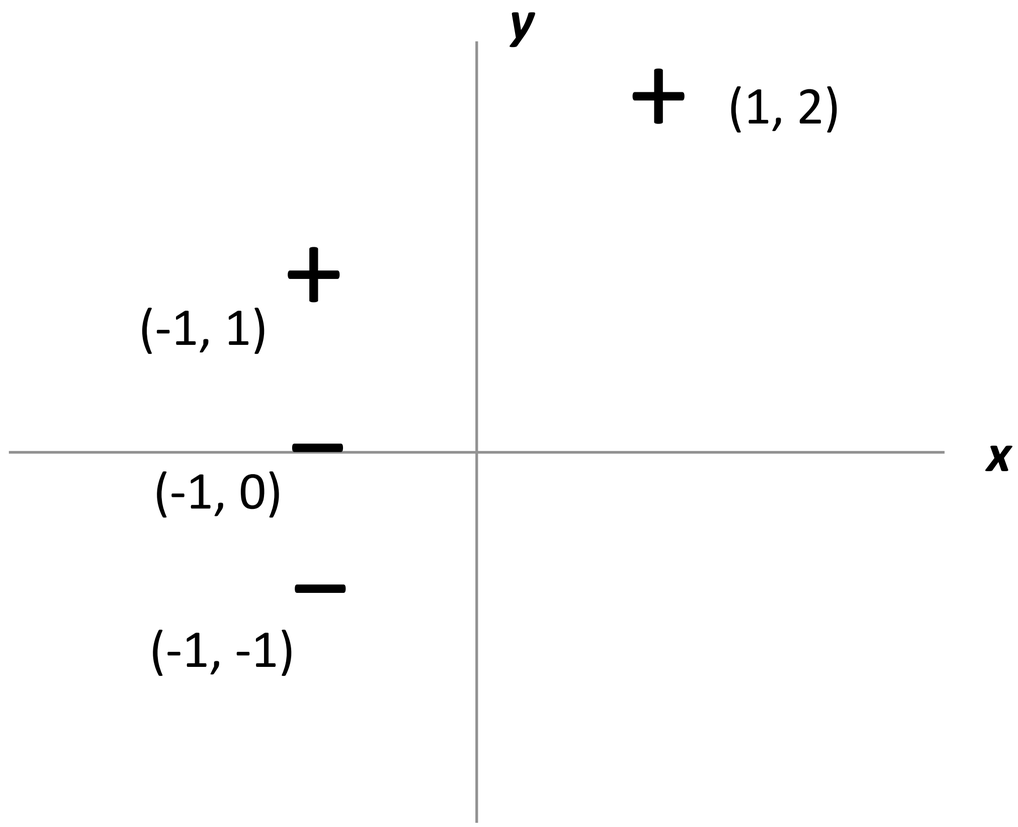
Perceptron as minimizing loss

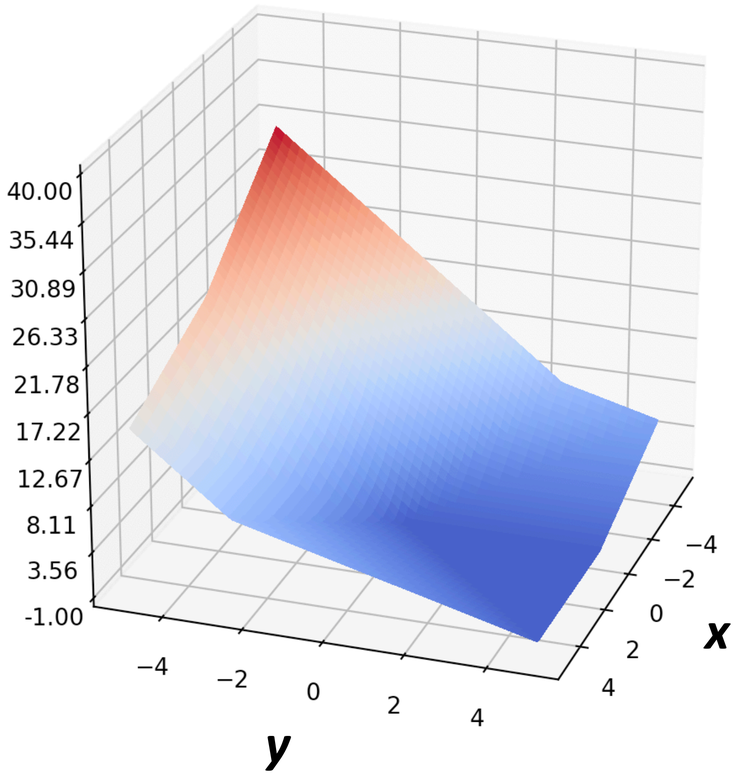
Perceptron as minimizing loss

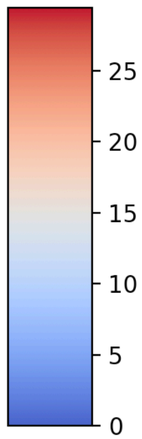
Perceptron Loss Demonstration



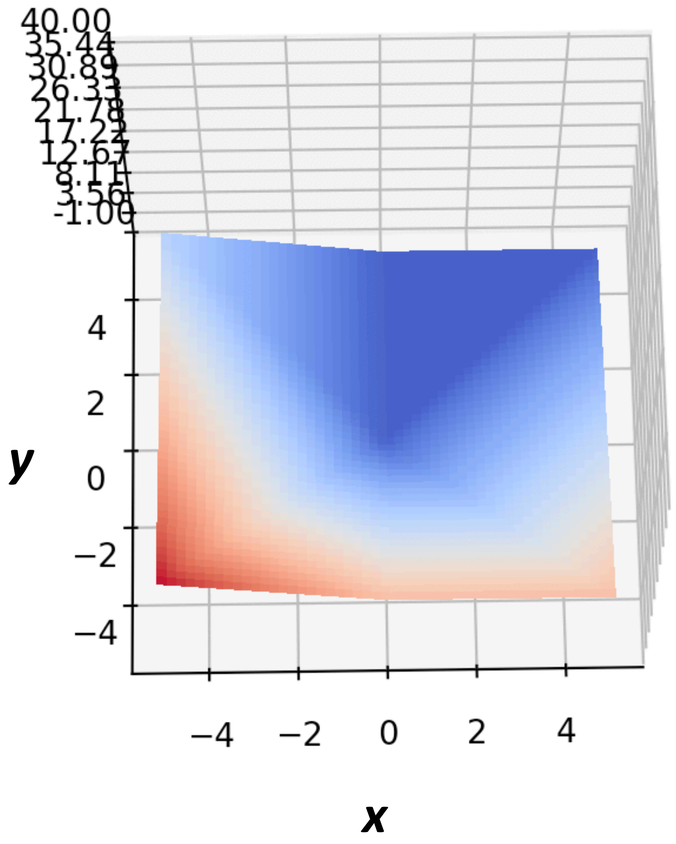


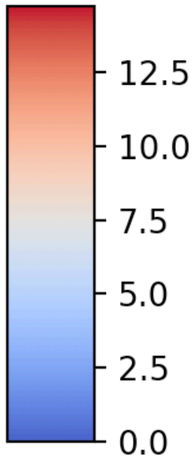


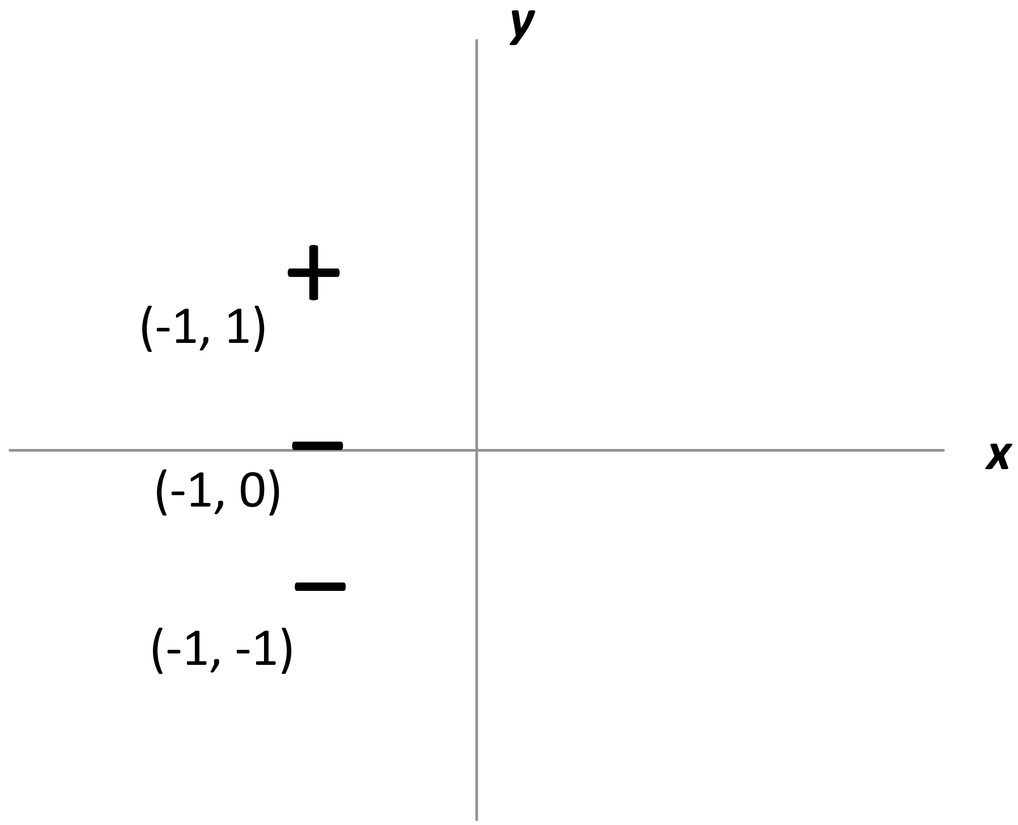




Perceptron Loss Demonstration







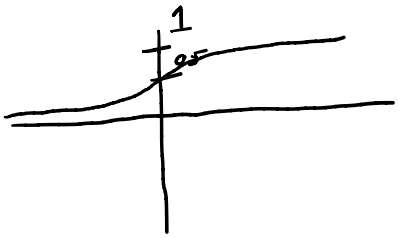
Logistic Regression

Discriminative probabilistic model $P(y|\bar{x})$

(generative: $P(\bar{x}, y)$)

$$P(y=+1|\bar{x}) = \frac{e^{\bar{w}^T f(\bar{x})}}{1+e^{\bar{w}^T f(\bar{x})}}$$

logistic $\frac{e^x}{1+e^x}$



$$P(y=-1|\bar{x}) = \frac{1}{1+e^{\bar{w}^T f(\bar{x})}}$$

Decision boundary?

return +1 if $P(y=+1|\bar{x}) > 0.5$

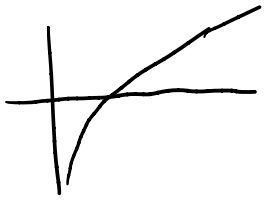
$\bar{w}^T f(\bar{x}) > 0$

Training

For a dataset $(\bar{x}^{(i)}, y^{(i)})_{i=1}^{D}$, want to maximize

$\prod_{i=1}^{D} P(y^{(i)} | \bar{x}^{(i)})$ maximum likelihood

$\max_{\bar{w}} \sum_{i=1}^{D} \log P(y^{(i)} | \bar{x}^{(i)})$ log likelihood



$\Rightarrow \min_{\bar{w}} \sum_{i=1}^{D} \underbrace{-\log P(y^{(i)} | \bar{x}^{(i)})}_{\text{loss}(\bar{x}^{(i)}, y^{(i)}, \bar{w})}$] training objective: negative log likelihood (NLL)

Need to compute $\frac{\partial}{\partial \bar{w}} \text{loss}(\bar{x}^{(i)}, y^{(i)}, \bar{w})$

LR Gradients

Assume $y^{(i)} = +1$

$$\frac{\partial}{\partial \bar{w}} \text{loss} = \frac{\partial}{\partial \bar{w}} \left[-\bar{w}^T f(\bar{x}) + \log\left(1 + e^{\bar{w}^T f(\bar{x})}\right) \right]$$

$$= -f(\bar{x}) + \frac{1}{1 + e^{\bar{w}^T f(\bar{x})}} \cdot e^{\bar{w}^T f(\bar{x})} \cdot f(\bar{x})$$

$$= f(\bar{x}) \left[-1 + \frac{e^{\bar{w}^T f(\bar{x})}}{1 + e^{\bar{w}^T f(\bar{x})}} \right] = f(\bar{x}) \left[P(y=+1 \mid \bar{x}) - 1 \right]$$

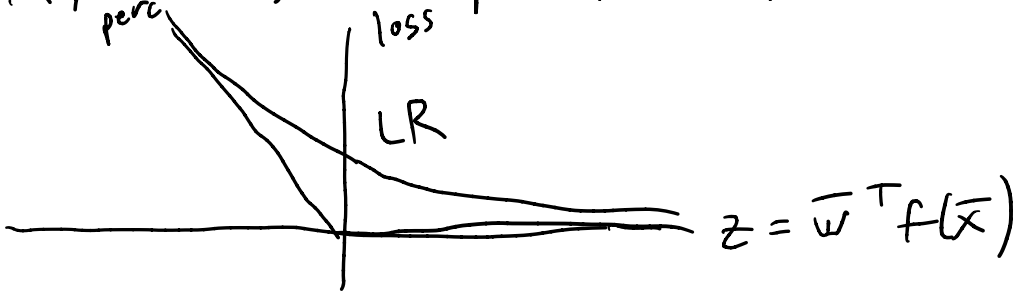
Update: $\bar{w} \leftarrow \bar{w} + \alpha f(\bar{x}) \left(1 - P(y=+1 \mid \bar{x})\right)$

$P(y=+1 \mid \bar{x}) \simeq 1$: no update

$P(y=+1 \mid \bar{x}) \simeq 0$: perceptron update

Let $z = \bar{w}^T f(\bar{x})$

NLL: $\log\left(1 + e^{z}\right) - z$



Sentiment Analysis

this movie was great! would watch again +

the movie was gross and overwrought, but I liked it +

this movie was not really very enjoyable —

- Bag-of-words doesn't seem sufficient (discourse structure, negation)
- There are some ways around this: extract bigram feature for "*not* X" for all X following the *not*

Sentiment with Bag-of-words Models

	Features	# of features	frequency or presence?	NB	ME	SVM
(1)	unigrams	16165	freq.	**78.7**	N/A	72.8
(2)	unigrams	"	pres.	81.0	80.4	**82.9**
(3)	unigrams+bigrams	32330	pres.	80.6	80.8	**82.7**
(4)	bigrams	16165	pres.	77.3	**77.4**	77.1
(5)	unigrams+POS	16695	pres.	81.5	80.4	**81.9**
(6)	adjectives	2633	pres.	77.0	**77.7**	75.1
(7)	top 2633 unigrams	2633	pres.	80.3	81.0	**81.4**
(8)	unigrams+position	22430	pres.	81.0	80.1	**81.6**

- Simple feature sets can do pretty well!

Bo Pang, Lillian Lee, Shivakumar Vaithyanathan (2002)

Sentiment with Bag-of-words Models

- 10 years later — revisited basic BoW classifiers vs. other methods

Method	RT-s	MPQA
MNB-uni	77.9	85.3
MNB-bi	**79.0**	**86.3**
SVM-uni	76.2	86.1
SVM-bi	77.7	**86.7**
NBSVM-uni	**78.1**	85.3
NBSVM-bi	**79.4**	**86.3**
RAE	76.8	85.7
RAE-pretrain	77.7	**86.4**
Voting-w/Rev.	63.1	81.7
Rule	62.9	81.8
BoF-noDic.	75.7	81.8
BoF-w/Rev.	76.4	84.1
Tree-CRF	77.3	86.1

Before neural nets had taken off — results weren't that great

Kim (2014) CNNs **81.5** **89.5**

Wang and Manning (2012)

Kim (2014)

Sentiment with Bag-of-words Models

- Stanford Sentiment Treebank (SST) binary classification
- Best systems now: large pretrained networks
- 90 -> 97 over the last 2 years

Model	Accuracy	Paper / Source	Code
XLNet-Large (ensemble) (Yang et al., 2019)	96.8	XLNet: Generalized Autoregressive Pretraining for Language Understanding	Official
MT-DNN-ensemble (Liu et al., 2019)	96.5	Improving Multi-Task Deep Neural Networks via Knowledge Distillation for Natural Language Understanding	Official
Snorkel MeTaL(ensemble) (Ratner et al., 2018)	96.2	Training Complex Models with Multi-Task Weak Supervision	Official
MT-DNN (Liu et al., 2019)	95.6	Multi-Task Deep Neural Networks for Natural Language Understanding	Official
Bidirectional Encoder Representations from Transformers (Devlin et al., 2018)	94.9	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	Official
...			
Neural Semantic Encoder (Munkhdalai and Yu, 2017)	89.7	Neural Semantic Encoders	
BLSTM-2DCNN (Zhou et al., 2017)	89.5	Text Classification Improved by Integrating Bidirectional LSTM with Two-dimensional Max Pooling	

https://github.com/sebastianruder/NLP-progress/blob/master/english/sentiment_analysis.md

Optimization Basics

Find $\bar{w}$ to minimize loss; search over space of params

loss $\mathcal{L}\left(\underbrace{(\bar{x}^{(i)}, y^{(i)})_{i=1}^{D}}_{\text{training data}}, \bar{w}\right) = \sum_{i=1}^{D} \mathcal{L}(\bar{x}^{(i)}, y^{(i)}, \bar{w})$

function of $\bar{w}$

Stochastic gradient descent: repeatedly pick example i

$$\bar{w} \leftarrow \bar{w} - \alpha \frac{\partial}{\partial \bar{w}} \mathcal{L}(i, \bar{w})$$

α: step size; $\mathcal{L}(i, \bar{w})$: loss on ith example

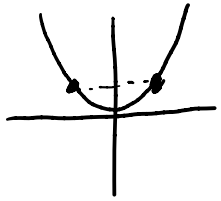
Step size Suppose $\mathcal{L}(i, \bar{w}) = w^2$ one feature $\bar{w} = [w]$

$w = -1$

$\frac{\partial}{\partial w} \mathcal{L} = 2w$

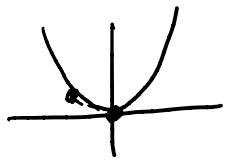
if $\alpha = 1$: $w = -1 - 1 \cdot 2(-1)$

$\Rightarrow w = 1$



Keep $\alpha = 1$: oscillates!

if $\alpha = 1/2$: $w \to 0$



Choosing Step Size

How to choose step size?

- Try them out: $1e^{0}$ $1e^{-1}$ $1e^{-2}$ ...
- Large → small, e.g. $1/t$ for epoch t ($1/\sqrt{t}$ ...)
  (fixed schedule)

  or decrease step size when performance stagnates on held-out data

Newton's method: $\bar{w} \leftarrow \bar{w} - \underbrace{\left(\frac{\partial^2}{\partial \bar{w}^2}\mathcal{L}\right)^{-1}}_{\text{inverse Hessian}} \frac{\partial}{\partial \bar{w}}\mathcal{L}$ very expensive to compute

DL

Adagrad, Adadelta, **Adam**: "adaptive" methods

approximations to the inverse Hessian (linear in # feats)

Regularization: don't really use

WEEK 2

Multiclass & Neural Networks

Segments in this section

seg-9.pdf - Multiclass Classification p. 37
seg-10.pdf - Multiclass Perceptron & Logistic Regression p. 39
seg-11.pdf - Examples of Text Classification p. 41
seg-11a.pdf - Fairness in Classification p. 48
seg-12.pdf - Neural Networks p. 53
seg-13.pdf - Neural Networks: Visualization p. 55
seg-14.pdf - Vectorization, Softmax & Backpropagation p. 58
seg-15.pdf - Computation Graphs & PyTorch p. 66
seg-16.pdf - Training Neural Networks (batching, init, dropout, optimizers) p. 74

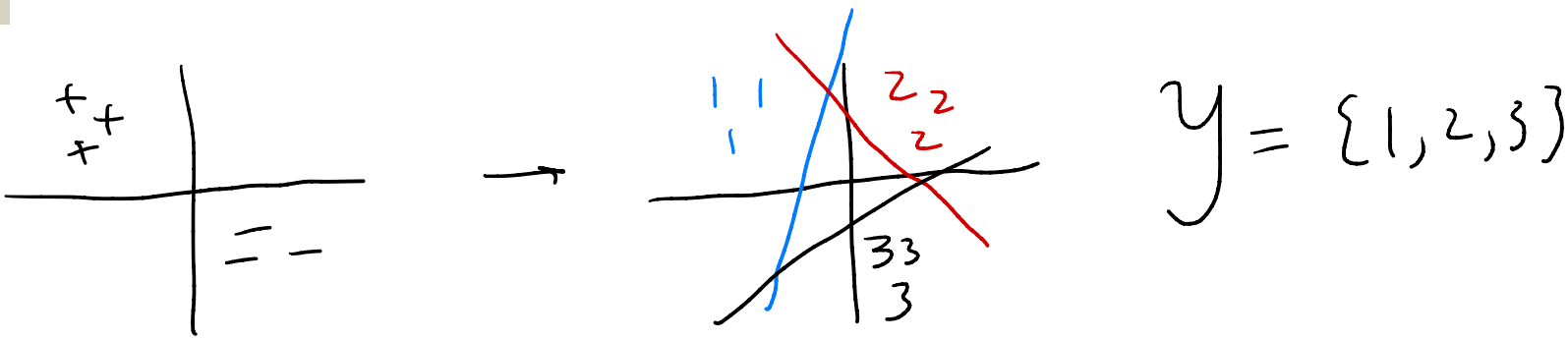
Key points (added notes)

- Multiclass: different weights (w_y) vs different features f(x,y); softmax gradient = gold features - expected features.
- Fairness: proxy features (ZIP code, dialect) cause discrimination; accuracy alone hides it.
- FFNN: P(y|x) = softmax(W g(V f(x))); backprop = chain rule; PyTorch nn.Module, zero_grad / backward / step.
- Training: never initialize to zeros, Glorot init, dropout, Adam, gradient clipping.

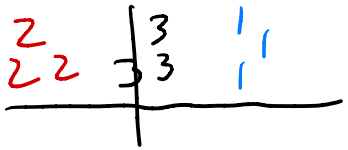
EXAM: pick a correct FFNN implementation (dimensions + nonlinearity), softmax arithmetic.

Note: segments 9, 10 and 12 are handwritten.

Multiclass Classification



one-vs-all: n binary classifiers



Two techniques

One weight vector per class (different weights, DW)

or different features (DF) per class

DW: $\underset{y \in Y}{\operatorname{argmax}} \; \overline{w}_y^T f(\overline{x})$ (NN)

DF: $\underset{y \in Y}{\operatorname{argmax}} \; \overline{w}^T f(\overline{x}, y)$ (hypothesized y)

Topic Classification

$\bar{x}$ = too many drug trials, too few patients

$y = \{\text{health}, \text{sports}, \text{science}\}$

$f(\bar{x})$ = bag-of-unigrams [drug, patients, baseball]

$f(\bar{x}) = [1, 1, 0]$

DW: $\bar{w}_{\text{health}} = [2, 5.6, -3]$ $\bar{w}_{\text{sports}} = [1.2, -3.1, 5.7]$

$\Rightarrow 7.6$ $\Rightarrow -1.9$

DF: $f(\bar{x}, y) = f(\bar{x})$ replicated for each class

$f(\bar{x}, y = \text{Health}) = [1, 1, 0, 0, 0, 0, 0, 0, 0]$

$f(\bar{x}, y = \text{Sports}) = [0, 0, 0, 1, 1, 0, 0, 0, 0]$

Indicator (sent contains word i $\wedge$ y = Sports)

$\bar{w} = [2, 5.6, -3, 1.2, -3.1, 5.7, \ldots]$

Multiclass Perceptron + Logistic Regression $y = \{1,2,3\}$

Multiclass Perceptron

for t in epochs

  for i in data (diff weights)

$$y_{pred} = \arg\max_y \bar{w}_y^T f(\bar{x})$$

if $y_{pred} \neq y^{(i)}$

$$\bar{w}_{y_{pred}} \leftarrow \bar{w}_{y_{pred}} - \alpha f(\bar{x})$$

$$\bar{w}_{y^{(i)}} \leftarrow \bar{w}_{y^{(i)}} + \alpha f(\bar{x})$$

DF: $\bar{w} \leftarrow \bar{w} + \alpha f(\bar{x}, y^{(i)})$

Ex: [1,1,0] $y=1$ health

too many drug trials, too few patients

[1,0,1] $y=2$ sports

baseball players taking drugs

Assume "default" pred is $y=3$

[0 0 0 { 0 0 0 { 0 0 0]

$\bar{w}_{y_1}$ $\bar{w}_{y_2}$ $\bar{w}_{y_3}$

Update on [1,1,0], 1

$y_{pred} = 3$ $y^{(i)} = 1$ $\alpha = 1$

[1,1,0 { 0 0 0 { -1 -1 0]

$y_{pred} = 1$ $y^{(i)} = 2$

[0,1,-1 { 1,0,1 { -1 -1 0]

Multiclass LR

DF

$$P(y=\hat{y}\mid\bar{x}) = \frac{e^{\bar{w}^T f(\bar{x},\hat{y})}}{\sum_{y'\in y} e^{\bar{w}^T f(\bar{x},y')}}$$

↑ must sum to 1 over y

in binary LR:

$y=+1: e^{\bar{w}^T f(\bar{x})}$

$y=-1: 1 = e^{\bar{w}^T \bar{0}}$

$$\frac{\partial}{\partial \bar{w}} loss(\bar{x}^{(i)}, y^{(i)}, \bar{w}) = -f(\bar{x}, y^{(i)}) + \sum_{y'\in y} P(y'\mid\bar{x}) f(\bar{x}, y')$$

$P(y^{(i)}\mid\bar{x})\approx 1: \quad -f(\bar{x}, y^{(i)}) + f(\bar{x}, y^{(i)}) \approx 0$

$P(y_{bad}\mid\bar{x})\approx 1: \quad -f(\bar{x}, y^{(i)}) + f(\bar{x}, y_{bad})$

SGD: $-\alpha : \frac{\partial}{\partial \bar{w}}$

Text Classification

A Cancer Conundrum: Too Many Drug Trials, Too Few Patients

Breakthroughs in immunotherapy and a rush to develop profitable new treatments have brought a crush of clinical trials scrambling for patients.

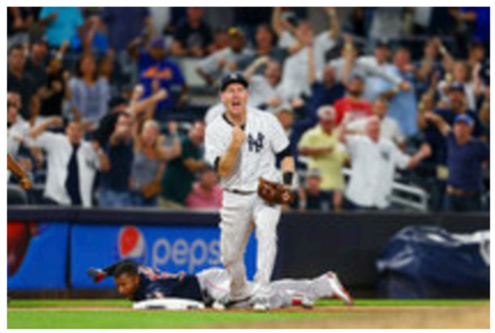
By GINA KOLATA

→ Health

Yankees and Mets Are on Opposite Tracks This Subway Series

As they meet for a four-game series, the Yankees are playing for a postseason spot, and the most the Mets can hope for is to play spoiler.

By FILIP BONDY



→ Sports

~20 classes

- 20 Newsgroups, Reuters, Yahoo! Answers, …

Textual Entailment

- Three-class task over sentence pairs
- Not clear how to do this with simple bag-of-words features

A soccer game with multiple males playing.

ENTAILS

Some men are playing a sport.

A black race car starts up in front of a crowd of people.

CONTRADICTS

A man is driving down a lonely road

A smiling costumed woman is holding an umbrella.

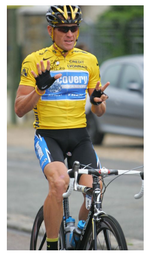
NEUTRAL

A happy woman in a fairy costume holds an umbrella.

Bowman et al. (2015)

Entity Disambiguation/Entity Linking

Although he originally won the event, the United States Anti-Doping Agency announced in August 2012 that they had disqualified **Armstrong** from his seven consecutive Tour de France wins from 1999–2005.



Lance Edward Armstrong is an American former professional road cyclist

Armstrong County is a county in Pennsylvania…

?

?

- 4,500,000 classes (all articles in Wikipedia)
- Needs a very different structure for $f(\boldsymbol{x},\boldsymbol{y})$ to accommodate so many classes

Authorship Attribution

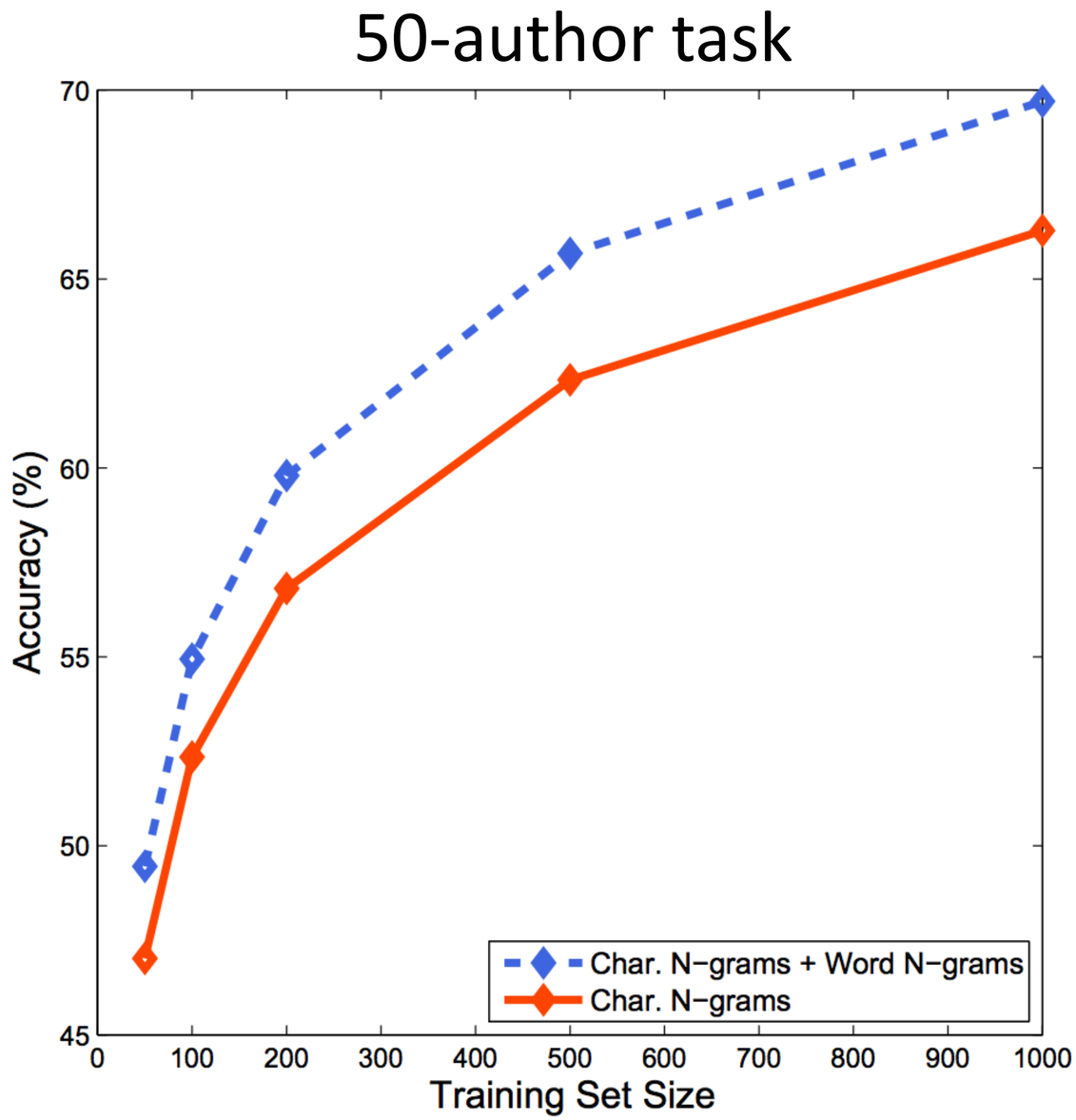
- Statistical methods date back to 1930s and 1940s
  - Based on handcrafted heuristics like stopword frequencies
  - Early work: Shakespeare's plays, Federalist papers (Hamilton v. Madison)

- Twitter: given a bunch of tweets, can we figure out who wrote them?
  - Schwartz et al. EMNLP 2013: 500M tweets, take 1000 users with at least 1000 tweets each

- Task: given a held-out tweet by one of the 1000 authors, who wrote it?

Authorship Attribution

- SVM with character 4-grams, words 2-grams through 5-grams



Schwartz et al. (2013)

Authorship Attribution

- k-signature: n-gram that appears in k% of the authors tweets but not appearing for anyone else — suggests why these are so effective

Signature Type	10%-signature	Examples
Character n-grams	‘ ^_^”	REF oh ok ^_^ Glad you found it!
		Hope everyone is having a good afternoon ^_^
		REF Smirnoff lol keeping the goose in the freezer ^_^
	‘**yew** ’	gurl **yew** serving me tea nooch
		REF about wen **yew** and ronnie see each other
		REF lol so **yew** goin to check out tini’s tonight huh???

Schwartz et al. (2013)

Authorship Attribution

- k-signature: n-gram that appears in k% of the authors tweets but not appearing for anyone else — suggests why these are so effective

Word n-grams	.. lal	REF aww those are cool where u get those.. how do ppl react.. lal
		Ludas album is gone be hott.. lal
		Dayum refs don't get injury timeouts.. lal.. get him off the field..
	smoochies , e3	I'm just back after takin' a very long, icy cold shower.....Shivering smoochies,E3 http://bit.ly/4CzzP9
		A blue stout or two would be nice as well, Purr!Blue smooth smoochies,E3 http://bit.ly/75D4fO
		That is soooooooooooooooooo unfair!Double smoochies,E3 http://bit.ly/07sXRGX

Schwartz et al. (2013)

Fairness in Classification

- Classifiers can be used to make real-world decisions:
  - Who gets an interview?
  - Who should we lend money to?
  - Is this online activity suspicious?
  - ~~Is someone a criminal based on their face?~~ **Don't do this!**

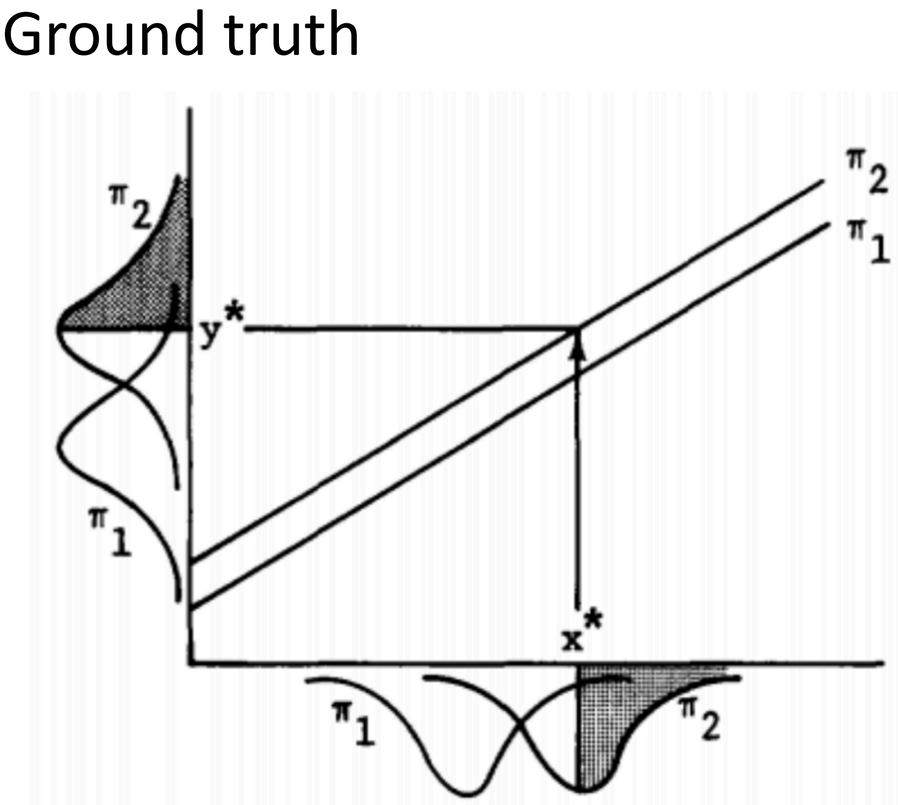
- Humans making these decisions are typically subject to anti-discrimination laws; how do we ensure classifiers are *fair* in the same way?
- Many other factors to consider when deploying classifiers in the real world (e.g., impact of a false positive vs. a false negative) but we'll focus on fairness here

Evaluating Fairness

Idea 1: Classifiers need to be evaluated beyond just accuracy

- T. Anne Cleary (1966-1968): a test is biased if prediction on a subgroup makes *consistent* nonzero prediction errors compared to the aggregate

- Individuals of X group could still score lower on average. But the *errors* should not be consistently impacting X



- Member of π_1 has a test result higher than a member of π_2 for the same ground truth ability. Test penalizes π_2

Hutchinson and Mitchell (2018)

Evaluating Fairness

Idea 1: Classifiers need to be evaluated beyond just accuracy

- Thorndike (1971), Petersen and Novik (1976): fairness in classification: ratio of predicted positives to ground truth positives must be approximately the same for each group
  - Group 1: 50% positive movie reviews. Group 2: 60% positive movie reviews
  - A classifier classifying 50% positive in both groups is unfair, regardless of accuracy

- Allows for different criteria across groups: imposing different classification thresholds actually can give a fairer result

- Can’t we just make our classifiers not depend on sensitive features like gender?

Petersen and Novik (1976)

Hutchinson and Mitchell (2018)

Discrimination

Idea 2: It is easy to build classifiers that discriminate even *without meaning to*

- A feature might correlate with minority group X and penalize that group:
  - Bag-of-words features can identify particular dialects of English like AAVE or code-switching (using two languages). Impacts classification on social media, etc.
  - ZIP code as a feature is correlated with race

- Reuters: "Amazon scraps secret AI recruiting tool that showed bias against women"
  - "Women's X" organization, women's colleges were negative-weight features
  - Accuracy will not catch these problems, very complex to evaluate depending on what humans did in the **actual** recruiting process

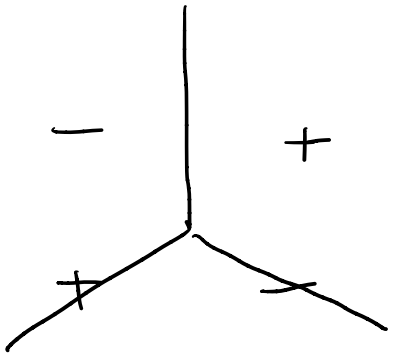
Credit: https://www.reuters.com/article/us-amazon-com-jobs-automation-insight/amazon-scraps-secret-ai-recruiting-tool-that-showed-bias-against-women-idUSKCN1MK08G

Takeaways

- What minority groups in the population should I be mindful of? (Review sentiment: movies with female directors, foreign films, …)
- Can I check one of these fairness criteria?
- Do aspects of my system or features it uses introduce potential correlations with protected classes or minority groups?

Neural Nets

	g	b	n
good	1	0	0
bad	0	1	0
not good	1	0	1
not bad	0	1	1



NNs: transform the data into a latent feature space

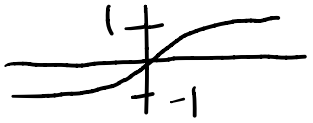
$\bar{w}^T f(\bar{x})$ replace $f(\bar{x})$ with a nonlinear function of the original $f(\bar{x})$

Define $\bar{z} = g(V f(\bar{x}))$. Classify with $\bar{w}^T \bar{z}(\bar{x})$

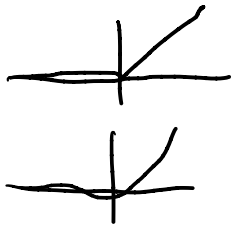
g: nonlinearity; V: dxn matrix; $f(\bar{x})$: n-dimensional feat vector

How can $V \circ g$ give us useful latent features?

$g = \tanh$



$g = \text{ReLU}$



NN example

Suppose $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$

$g = \tanh$

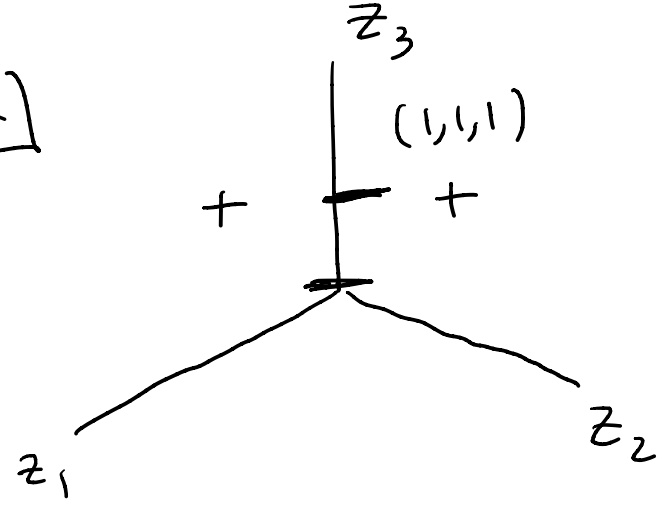
$\tanh(0) = 0$, $\tanh(1) \approx 1$

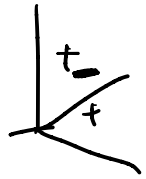
$\tanh(2) \approx 1$

$\bar{z} = [\tanh(x_1), \tanh(x_2), \tanh(x_1 + x_2)]$

$\bar{z} = g(V f(\bar{x}))$

(x_1, x_2)





Neural Networks

$$\mathbf{z} = g(V f(\mathbf{x}) + \mathbf{b})$$

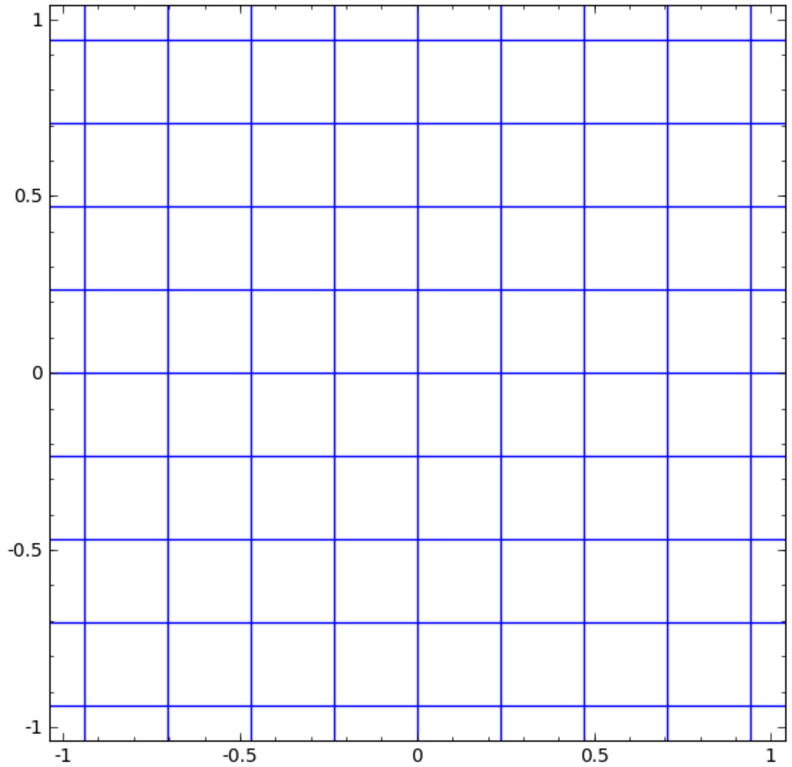
Nonlinear transformation

Warp space

Shift

$$y_{\text{pred}} = \text{argmax}_y \mathbf{w}_y^\top \mathbf{z}$$

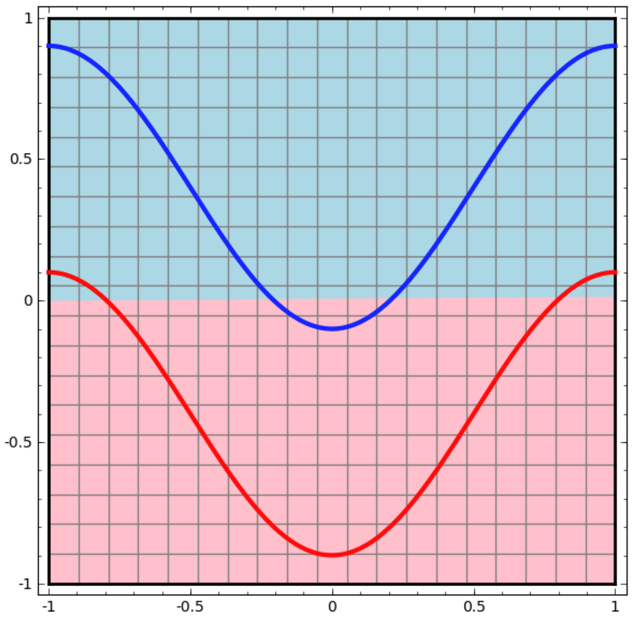
- Ignore shift / +**b** term for the rest of the course



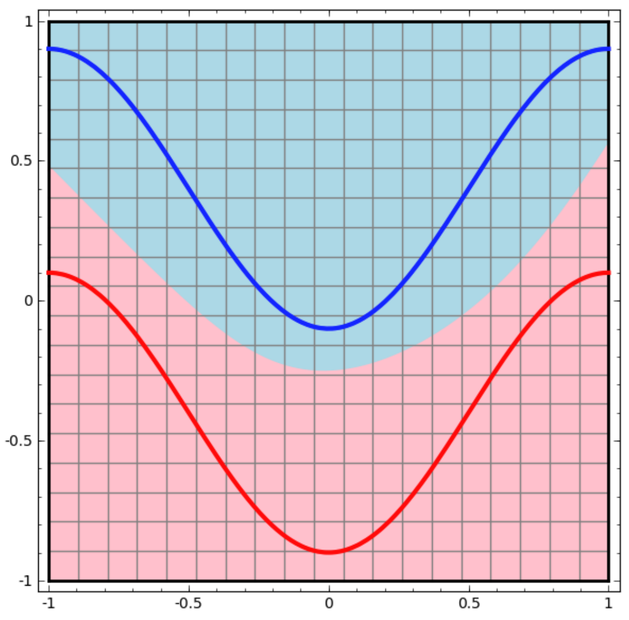
Taken from http://colah.github.io/posts/2014-03-NN-Manifolds-Topology/

Neural Networks

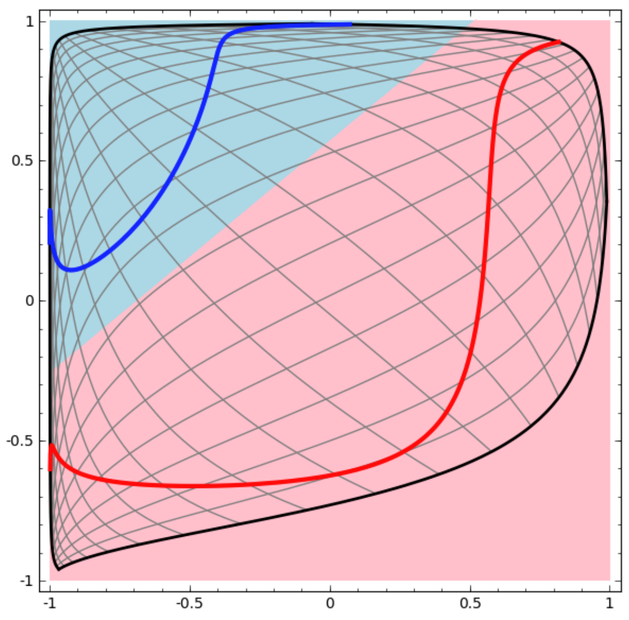
Linear classifier



Neural network



Linear classification in the transformed space!



Taken from http://colah.github.io/posts/2014-03-NN-Manifolds-Topology/

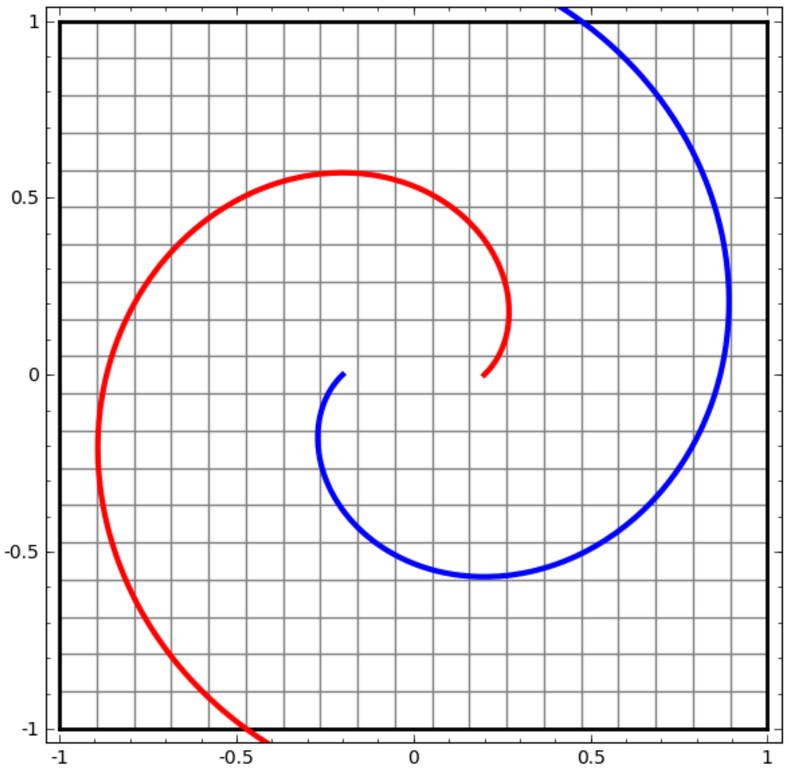
Deep Neural Networks

$$\mathbf{z}_1 = g(V_1 f(\mathbf{x}))$$

$$\mathbf{z}_2 = g(V_2 \mathbf{z}_1)$$

$$\dots$$

$$y_{\text{pred}} = \text{argmax}_y \mathbf{w}_y^\top \mathbf{z}_n$$



Taken from http://colah.github.io/posts/2014-03-NN-Manifolds-Topology/

Vectorization and Softmax

$$P(y|\mathbf{x}) = \frac{\exp(\mathbf{w}_y^\top f(\mathbf{x}))}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{w}_{y'}^\top f(\mathbf{x}))}$$

- Single scalar probability

- Three classes, "different weights"

$$\begin{array}{lll} \mathbf{w}_1^\top f(\mathbf{x}) & \text{-1.1} & \text{0.036} \\ \mathbf{w}_2^\top f(\mathbf{x}) = & \text{2.1} & \xrightarrow{\text{softmax}} \quad \text{0.89} \\ \mathbf{w}_3^\top f(\mathbf{x}) & \text{-0.4} & \text{0.07} \end{array}$$

class probs

- Softmax operation = "exponentiate and normalize"

- We write this as: $\mathrm{softmax}(W f(\mathbf{x}))$

Logistic Regression with NNs

$$P(y|\mathbf{x}) = \frac{\exp(\mathbf{w}_y^\top f(\mathbf{x}))}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{w}_y^\top f(\mathbf{x}))}$$

- Single scalar probability

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W f(\mathbf{x}))$$

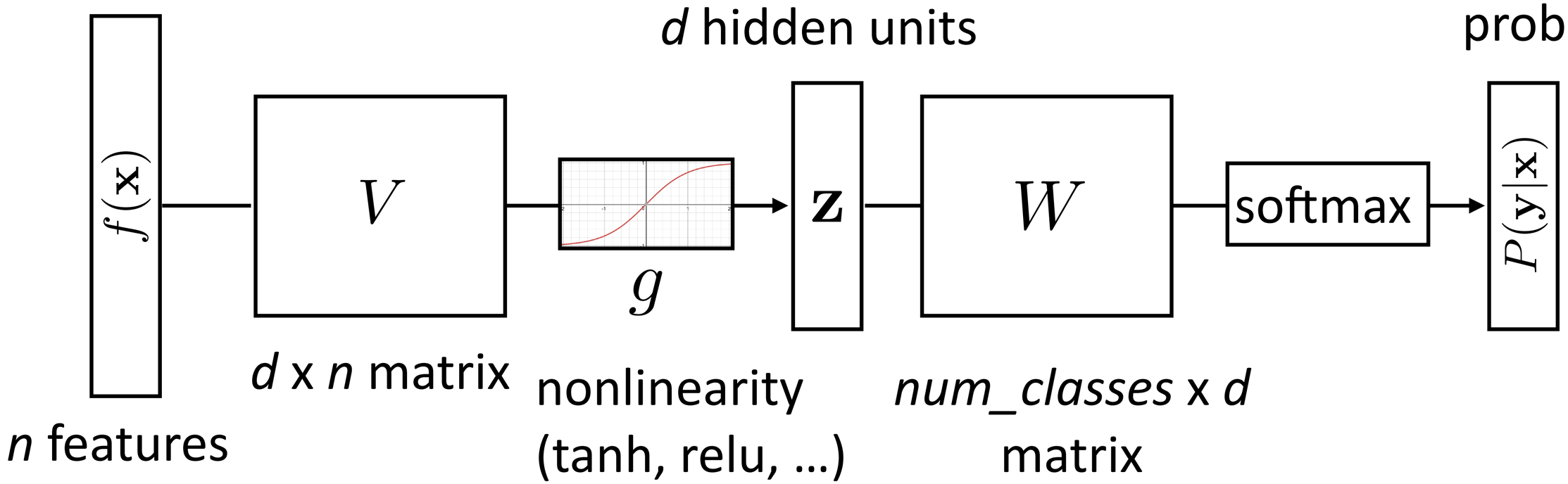
- Weight vector per class; W is [num classes x num feats]

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$$

- Now one hidden layer

Neural Networks for Classification

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\, g(V\, f(\mathbf{x})))$$



Training Neural Networks

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\mathbf{z}) \qquad \mathbf{z} = g(V f(\mathbf{x}))$$

- Maximize log likelihood of training data

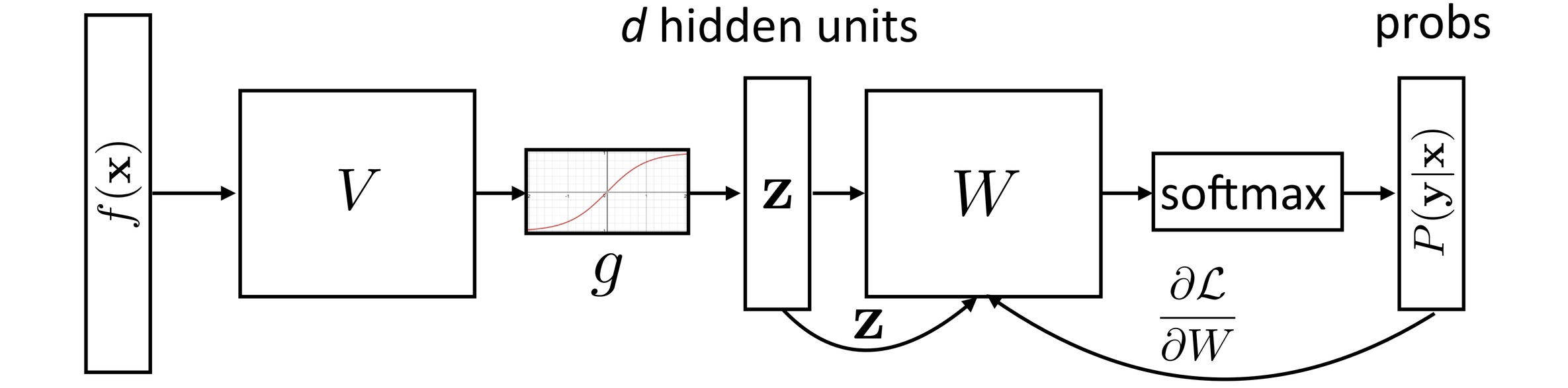
$$\mathcal{L}(\mathbf{x}, i^*) = \log P(y = i^*|\mathbf{x}) = \log\left(\text{softmax}(W\mathbf{z}) \cdot e_{i^*}\right)$$

- *i**: index of the gold label
- e_i: 1 in the *i*th row, zero elsewhere. Dot by this = select *i*th index

$$\mathcal{L}(\mathbf{x}, i^*) = W\mathbf{z} \cdot e_{i^*} - \log \sum_j \exp(W\mathbf{z}) \cdot e_j$$

Backpropagation Picture

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\, g(V\, f(\mathbf{x})))$$

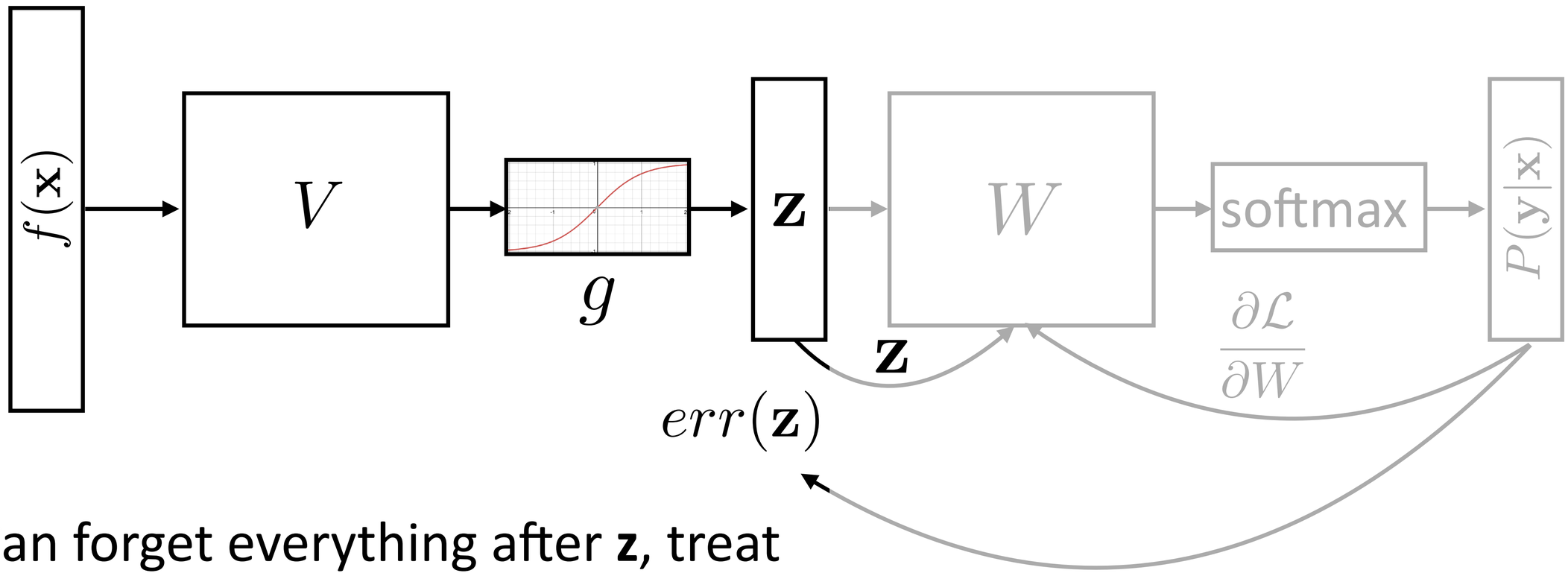


n features

- Gradient w.r.t. *W*: looks like logistic regression, can be computed treating **z** as the features

Backpropagation Picture

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\, g(V\, f(\mathbf{x})))$$



- Can forget everything after **z**, treat it as the output and keep backpropping

Computing Gradients with Backprop

$$\mathcal{L}(\mathbf{x}, i^*) = W\mathbf{z} \cdot e_{i^*} - \log \sum_j \exp(W\mathbf{z}) \cdot e_j$$

$$\mathbf{z} = g(Vf(\mathbf{x}))$$

Activations at hidden layer

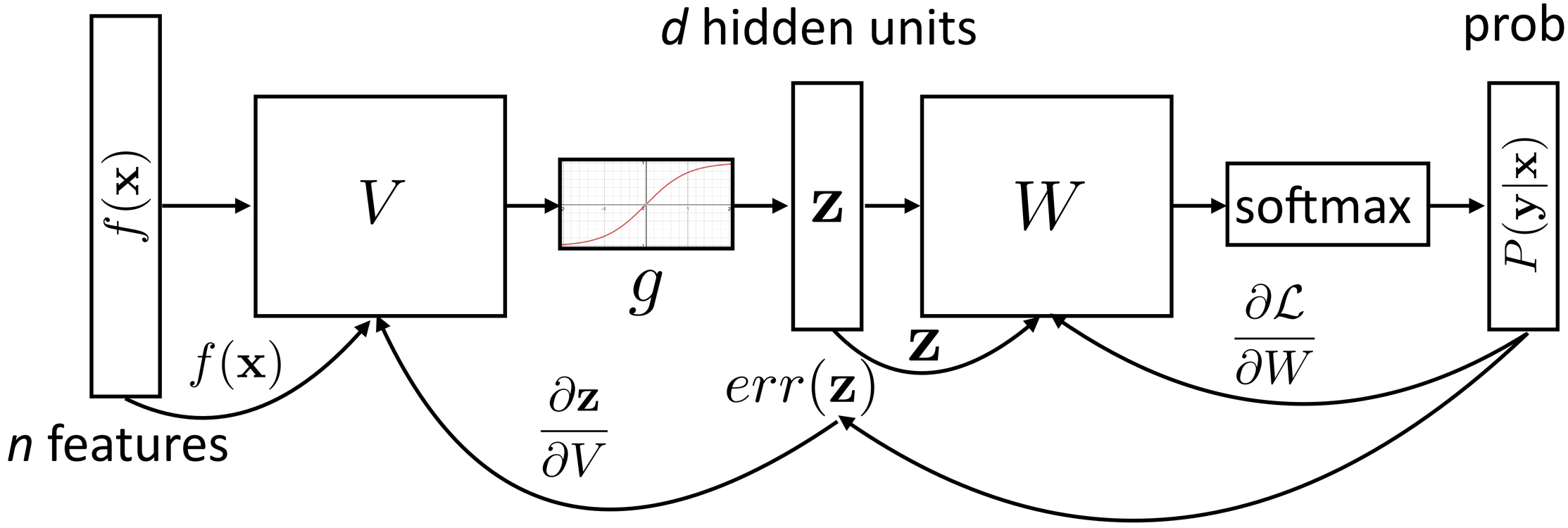
- Gradient with respect to *V*: apply the chain rule

$$\frac{\partial \mathcal{L}(\mathbf{x}, i^*)}{\partial V_{ij}} = \boxed{\frac{\partial \mathcal{L}(\mathbf{x}, i^*)}{\partial \mathbf{z}}} \boxed{\frac{\partial \mathbf{z}}{\partial V_{ij}}} \qquad \frac{\partial \mathbf{z}}{\partial V_{ij}} = \boxed{\frac{\partial g(\mathbf{a})}{\partial \mathbf{a}}} \boxed{\frac{\partial \mathbf{a}}{\partial V_{ij}}} \qquad \mathbf{a} = Vf(\mathbf{x})$$

- First term: *err*(**z**); represents gradient w.r.t. **z**
- First term: gradient of nonlinear activation function at ***a*** (depends on current value)
- Second term: gradient of linear function

Backpropagation Picture

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\, g(V\, f(\mathbf{x})))$$



- Combine backward gradients with forward-pass products

Computation Graphs

- Computing gradients is hard!
- Automatic differentiation: instrument code to keep track of derivatives

```
y = x * x    ⟶ (codegen)    (y,dy) = (x * x, 2 * x * dx)
```

- Computation is now something we need to reason about symbolically; use a library like PyTorch (or Tensorflow)

PyTorch

- Framework for defining computations that provides easy access to derivatives

- Module: defines a neural network (can use wrap other modules which implement predefined layers)

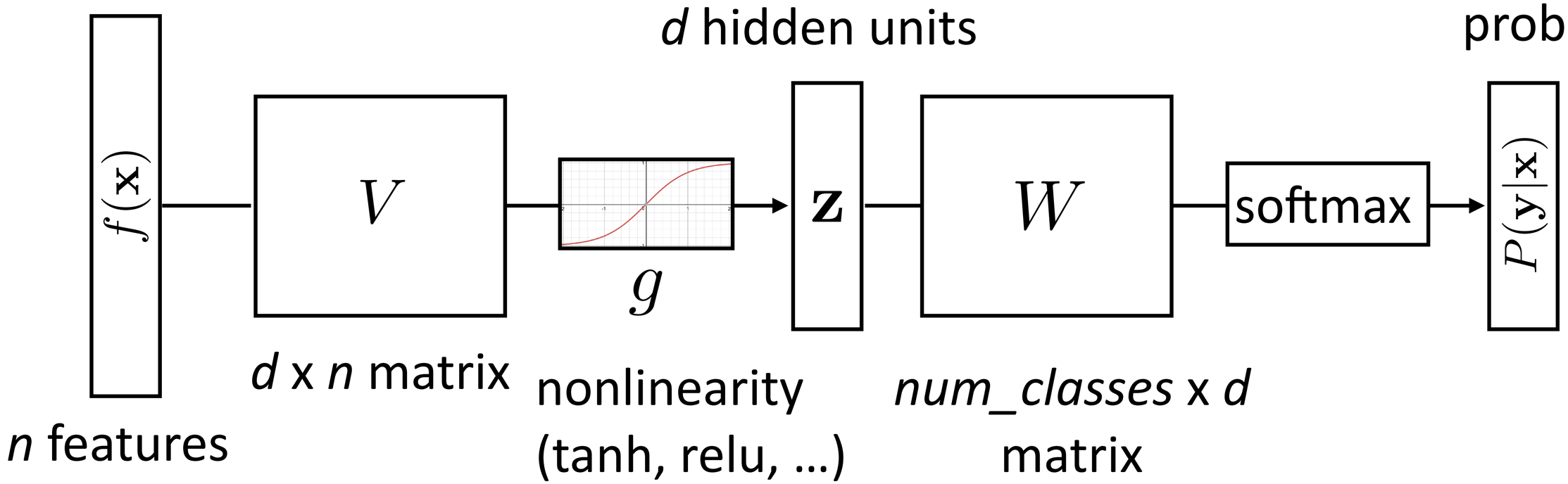
- If forward() uses crazy math, you have to write backward yourself

torch.nn.Module

```
# Takes an example x and computes result
forward(x):
    …
# Computes gradient after forward() is called
backward(): # produced automatically
    …
```

Neural Networks for Classification

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\, g(V\, f(\mathbf{x})))$$



Computation Graphs in PyTorch

- Define forward pass for $P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W g(V f(\mathbf{x})))$

```
class FFNN(nn.Module):
    def __init__(self, input_size, hidden_size, out_size):
        super(FFNN, self).__init__()
        self.V = nn.Linear(input_size, hidden_size)
        self.g = nn.Tanh() # or nn.ReLU(), sigmoid()...
        self.W = nn.Linear(hidden_size, out_size)
        self.softmax = nn.Softmax(dim=0)

def forward(self, x):
    return self.softmax(self.W(self.g(self.V(x))))
```

(syntactic sugar for forward)

Input to Network

- Whatever you define with torch.nn needs its input as some sort of tensor, whether it's integer word indices or real-valued vectors

```
def form_input(x) -> torch.Tensor:
    # Index words/embed words/etc.
    return torch.from_numpy(x).float()
```

- torch.Tensor is a different datastructure from a numpy array, but you can translate back and forth fairly easily

- Note that **translating out of PyTorch will break backpropagation**; don't do this inside your Module

Training and Optimization

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\,g(V\,f(\mathbf{x})))$$

one-hot vector
of the label
(e.g., [0, 1, 0])

```
ffnn = FFNN(inp, hid, out)
optimizer = optim.Adam(ffnn.parameters(), lr=lr)
for epoch in range(0, num_epochs):
   for (input, gold_label) in training_data:
      ffnn.zero_grad() # clear gradient variables
      probs = ffnn.forward(input)
      loss = torch.neg(torch.log(probs)).dot(gold_label)
      loss.backward()
      optimizer.step()
```

negative log-likelihood of correct answer

Computing Gradients with Backprop

```
class FFNN(nn.Module):
    def __init__(self, inp, hid, out):
        super(FFNN, self).__init__()
        self.V = nn.Linear(inp, hid)
        self.g = nn.Tanh()
        self.W = nn.Linear(hid, out)
        self.softmax = nn.Softmax(dim=0)
        nn.init.uniform(self.V.weight)
```

- Initializing to a nonzero value is critical!

Training a Model

Define modules, etc.

Initialize weights and optimizer

For each epoch:

For each batch of data:

Zero out gradient

Compute loss on batch

Autograd to compute gradients and take step on optimizer

[Optional: check performance on dev set to identify overfitting]

Run on dev/test set

Batching

- Batching data gives speedups due to more efficient matrix operations

- Need to make the computation graph process a batch at the same time

```
# input is [batch_size, num_feats]
# gold_label is [batch_size, num_classes]
def make_update(input, gold_label)
    ...
    probs = ffnn.forward(input) # [batch_size, num_classes]
    loss = torch.sum(torch.neg(torch.log(probs)).dot(gold_label))
    ...
```

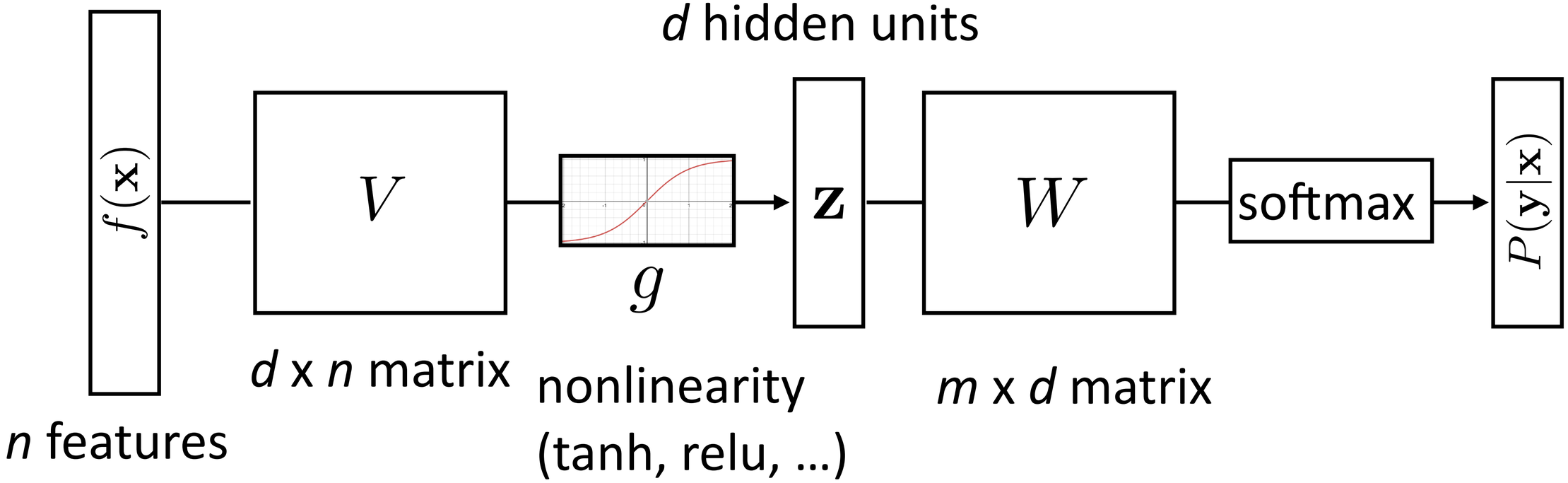
- Batch sizes from 1-100 often work well

Training Basics

- Basic formula: compute gradients on batch, use first-order optimization method (SGD, Adagrad, etc.)
- How to initialize? How to regularize? What optimizer to use?
- This segment: some practical tricks. Take deep learning or optimization courses to understand this further

How does initialization affect learning?

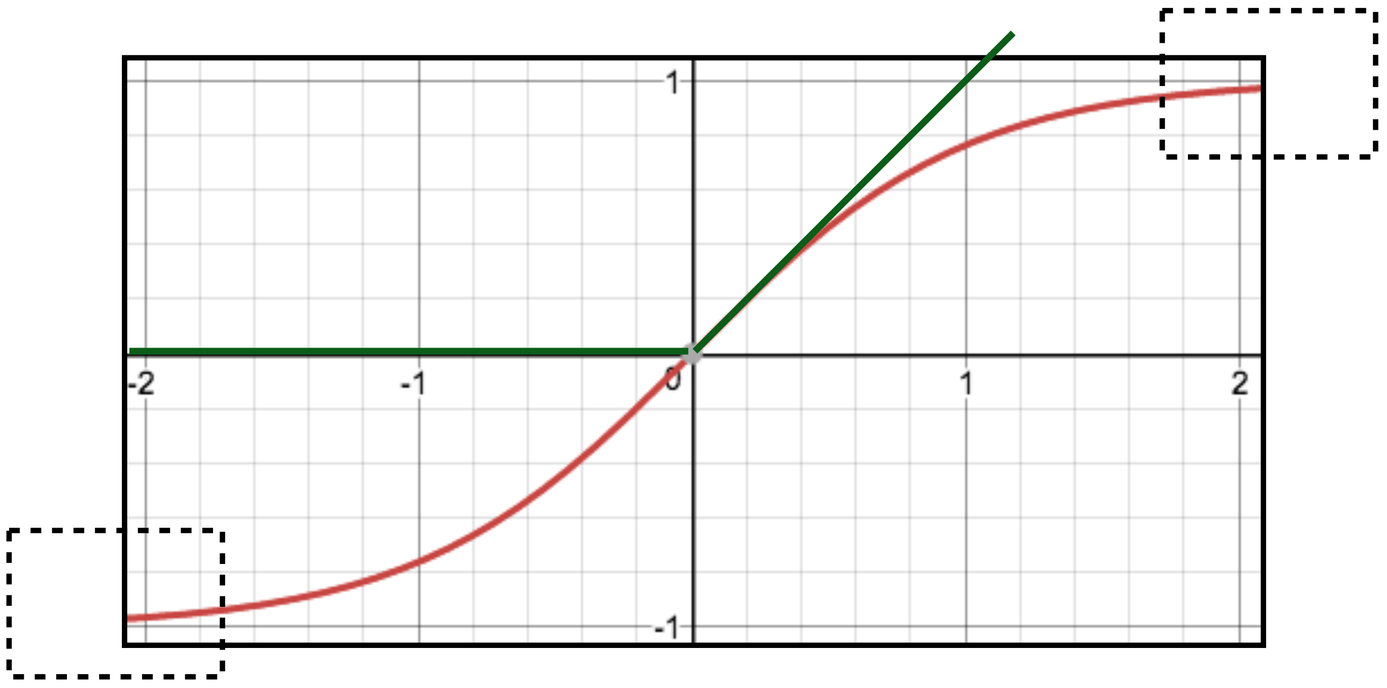
$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\, g(V\, f(\mathbf{x})))$$



- How do we initialize V and W? What consequences does this have?
- *Nonconvex* problem, so initialization matters!

How does initialization affect learning?

- Nonlinear model…how does this affect things?



- If cell activations are too large in absolute value, gradients are small

- ReLU: larger dynamic range (all positive numbers), but can produce big values, can break down if everything is too negative

Initialization

1) Can't use zeroes for parameters to produce hidden layers: all values in that hidden layer are always 0 and have gradients of 0, never change

2) Initialize too large and cells are saturated

- Can do random uniform / normal initialization with appropriate scale
  - Glorot initializer: $U\left[-\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}, +\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}\right]$
    - Want variance of inputs and gradients for each layer to be the same
- Batch normalization (Ioffe and Szegedy, 2015): periodically shift+rescale each layer to have mean 0 and variance 1 over a batch (useful if net is deep)

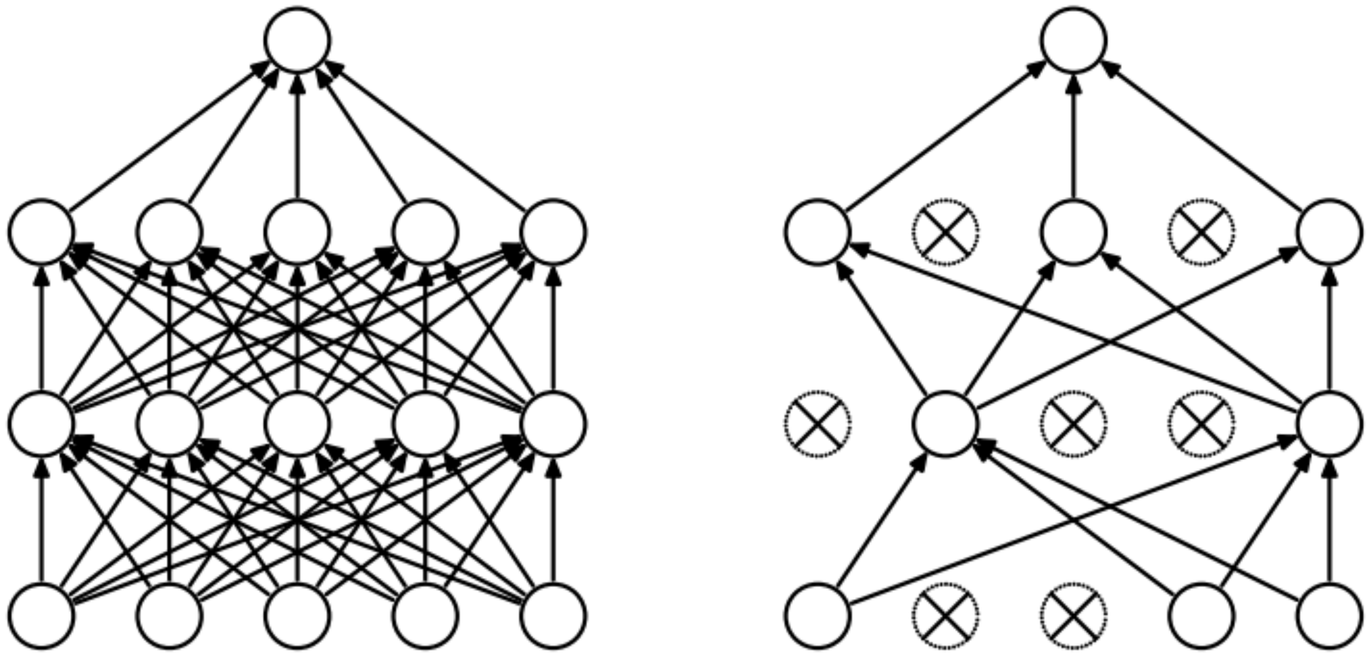
Dropout

- Probabilistically zero out parts of the network during training to prevent overfitting, use whole network at test time

- Form of stochastic regularization

- Similar to benefits of ensembling: network needs to be robust to missing signals, so it has redundancy

- One line in Pytorch/Tensorflow



(a) Standard Neural Net

(b) After applying dropout.

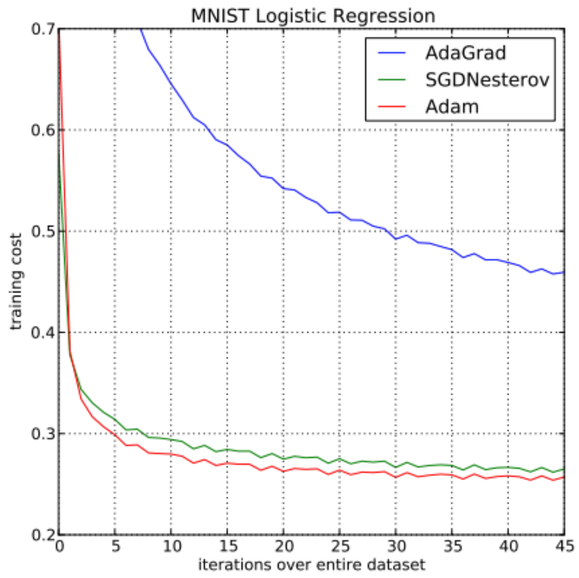
Srivastava et al. (2014)

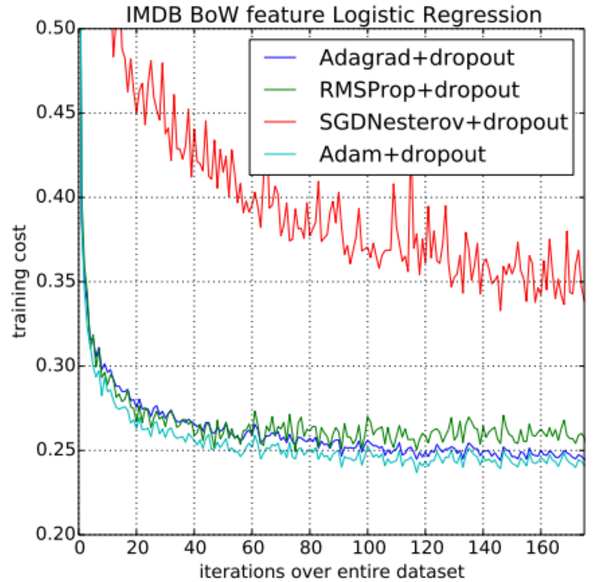
Optimizer

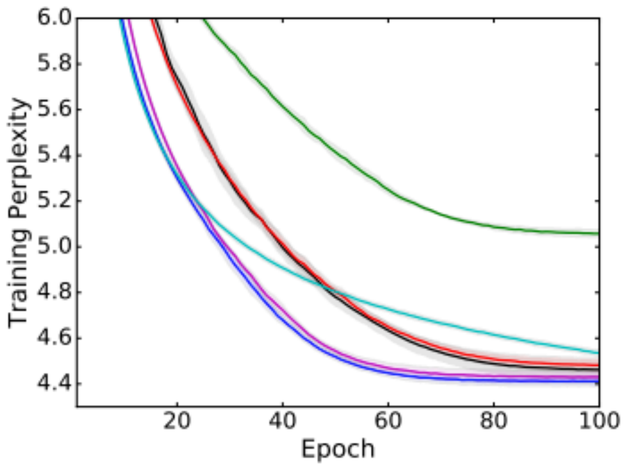
- Adam (Kingma and Ba, ICLR 2015): very widely used. Adaptive step size + momentum

- Wilson et al. NeurIPS 2017: adaptive methods can actually perform badly at test time (Adam is in pink, SGD in black)

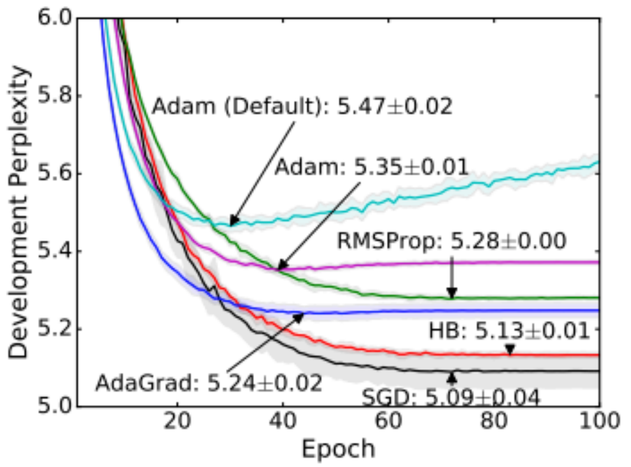
- One more trick: **gradient clipping** (set a max value for your gradients)







(e) Generative Parsing (Training Set)



(f) Generative Parsing (Development Set)

WEEK 3

Word Embeddings

Segments in this section

seg-17.pdf - Word Embeddings p. 82
seg-18.pdf - Skip-gram p. 84
seg-19.pdf - word2vec Variants, GloVe, fastText p. 87
seg-20.pdf - Bias in Word Embeddings p. 96
seg-21.pdf - Applying Embeddings; Deep Averaging Networks p. 99

Key points (added notes)

- Distributional hypothesis: you shall know a word by the company it keeps.
- Skip-gram P(c|w) = exp(v_w.c_c)/sum; CBOW predicts the word from summed context; negative sampling ~ PMI factorization.
- GloVe (log co-occurrence regression), fastText (character n-grams).
- Bolukbasi debiasing neutralizes a gender direction; Gonen & Goldberg: bias remains. DANs = average embeddings + FFNN.

EXAM: list skip-gram pairs, compute softmax over contexts, dot product vs cosine, parameter counts / runtime O(|V|d).

Note: segments 17 and 18 are handwritten.

Word Embeddings

$V = 10{,}000$ words

movie was good $\xrightarrow{\text{BoW}}$ $[0\ 1\ 0\ 0\ 0\ 1\ 0\ 0\ \ldots\ 1\ \ldots]$ (was, good, movie)

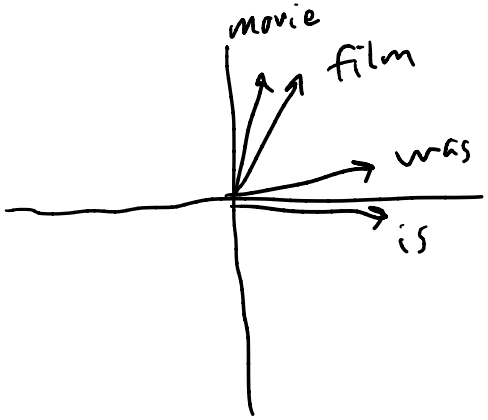
10,000-dim vector

$= [0 - 1 - 0] + [0\ 1\ 0\ -] + [0 - 1 - 0]$ (movie, was, good)

film is great = $[1\ \ldots\ 1\ \ldots\ 1]$ (film, is, great)

orthogonal to "movie was good"

Word embeddings: low-dimensional representations of words [50 - 300] capturing their similarity



How to learn embeddings

JR Firth 1957 Distributional hypothesis

"You shall know a word by the company it keeps"

I watched the movie
I watched the film
The film inspired me
The movie inspired me

I developed the film
in the darkroom

Brown Clusters, ...

Mikolov et al. 2013 "word2vec"

Each word w → $\bar{v}_w$ word vector
→ $\bar{c}_w$ context vector

Predict each word's context given that word

Skip-gram

Input: large corpus of sentences

Output: $\bar{v}_w$, $\bar{c}_w$ for each word type w

Hyperparams: word vector dim d ($\sim 50-300$)
window size k (assume $k=1$)

The film inspired

word	context
film	inspired
film	The

Take all neighbors of each word token up to k positions away

Skip-gram: probabilistic model of context | word

$$P(\text{context}=y \mid \text{word}=x) = \frac{\exp(\bar{v}_x \cdot \bar{c}_y)}{\sum_{y' \in V} \exp(\bar{v}_x \cdot \bar{c}_{y'})}$$

$\bar{v}, \bar{c}$ model params

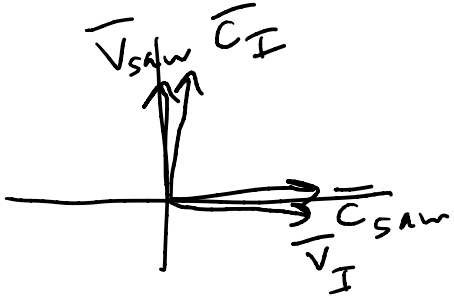
If $\bar{v}_x$ is similar to $\bar{c}_y$, y is likely to be in x's context

sum over vocab $\rightarrow y' \in V$

$2 \cdot |V| \times d$ params in model

Ex Corpus = I saw

$\bar{v}_I = [1,0]$ $\bar{v}_{saw} = [0,1]$



word	context
I	saw
saw	I

If $\bar{c}_{saw} = [1,0]$ and $\bar{c}_I = [0,1]$, what is $P(\text{context} \mid \text{word} = \text{saw})$?

$\exp(\bar{v}_{saw} \cdot \bar{c}_I) \approx 3$ $\exp(\bar{v}_{saw} \cdot \bar{c}_{saw}) \approx 1$ ↑ Vocab

$P(\text{context} = I \mid \text{word} = saw) = \frac{3}{4}$ saw|saw $\frac{1}{4}$

Training

Maximize $\sum_{(x,y)} \log P(\text{context}=y \mid \text{word}=x)$

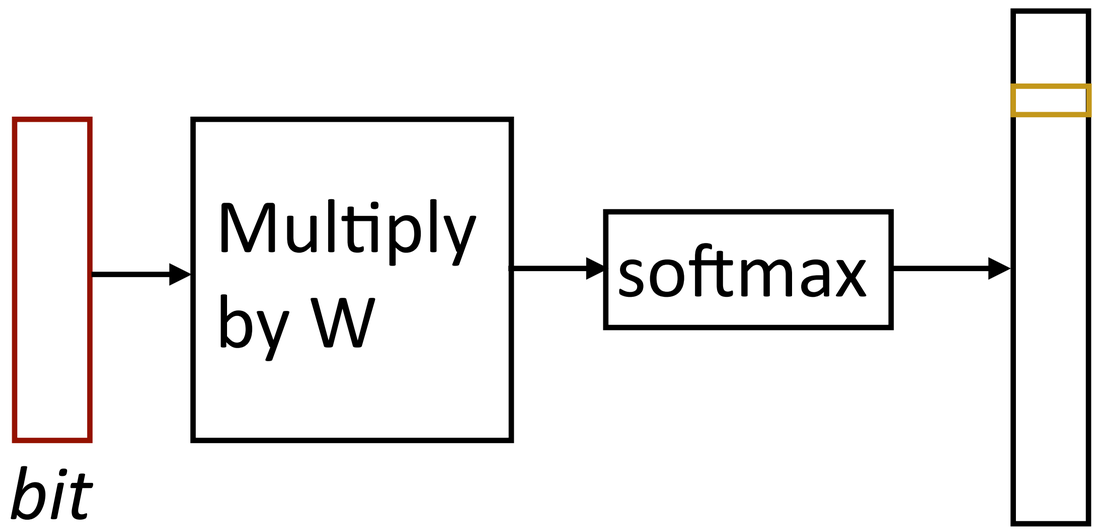
pairs in data

"Impossible" problem: cannot drive $P \to 1$

Initialize params randomly

Skip-Gram

- Predict each word of context from word in turn, up to distance *k*

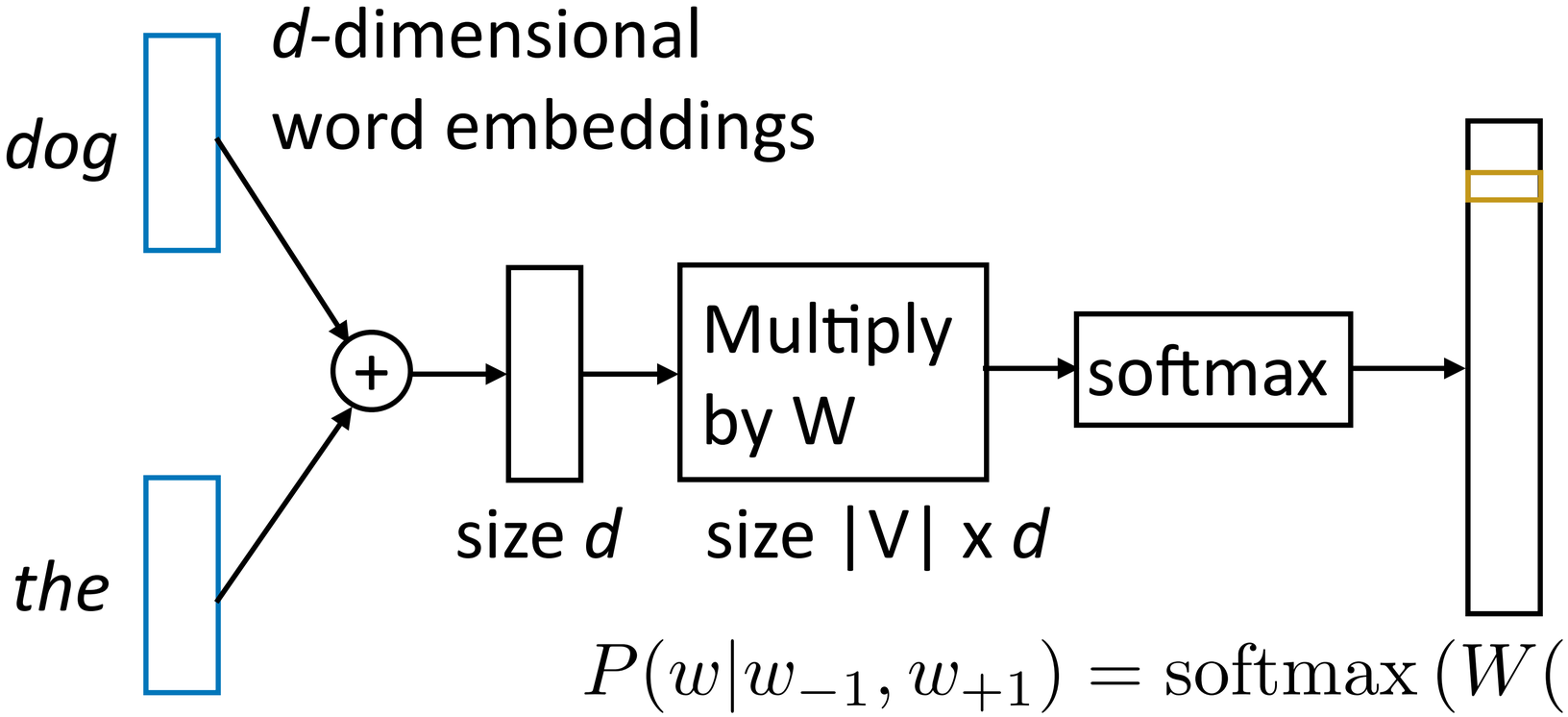


- Another training example: *bit* -> *the*
- Parameters: *d* x |V| word vectors, |V| x *d* context vectors (stacked into a matrix W)
- Why skip-gram? With window size >1, we predict a context word skipping over intermediate words

Mikolov et al. (2013)

Continuous Bag-of-words

- Predict *word* from multiple words of *context*



gold label = *bit,*
no manual labeling
required!

$$P(w|w_{-1}, w_{+1}) = \text{softmax}\left(W(c(w_{-1}) + c(w_{+1}))\right)$$

- Parameters: *d* x |V| (one *d*-length context vector per voc word),
  |V| x *d* word vectors (in matrix W)

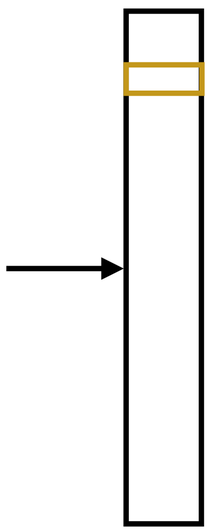
Mikolov et al. (2013)

Hierarchical Softmax

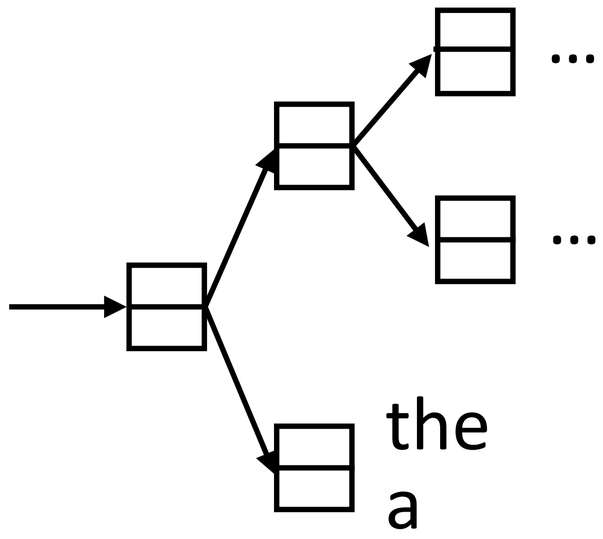
CBOW: $P(w|w_{-1}, w_{+1}) = \text{softmax}\left(W(c(w_{-1}) + c(w_{+1}))\right)$

Skip-gram: $P(w'|w) = \text{softmax}(We(w))$

- Matmul + softmax over |V| is very slow to compute for both techniques



- Standard softmax:
  |V| dot products of size *d*



- Hierarchical softmax:
  log(|V|) dot products of size *d,*
  |V| x *d* parameters

- Huffman encode vocabulary, use binary classifiers to decide which branch to take
- log(|V|) binary decisions

Mnih and Hinton (2008)

Skip-gram with Negative Sampling

- Are there alternative ways to learn vectors while avoiding O(|V|) term?
- Take (word, context) pairs and classify them as "real" or not. Create random negative examples by sampling from unigram distribution

(*bit, the*) => +1

(*bit, cat*) => -1

(*bit, a*) => -1

(*bit, fish*) => -1

$$P(y=1|w,c) = \frac{e^{w \cdot c}}{e^{w \cdot c} + 1}$$

words in similar contexts select for similar *c* vectors

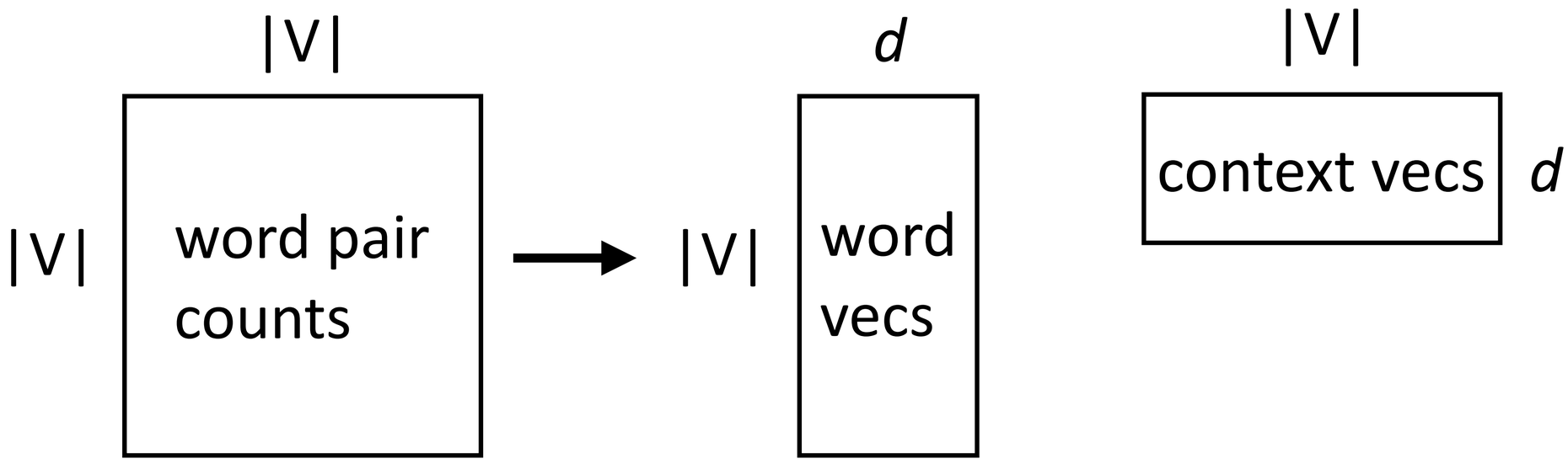
- *d* x |V| vectors, *d* x |V| context vectors (same # of params as before)
- Objective = $\log P(y=1|w,c) + \frac{1}{k}\sum_{i=1}^{n} \log P(y=0|w_i,c)$

sampled

Mikolov et al. (2013)

Connections with Matrix Factorization

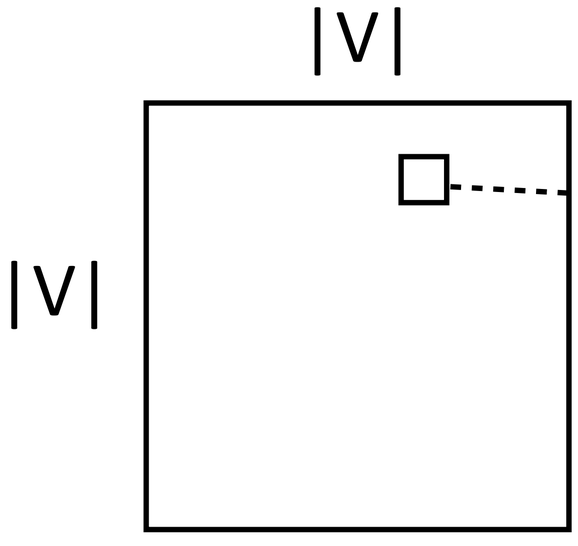
- Skip-gram model looks at word-word co-occurrences and produces two types of vectors



- Looks almost like a matrix factorization…can we interpret it this way?

Levy et al. (2014)

Skip-gram as Matrix Factorization



num negative samples

$$M_{ij} = \text{PMI}(w_i, c_j) - \log k$$

$$\text{PMI}(w_i, c_j) = \frac{P(w_i, c_j)}{P(w_i)P(c_j)} = \frac{\frac{\text{count}(w_i, c_j)}{D}}{\frac{\text{count}(w_i)}{D}\frac{count(c_j)}{D}}$$

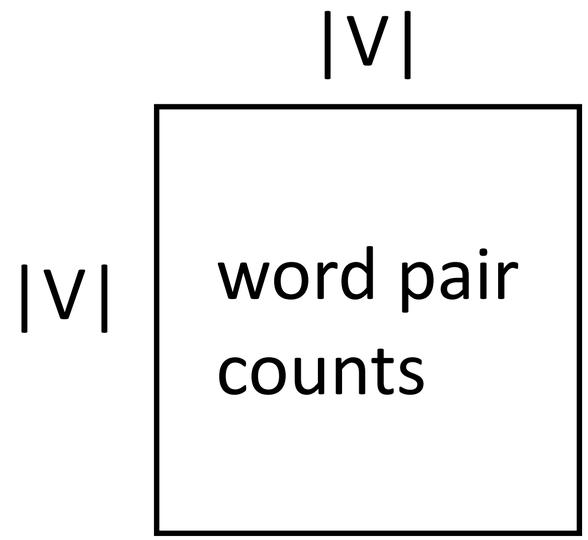
Skip-gram objective *exactly* corresponds to factoring this matrix:

- *If* we sample negative examples from the unigram distribution over words
- …and it's a *weighted* factorization problem (weighted by word freq)

Levy et al. (2014)

GloVe (Global Vectors)

- Also operates on counts matrix, weighted regression on the log co-occurrence matrix



- Objective = $\sum_{i,j} f(\text{count}(w_i, c_j)) \left(w_i^\top c_j + a_i + b_j - \log \text{count}(w_i, c_j)\right)^2$
- Constant in the dataset size (just need counts), quadratic in voc size
- By far the most common word vectors used today (5000+ citations)

Pennington et al. (2014)

fastText: Sub-word Embeddings

- Same as SGNS, but break words down into n-grams with n = 3 to 6

  where:
  3-grams: <wh, whe, her, ere, re>
  4-grams: <whe, wher, here, ere>,
  5-grams: <wher, where, here>,
  6-grams: <where, where>

- Replace $w \cdot c$ in skip-gram computation with $\left(\sum_{g \in \text{ngrams}} w_g \cdot c \right)$

Bojanowski et al. (2017)

Pre-trained Models: ELMo, GPT, BERT

- These encode “subwords” rather than words. Underscore indicates that the following token continues the existing word

  and there were no re_ fueling stations anywhere

  one of the city ’s more un_ princi_ pled real estate agents

- Any word is either in the subword vocabulary or can be expressed as a sequence of subwords in the vocabulary

- Embeddings are computed using RNNs and Transformers. We can’t just look up an embedding for each word, but actually need to run a model

- Learn embeddings through language modeling (discussed in the second half of the course)

Bias in Word Embeddings

- Word embeddings are trained from the whole web, and this data encodes biases about the real world

- Identify *she* - *he* axis in word vector space, project words onto this axis

- Nearest neighbor of (b - a + c)

Extreme *she* occupations

1. homemaker	2. nurse	3. receptionist
4. librarian	5. socialite	6. hairdresser
7. nanny	8. bookkeeper	9. stylist
10. housekeeper	11. interior designer	12. guidance counselor

Extreme *he* occupations

1. maestro	2. skipper	3. protege
4. philosopher	5. captain	6. architect
7. financier	8. warrior	9. broadcaster
10. magician	11. figher pilot	12. boss

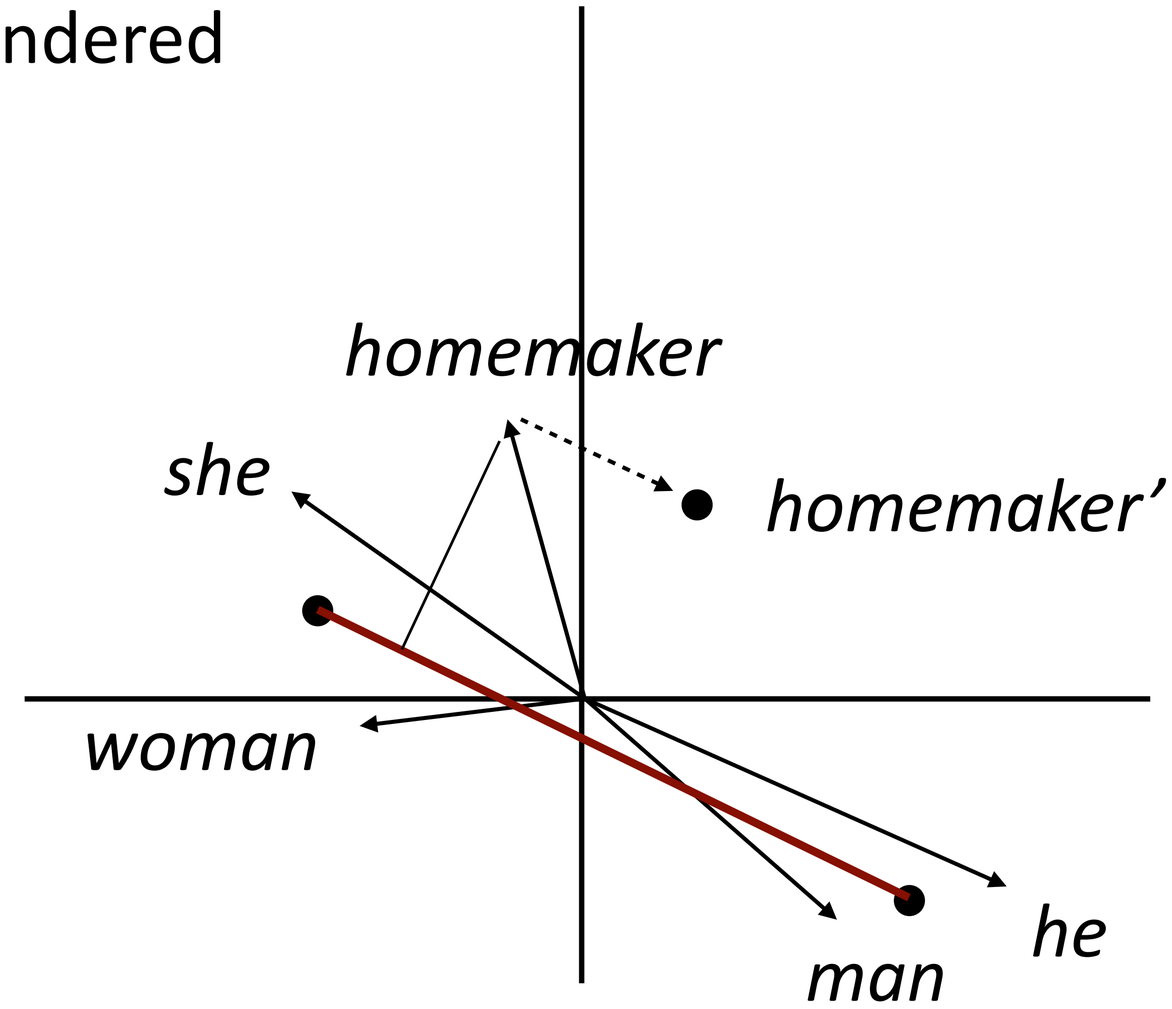
Bolukbasi et al. (2016)

Racial Analogies	
black → homeless	caucasian → servicemen
caucasian → hillbilly	asian → suburban
asian → laborer	black → landowner
Religious Analogies	
jew → greedy	muslim → powerless
christian → familial	muslim → warzone
muslim → uneducated	christian → intellectually

Manzini et al. (2019)

Debiasing

- Identify gender subspace with gendered words
- Project words onto this subspace
- Subtract those projections from the original word

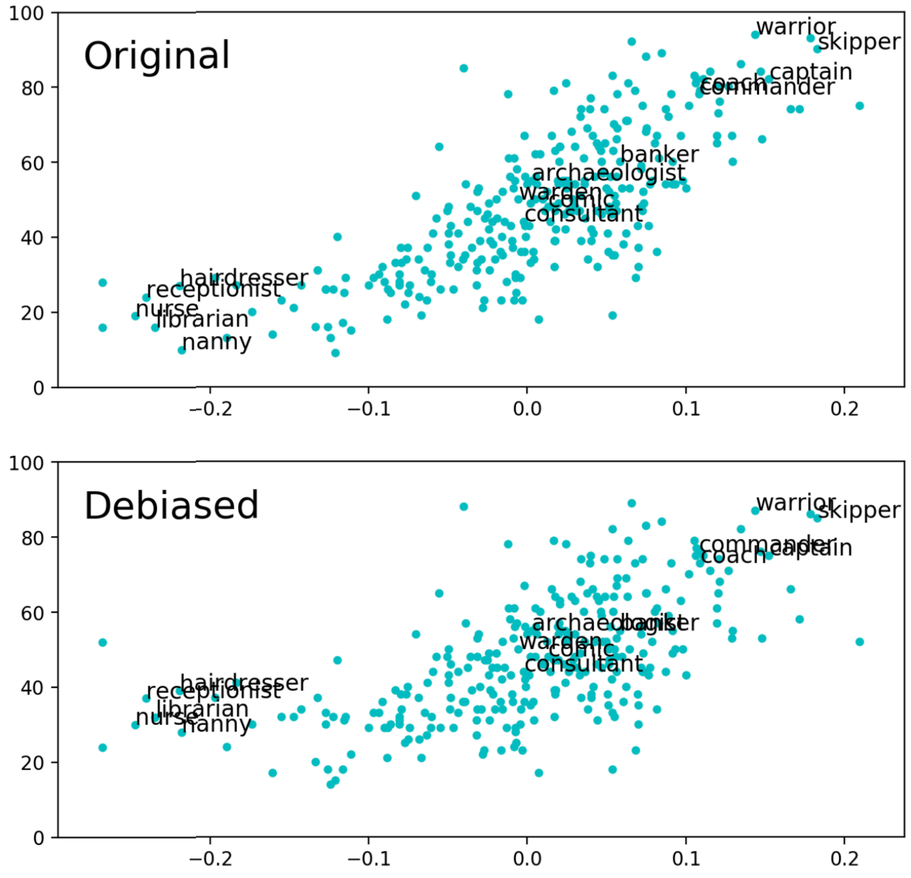


Bolukbasi et al. (2016)

Hardness of Debiasing

- Not that effective…and the male and female words are still clustered together

- Bias pervades the word embedding space and isn't just a local property of a few words



(a) The plots for HARD-DEBIASED embedding, before (top) and after (bottom) debiasing.

Gonen and Goldberg (2019)

Applying Embeddings

- First layer of your network: map from word indices to embeddings

- Approach 1: learn these embeddings as parameters from your data
  - Often works pretty well

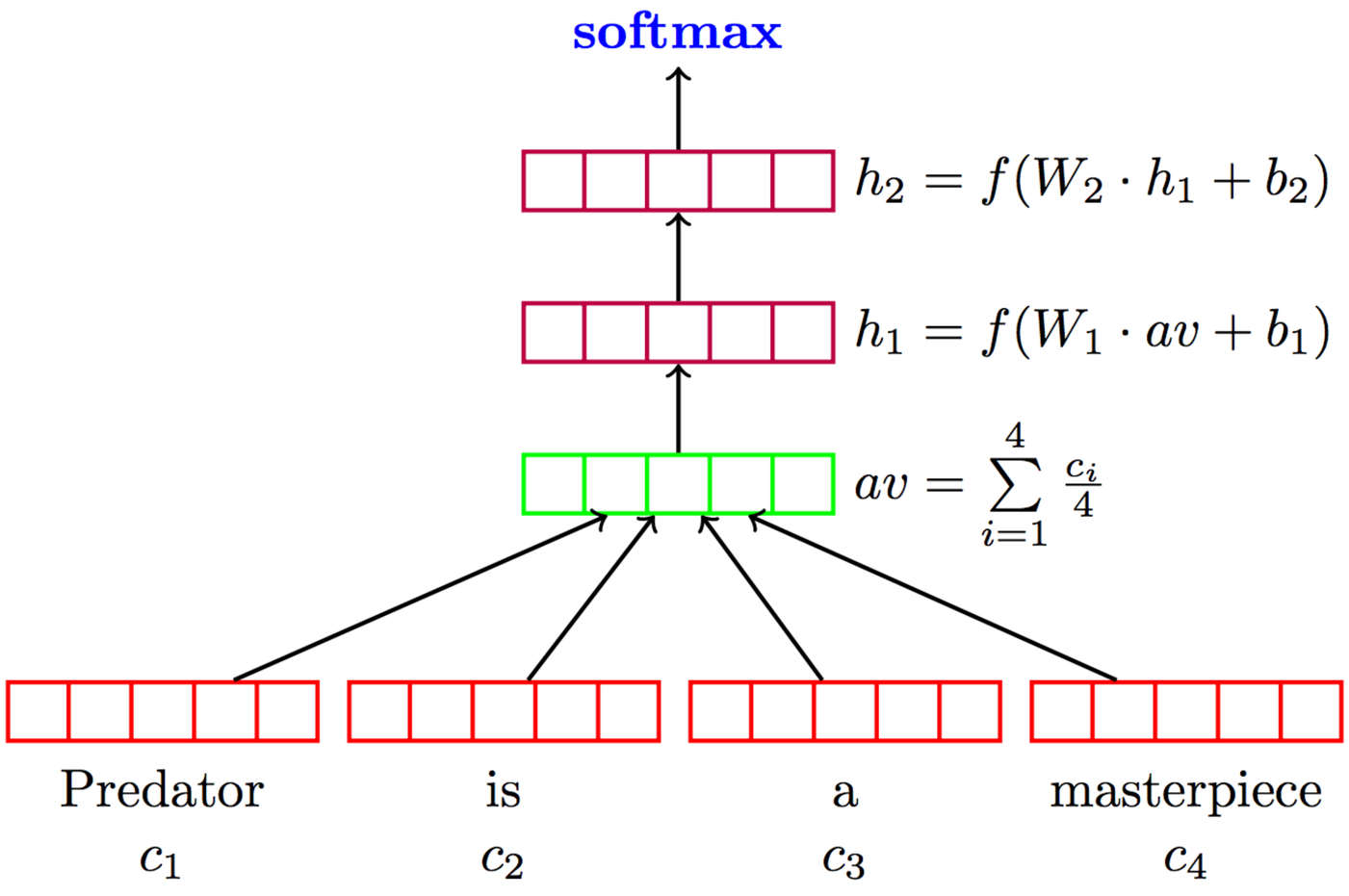
- Approach 2: initialize word embeddings using GloVe, keep fixed
  - Faster because no need to update these parameters

- Approach 3: initialize word embeddings GloVe, fine-tune
  - Works best for some tasks

- Can also evaluate embeddings intrinsically on tasks like word similarity

Deep Averaging Networks

- Deep Averaging Networks: feedforward neural network on average of word embeddings from input

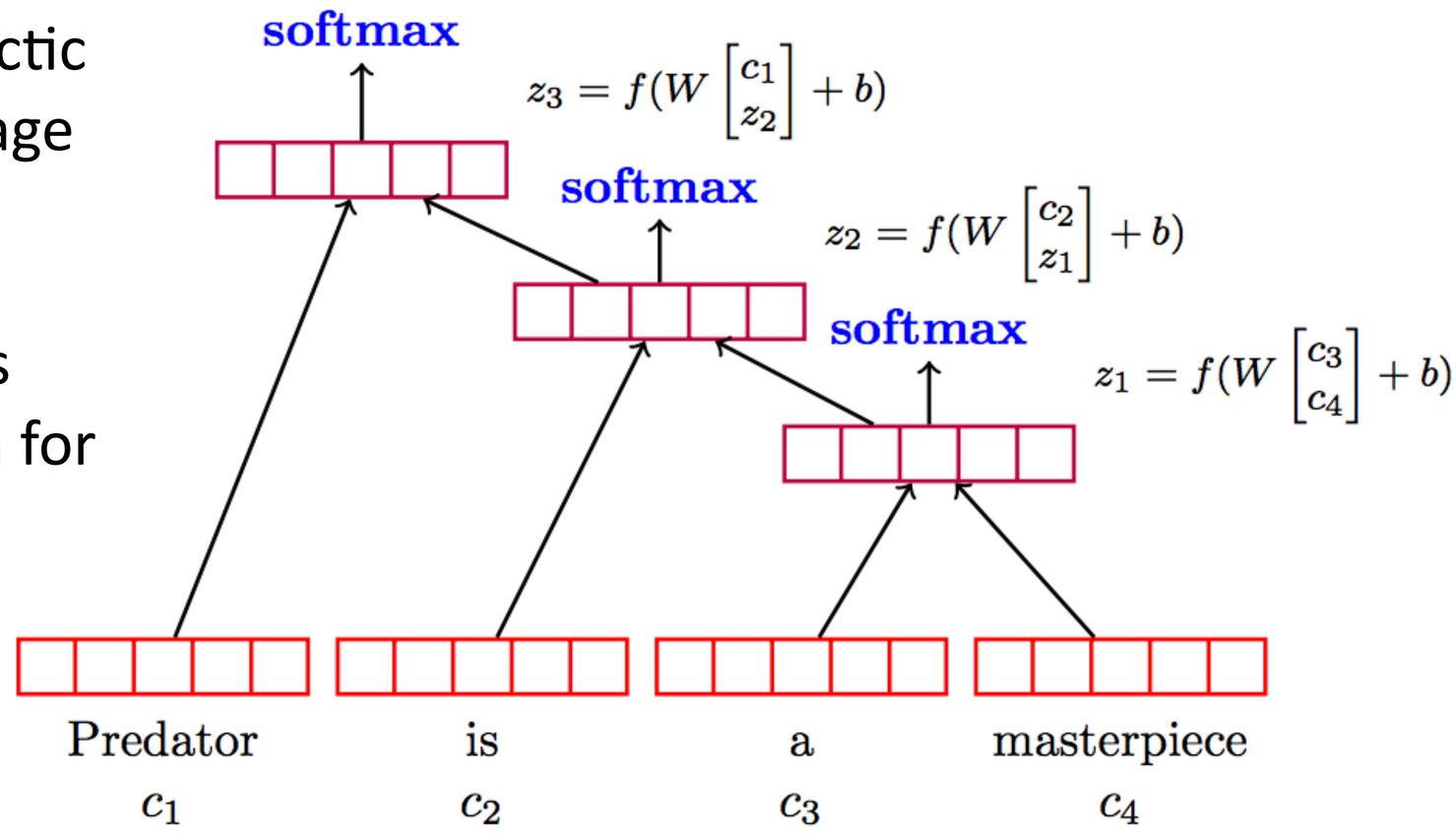


Iyyer et al. (2015)

Deep Averaging Networks

- Contradicts a widely-held view that we need to model syntactic structure to represent language

- Simple averaging can work as well as syntactic composition for some problems!



Iyyer et al. (2015)

Deep Averaging Networks

No pretrained embeddings

Bag-of-words

Tree-structured neural networks

Model	RT	SST fine	SST bin	IMDB	Time (s)	
DAN-ROOT	—	46.9	85.7	—	**31**	Iyyer et al. (2015)
DAN-RAND	77.3	45.4	83.2	88.8	136	
DAN	80.3	47.7	86.3	89.4	136	
NBOW-RAND	76.2	42.3	81.4	88.9	91	Wang and Manning (2012)
NBOW	79.0	43.6	83.6	89.0	91	
BiNB	—	41.9	83.1	—	—	
NBSVM-bi	79.4	—	—	91.2	—	
RecNN*	77.7	43.2	82.4	—	—	Kim (2014)
RecNTN*	—	45.7	85.4	—	—	
DRecNN	—	49.8	86.6	—	431	
TreeLSTM	—	**50.6**	86.9	—	—	
DCNN*	—	48.5	86.9	89.4	—	
PVEC*	—	48.7	87.8	**92.6**	—	
CNN-MC	**81.1**	47.4	**88.1**	—	2,452	
WRRBM*	—	—	—	89.2	—	

Iyyer et al. (2015)

Deep Averaging Networks

Sentence	DAN	DRecNN	Ground Truth
who knows what exactly godard is on about in this film, but his words and images do n't have to add up to mesmerize you.	positive	positive	positive
it's so good that its relentless, polished wit can withstand not only inept school productions, but even oliver parker's movie adaptation	negative	positive	positive
too bad, but thanks to some lovely comedic moments and several fine performances, it's not a total loss	negative	negative	positive
this movie was not good	negative	negative	negative
this movie was good	positive	positive	positive
this movie was bad	negative	negative	negative
the movie was not bad	negative	negative	positive

- Will return to compositionality with syntax and LSTMs

Iyyer et al. (2015)

WEEK 4

Language Modeling & Self-Attention

Segments in this section

seg-41.pdf - Language Modeling (n-grams) p. 105
seg-42.pdf - Smoothing in n-gram LMs p. 107
seg-lm3-lm-evaluation.pdf - LM Evaluation (Perplexity) p. 109
seg-43.pdf - Neural Language Models p. 110
seg-lm5-rnns-and-their-shortcomings.pdf - RNNs and their Shortcomings p. 111
seg-lm6-attention.pdf - Attention p. 115
seg-lm7-self-attention.pdf - Self-Attention p. 121
seg-lm8-multihead-self-attention.pdf - Multi-head Self-Attention p. 128
seg-lm9-position-encoding.pdf - Positional Encoding p. 133

Key points (added notes)

- n-gram LM: P(w_i | previous n-1 words); MLE = count(a,b)/count(a); smoothing (absolute discounting, Kneser-Ney).
- Perplexity = exp(average NLL), lower is better (= geometric mean of 1/P).
- RNNs: vanishing gradients, sequential (slow), recency bias.
- Self-attention: softmax(QK^T / sqrt(d_k)) V; multiple heads; positional encodings (sinusoidal, learned, relative, ALiBi).

EXAM: bigram probabilities from count tables, effect of adding data, why an RNN mislabels a review.

Note: segments 41-43 are handwritten.

N-gram Language Modeling

$$P(\bar{w}) = P(w_1, \cdots, w_m) = P(w_1)\,P(w_2|w_1)\,P(w_3|w_1 w_2) \cdots$$

n-gram LM:

$$P(\bar{w}) = \prod_{i=1}^{m} P(w_i \mid \underbrace{w_{i-n+1}, \cdots, w_{i-1}}_{\text{previous } n-1 \text{ words}})$$

$n \approx 3-7$

2-gram LM: $P(w_1|\langle s\rangle)\,P(w_2|w_1)\,P(w_3|w_2)\cdots$

w_3 cond. indep. of $w_1 \mid w_2$

n-gram LM $\Leftrightarrow$ n-1-order Markov model

3-gram LM: $P(w_1|\langle s\rangle\langle s\rangle)\,P(w_2|\langle s\rangle w_1)\,P(w_3|w_1 w_2)\cdots$

2-gram: multinomial distributions, $|V| \times |V|$ params

$P(w \mid the) =$ 0.001 house

0.0005 dog

0.0005 cat

⋮

very flat distribution!

Parameter estimation MLE from a large corpus

$$P(dog \mid the) = \frac{count(the, dog)}{count(the)}$$

Why ① Generation: machine translation

② Grammatical error correction

③ Way to build "word2vec++"

Smoothing in N-gram LMs

5-gram models work well!

(2) $P(w \mid to)$ (3) $P(w \mid go\ to)$ (5) $P(w \mid want\ to\ go\ to)$ → Austin

(5) $P(w \mid hate\ to\ go\ to)$ → class

$P(Austin \mid to) > 0$ seen in data

$P(Austin \mid want\ to\ go\ to) = 0$ if corpus isn't huge

Absolute Discounting

Reserve mass from seen 5-grams to allocate to unseen 5-grams

$0 < K < 1$

$$P_{AD}(\text{Austin} \mid \text{want to go to}) = \frac{\text{count}(w\ t\ g\ t\ A) - K}{\text{count}(w\ T\ g\ T)} + \lambda \overbrace{P_{AD}(A \mid t\ g\ t)}^{4\text{-gram}}$$

λ set to make this normalize

$K = 0.2$

# of word types seen in this context times K

$$\lambda = \frac{0.6}{4}$$

want to go to → Maui 2 → 1.8

want to go to → Class 1 → 0.8

want to go to → campus 1 → 0.8

count = 4

$$P_{AD}(A \mid t\ g\ t) = \cdots + \lambda' P_{AD}(A \mid g\ t)$$

Kneser-Ney smoothing

$$\cdots P(A) > 0$$

LM Evaluation

- Accuracy doesn't make sense as an evaluation metric — predicting the next word is generally impossible so accuracy values would be very low
- Instead, evaluate LMs on the **likelihood of held-out data** (averaged to normalize for length)

$$\frac{1}{n}\sum_{i=1}^{n}\log P(w_i|w_1,\ldots,w_{i-1})$$

- **Perplexity**: exp(average negative log likelihood). Lower is better.
  - Suppose we have probs 1/4, 1/3, 1/4, 1/3 for 4 predictions
  - Avg NLL (base e) = 1.242 Perplexity = 3.464 <== geometric mean of denominators
- Perplexity numbers usually range from 10-200, depending on model quality. They're standard in LM research but not used much elsewhere.

Neural Language Models

$$P(\bar{w}) = P(w_1)P(w_2|w_1)\cdots \underbrace{P(w_n|w_1 \cdots w_{n-1})}$$

model with a neural net

Very basic neural LM: $P(w_i|w_{i-1}) = \frac{e^{v_i \cdot c_{i-1}}}{\sum_{w' \in V} e^{v_w \cdot c_{i-1}}}$

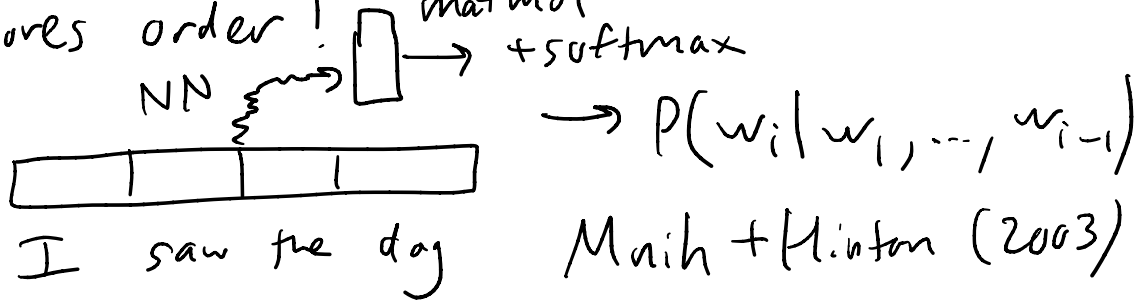
skip-gram

More generally: $P(w_i|w_1 \cdots w_{i-1}) = \text{softmax}(U_{w_i} \cdot f(w_1 \cdots w_{i-1}))$

f: neural net to embed the context

f is a DAN? Ignores order!

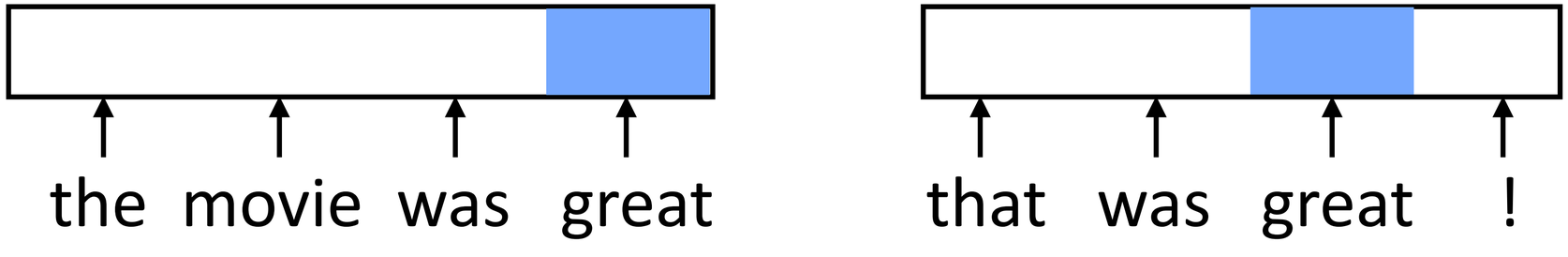
f is a FFNN?



Mnih + Hinton (2003)

RNN Motivation

- Feedforward NNs can’t handle variable length input: each position in the feature vector has fixed semantics



- These don’t look related (*great* is in two different orthogonal subspaces)

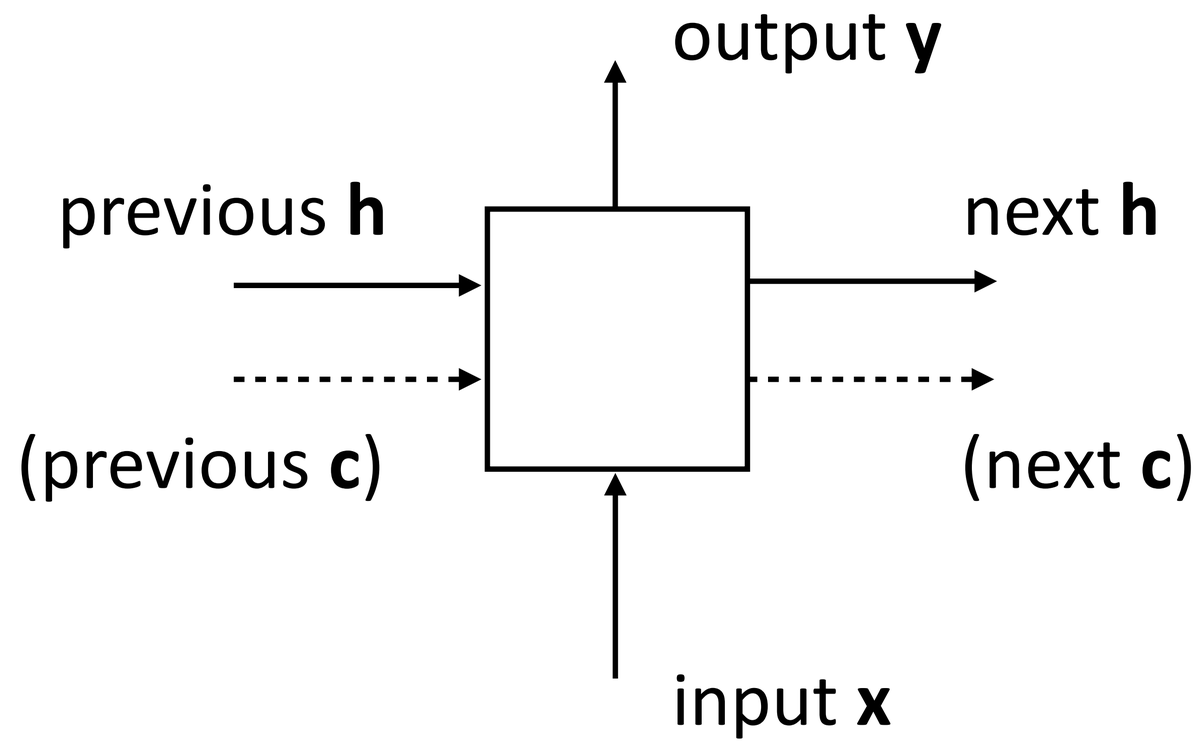
- Instead, we need to:

  1) Process each word in a uniform way

  2) …while still exploiting the context that that token occurs in

RNN Abstraction

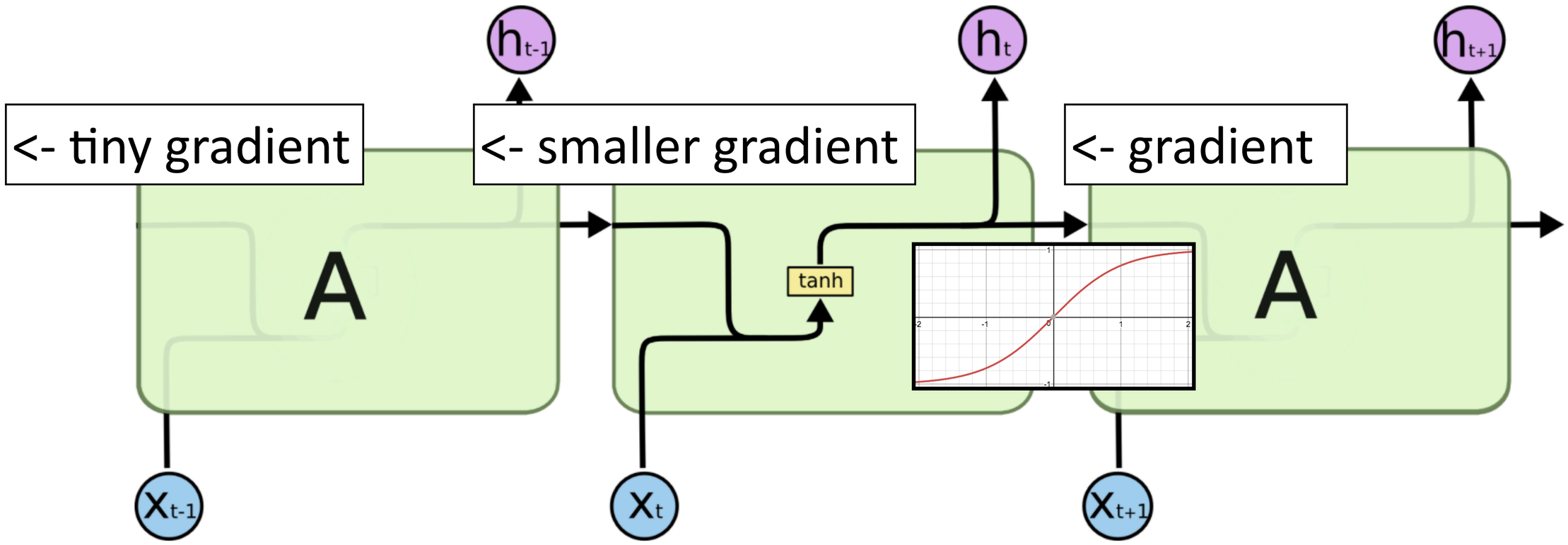
- Cell that takes some input **x**, has some hidden state **h**, and updates that hidden state and produces output **y** (all vector-valued)
- Optionally: cell state **c** (used in LSTMs but not all architectures)



- Example:

the movie was great it was not great

Vanishing Gradient

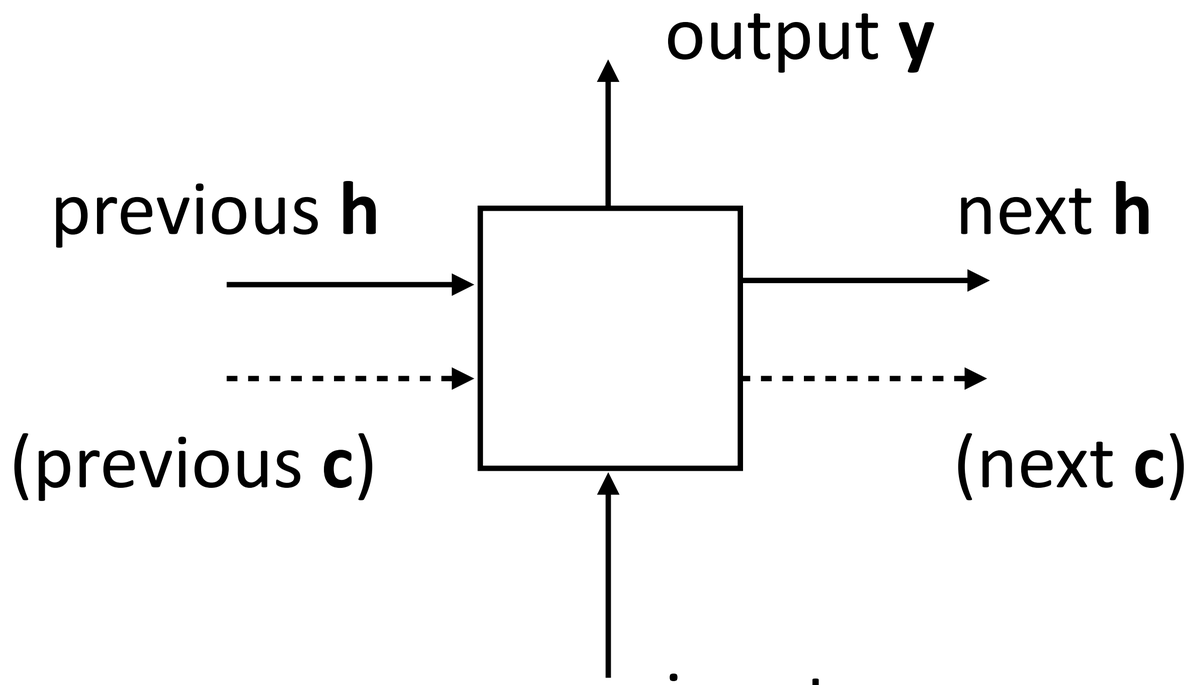


$$\mathbf{h}_t = \tanh(W\mathbf{x}_t + V\mathbf{h}_{t-1} + \mathbf{b}_h)$$

- Gradient diminishes going through tanh; if not in [-2, 2], gradient is almost 0
- Repeated multiplication by V causes problems

http://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNNs: Why not?



- Vanishing gradient makes it hard to learn. LSTMs can help…but not enough*
- Slow. They do not parallelize and there are O(n) non-parallel operations to encode n items
- Solution: Transformers. They can scale to thousands of words!

*This is somewhat addressed by recent innovations like state-space models

Attention

- “Attention is all you need”: title of the Transformer paper by Vaswani et al.
- Key mechanism for accessing relevant information in a context to make predictions
- This segment: example of how attention can impact language modeling

Attention: Running Example

- Fixed-length sequence of As and Bs

AAAAAAA

ABAAAAB

ABAABAB

AAAABAB

BAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAB

- All As = last letter is A; any B = last letter is B
- **Attention**: method to access arbitrarily* far back in context from this point
- RNNs generally struggle with this; remembering context for many positions is hard (though of course they can do this simplified example — you can even hand-write weights to do it!)

Keys and Query

Attention

Attention

keys k_i

[1, 0] [1, 0] [0, 1] [1, 0]

A A B A

query: q = [0, 1] (we want to find Bs)

We can make attention more peaked by amplifying the embeddings

$k_i = W^K e_i$ $W^K = \begin{matrix} 10 & 0 \\ 0 & 10 \end{matrix}$ [10, 0][10, 0][0, 10][10, 0]

0 0 1 0

What will new attention values be with these keys?

Attention, Formally

- Original “dot product” attention: $s_i = k_i^T q$
- Scaled dot product attention: $s_i = k_i^T W q$
- Equivalent to having two weight matrices: $s_i = (W^K k_i)^T (W^Q q)$

- Other forms exist: Luong et al. (2015), Bahdanau et al. (2014) present some variants (originally for machine translation)
- We will see that the real attention computation actually has three matrices (one for values as well); we’ll come back to this later

Self-Attention

- Self-attention: builds on the idea of attention. **Every word in a sequence is both a key and a query simultaneously**

Q: seq len x d matrix (d = embedding dimension = 2 for these slides)

K: seq len x d matrix

$W^Q = \begin{matrix} 0 & 1 \\ 0 & 1 \end{matrix}$ no matter what the value is, we're going to look for Bs

$W^K = \begin{matrix} 10 & 0 \\ 0 & 10 \end{matrix}$ "booster" as before

Note: there are many ways to set up these weights that will be equivalent to this

Self-Attention

$$E = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad W^Q = \begin{matrix} 0 & 1 \\ 0 & 1 \end{matrix} \qquad W^K = \begin{matrix} 10 & 0 \\ 0 & 10 \end{matrix}$$

Self-Attention (Vaswani et al.)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

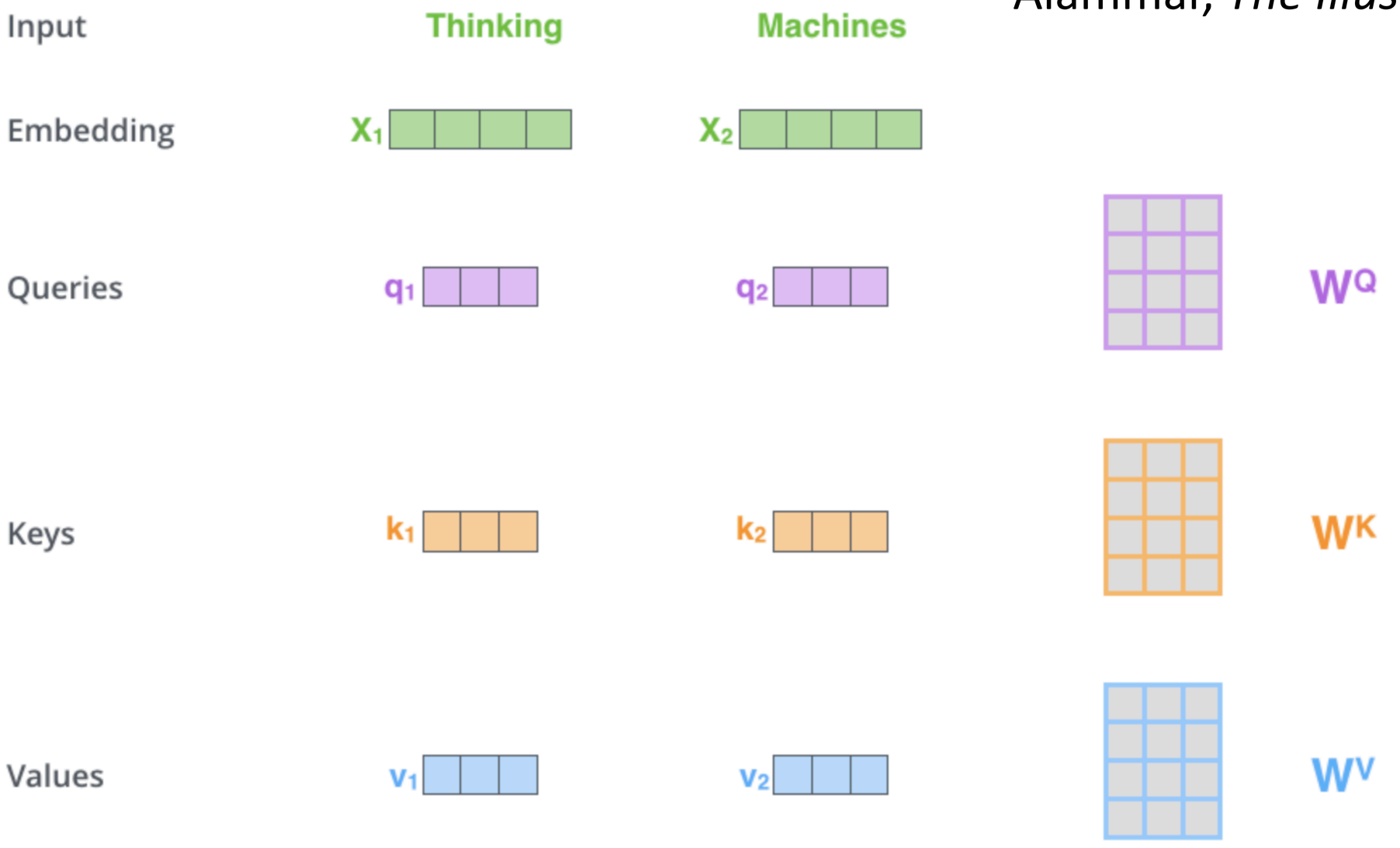
$Q = EW^Q$, $K = EW^K$, $V = EW^V$

- Normalizing by $\sqrt{d_k}$ helps control the scale of the softmax, makes it less peaked
- This is just one ***head*** of self-attention — produce multiple heads via randomly initialize parameter matrices (more in a bit)
- What does self-attention produce?
  - Square attention matrix * input = same dimension as the input.
  - Computes a contextualized encoding for each word, preserving the length of the sequence

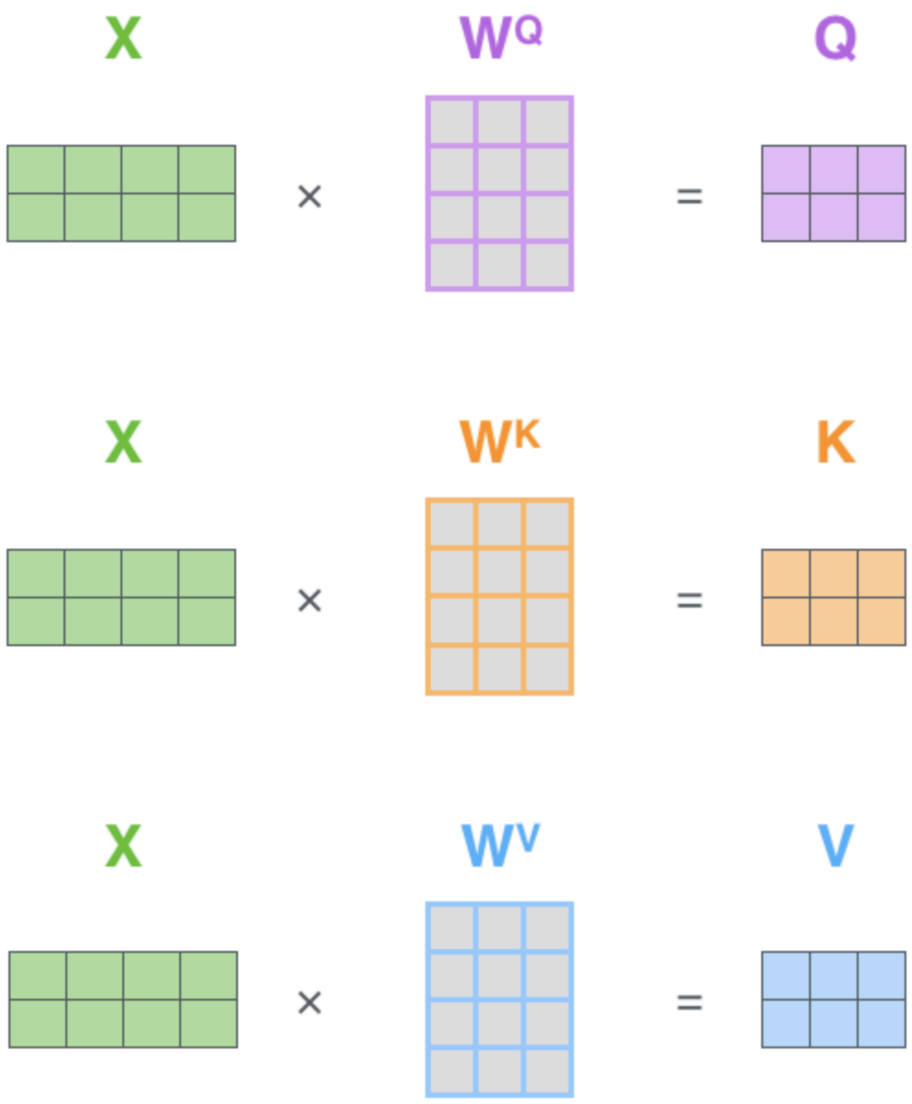
Vaswani et al. (2017)

Self-Attention (Alammar)

Alammar, *The Illustrated Transformer*

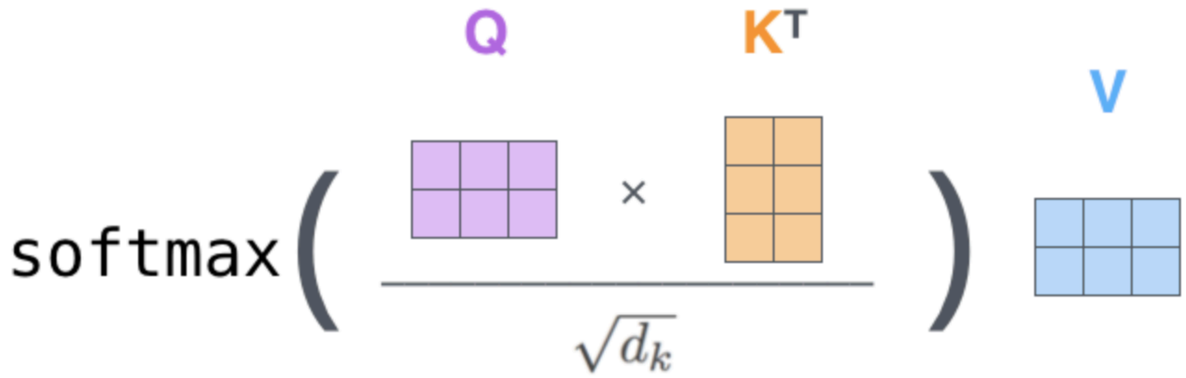


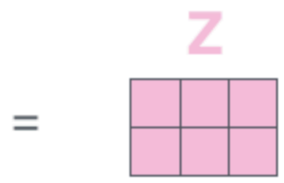
Self-Attention (Alammar)



Alammar, *The Illustrated Transformer*

sent len x sent len (attn for each word to each other)



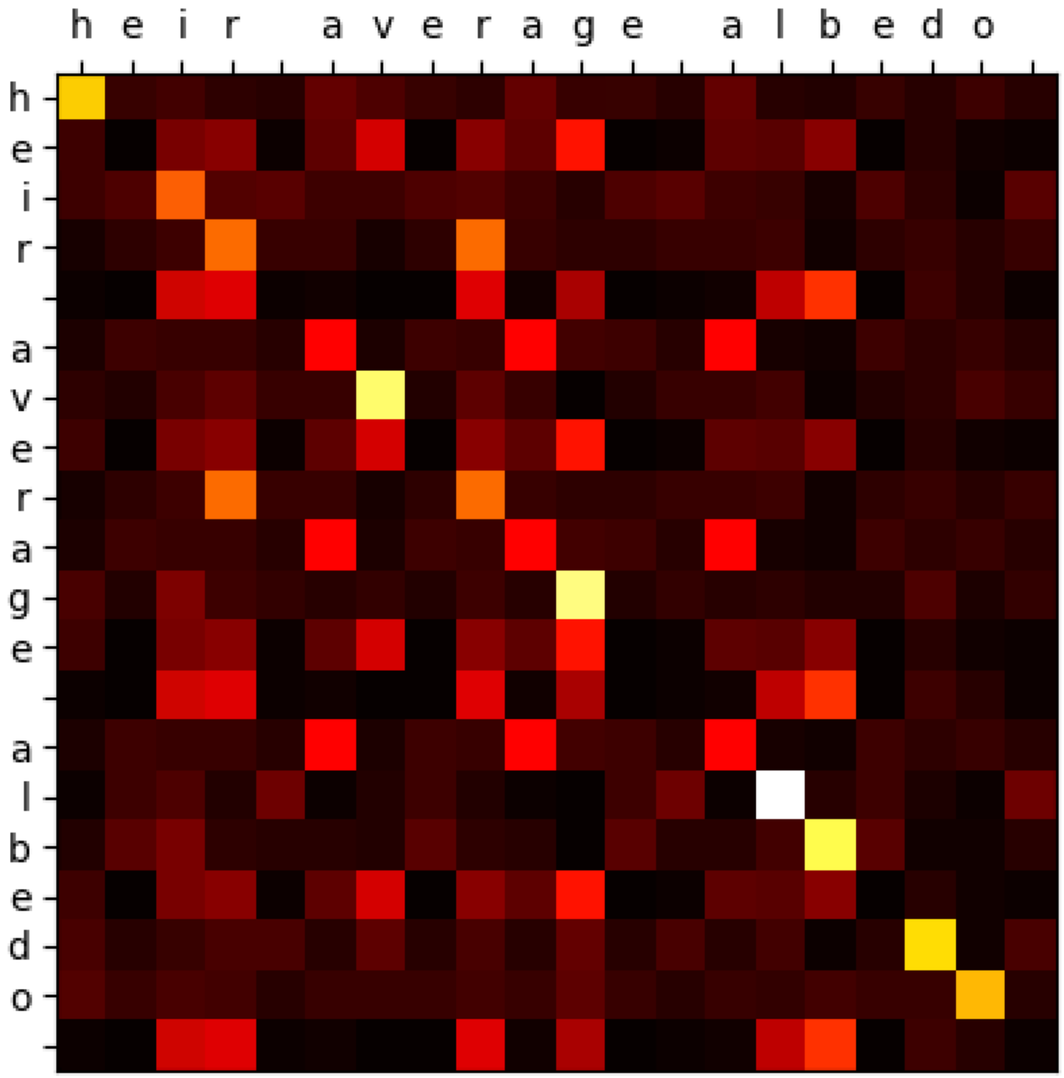


sent len x hidden dim

Z is a weighted combination of V rows

Attention Maps

- Example visualization of attention matrix A (from assignment)
- Each row: distribution over what that token attends to. E.g., the first “v” attends very heavily to itself (bright yellow box)
- This only depicts a single head of self-attention. Recall there are many heads and many layers, and much of the computation happens in FFNNs



Properties of Self-Attention

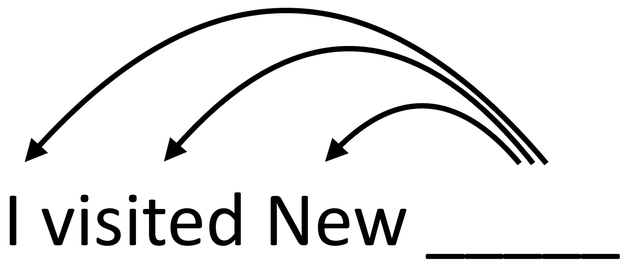
Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

- n = sentence length, d = hidden dim, k = kernel size, r = restricted neighborhood size
- **Quadratic complexity**, but O(1) sequential operations (not linear like in RNNs) and O(1) "path" for words to inform each other

Vaswani et al. (2017)

Peaky Attention

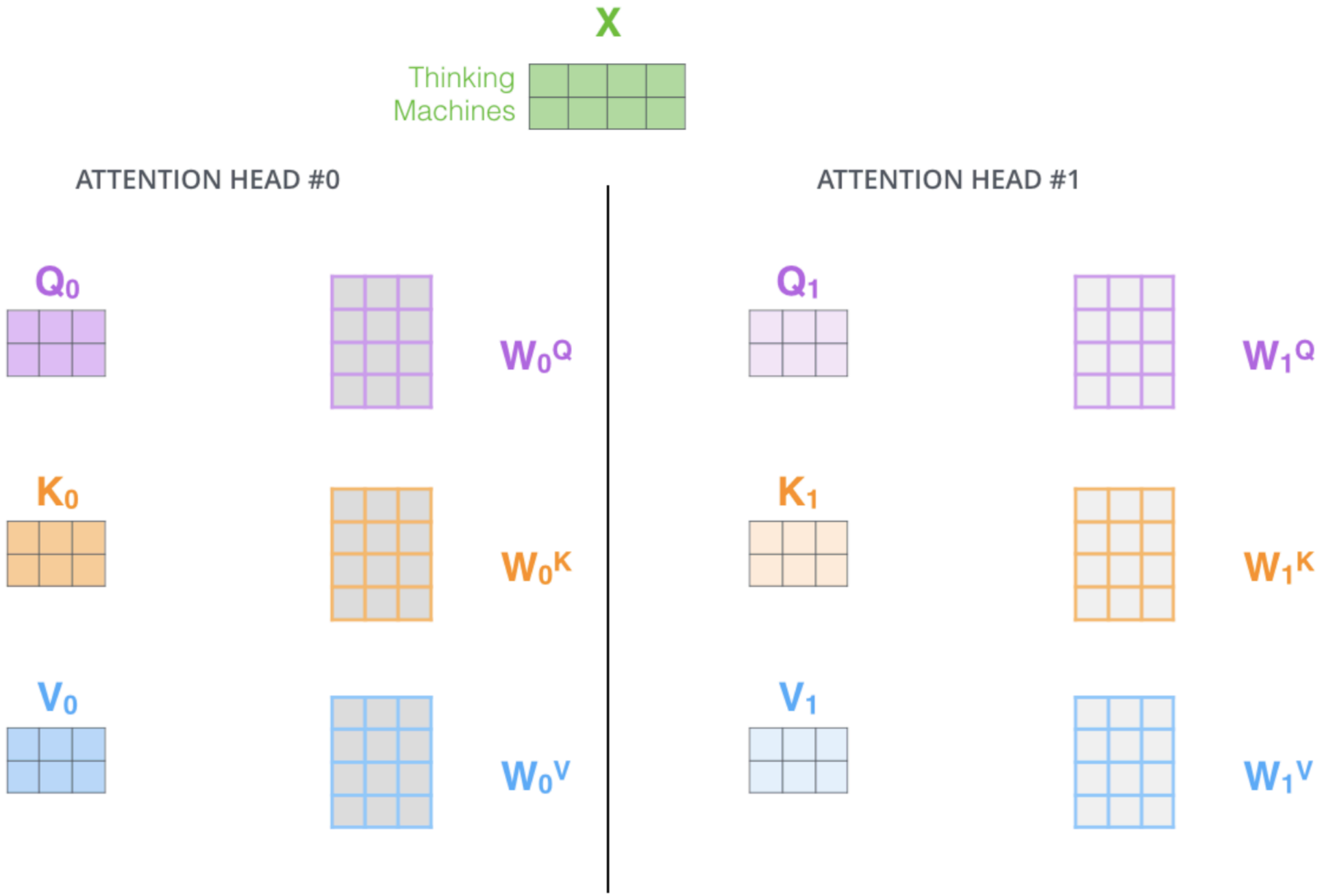
- Although attention can theoretically learn to attend to multiple tokens, in practice softmax distributions become peaked:



- Having many layers of self-attention can help, but what about within a single layer?

Multi-head Self-Attention

- Solution: multiple *heads* to do independent "copies" of attention



Alammar, *The Illustrated Transformer*

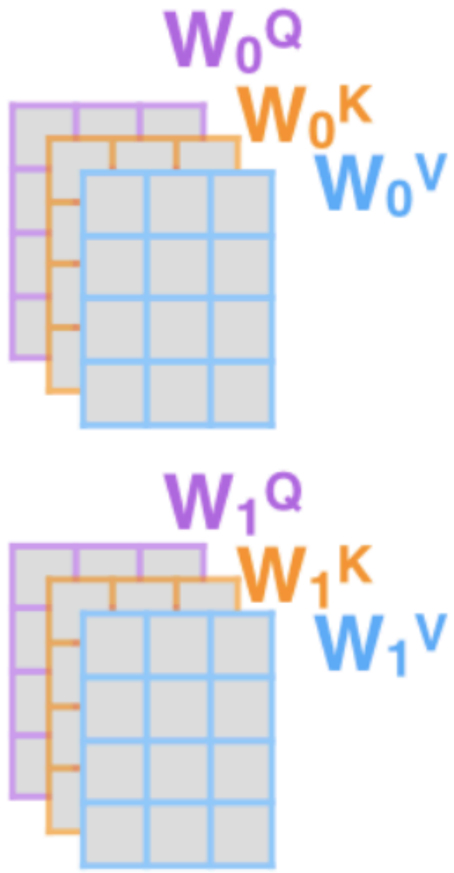
Multi-head Self-Attention

1) This is our input sentence*

2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

Thinking Machines



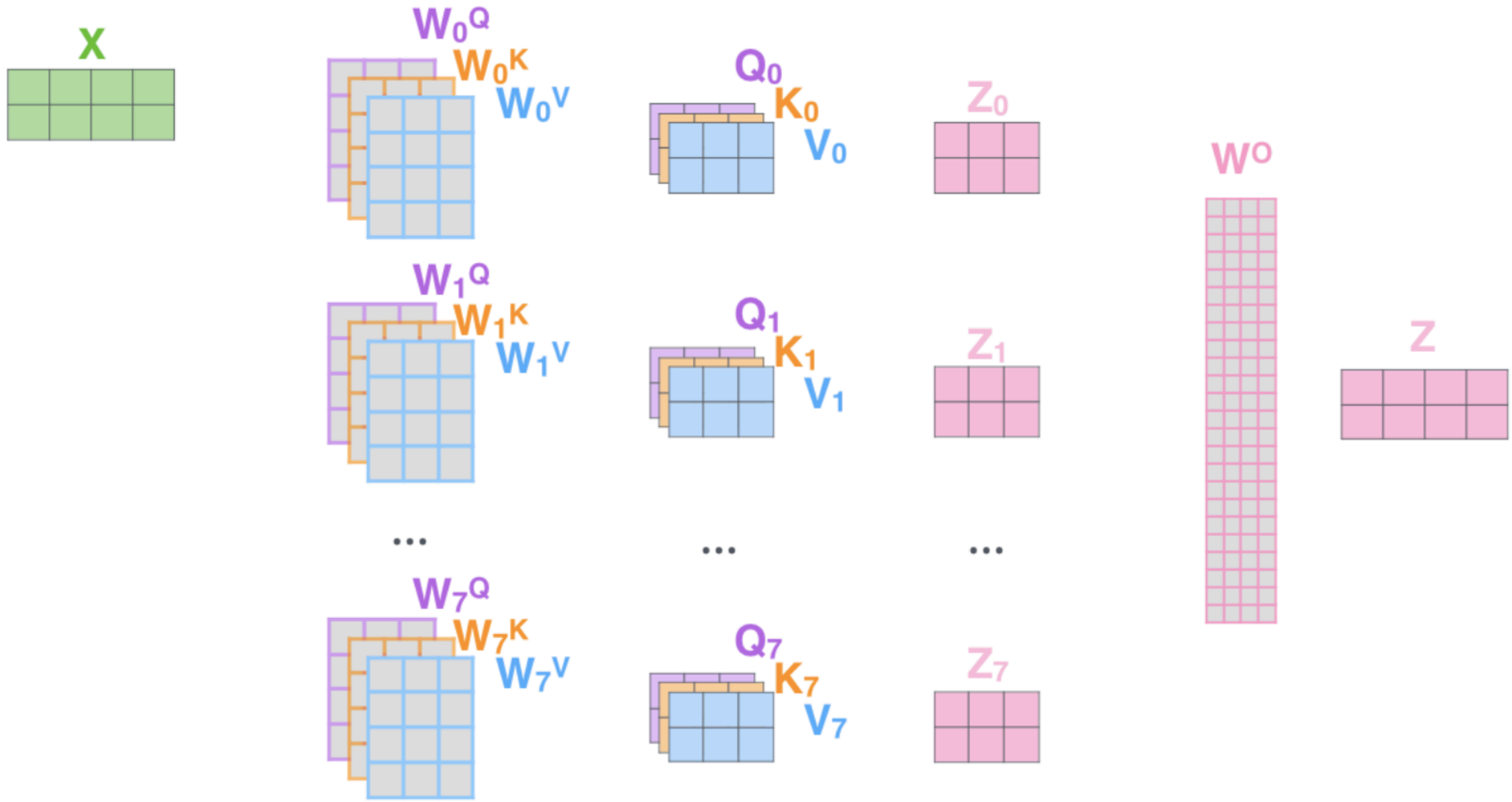
Alammar, *The Illustrated Transformer*

Multi-head Self-Attention

3) Split into 8 heads. We multiply X or R with weight matrices

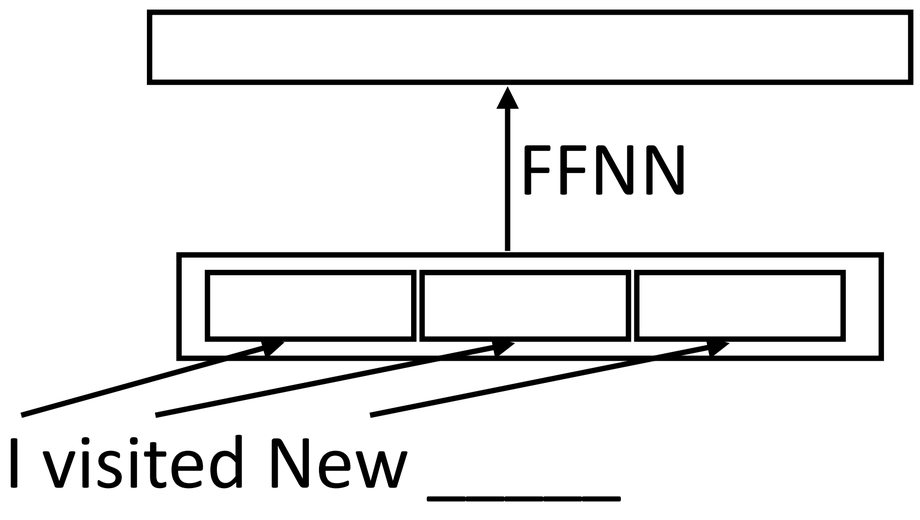
4) Calculate attention using the resulting Q/K/V matrices

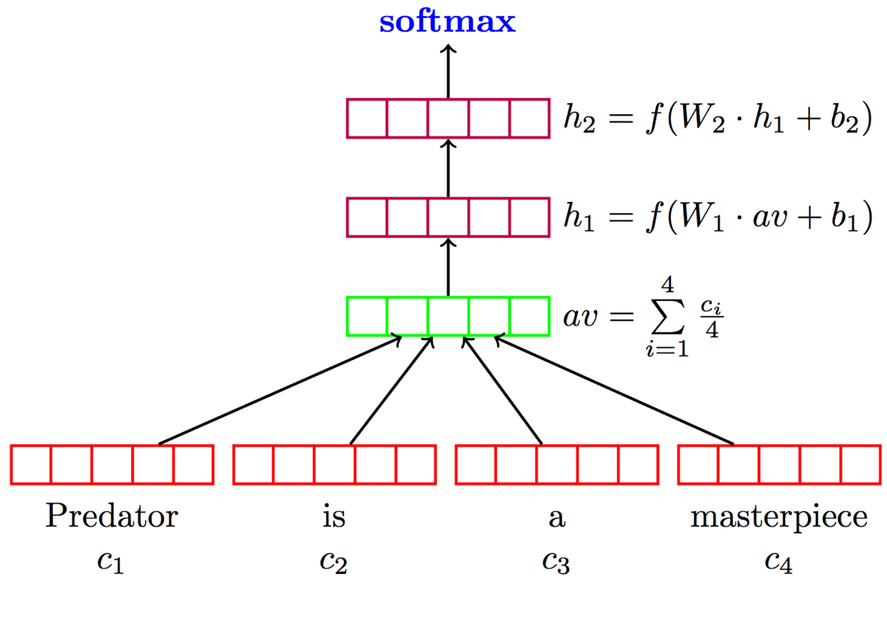
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



Alammar, *The Illustrated Transformer*

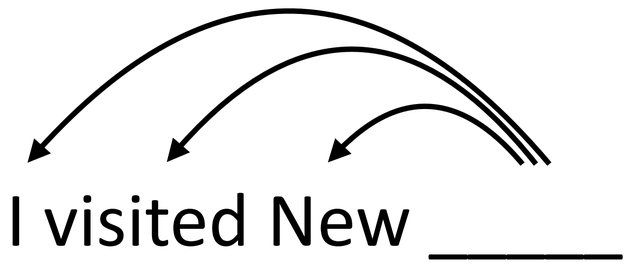
What does this give us?





FFNN DAN

Self-attention:



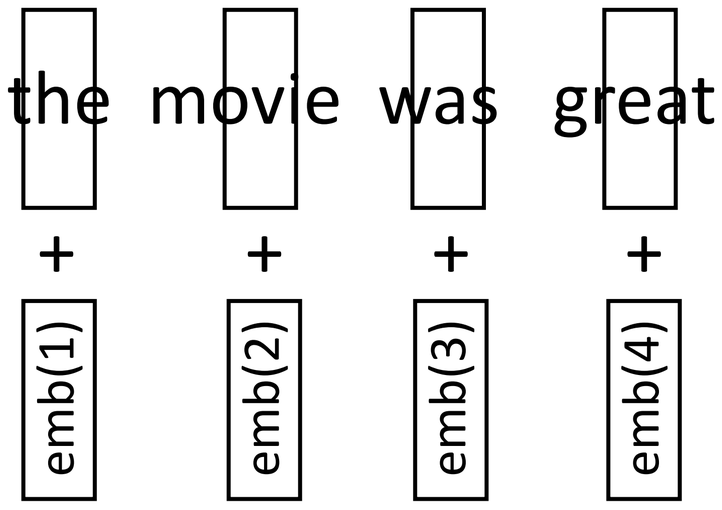
- One head can attend to *New*, another can attend to *visited*; over the course of multiple layers, this information can combine

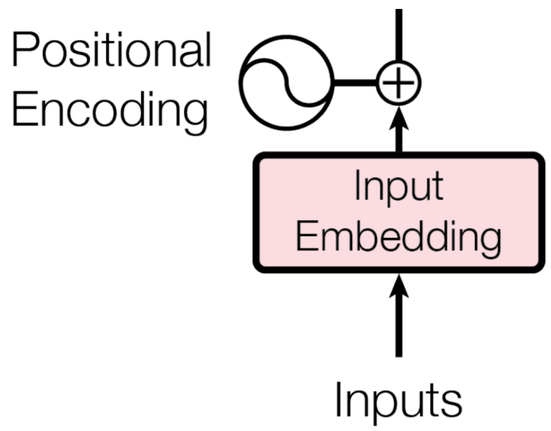
Positional Encoding

- *visited* and *ate* actually look the same from the perspective of self-attention, unless we provide position information

- *Positional encoding* refers to a family of schemes to provide this information to Transformer models

Positional Encoding



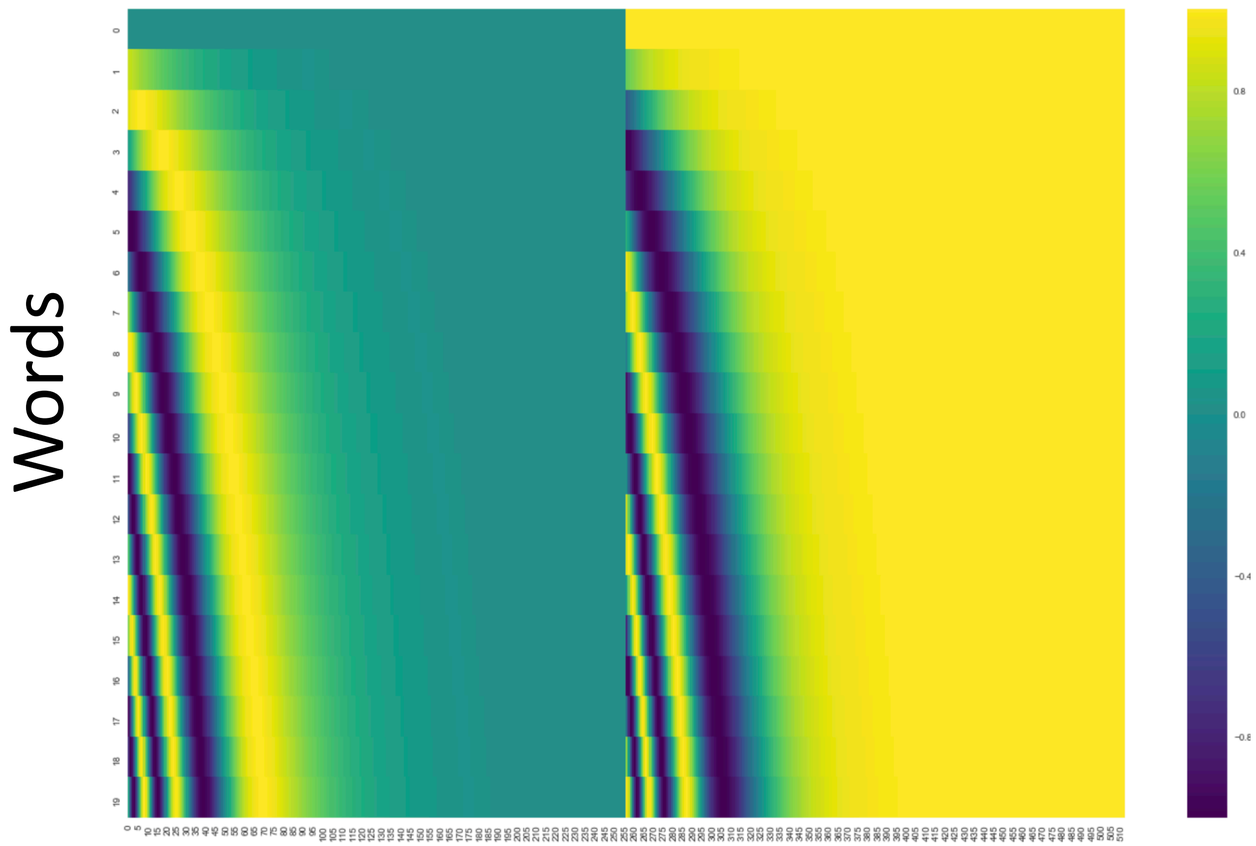


- Encode each sequence position as an integer, add it to the word embedding vector

- What are some drawbacks of this?

Multi-head Self-Attention

- Alternative from Vaswani et al.: sines/cosines of different frequencies (closer words get higher dot products by default)



$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

Positional Encoding: Variants

- Relative positional encoding (used in T5): self-attention computation depends on the distance between two tokens

- ALiBi (Press et al., 2022): $\text{softmax}(\mathbf{q}_i \mathbf{K}^\top + m \cdot [-(i-1), ..., -2, -1, 0])$
  - Adds a linear bias to disprefer attending farther back, use different slope *m* for each attention head

- No positional encoding (NoPE) actually works too! (Kazemnejad et al., 2023)
  - Transformers with self-attention into the past can learn to "count" tokens; each token can determine what position it is after a single layer.

WEEK 5

Transformers & Decoding

Segments in this section

seg-trans1-transformer-architecture.pdf - Transformer Architecture p. 138
seg-trans2-using-transformers.pdf - Using Transformers p. 143
seg-trans3-transformer-language-models.pdf - Transformer Language Models p. 147
seg-trans4-transformer-extensions.pdf - Transformer Extensions p. 154
seg-subword-segmentation.pdf - Subword Segmentation (BPE, WordPiece) p. 159
seg-infer1-beam-search.pdf - Decoding: Beam Search p. 163
seg-infer2-nucleus-sampling.pdf - Decoding: Nucleus Sampling p. 167

Key points (added notes)

- Block = MHA + Add&Norm + FFN + Add&Norm; ~12 d^2 parameters per layer (the FFN holds most).
- Transformer LM: targets are inputs shifted by one; causal mask (-inf above the diagonal).
- Shapes: attention [batch, seq, seq]; outputs [batch, seq, vocab].
- Decoding: greedy, beam search (MT), sampling, nucleus (top-p). Beam search degenerates on open-ended text.
- BPE merges the most frequent adjacent pair; WordPiece / unigram LM alternative.

EXAM: parameter counting for modified Transformers, shape questions, beam / nucleus by hand.

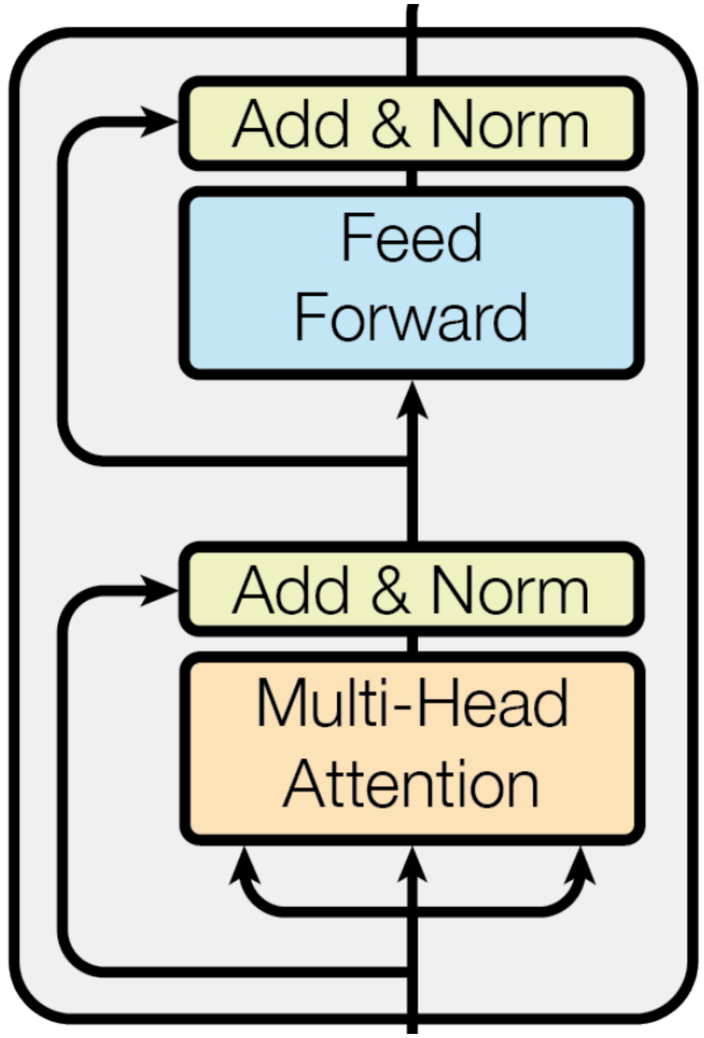
Note: the subword segmentation segment is not listed in the README; it is placed here next to the Transformer material. The Assignment 3 walkthrough is not in the folder.

Transformer Architecture

- Self-attention is not the whole story; we'll describe what goes into a Transformer
- Alternate multi-head self-attention with feedforward layers that **operate over each word individually**

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

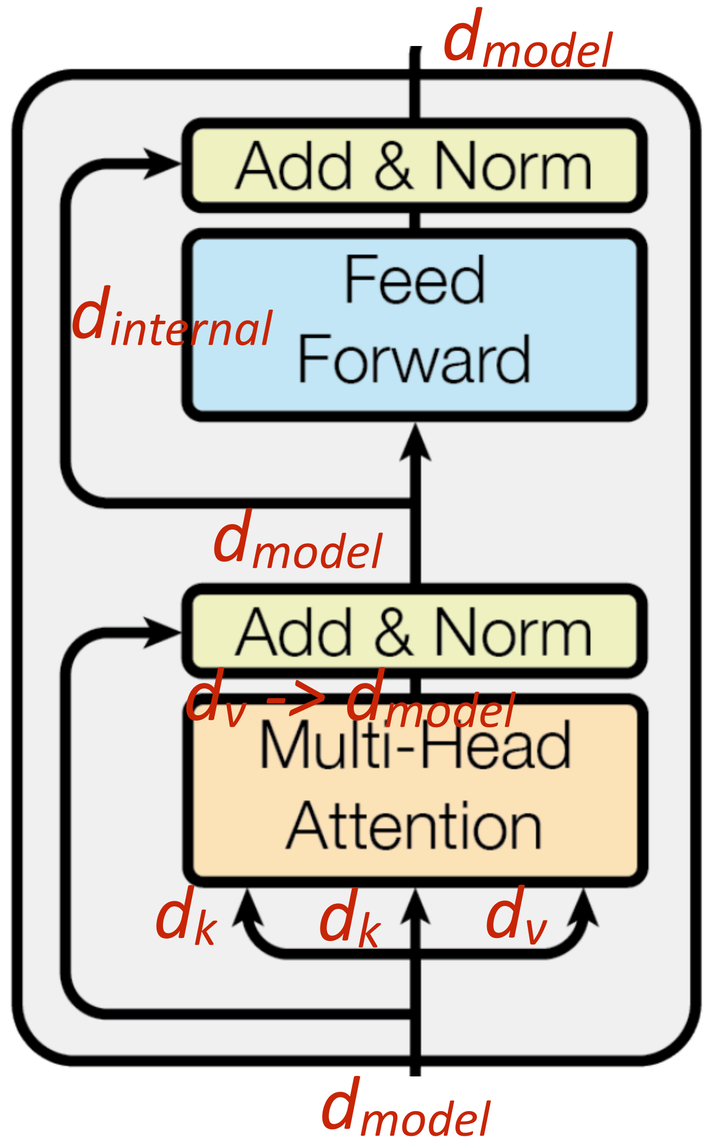
  - These feedforward layers are where most of the parameters are
- Residual connections in the model: input of a layer is added to its output
- Layer normalization: controls the scale of different layers in very deep networks



Transformer Architecture

- Vectors: d_{model}
- Queries/keys: d_k, always smaller than d_{model}
- Values: separate dimension d_v, output is multiplied by W^O which is d_v x d_{model} so we can get back to d_{model} before the residual
- FFN can explode the dimension with W_1 and collapse it back with W_2

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



Vaswani et al. (2017)

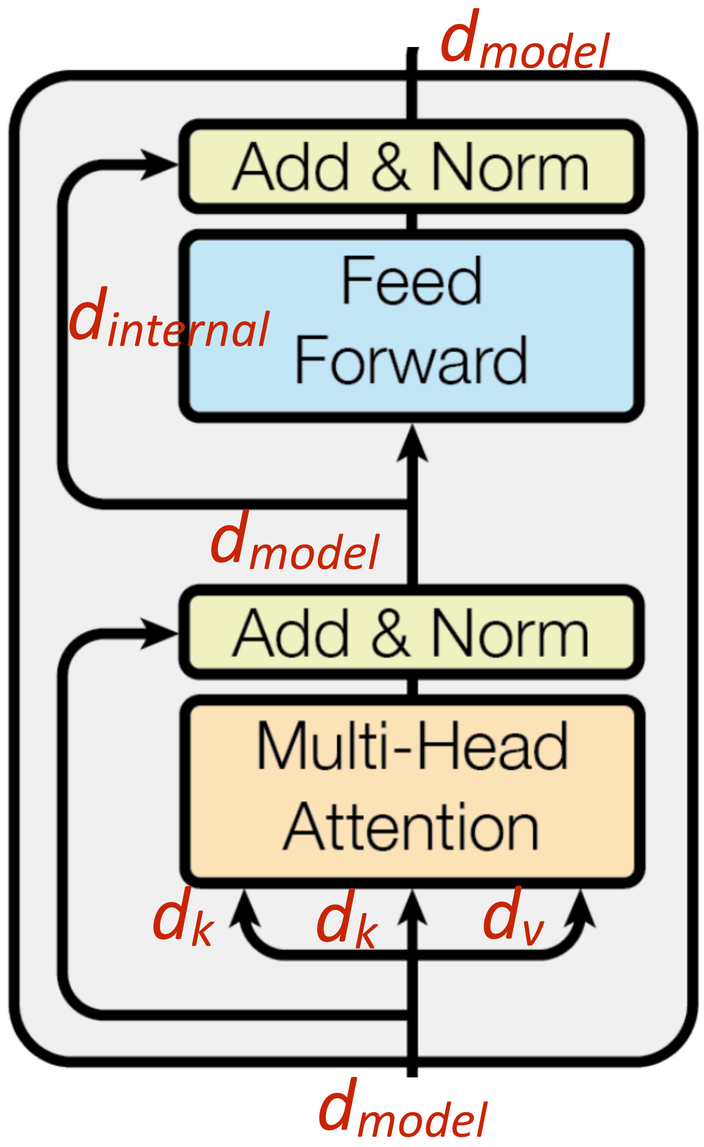
Transformer Architecture: Sizes

	N	d_{model}	d_{ff}	h	d_k	d_v
base	6	512	2048	8	64	64

- From Vaswani et al.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128

- From GPT-3; d_{head} is our d_k



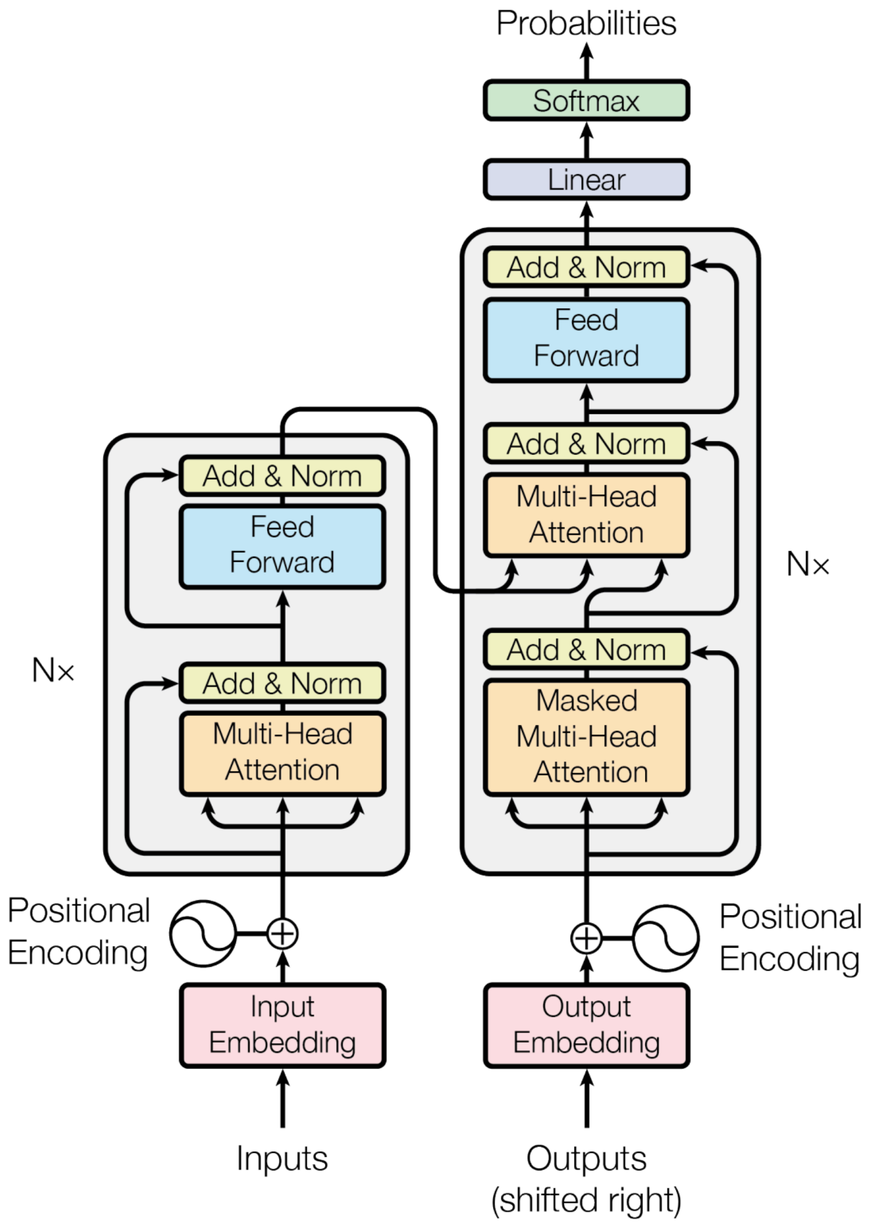
Vaswani et al. (2017)

Transformer Architecture: Sizes

1	description	FLOPs / update	% FLOPS MHA	% FLOPS FFN	% FLOPS attn	% FLOPS logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

Credit: Stephen Roller on Twitter

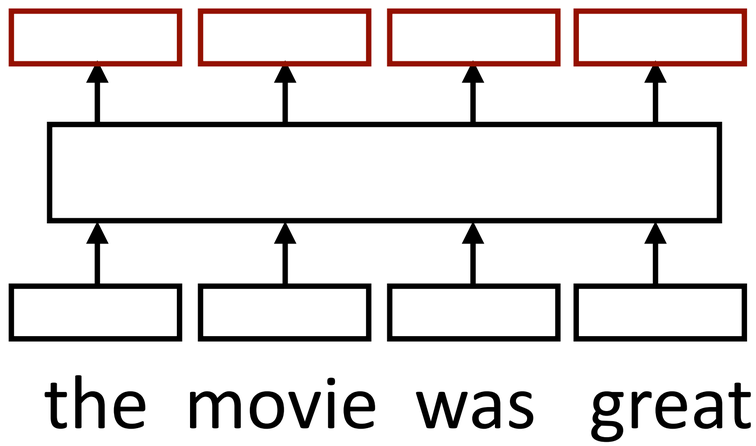
Transformer: Complete Model



- Original Transformer paper presents an **encoder-decoder** model
- Right now we don't need to think about both of these parts — will return in the context of sequence-to-sequence models later
- Can turn the encoder into a decoder-only model through use of a triangular causal attention mask (only allow attention to previous tokens)

Vaswani et al. (2017)

What do Transformers produce?

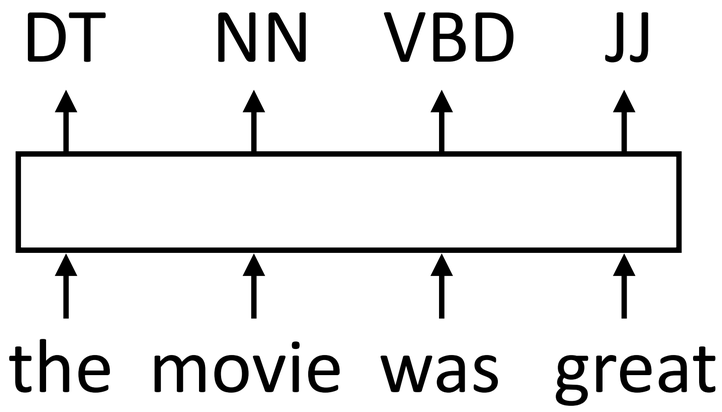


- Encoding of each word — can pass this to another layer to make a prediction (like predicting the next word for language modeling)

- Like RNNs, Transformers can be viewed as a transformation of a sequence of vectors into a sequence of context-dependent vectors

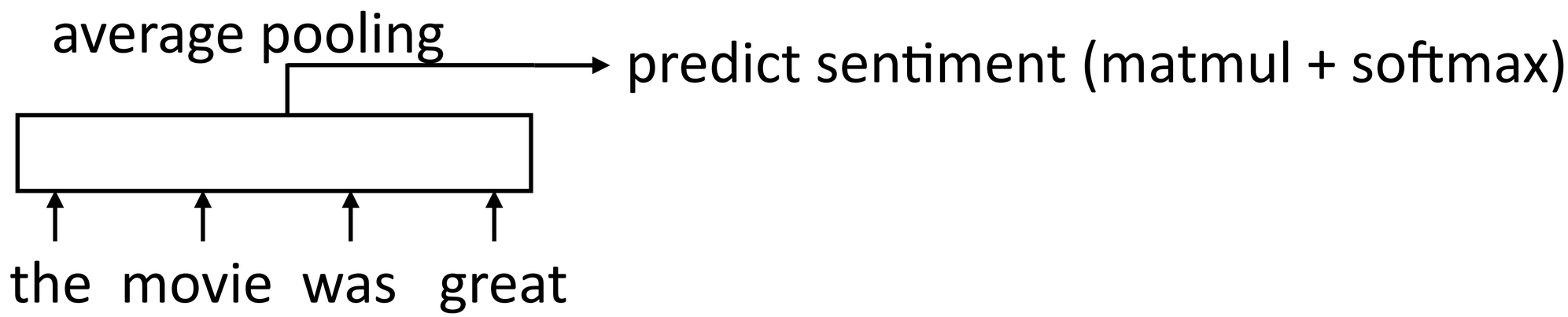
Transformer Uses

- Transducer: make some prediction for each element in a sequence

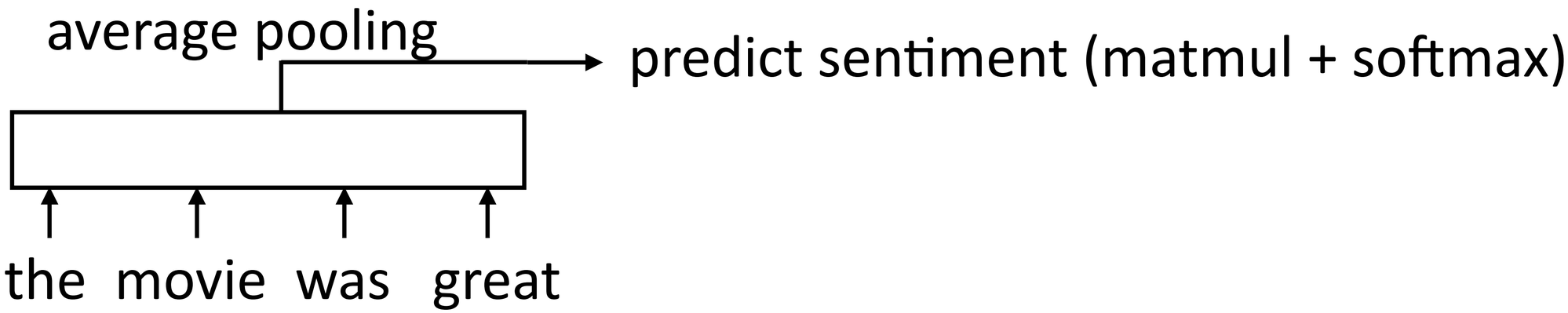


output **y** = score for each tag, then softmax

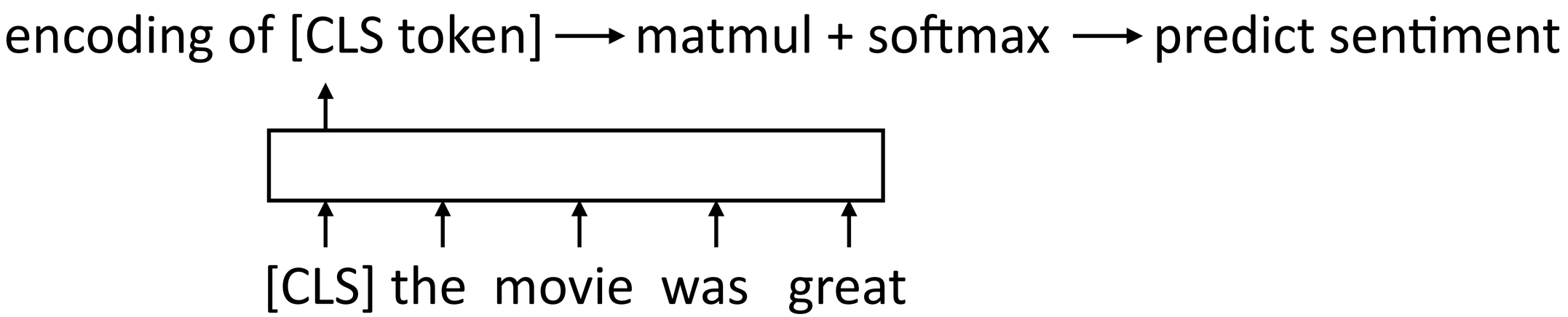
- Classifier: encode a sequence into a fixed-sized vector and classify that



Transformer Uses



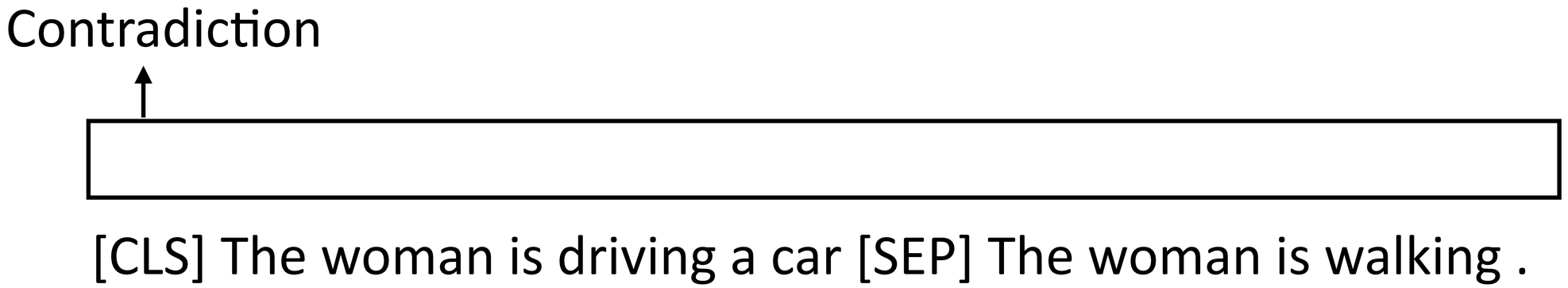
- Alternative: use a placeholder [CLS] token at the start of the sequence.



- Because [CLS] attends to everything with self-attention, it can do the pooling for you!

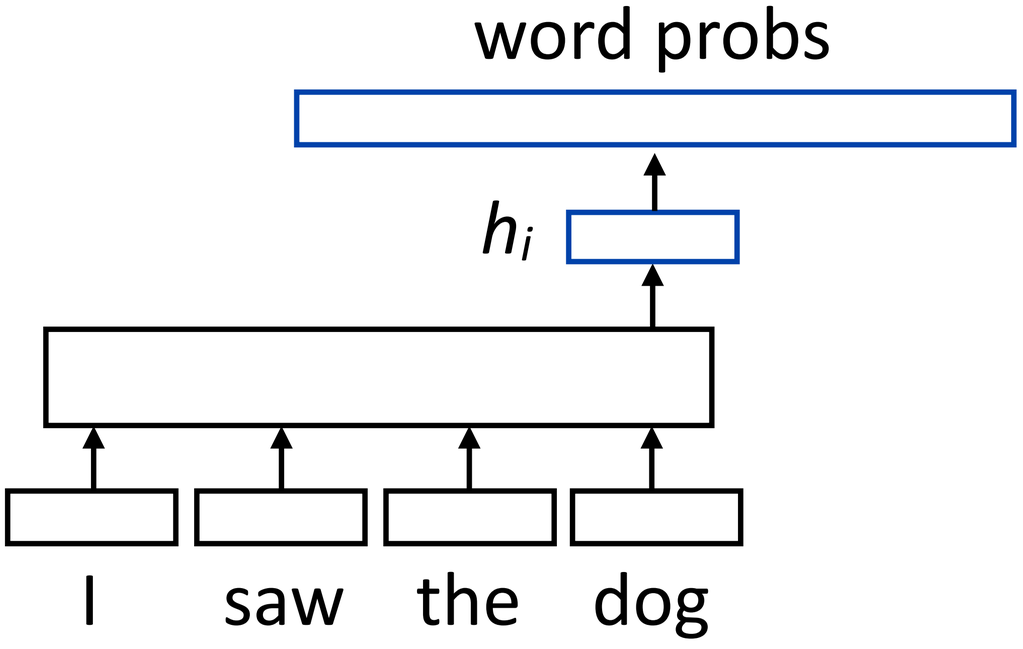
Transformer Uses

- Sentence **pair** classifier: feed in two sentences and classify something about their relationship



- Transformers are particularly good at sentence **pair** tasks because they can capture alignment between words

Transformer Language Modeling



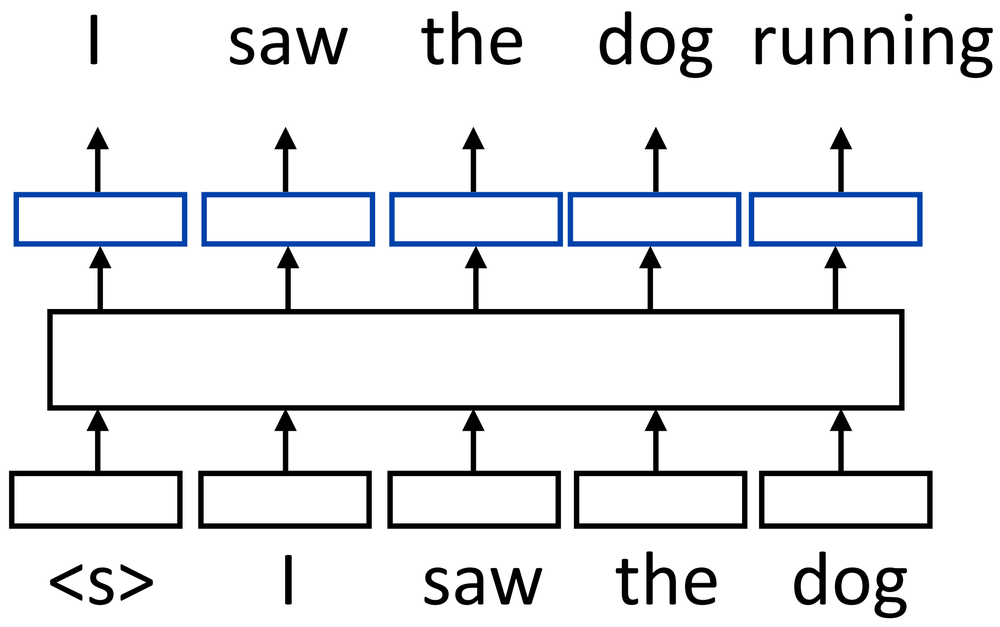
$$P(w|\text{context}) = \frac{\exp(\mathbf{w} \cdot \mathbf{h_i})}{\sum_{w'} \exp(\mathbf{w}' \cdot \mathbf{h_i})}$$

equivalent to

$$P(w|\text{context}) = \text{softmax}(W\mathbf{h}_i)$$

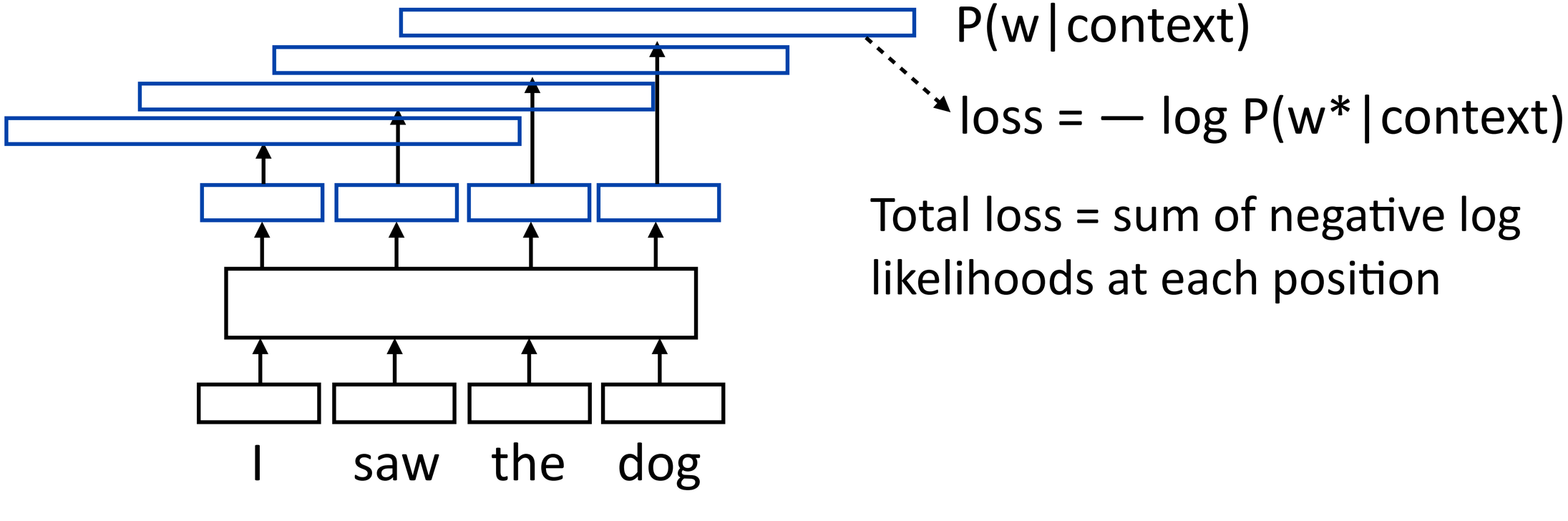
- h_i is the embedding of *dog* produced by the Transformer
- W is a (vocab size) x (hidden size) matrix; linear layer in PyTorch (rows are word embeddings)

Training Transformer LMs



- Input is a sequence of words, output is those words shifted by one
- Allows us to train on predictions across several timesteps simultaneously (similar to batching but this is NOT what we refer to as batching)

Training Transformer LMs



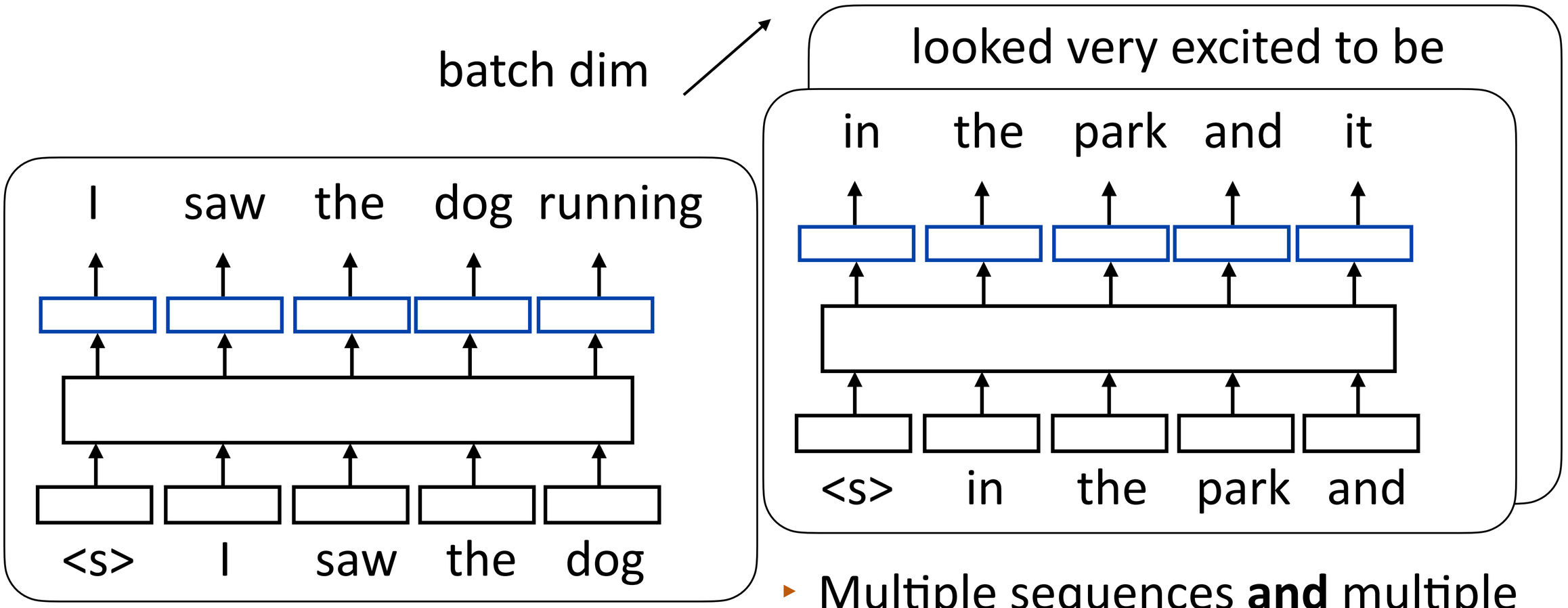

```
loss_fcn = nn.NLLLoss()
loss += loss_fcn(log_probs, ex.output_tensor)
```

[seq len, num output classes] [seq len]

- Batching is a little tricky with NLLLoss: need to collase [batch, seq len, num classes] to [batch * seq len, num classes]. You do not need to batch

Batched LM Training

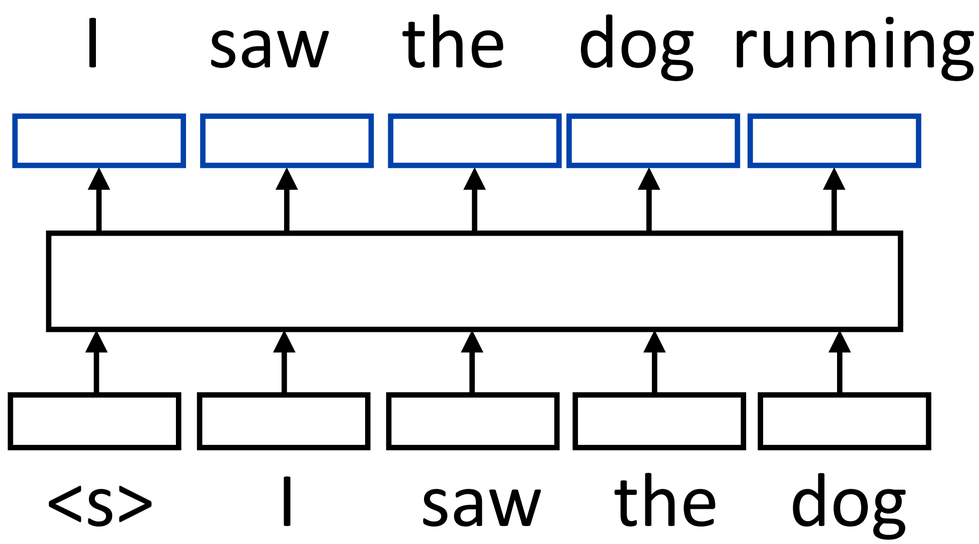
I saw the dog running | in the park and it | looked very excited to be | there



- Multiple sequences **and** multiple timesteps per sequence

A Small Problem with Transformer LMs

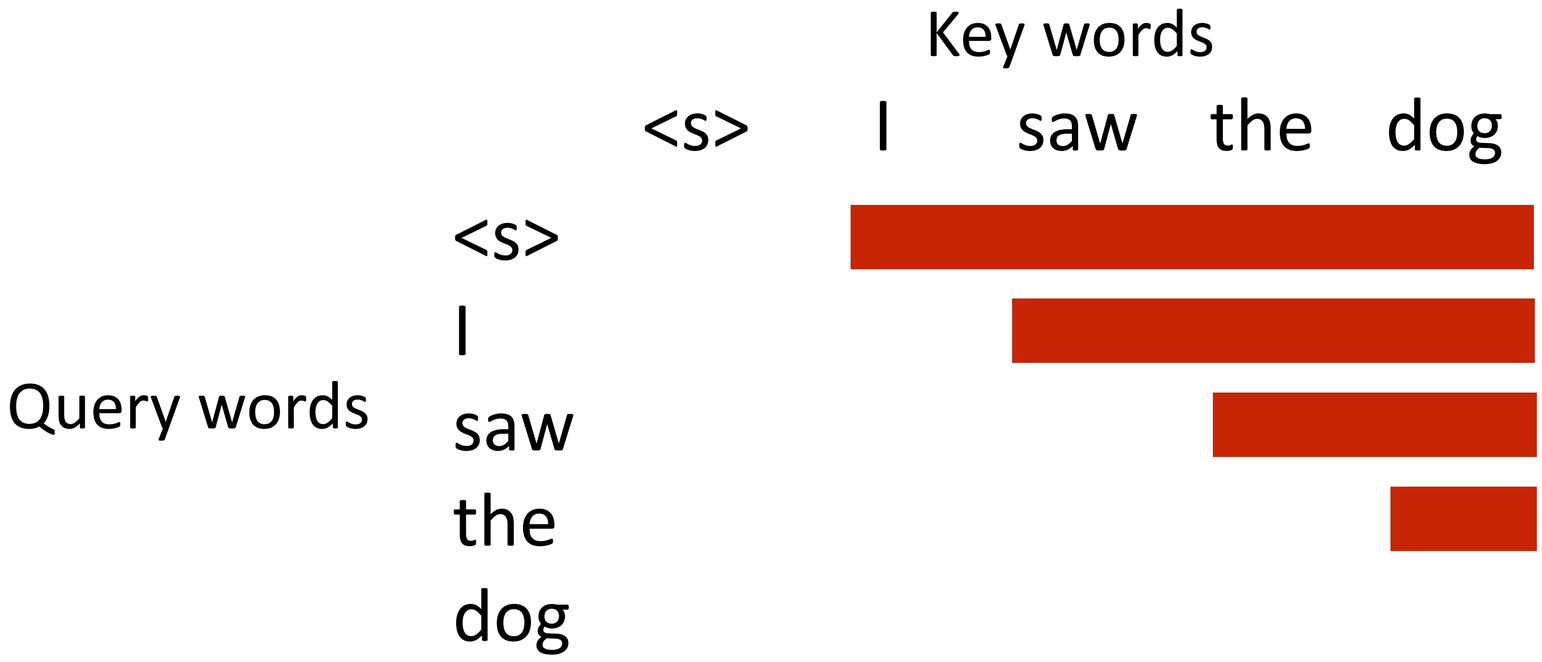
- This Transformer LM as we've described it will *easily* achieve perfect accuracy. Why?



- With standard self-attention: "I" attends to "saw" and the model is "cheating". How do we ensure that this doesn't happen?

Attention Masking

- What do we want to prohibit?



- This is called a **causal** mask (also, causal self-attention / causal Transformers). Only things in the “past” can influence the “present”

Implementation in PyTorch

- nn.TransformerEncoder can be built out of nn.TransformerEncoderLayers, can accept an input and a mask for language modeling:

```
# Inside the module; need to fill in size parameters
layers = nn.TransformerEncoderLayer([...])
transformer_encoder = nn.TransformerEncoder(encoder_layers, num_layers=[...])
[. . .]
# Inside forward(): puts negative infinities in the red part
mask = torch.triu(torch.ones(len, len) * float('-inf'), diagonal=1)
output = transformer_encoder(input, mask=mask)
```

Scaling Laws

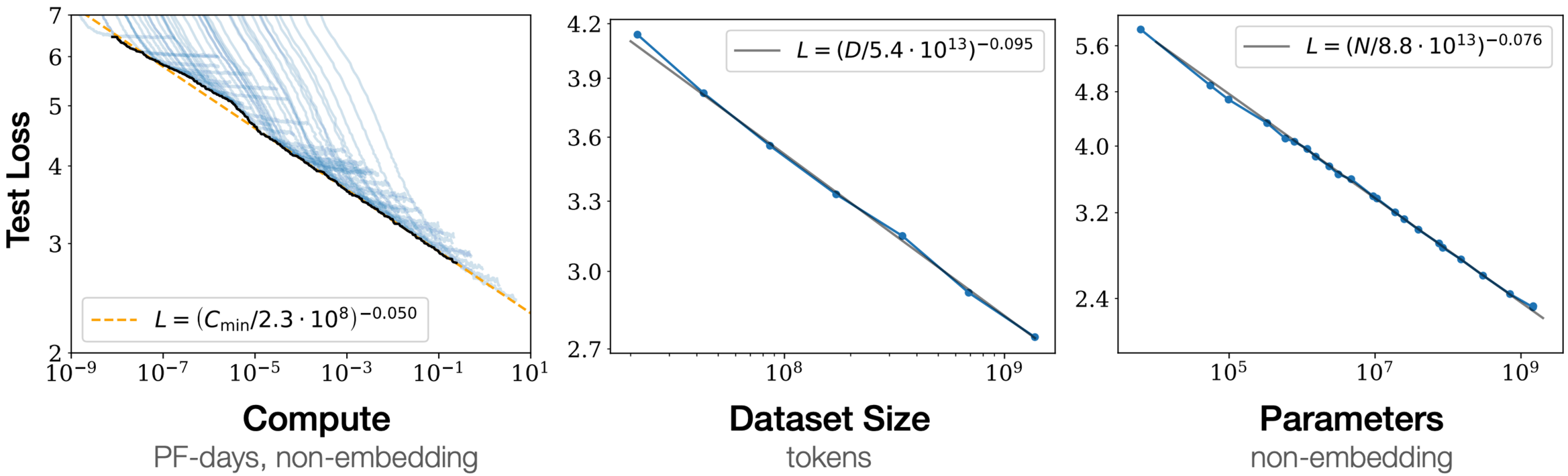


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

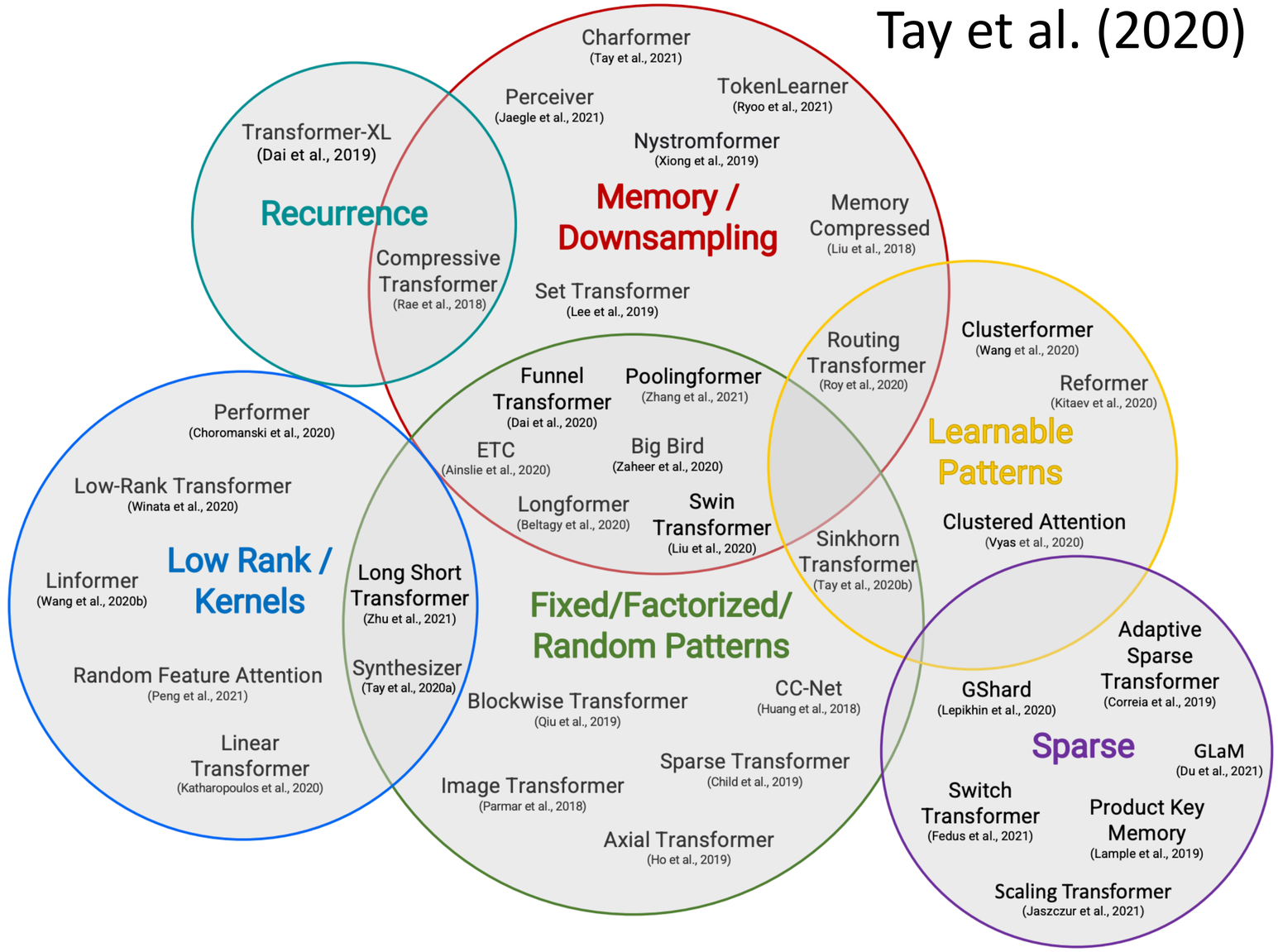
- Transformers scale really well!

Kaplan et al. (2020)

Transformer Runtime

- Even though most parameters and FLOPs are in feedforward layers, Transformers are still limited by quadratic complexity of self-attention
- Many ways proposed to handle this

Tay et al. (2020)



Performers

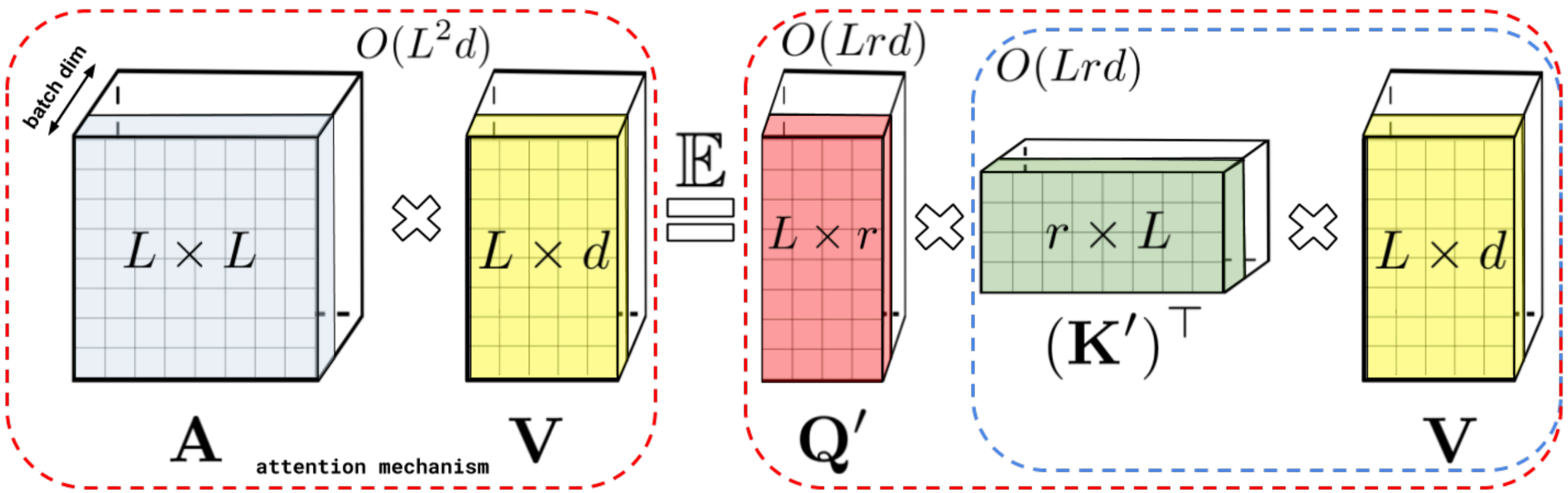
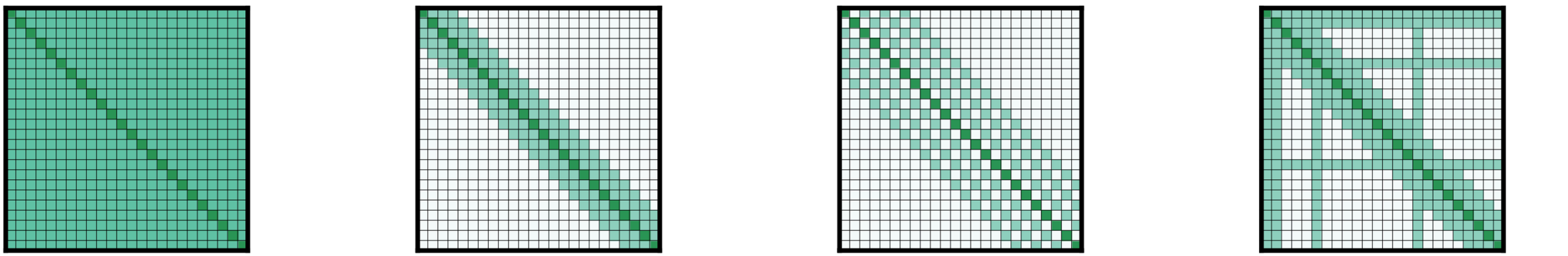


Figure 1: Approximation of the regular attention mechanism $\mathbf{AV}$ (before $\mathbf{D}^{-1}$-renormalization) via (random) feature maps. Dashed-blocks indicate order of computation with corresponding time complexities attached.

- No more len^2 term, but we are fundamentally approximating the self-attention mechanism (cannot form **A** and take the softmax)

Choromanski et al. (2020)

Longformer



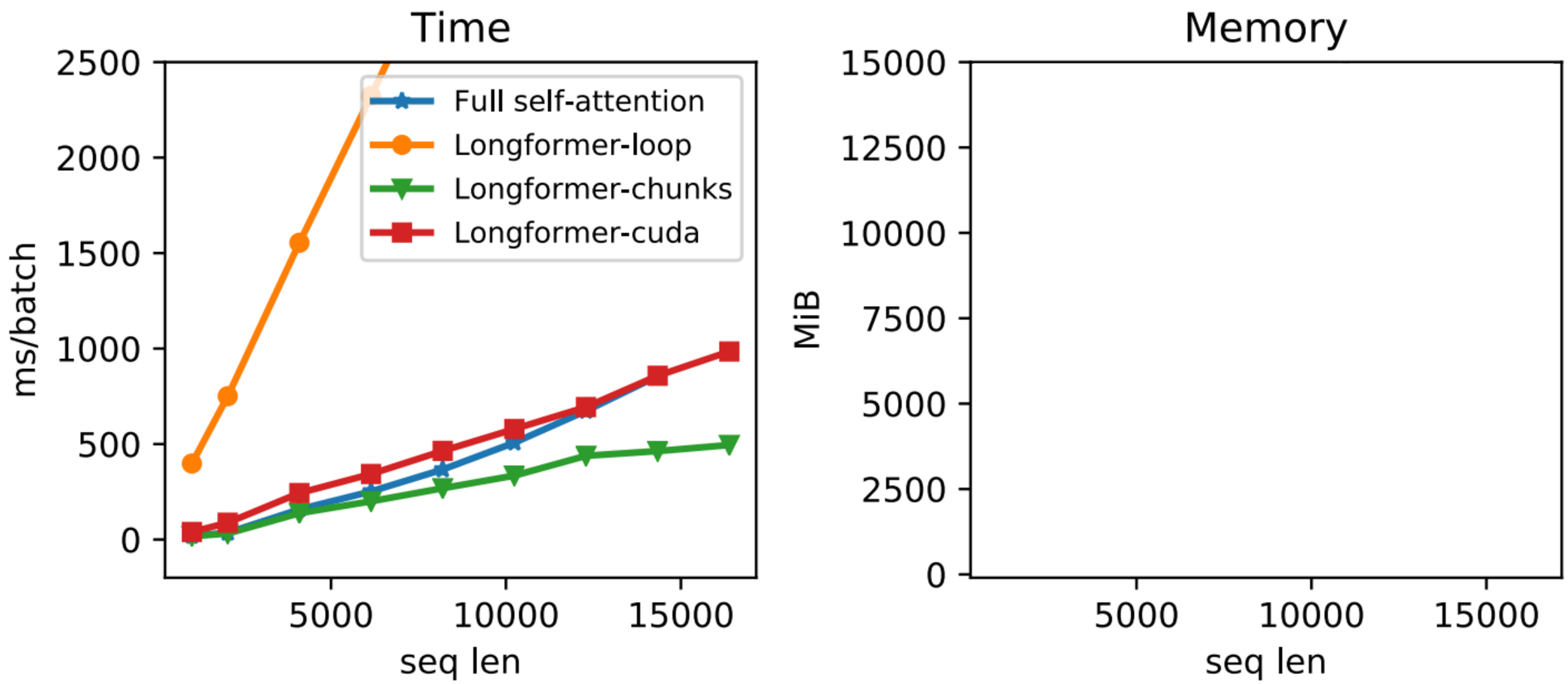
(a) Full n^2 attention (b) Sliding window attention (c) Dilated sliding window (d) Global+sliding window

Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

- Use several pre-specified self-attention patterns that limit the number of operations while still allowing for attention over a reasonable set of things
- Scales to 4096-length sequences

Beltagy et al. (2021)

Attention Maps



- Loop = non-vectorized version
- What will the memory profile look like?

Beltagy et al. (2021)

Handling Rare Words

- Words are a difficult unit to work with: copying can be cumbersome, word vocabularies get very large
- Character-level models don't work well
- Compromise solution: use subword tokens, which may be full words but may also be parts of words

Input: _the **_eco tax** _port i co _in _Po nt - de - Bu is …

Output: _le _port ique **_éco taxe** _de _Pont - de - Bui s

- Can achieve transliteration with this, subword structure makes some translations easier to achieve

Sennrich et al. (2016)

Byte Pair Encoding (BPE)

- Start with every individual byte (character) as its own symbol

```
for i in range(num_merges):
  pairs = get_stats(vocab)
  best = max(pairs, key=pairs.get)
  vocab = merge_vocab(best, vocab)
```

- Count bigram character cooccurrences in dictionary
- Merge the most frequent pair of adjacent characters
- Vocabulary stats are weighted over a large corpus
- Doing 30k merges => vocabulary of 30000 word pieces. Includes many whole words:

and there were no re_ fueling stations anywhere

one of the city ’s more un_ princi_ pled real estate agents

Sennrich et al. (2016)

Word Pieces

- Alternative to BPE

while voc size < target voc size:

Build a language model over your corpus

Merge pieces that lead to highest improvement in language model perplexity

- Issues: what LM to use? How to make this tractable?
- SentencePiece library from Google: unigram LM

Schuster and Nakajima (2012), Wu et al. (2016)

Comparison

(a)	**Original:**	furiously
	BPE:	_fur | ious | ly
	Unigram LM:	_fur | ious | ly
(b)	**Original:**	tricycles
	BPE:	_t | ric | y | cles
	Unigram LM:	_tri | cycle | s
(c)	**Original:**	Completely preposterous suggestions
	BPE:	_Comple | t | ely | _prep | ost | erous | _suggest | ions
	Unigram LM:	_Complete | ly | _pre | post | er | ous | _suggestion | s

- BPE produces less linguistically plausible units than word pieces (unigram LM)
- Some evidence that unigram LM works better in pre-trained transformer models
- Other work explores ensembling across multiple tokenizations

Bostrom and Durrett (2020)

Decoding Strategies

- LMs place a distribution $P(y_i \mid y_1, \ldots, y_{i-1})$
- seq2seq models place a distribution $P(y_i \mid \mathbf{x}, y_1, \ldots, y_{i-1})$
- Generation from both models looks similar; how do we do it?
  - Option 1: max y_i $P(y_i \mid y_1, \ldots, y_{i-1})$ — take greedily best option
  - Option 2: use beam search to find the sequence with the highest prob.
  - Option 3: sample from the model; draw y_i from that distribution
- This segment: beam search

Beam Search

Beam Search

Beam Search: Applications

- Beam search is used frequently in seq2seq conditional generation settings like machine translation where finding the highest-probability hypothesis is really important

- Better models make beam search less crucial. The models are better at "planning ahead" internally so it's less likely that they assign a sequence $y_1 \ldots y_{n-1}$ high probability if there's no good y_n for it

Decoding Strategies

- LMs place a distribution $P(y_i \mid y_1, \ldots, y_{i-1})$
- seq2seq models place a distribution $P(y_i \mid \mathbf{x}, y_1, \ldots, y_{i-1})$
- Generation from both models looks similar; how do we do it?
  - Option 1: max y_i $P(y_i \mid y_1, \ldots, y_{i-1})$: greedily take best option
  - Option 2: use beam search to find the sequence with the highest prob.
  - Option 3: sample from the model; draw y_i from that distribution
- Beam search is great for applications like machine translation or question answering where the answers are somewhat constrained. But LLMs are increasingly being used for **open-ended** generation tasks where there is not one right answer. How do these compare here?

Story Generation (GPT-2)

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, *b*=32:
"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

- Beam search degenerates and starts repeating. If you see a fragment repeated 2-3x, it has very high probability to keep repeating
- Sampling is too noisy, introduces many grammatical errors

Holtzman et al. (2019)

Drawbacks of Beam Search: Degeneration

- Beam search fails because the model is *locally normalized*
- Let's look at all the individual decisions that get made here

Beam Search, *b*=32:
"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

P(*Nacional | … Universidad*) is high

P(*Autónoma | … Universidad Nacional*) is high

P(*de | … Universidad Nacional Autónoma*) is high

P(*México | Universidad Nacional Autónoma de*) is high

P(*/ | … México*) and P(*Universidad | … México /*) — these probabilities may be low. But those are just 2/6 words of the repeating fragment

- Each word is likely given the previous words, but the sequence is bad. (Globally-normalized models can judge this, but are computationally hard to use)

Holtzman et al. (2019)

Drawbacks of Sampling: Long Tail

- Sampling is “too random”

Pure Sampling:
They were cattle called Bolivian Cavalleros; they live in a remote desert uninterrupted by town and they speak huge, beautiful, paradisiacal Bolivian linguistic thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV

P(*y* | … they live in a remote desert uninterrupted by)

0.01 roads
0.01 towns
0.01 people
0.005 civilization

Good options, maybe accounting for 90% of the total probability mass. So a 90% chance of getting something good

…
0.0005 town

Long tail with 10% of the mass

- On average, every 10 words we will get something from the 10% tail of the distribution. 100 words -> 1%. Such words can really derail us!

Holtzman et al. (2019)

Nucleus Sampling

P(*y* | … they live in a remote desert uninterrupted by)

0.01 roads

0.01 towns

0.01 people

0.005 civilization

renormalize and sample

cut off after *p*% of mass

- Define a threshold *p.* Keep the most probable options account for *p*% of the probability mass (the *nucleus*), then sample among these.
- To implement: sort options by probability, truncate the list once the total exceeds *p*, then renormalize and sample from it

Holtzman et al. (2019)

Nucleus Sampling

Method	Perplexity	Self-BLEU4	Repetition %	HUSE
Human	12.38	0.31	0.28	-
Greedy	1.50	0.50	73.66	-
Beam, b=16	1.48	0.44	28.94	-
Stochastic Beam, b=16	19.20	0.28	0.32	-
Pure Sampling	22.73	0.28	0.22	0.67
Sampling, t=0.9	10.25	0.35	0.66	0.79
Top-k=40	6.88	0.39	0.78	0.19
Top-k=640	13.82	**0.32**	**0.28**	0.94
Top-k=40, t=0.7	3.48	0.44	8.86	0.08
Nucleus p=0.95	**13.13**	**0.32**	0.36	**0.97**

- Nucleus: decent perplexity, doesn't have bad repetitions like greedy/ beam do, HUSE (metric that incorporates human evaluation) is much higher, indicates naturalness

Holtzman et al. (2019)

Decoding Strategies

- LMs place a distribution $P(y_i \mid y_1, \ldots, y_{i-1})$
- seq2seq models place a distribution $P(y_i \mid \mathbf{x}, y_1, \ldots, y_{i-1})$
- Generation from both models looks similar; how do we do it?
  - Option 1: max y_i $P(y_i \mid y_1, \ldots, y_{i-1})$ — take greedily best option
  - Option 2: use beam search to find the sequence with the highest prob.
  - ~~Option 3: sample from the model; draw y_i from that distribution~~
  - Option 4: nucleus sampling

WEEK 6

Pre-training & Seq2seq

Segments in this section

seg-68.pdf - BERT: Masked Language Modeling p. 175
seg-69.pdf - BERT: Model and Applications p. 180
seg-s2s1-seq2seq.pdf - Encoder-Decoder (seq2seq) Models p. 191
seg-s2s2-bart.pdf - BART p. 195
seg-s2s3-t5.pdf - T5 p. 200

Key points (added notes)

- BERT: masked LM (15%, 80/10/10) + next sentence prediction; [CLS] for classification; bidirectional; cannot generate.
- RoBERTa: dynamic masking, no NSP, more data.
- Seq2seq: encoder + decoder with cross-attention; teacher forcing.
- BART: denoising seq2seq (infilling, permutation). T5: span corruption with sentinels, text-to-text, C4.

EXAM: what would improve BERT, BERT vs BART capabilities.

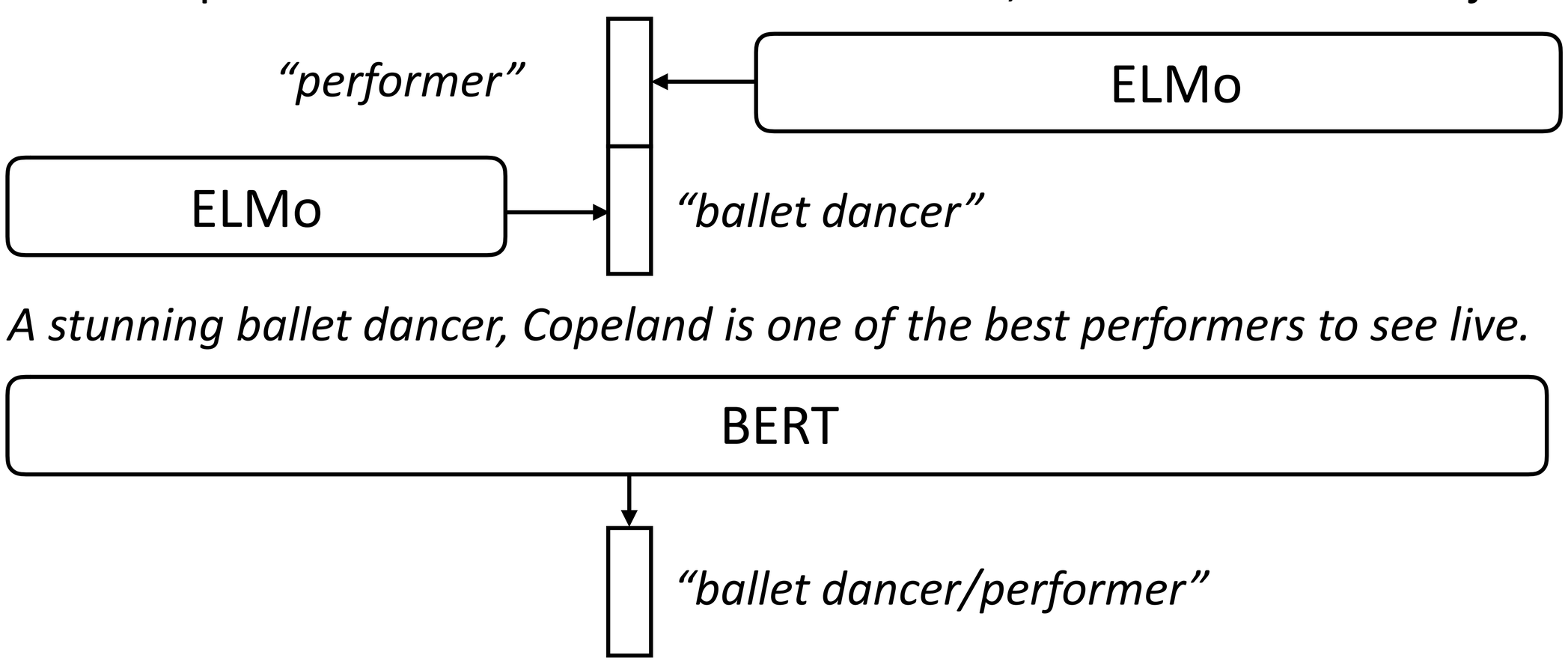
Note: segment 59 listed in the README is not in the folder.

BERT

- AI2 made ELMo in spring 2018, GPT (transformer-based ELMo) was released in summer 2018, BERT came out October 2018
- Four major changes compared to ELMo:
  - Transformers instead of LSTMs
  - Bidirectional model with “Masked LM” objective instead of normal LM
  - Fine-tune instead of freeze at test time
  - Operates over word pieces (byte pair encoding)

BERT

- ELMo is a unidirectional model (as is GPT): we can concatenate two unidirectional models, but is this the right thing to do?
- ELMo reprs look at each direction in isolation; BERT looks at them jointly

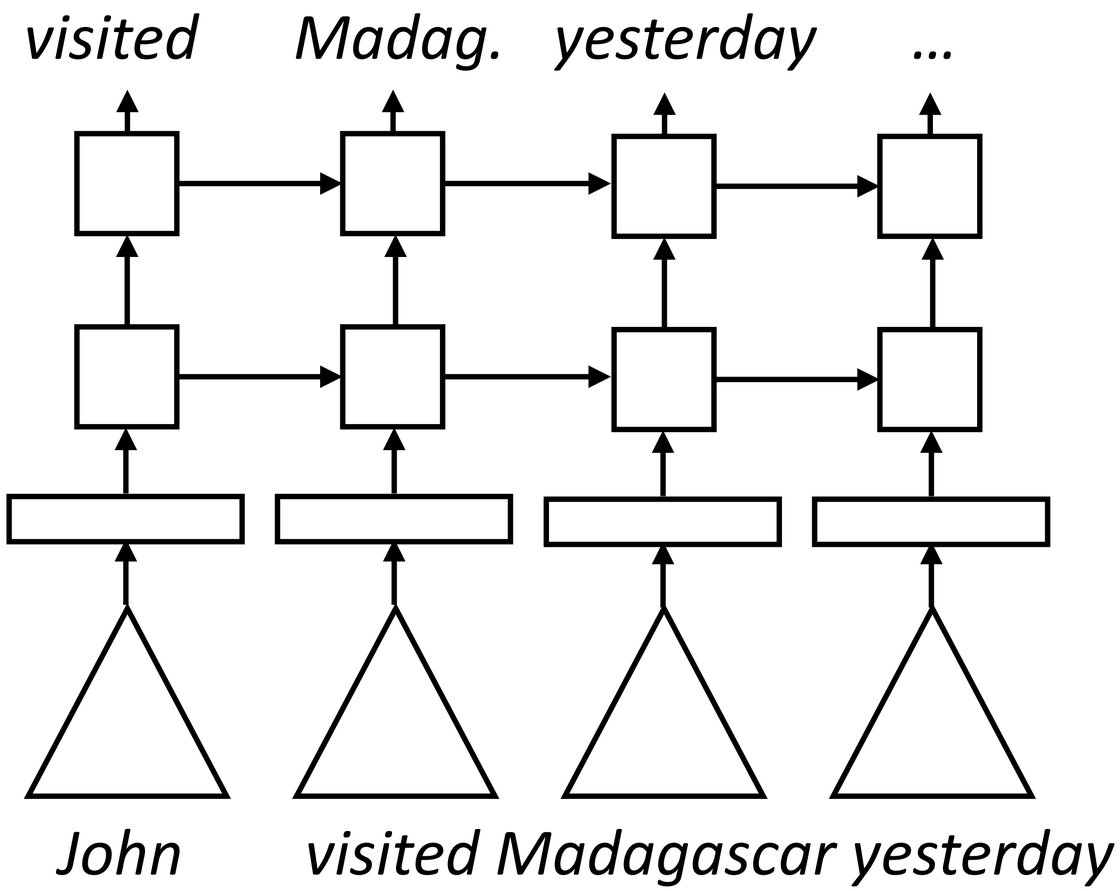


Devlin et al. (2019)

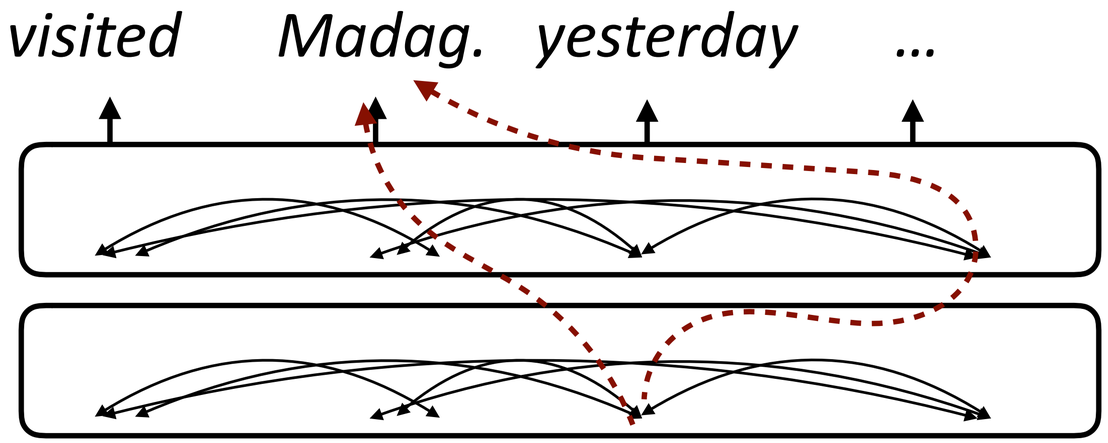
Bidirectional Modeling

- How to learn a “deeply bidirectional” model? What happens if we just replace an LSTM with a transformer?

ELMo (Language Modeling)



BERT



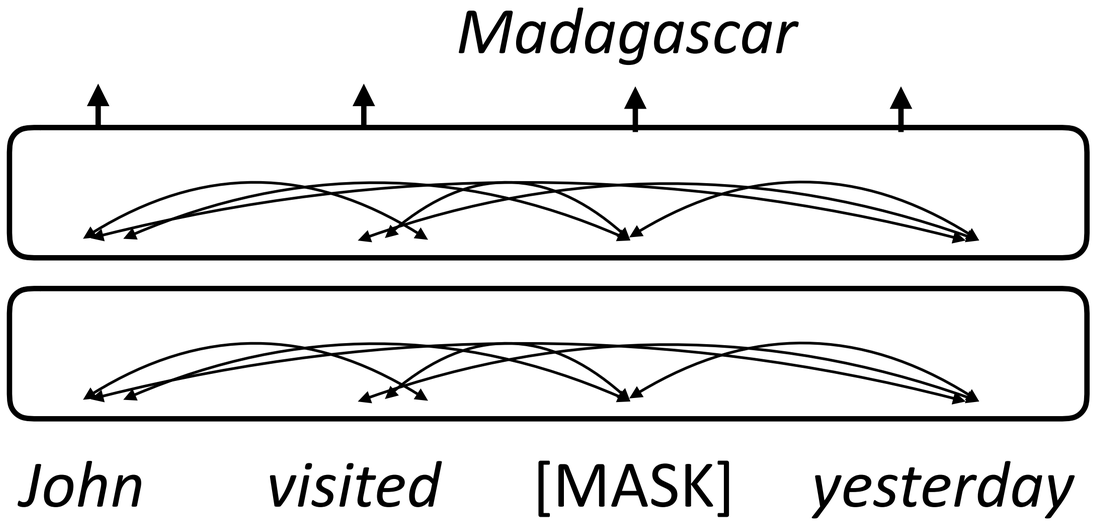
- You could do this with a “one-sided” transformer, but this “two-sided” model can cheat

Devlin et al. (2019)

Masked Language Modeling

- How to prevent cheating? Next word prediction fundamentally doesn't work for bidirectional models, instead do *masked language modeling*

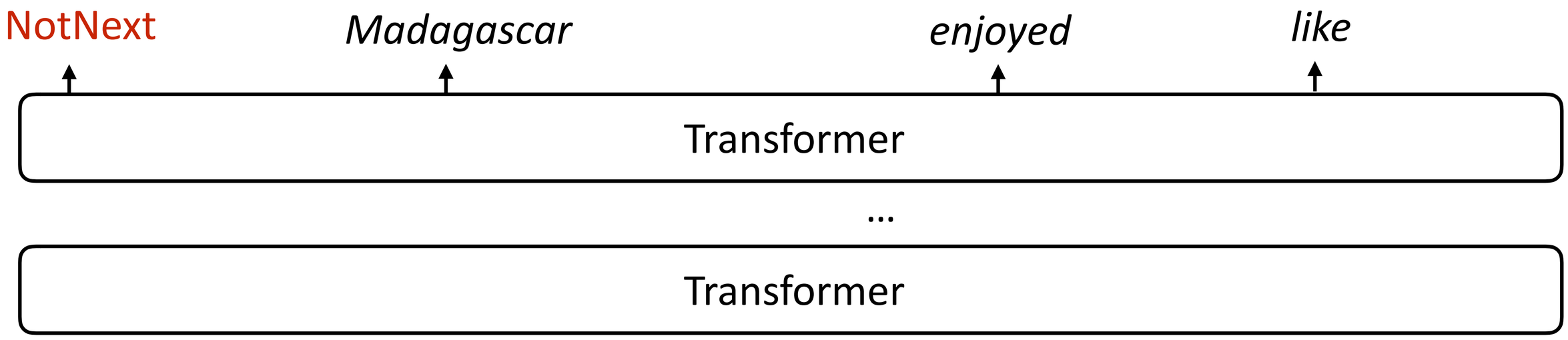
- BERT formula: take a chunk of text, mask out 15% of the tokens, and try to predict them



Devlin et al. (2019)

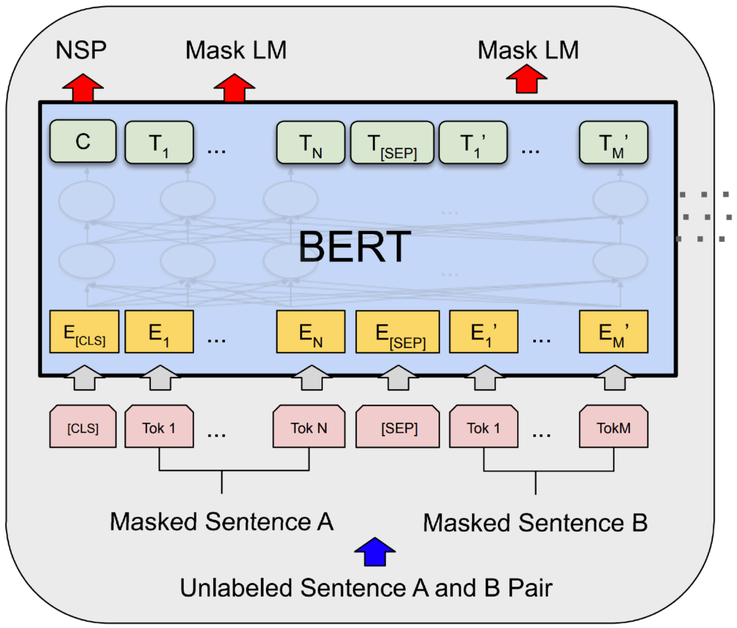
BERT Objective

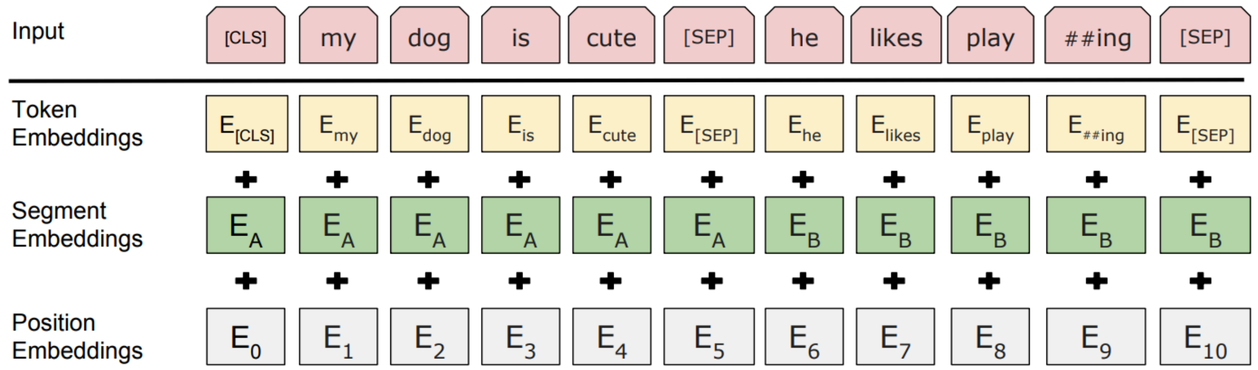
- Input: [CLS] Text chunk 1 [SEP] Text chunk 2
- 50% of the time, take the true next chunk of text, 50% of the time take a random other chunk. Predict whether the next chunk is the “true” next
- BERT objective: masked LM + next sentence prediction



BERT: Model and Applications

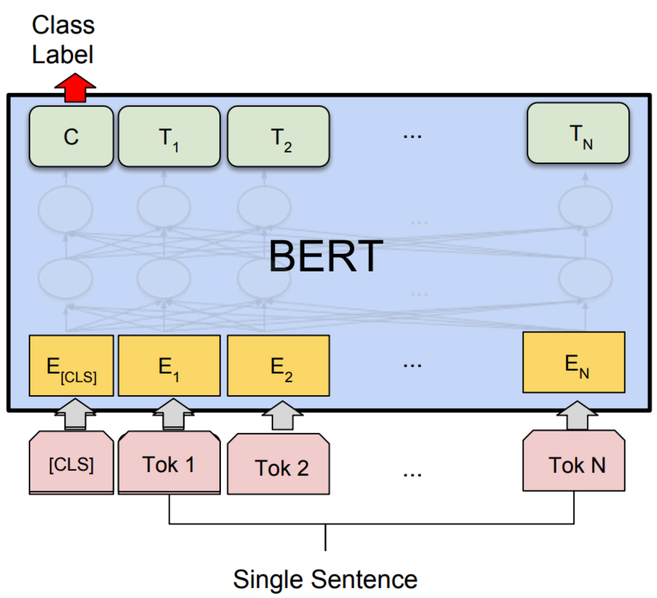
- BERT Base: 12 layers, 768-dim per wordpiece token, 12 heads. Total params = 110M
- BERT Large: 24 layers, 1024-dim per wordpiece token, 16 heads. Total params = 340M
- Positional embeddings and segment embeddings, 30k word pieces
- This is the model that gets **pre-trained** on a large corpus



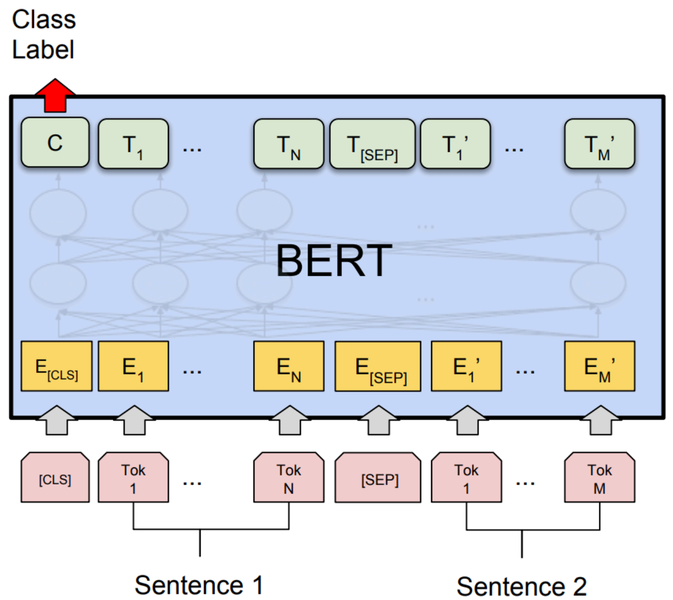


Devlin et al. (2019)

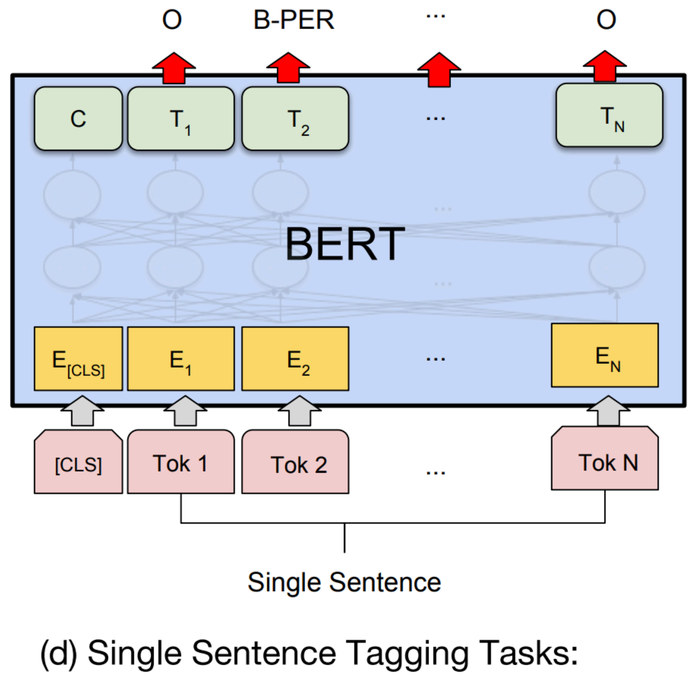
What can BERT do?



(b) Single Sentence Classification Tasks: SST-2, CoLA



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



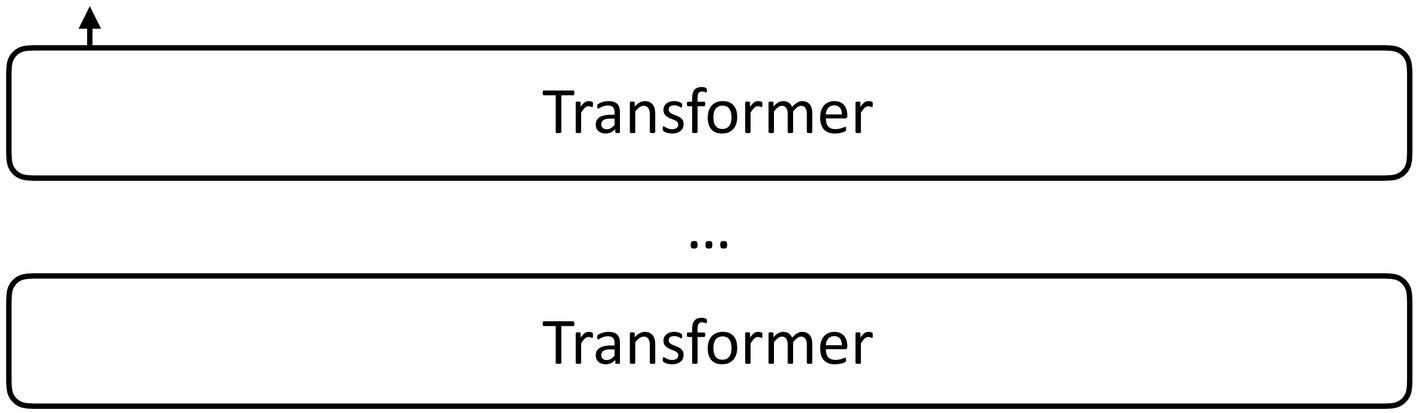
(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

- Artificial [CLS] token is used as the vector to do classification from
- Sentence pair tasks (entailment): feed both sentences into BERT
- BERT can also do tagging by predicting tags at each word piece

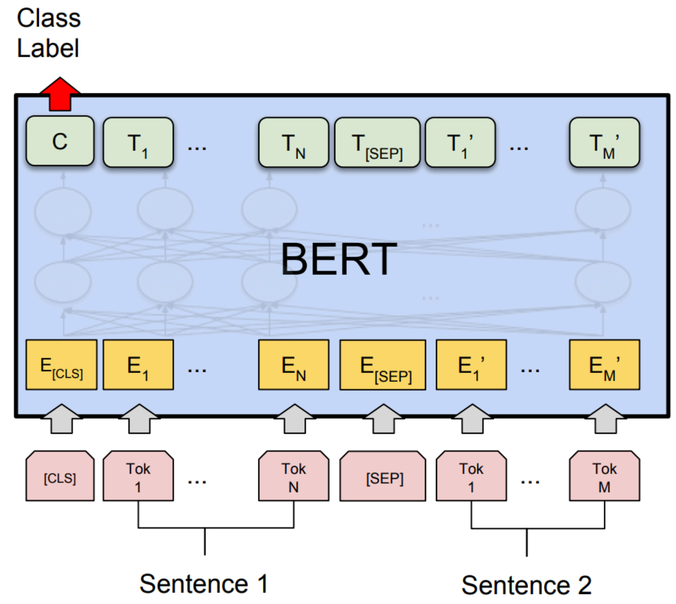
Devlin et al. (2019)

What can BERT do?

Entails (first sentence implies second is true)



[CLS] A boy plays in the snow [SEP] A boy is outside



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

- How does BERT model sentence pair tasks?
- Transformers can capture interactions between the two sentences (even though the NSP objective doesn't really cause this to happen)

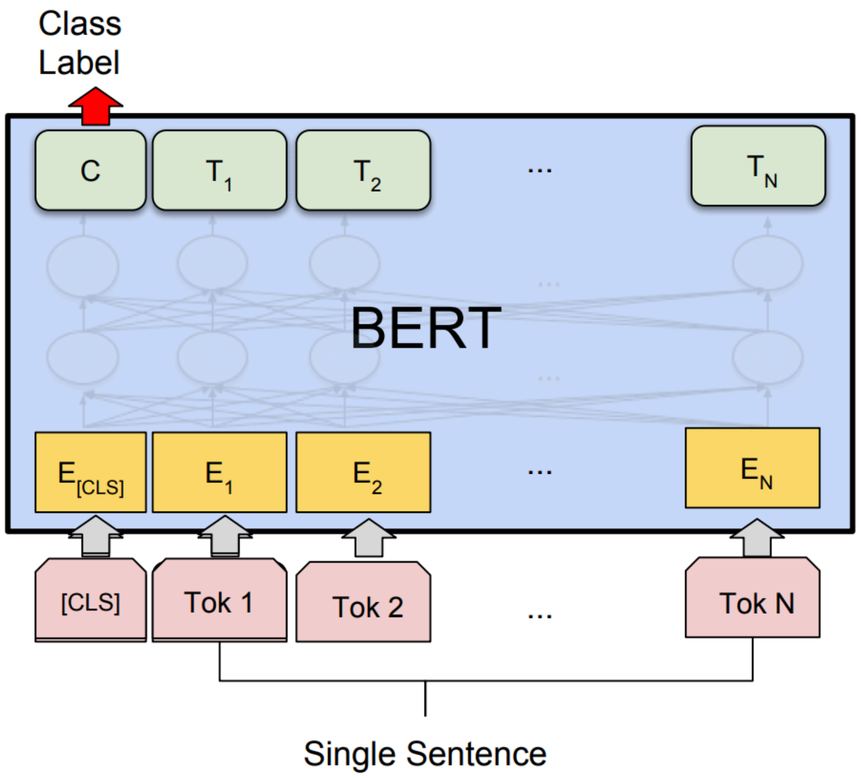
Devlin et al. (2019)

What can BERT NOT do?

- BERT **cannot** generate text (at least not in an obvious way)
  - Can fill in MASK tokens, but can't generate left-to-right (you can put MASK at the end repeatedly, but this is slow)
- Masked language models are intended to be used primarily for "analysis" tasks

Devlin et al. (2019)

Fine-tuning BERT



(b) Single Sentence Classification Tasks: SST-2, CoLA

- Fine-tune for 1-3 epochs, small learning rate
- Large changes to weights up here (particularly in last layer to route the right information to [CLS])
- Smaller changes to weights lower down in the transformer
- Small LR and short fine-tuning schedule mean weights don't change much
- More complex "triangular learning rate" schemes exist

Fine-tuning BERT

Pretraining	Adaptation	NER CoNLL 2003	SA SST-2	Nat. lang. inference MNLI	Nat. lang. inference SICK-E	Semantic textual similarity SICK-R	Semantic textual similarity MRPC	Semantic textual similarity STS-B
Skip-thoughts	❄️	-	81.8	62.9	-	86.6	75.8	71.8
ELMo	❄️	91.7	**91.8**	**79.6**	**86.3**	**86.1**	**76.0**	**75.9**
	🔥	**91.9**	91.2	76.4	83.3	83.3	74.7	75.5
	Δ=🔥-❄️	0.2	-0.6	-3.2	-3.3	-2.8	-1.3	-0.4
BERT-base	❄️	92.2	93.0	**84.6**	84.8	86.4	78.1	82.9
	🔥	**92.4**	**93.5**	**84.6**	**85.8**	**88.7**	**84.8**	**87.1**
	Δ=🔥-❄️	0.2	0.5	0.0	1.0	2.3	6.7	4.2

- BERT is typically better if the whole network is fine-tuned, unlike ELMo

Peters et al. (2019)

Evaluation

Corpus	|Train|	|Test|	Task	Metrics	Domain
			Single-Sentence Tasks		
CoLA	8.5k	**1k**	acceptability	Matthews corr.	misc.
SST-2	67k	1.8k	sentiment	acc.	movie reviews
			Similarity and Paraphrase Tasks		
MRPC	3.7k	1.7k	paraphrase	acc./F1	news
STS-B	7k	1.4k	sentence similarity	Pearson/Spearman corr.	misc.
QQP	364k	**391k**	paraphrase	acc./F1	social QA questions
			Inference Tasks		
MNLI	393k	**20k**	NLI	matched acc./mismatched acc.	misc.
QNLI	105k	5.4k	QA/NLI	acc.	Wikipedia
RTE	2.5k	3k	NLI	acc.	news, Wikipedia
WNLI	634	**146**	coreference/NLI	acc.	fiction books

Wang et al. (2019)

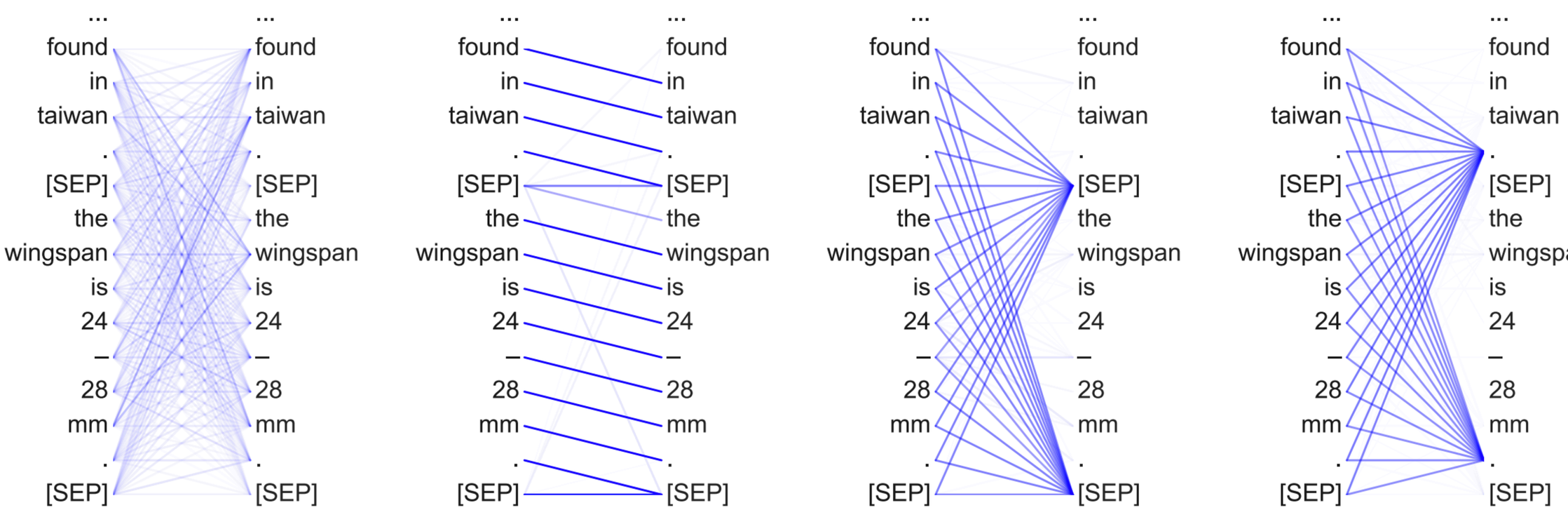
Evaluation

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**91.1**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**81.9**

- Huge improvements over prior work (even compared to ELMo)
- Effective at “sentence pair” tasks: textual entailment (does sentence A imply sentence B), paraphrase detection

Devlin et al. (2019)

Analysis



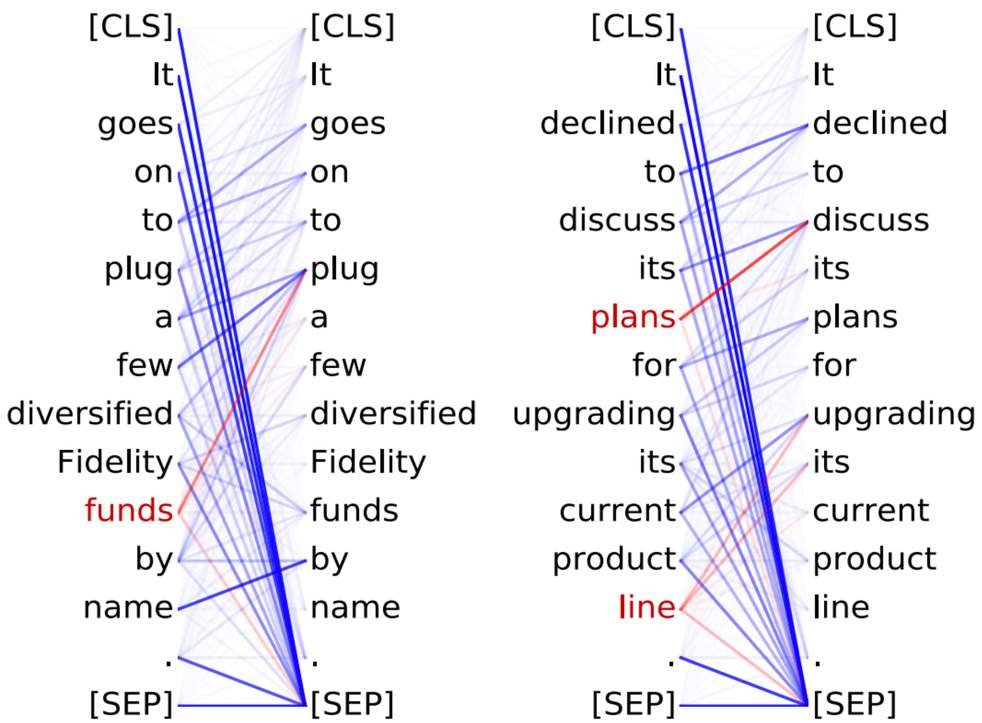
- Heads on transformers learn interesting and diverse things: content heads (attend based on content), positional heads (based on position), etc.

Clark et al. (2019)

Analysis

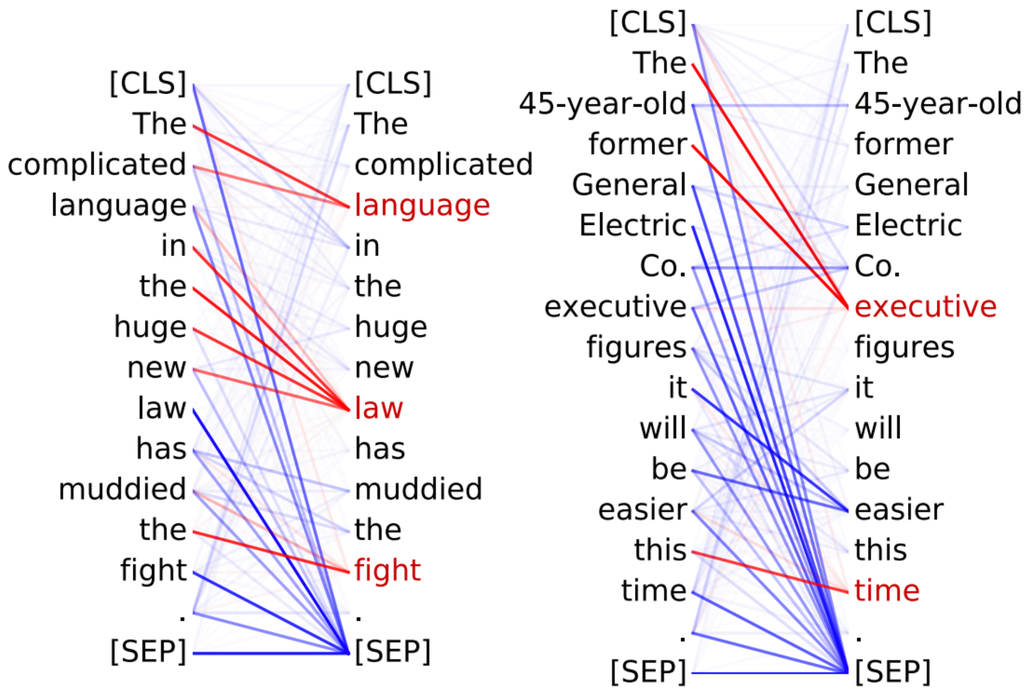
Head 8-10

- **Direct objects** attend to their verbs
- 86.8% accuracy at the `dobj` relation



Head 8-11

- **Noun modifiers** (e.g., determiners) attend to their noun
- 94.3% accuracy at the `det` relation



- Still way worse than what supervised parsing systems can do, but interesting that this is learned organically

RoBERTa

- "Robustly optimized BERT"
- 160GB of data instead of 16 GB
- Dynamic masking: standard BERT uses the same MASK scheme for every epoch, RoBERTa recomputes them

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

- New training + more data = better performance
- For this and more: check out Huggingface Transformers or fairseq

Liu et al. (2019)

Encoder-Decoder (seq2seq) Models

- Can view many tasks as mapping from an input sequence of tokens to an output sequence of tokens
- Syntactic parsing

The dog ran ⟶ `(S (NP (DT the) (NN dog) ) (VP (VBD ran) ) )`

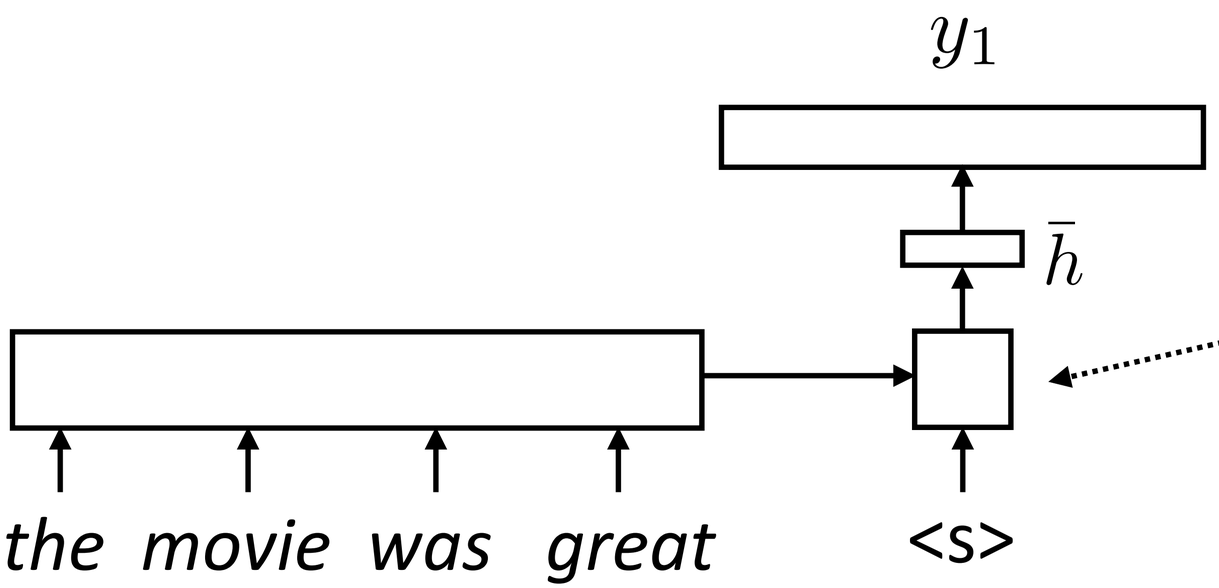
- Semantic parsing:

What states border Texas ⟶ `λ x state( x ) ∧ borders( x , e89 )`

- Machine translation, summarization, dialogue can all be viewed in this framework as well; our examples will be MT for now
- This is slightly different from language modeling (“decoder-only”) because the input and output vocabularies can be different. Modern language models like ChatGPT can model all this with a shared vocabulary.

Seq2seq Models

- Generate next word conditioned on previous output as well as input
- W size is |vocab| x |hidden state|, softmax over entire vocabulary



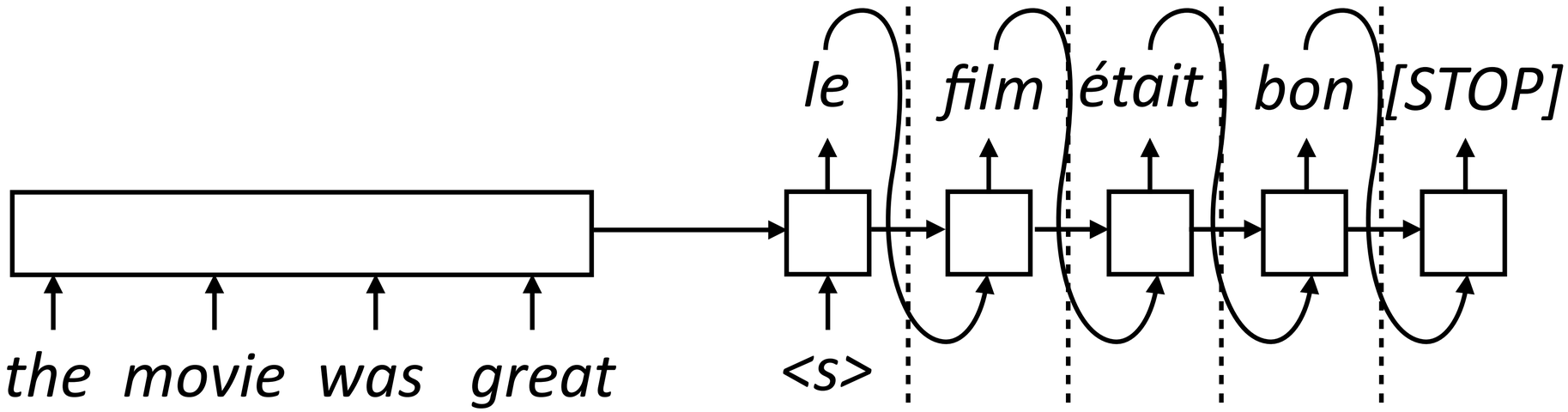
$$P(y_i|\mathbf{x}, y_1, \ldots, y_{i-1}) = \text{softmax}(W\bar{h})$$

$$P(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^{n} P(y_i|\mathbf{x}, y_1, \ldots, y_{i-1})$$

Decoder has separate parameters from encoder

- Example: translate this input **x** into a French output **y**

Seq2seq Models: Inference and Training

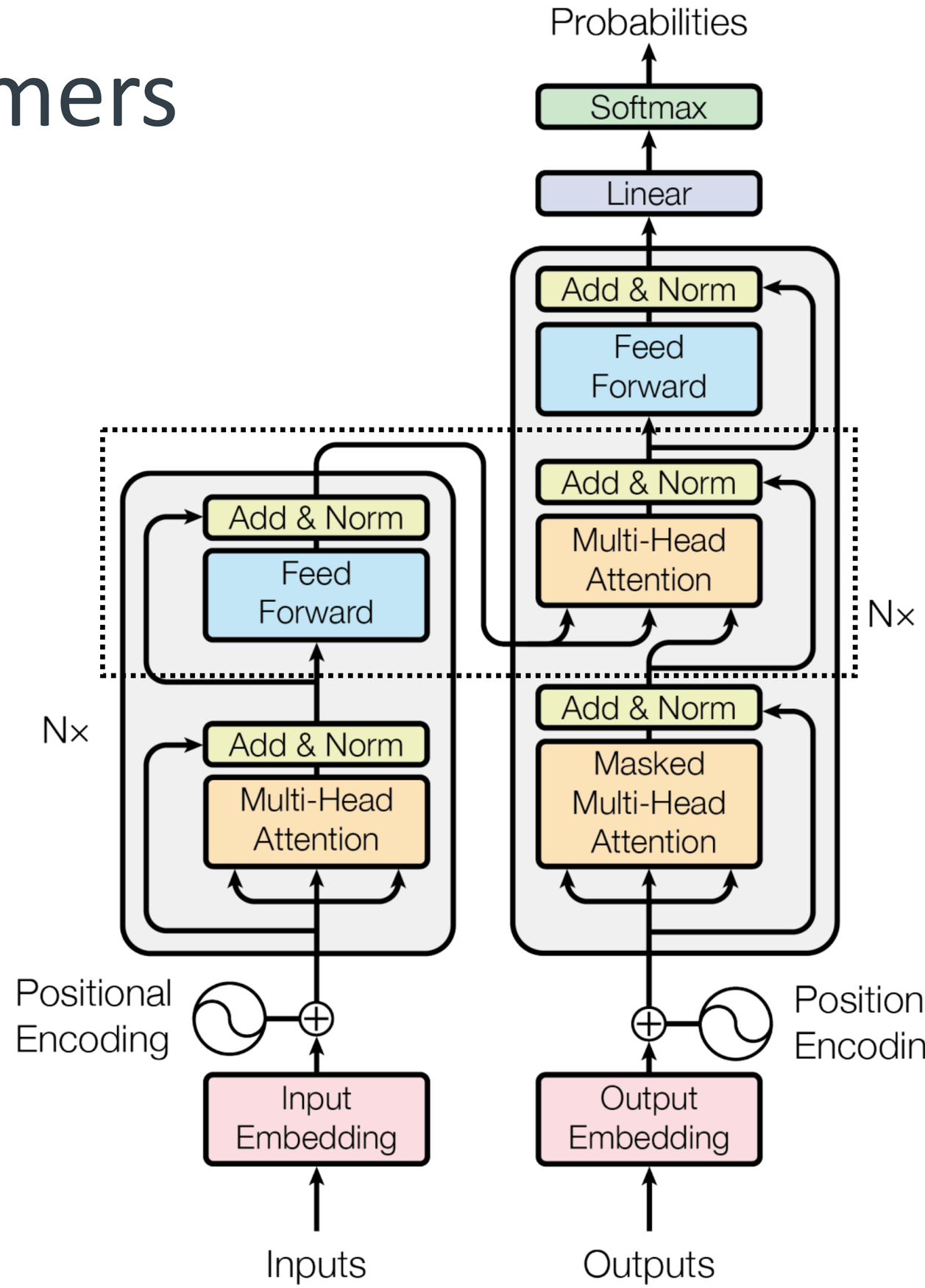


- **Inference**: need to compute the argmax over the word predictions and then feed that to the next Transformer call
- Decoder is advanced one state at a time until [STOP] is reached
- The encoder can just be run a single time

- **Training**: same as language model training, maximize the probability of the gold sequence ***y*** (now conditioned on the input ***x***)

Seq2seq Transformers

- Encoder-decoder Transformer includes a separate multi-head attention computation that attends to the encoder inputs
- Otherwise, behaves very similarly to the Transformer we've seen before

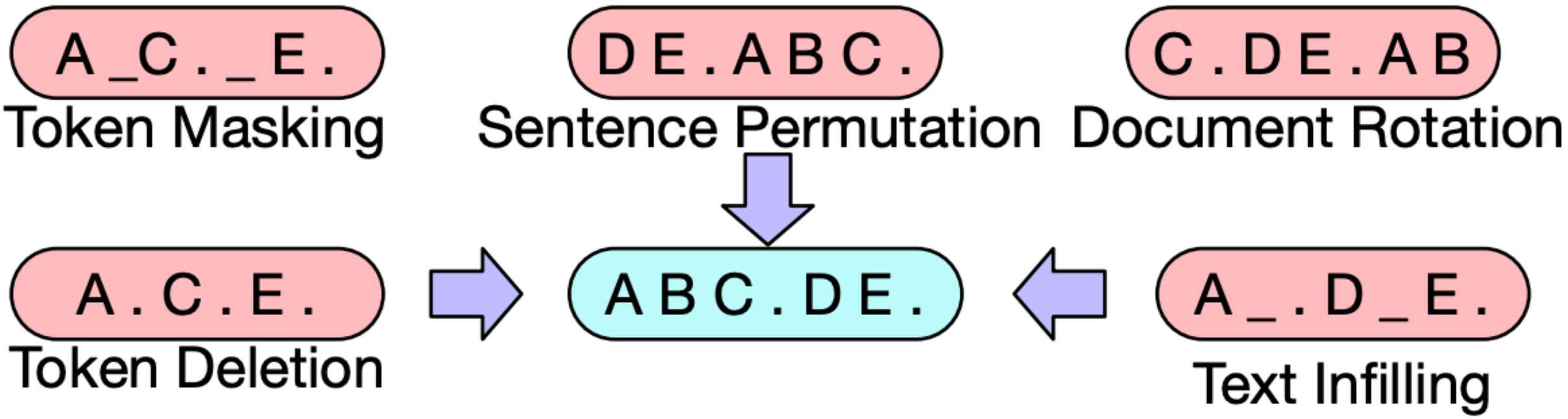


Vaswani et al. (2017)

Seq2seq Pre-training

- LMs P(**w**): trained unidirectionally
- Masked LMs: trained bidirectionally but with masking
- How can we pre-train a model for P(**y**|**x**)?
- Why was BERT effective? Predicting a mask requires some kind of text "understanding":
- What would it take to impart the same "skills" for sequence prediction?

BART



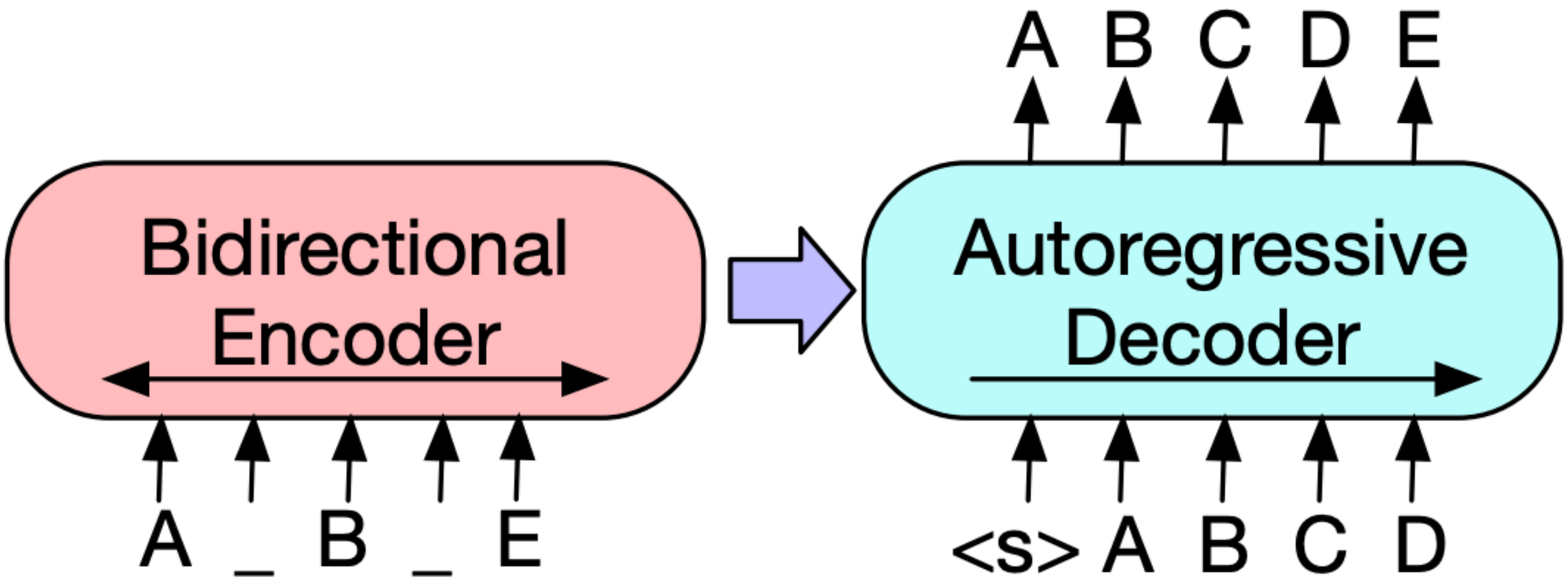
Infilling is longer spans than masking

- Several possible strategies for corrupting a sequence are explored in the BART paper

Lewis et al. (2019)

BART

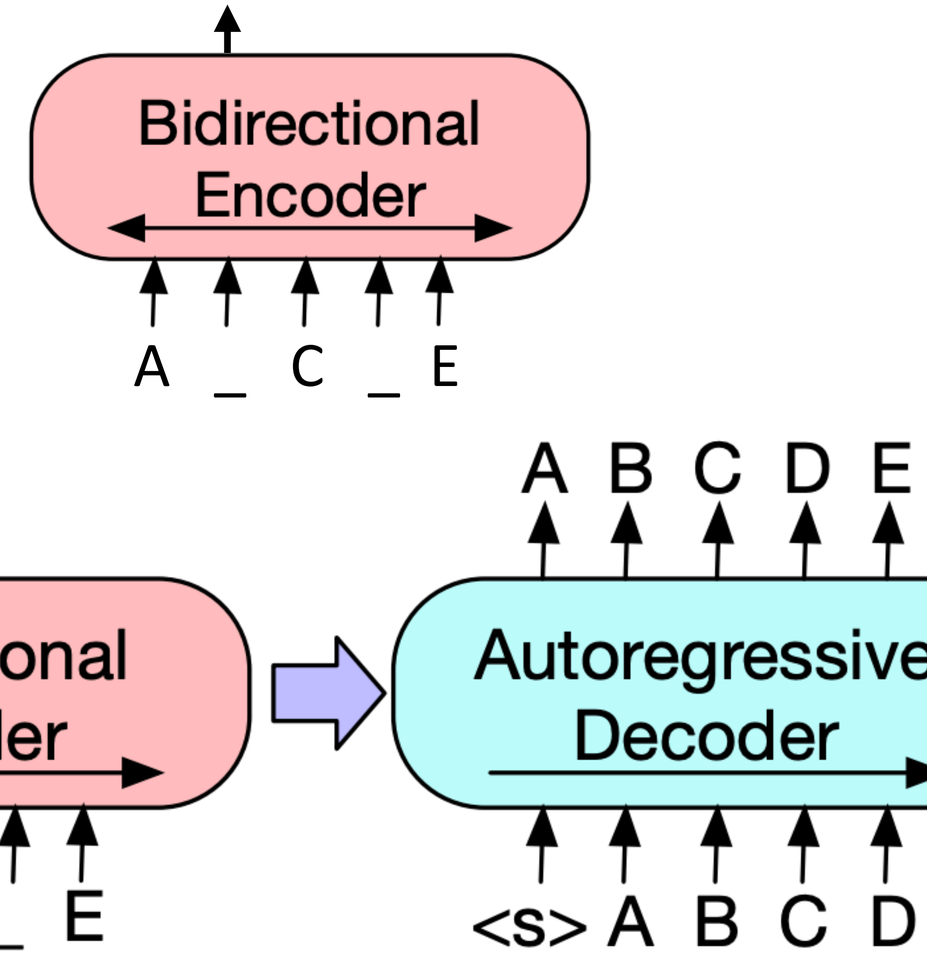
- Sequence-to-sequence Transformer trained on this data: permute/ make/delete tokens, then predict full sequence autoregressively



Lewis et al. (2019)

BERT vs. BART

- BERT: only parameters are an encoder, trained with masked language modeling objective. Cannot generate text or do seq2seq tasks



- BART: both an encoder and a decoder. Can also use just the encoder wherever we would use BERT

Lewis et al. (2019)

BART for Summarization

- **Pre-train** on the BART task: take random chunks of text, noise them according to the schemes described, and try to “decode” the clean text
- **Fine-tune** on a summarization dataset: a news article is the input and a summary of that article is the output (usually 1-3 sentences depending on the dataset)

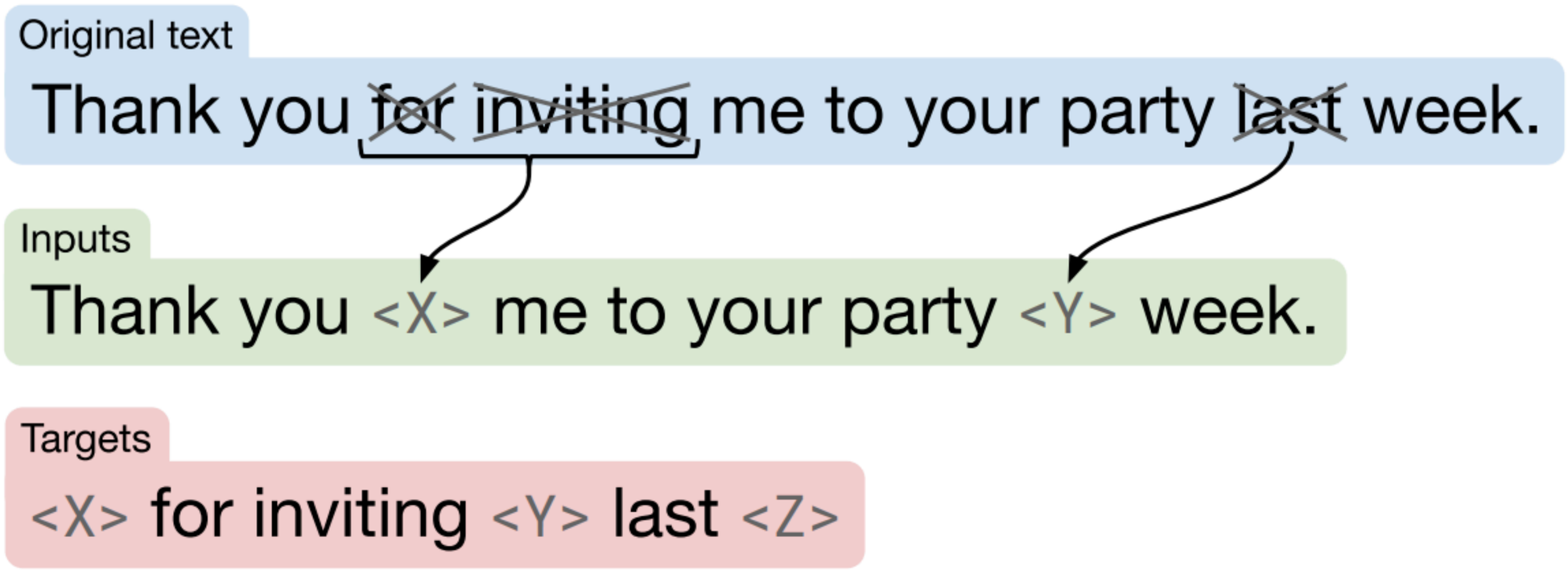
This is the first time anyone has been recorded to run a full marathon of 42.195 kilometers (approximately 26 miles) under this pursued landmark time. It was not, however, an officially sanctioned world record, as it was not an ”open race” of the IAAF. His time was 1 hour 59 minutes 40.2 seconds. Kipchoge ran in Vienna, Austria. It was an event specifically designed to help Kipchoge break the two hour barrier.

Kenyan runner Eliud Kipchoge has run a marathon in less than two hours.

Lewis et al. (2019)

T5

- Pre-training: similar denoising scheme to BART (they were released within a week of each other in fall 2019)
- Input: text with gaps. Output: a series of phrases to fill those gaps.



Raffel et al. (2019)

T5

Number of tokens	Repeats	GLUE	CNNDM	EnDe	EnFr	EnRo
★ Full dataset	0	**83.28**	**19.24**	**26.98**	**39.82**	**27.65**
2^{29}	64	**82.87**	**19.19**	**26.83**	**39.74**	**27.63**
2^{27}	256	82.62	**19.20**	**27.02**	**39.71**	27.33
2^{25}	1,024	79.55	18.57	26.38	39.56	26.80
2^{23}	4,096	76.34	18.33	26.37	38.84	25.81

summarization

machine translation

- Colossal Cleaned Common Crawl: 750 GB of text
- T5 was one of the first papers to heavily focus on the scale and quality of pre-training data
- Another key focus: multitasking between generation tasks

Raffel et al. (2019)

Successes of T5: UnifiedQA

- How can we handle a task like QA by framing it as a seq2seq problem?

Dataset	SQuAD 1.1
Input	At what speed did the turbine operate? \n (Nikola_Tesla) On his 50th birthday in 1906, Tesla demonstrated his 200 horsepower (150 kilowatts) 16,000 rpm bladeless turbine. ...
Output	16,000 rpm

- Format: *Question \n Passage* —> *Answer*
  encoder decoder

Khashabi et al. (2020)

Successes of T5: UnifiedQA

AB	**Dataset**	NarrativeQA
	Input	What does a drink from narcissus's spring cause the drinker to do? \n Mercury has awakened Echo, who weeps for Narcissus, and states that a drink from Narcissus's spring causes the drinkers to ``Grow dotingly enamored of themselves.'' ...
	Output	fall in love with themselves

Abstractive question, requires generating *free-form answer*

- Past work: different architectures for every QA formulation. (Span selection, answer generation, multiple choice, …)
- Now: one 11B parameter T5 model

Khashabi et al. (2020)

Successes of T5: UnifiedQA

Multiple choice

Yes/no

MC	**Dataset**	MCTest
	Input	Who was Billy? \n (A) The skinny kid (B) A teacher (C) A little kid (D) The big kid \n Billy was like a king on the school yard. A king without a queen. He was the biggest kid in our grade, so he made all the rules during recess. ...
	Output	The big kid
YN	**Dataset**	BoolQ
	Input	Was America the first country to have a president? \n (President) The first usage of the word president to denote the highest official in a government was during the Commonwealth of England ...
	Output	no

- Past work: different architectures for every QA formulation. (Span selection, answer generation, multiple choice, …)
- Now: one 11B parameter T5 model
- Gives strong results across all these QA tasks when fine-tuned on them together

Khashabi et al. (2020)

WEEKS 7-8

Structured Prediction: Tagging & Parsing

Segments in this section

seg-22.pdf - Part-of-Speech Tagging p. 206
seg-23.pdf - Sequence Labeling with Classifiers p. 208
seg-24.pdf - Hidden Markov Models p. 211
seg-25.pdf - HMM Parameter Estimation p. 212
seg-26.pdf - HMM Inference: Viterbi p. 214
seg-28.pdf - Trigram Taggers & POS Tagging Results p. 217
seg-34.pdf - Constituency Parsing p. 224
seg-35.pdf - Probabilistic Context-Free Grammars p. 231
seg-36.pdf - CKY Algorithm p. 233
seg-37.pdf - Refining PCFGs: Markovization & Lexicalization p. 235
seg-38.pdf - Dependency Parsing p. 243
seg-39.pdf - Transition-based Dependency Parsing p. 253

Key points (added notes)

- POS tagging; independent classifiers give incoherent sequences, so use sequence models.
- HMM: P(y1) prod P(y_i|y_i-1) prod P(x_i|y_i) P(STOP|y_n); MLE by counting; Viterbi O(n k^2).
- PCFGs: rule probs = count(X->a)/count(X); CKY O(n^3 |G|) on a binarized grammar.
- Vertical / horizontal Markovization, lexicalization (head words).
- Dependency parsing, projectivity; arc-standard shift-reduce (Shift, Left-Arc, Right-Arc; 2n transitions).

EXAM: HMM sequence probability, counting valid tag sequences, Viterbi complexity; extract a grammar from trees, parse probability, Markovization effects.

Note: segments 22-26 and 35-36 are handwritten. The CRF / forward-backward segments (29-33) are not in the folder; see the study guide, Section 23.

Part-of-Speech Tagging

record → TTS
- noun
- verb

arms → IE
- noun
- verb

POS Tags:

Open class — new words can join these categories

Nouns: Proper (IBM, Italy)
Common (cat, snow)

Verbs: see, tweet

Adjectives

Adverbs: swiftly

Closed class

Determiners: the, some

Conjunctions: and, or

Pronouns

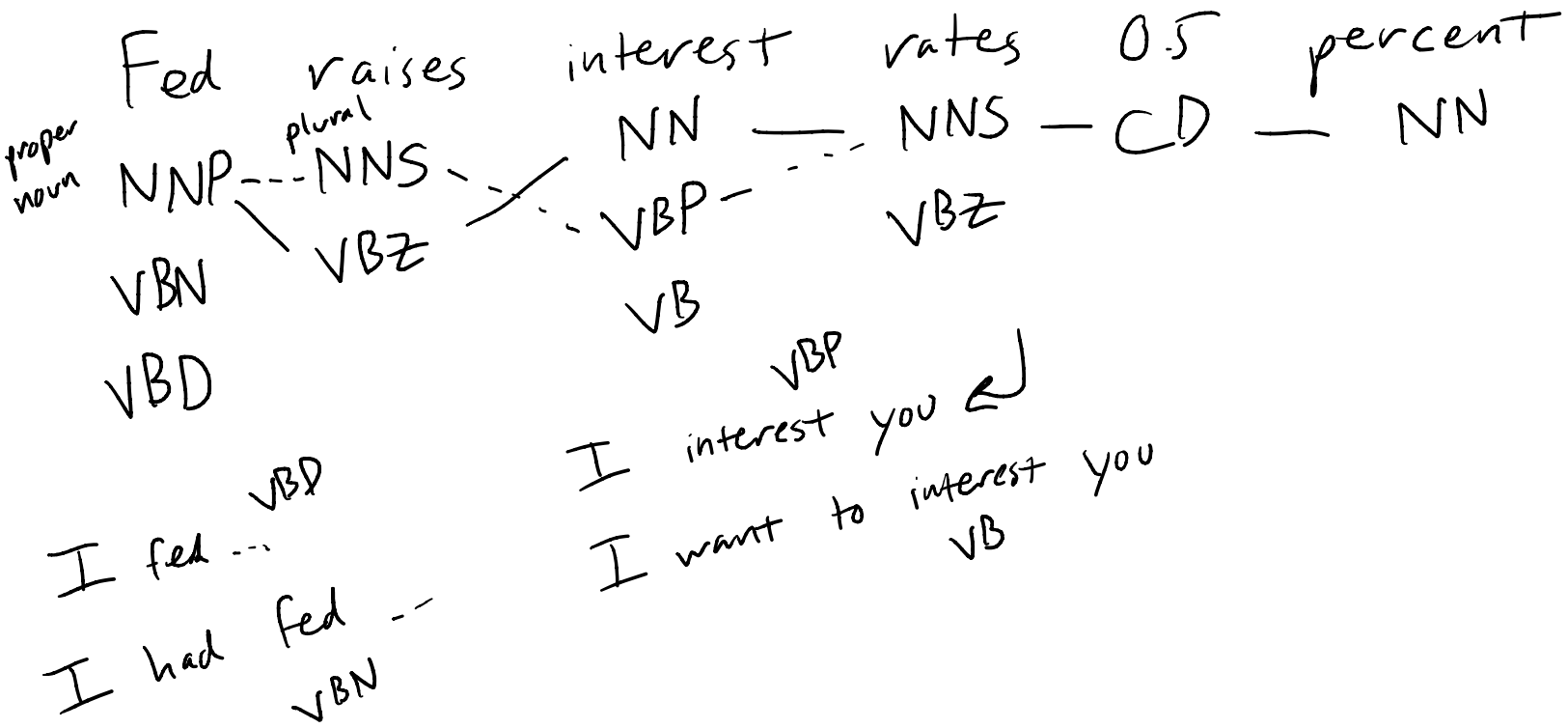
Auxiliaries: had [VERB]

Modals: could

Prepositions: up, in, to (like up-)

Particles: up made up

Example


Fed raises interest rates 0.5 percent
proper noun
plural
NNP
NNS
NN
NNS
CD
NN
VBN
VBZ
VBP
VBZ
VBD
VB
VBP
I interest you
I want to interest you
VB
VBD
I fed ...
I had fed ...
VBN

Sequence Labeling (Tagging), Tagging with Classifiers

Input $\bar{x} = (x_1, x_2, \ldots, x_n)$

Output $\bar{y} = (y_1, y_2, \ldots, y_n)$ one prediction per word

Structured classification Fed raises interest rates 0.5 percent

Predict each y_i independently w/ logistic regression

$P(y_i = y \mid \bar{x}, i)$ ← index we're predicting at

$f(\bar{x}) = [\underset{\text{Fed}}{1}\ 0\ 0\ \underset{\text{raises}}{1}\ 0\ 0\ 0 \ldots]$

BOW: $f(\bar{x}, y = \text{NN}) = [\underline{000 f(\bar{x}) 000} \mid \underset{\text{NN}}{\underline{100100}} \mid \underline{0000} \ldots]$

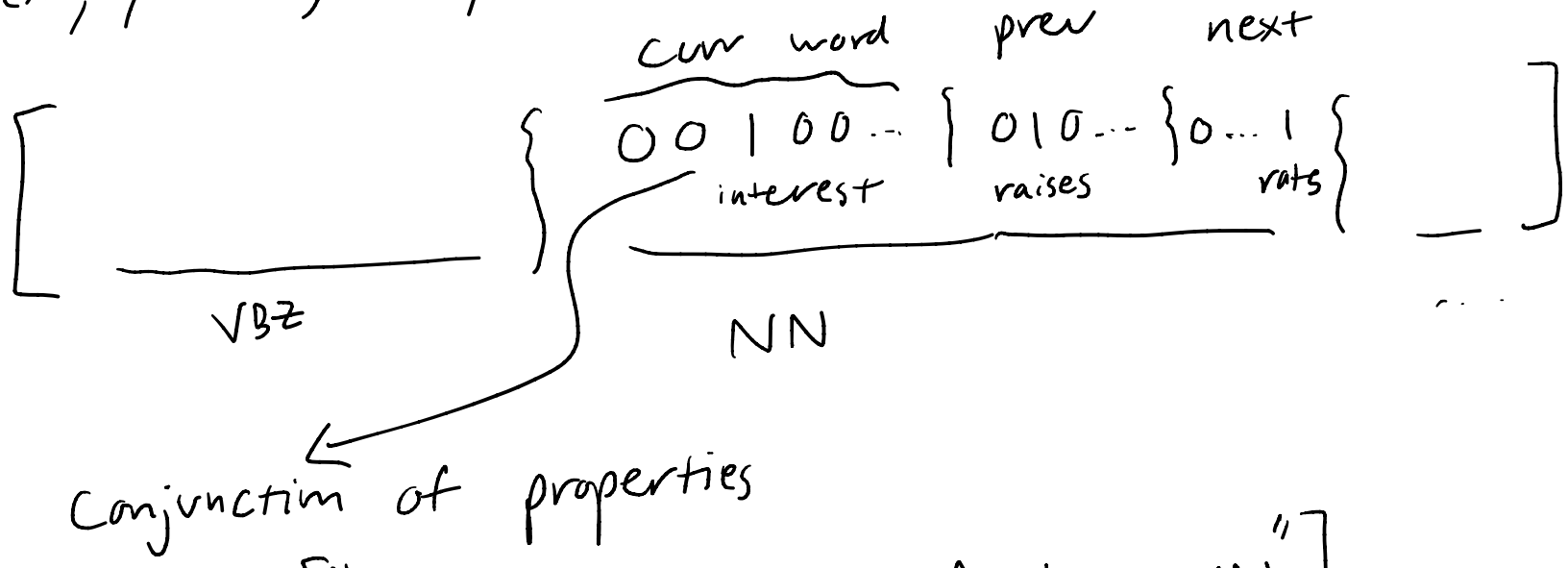
independent of i

Positional: $f(\bar{x}, y = \text{NN}, i = 3)$ $[0 \ldots \mid 001\underset{\text{NN}}{\underset{\text{interest}}{0}}0000 \mid 0 \ldots]$

Simple way: single feature on curr word

Positional features with context

$f(\bar{x}, y = NN, i = 3)$



Conjunction of properties

Indicator ["curr word = interest $\wedge$ tag = NN"]

"word" in a bag-of-words space

Problem w/ classification for tagging

indicators → classifier

What goes wrong?

VBZ ~~—/—~~ VBP

NNS ~~—/—~~ NN

Fed raises interest rates ...

predictions of a classifier may be incoherent

Sequence modeling:

Hidden Markov Models

Conditional Random Fields

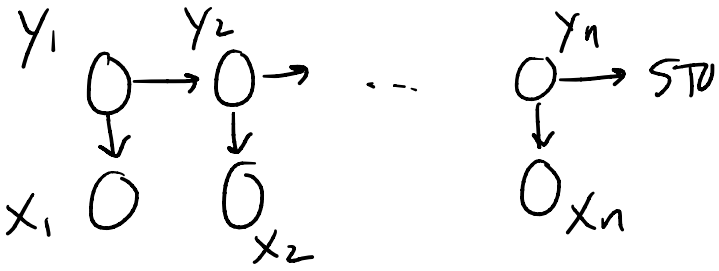
Hidden Markov Models

Generative sequence model

Tags $y_i \in \underset{\text{tags}}{T}$ words $x_i \in \underset{\text{vocabulary}}{V}$

Two steps
1. Param estimation
2. Inference

$$P(\bar{y}, \bar{x}) = P(y_1) P(x_1|y_1) P(y_2|y_1) P(x_2|y_2) \cdots P(\text{STOP}|y_n)$$



y's form a Markov process

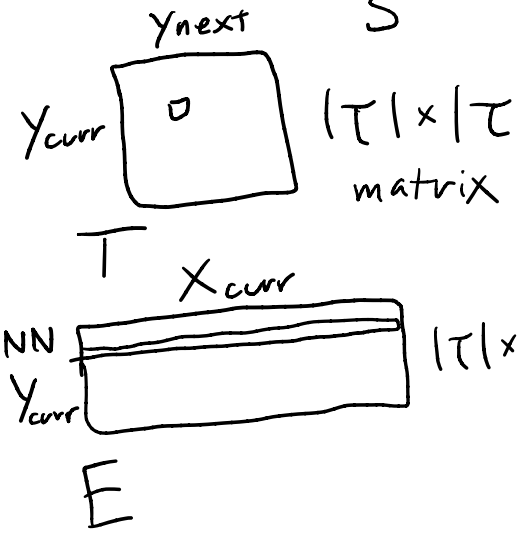
y_i is conditionally independent of $y_1 \cdots y_{i-2}$ given y_{i-1}

Parameters

$P(y_1)$ initial distribution — $|T|$-len vector — S

$P(y_i|y_{i-1})$ transitions — $|T| \times |T|$ matrix — T

$P(x_i|y_i)$ emissions — $|T| \times |V|$ — E



HMMs: Parameter Estimation

Labeled data $(\bar{x}^{(i)}, \bar{y}^{(i)})_{i=1}^{D}$

Maximize $\sum_i \log P(\bar{y}^{(i)}, \bar{x}^{(i)})$ generative (joint) likelihood

$$= \sum_i \log P(y_1^{(i)}) + \sum_i \sum_j \log P(x_j^{(i)} \mid y_j^{(i)}) + \sum_i \sum_j \log P(y_j^{(i)} \mid y_{j-1}^{(i)})$$

i: training data; j: sent index

MLE with frequency counts: biased coin w/ prob p of H

HHHT $\quad 3/4 = \arg\max_p 3 \log p + \log(1-p)$

HMM param estimation: Count + normalizing

Example

$\mathcal{T} = \{N, V, STOP\}$ $\quad \mathcal{V} = \{they, can, fish\}$

Data

N V STOP
they can

N V STOP
they fish

$S =$

N	2
V	0

↓

N	1
V	0

$T =$

	N	V	STOP
N	0	2	0
V	0	0	2

↓

0	1	0
0	0	1

$E =$

	they	can	fish
N	2		
V		1	1

↓c

	t	f
N	1	0	0
V	0	1/2	1/2

NOT P(V | can)

P(can | V)

Smoothing add counts to avoid 0s

$T \rightarrow$

1	3	1
1	1	3

+1 everywhere $=$

1/5	3/5	1/5
1/5	1/5	3/5

V-V

↓

1	3/5	1/5	3/5
1	1/2	1/2	

Ex N V V
they can fish $\Rightarrow P(\bar{y}, \bar{x}) =$

HMM Inference: The Viterbi Algorithm

HMMs: model of $P(\bar{y}, \bar{x}) = P(y_1)\, P(x_1 | y_1)\, P(y_2 | y_1) \cdots$

Inference: $\underset{\bar{y}}{\text{argmax}}\ P(\bar{y} | \bar{x}) = \underset{\bar{y}}{\text{argmax}}\ \frac{P(\bar{y}, \bar{x})}{P(\bar{x})}$ ($P(\bar{x})$: constant w.r.t. $\bar{y}$)

$= \underset{\bar{y}}{\text{argmax}}\ P(\bar{y}, \bar{x}) = \underset{\bar{y}}{\text{argmax}}\ \log P(\bar{y}, \bar{x})$

$= \underset{\tilde{y}_1, \cdots, \tilde{y}_n}{\text{argmax}}\ \log P(\tilde{y}_1) + \log P(x_1 | \tilde{y}_1) + \log P(\tilde{y}_2 | \tilde{y}_1) + \cdots$

Viterbi Dynamic Program

Define $v_i(\tilde{y}) = n \times |T|$ n sent len, $|T|$ number of tags

score of the best path ending in $\tilde{y}$ at time i

Base: $v_1(\tilde{y}) = \log P(x_1 | \tilde{y}) + \log P(\tilde{y})$

Recurrence: $v_i(\tilde{y}) = \log P(x_i | \tilde{y}) + \max_{\tilde{y}_{prev}} \log P(\tilde{y} | \tilde{y}_{prev}) + v_{i-1}(\tilde{y}_{prev})$

Viterbi

for $i = 1 \dots n$

for $\tilde{y}$ in T:

compute $v_i(\tilde{y})$

Compute $v_{n+1}(\text{STOP})$, this $= \max_{\bar{y}} \log P(\bar{x}, \bar{y})$

Track "backpointers"

Example log probs

S = N: −1, V: −1

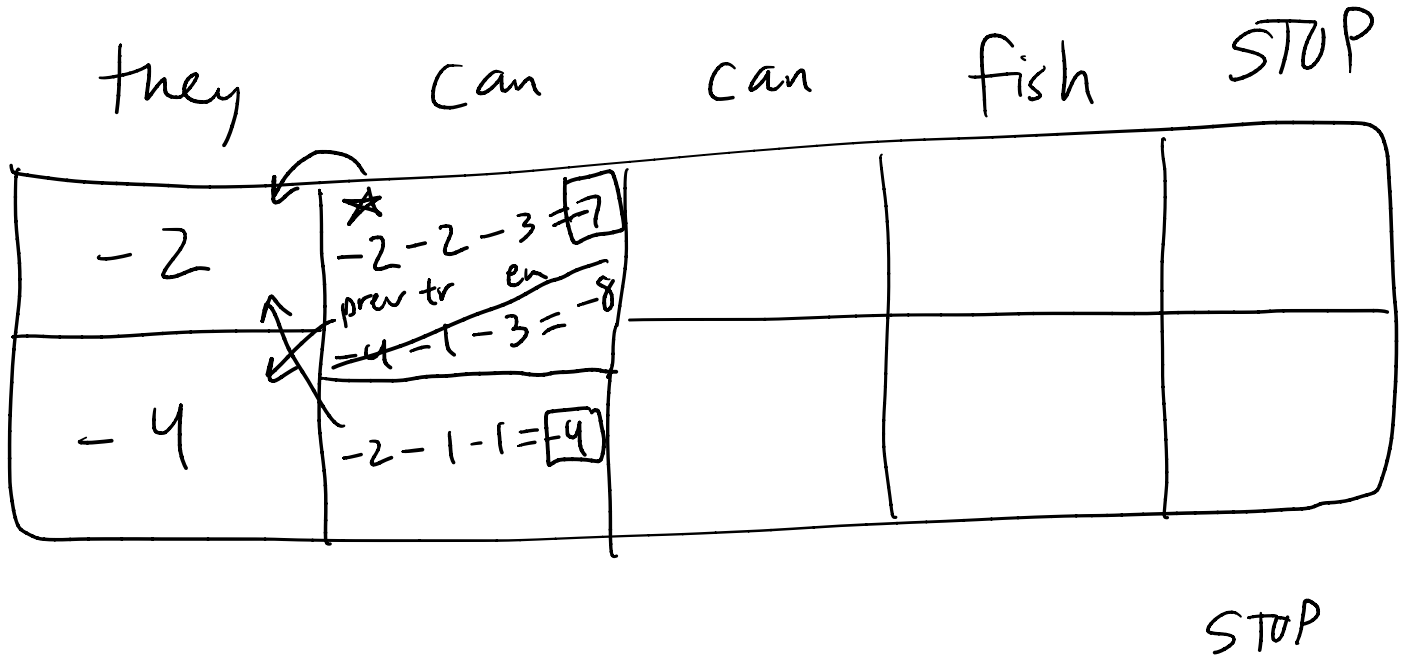
T =

	N	V	STOP
N	−2	−1	−1
V	−1	−1	−2

E =

	they	can	fish
N	−1	−3	−1
V	−3	−1	−1

$V_i(\tilde{y})$



Trigram Taggers

NNP VBZ NN NNS CD NN
Fed raises interest rates 0.5 percent

- Normal HMM "bigram" model: y_1 = NNP, y_2 = VBZ, …
- Trigram model: y_1 = (<S>, NNP), y_2 = (NNP, VBZ), …
- Probabilities now look like P((NNP, VBZ) | (<S>, NNP)) — more context! We know the verb is occurring two words after <S>
- Tradeoff between model capacity and data size — trigrams are a "sweet spot" for POS tagging

HMM POS Tagging

- Penn Treebank English POS tagging: 44 tags
- Baseline: assign each word its most frequent tag: ~90% accuracy
- Trigram HMM: ~95% accuracy / 55% on words not seen in train
- TnT tagger (Brants 1998, tuned HMM): 96.2% acc / 86.0% on unks
- MaxEnt tagger (Toutanova + Manning 2000): 96.9% / 87.0%
- State-of-the-art (BiLSTM-CRFs, BERT): 97.5% / 89%+

POS Errors

	JJ	NN	NNP	NNPS	RB	RP	IN	VB	VBD	VBN	VBP	Total
JJ	0	**177**	**56**	0	**61**	2	5	10	15	**108**	0	488
NN	**244**	0	**103**	0	12	1	1	29	5	6	19	525
NNP	**107**	**106**	0	**132**	5	0	7	5	1	2	0	427
NNPS	1	0	**110**	0	0	0	0	0	0	0	0	142
RB	**72**	21	7	0	0	16	**138**	1	0	0	0	295
RP	0	0	0	0	**39**	0	**65**	0	0	0	0	104
IN	11	0	1	0	**169**	**103**	0	1	0	0	0	323
VB	17	**64**	9	0	2	0	1	0	4	7	**85**	189
VBD	10	5	3	0	0	0	0	3	0	**143**	2	166
VBN	**101**	3	3	0	0	0	0	3	**108**	0	1	221
VBP	5	34	3	1	1	0	2	**49**	6	3	0	104
Total	626	536	348	144	317	122	279	102	140	269	108	3651

JJ/NN NN
official knowledge

VBD RP/IN DT NN
made up the story

RB VBD/VBN NNS
recently sold shares

(NN NN: tax cut, art gallery, …)

Toutanova + Manning (2000)

Remaining Errors

- Lexicon gap (word not seen with that tag in training): 4.5% of errors
- Unknown word: 4.5%
- Could get right: 16% (many of these involve parsing!)
- Difficult linguistics: 20%

VBD / VBP? (past or present?)

They set up absurd situations, detached from reality

- Underspecified / unclear, gold standard inconsistent / wrong: **58%**

adjective or verbal participle? JJ / VBN?

a $ 10 million fourth-quarter charge against discontinued operations

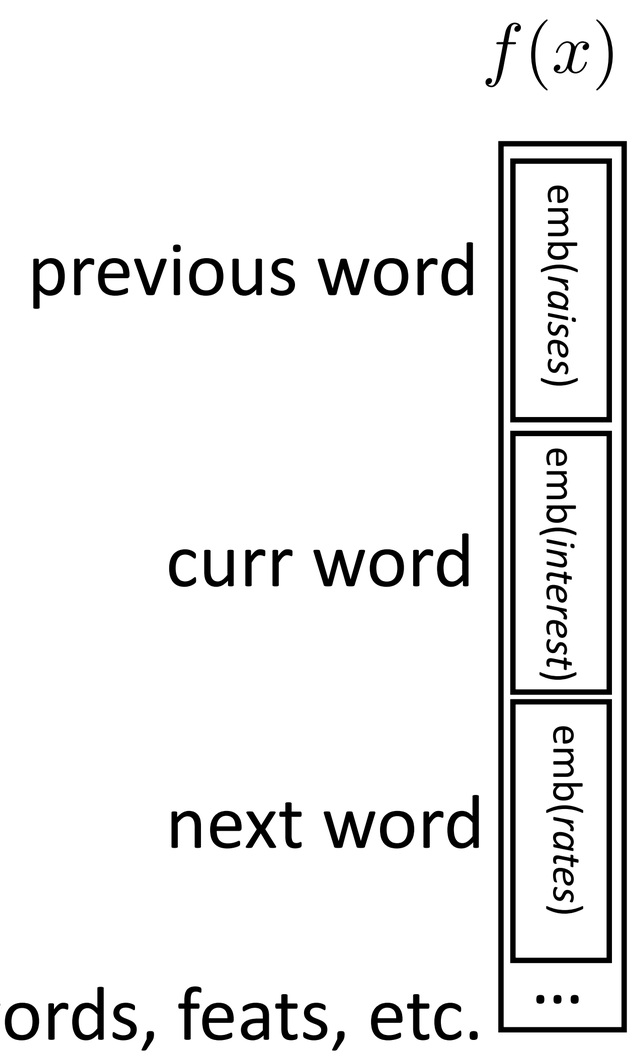
Manning 2011 “Part-of-Speech Tagging from 97% to 100%: Is It Time for Some Linguistics?”

POS with Feedforward Networks

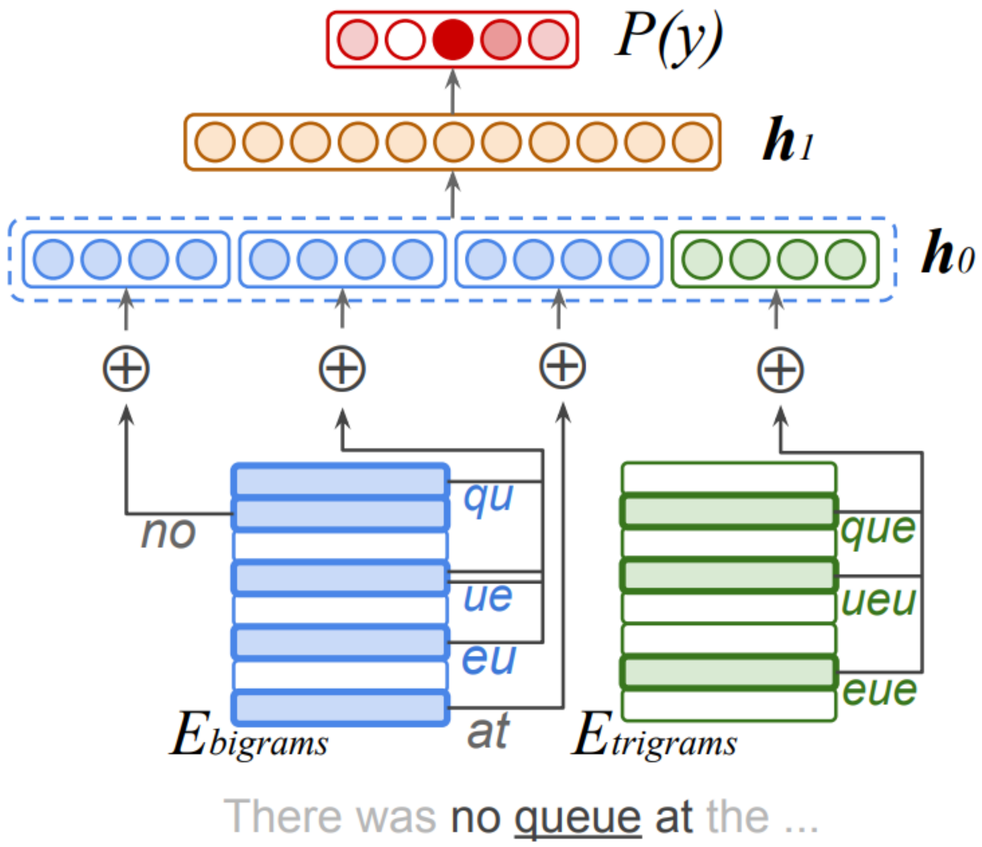
??

*Fed raises **interest** rates in order to …*

- Word embeddings for each word form input
- *f*(*x*) doesn't look like a bag-of-words, instead captures position-sensitive information



POS with Feedforward Networks



- Botha et al. (2017): small FFNNs for NLP tasks
- Use bag-of-character bigram + trigram embeddings for each word
- Hidden layer mixes these different signals and learns feature conjunctions

Botha et al. (2017)

POS with Feedforward Networks

- Works well on a range of languages
- Better than a RNN-based approach (Gillick et al., 2016)

Lang.	*L.R.*	*Mom.*	γ	*Steps*	*Acc.*
Small FF ($\frac{1}{2}$ Dim.) + Clusters					
bg	0.1	0.8	128k	210k	97.76
cs	0.05	0.9	32k	420k	98.06
da	0.05	0.9	16k	240k	95.33
en	0.05	0.8	8k	300k	93.06
fi	0.05	0.9	16k	390k	94.66
fr	0.08	0.9	128k	120k	95.28
de	0.08	0.9	16k	90k	92.13
el	0.08	0.9	16k	60k	97.42
id	0.08	0.9	8k	690k	92.15
it	0.05	0.9	64k	210k	97.42
fa	0.1	0.8	8k	510k	96.19
es	0.08	0.9	8k	60k	94.79
sv	0.1	0.8	16k	300k	95.76

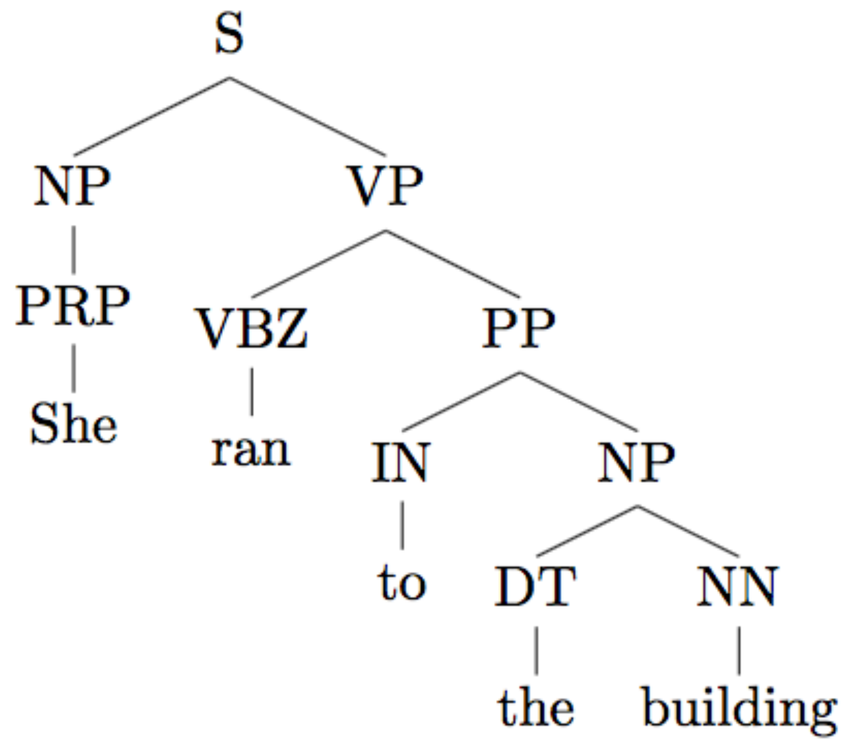
Botha et al. (2017)

Syntax

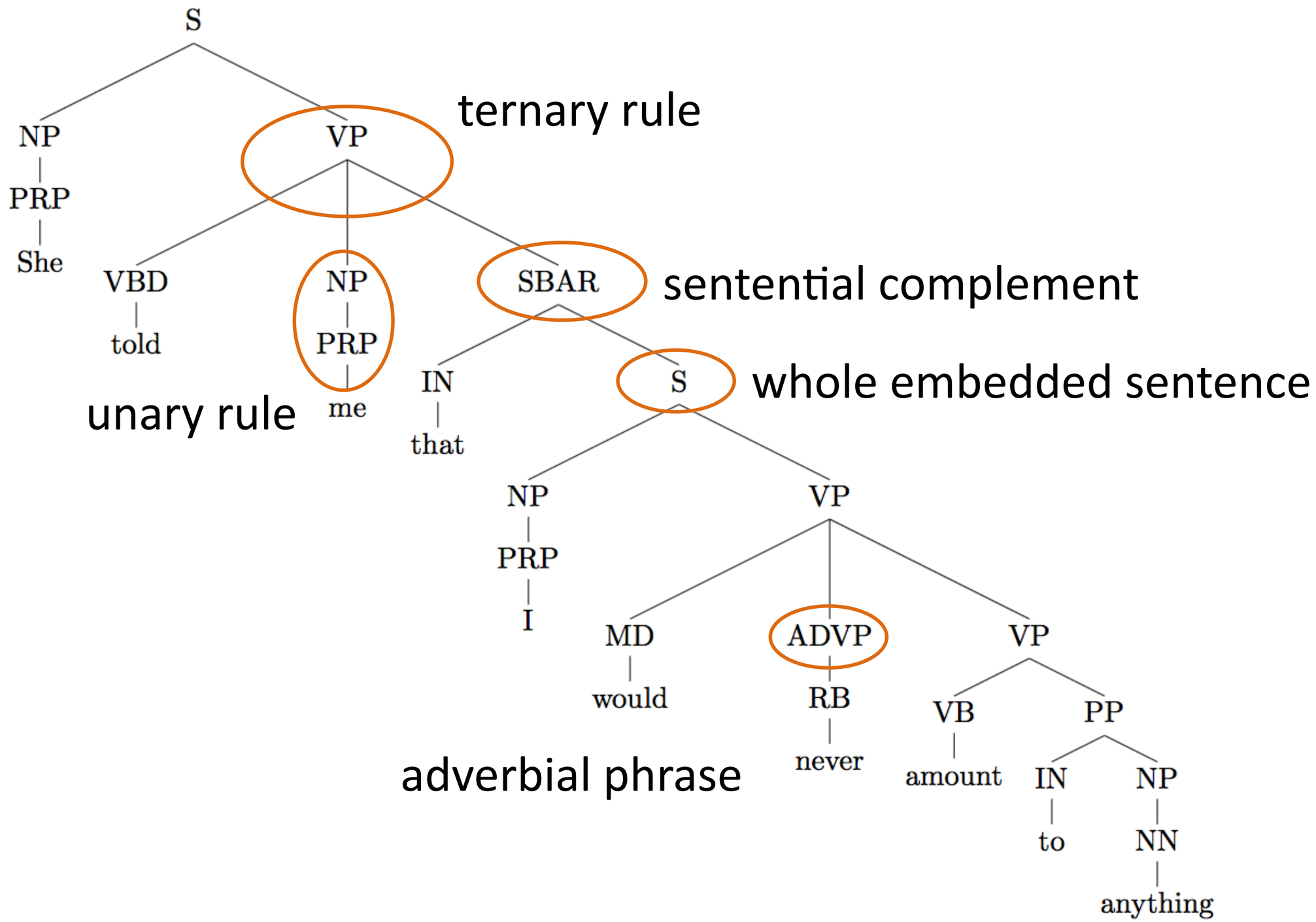
- Study of word order and how words form sentences
- Why do we care about syntax?
  - Multiple interpretations of words (noun or verb? *Fed raises*… example)
  - Recognize verb-argument structures (who is doing what to whom?)
  - Higher level of abstraction beyond words: some languages are SVO, some are VSO, some are SOV, parsing can canonicalize

Constituency Parsing

- Tree-structured syntactic analyses of sentences
- *Constituents*: (S)entence, (N)oun (P)hrases, (V)erb (P)hrases, (P)repositional (P)hrases, and more
- Bottom layer is POS tags
- Examples will be in English. Constituency makes sense for a lot of languages but not all



Constituency Parsing

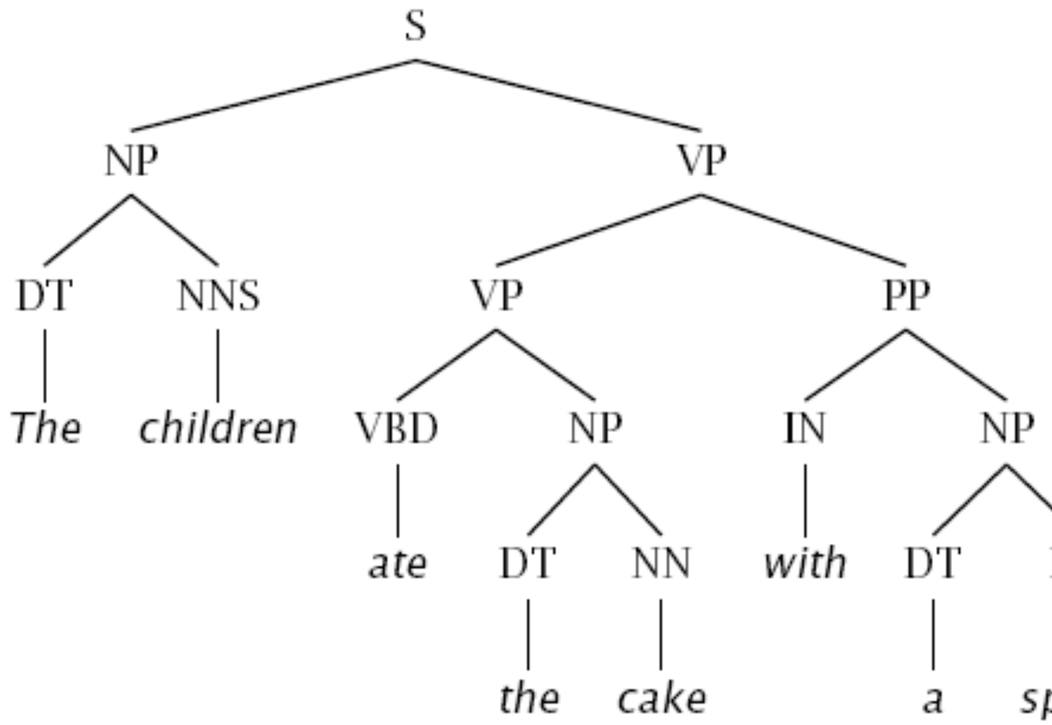


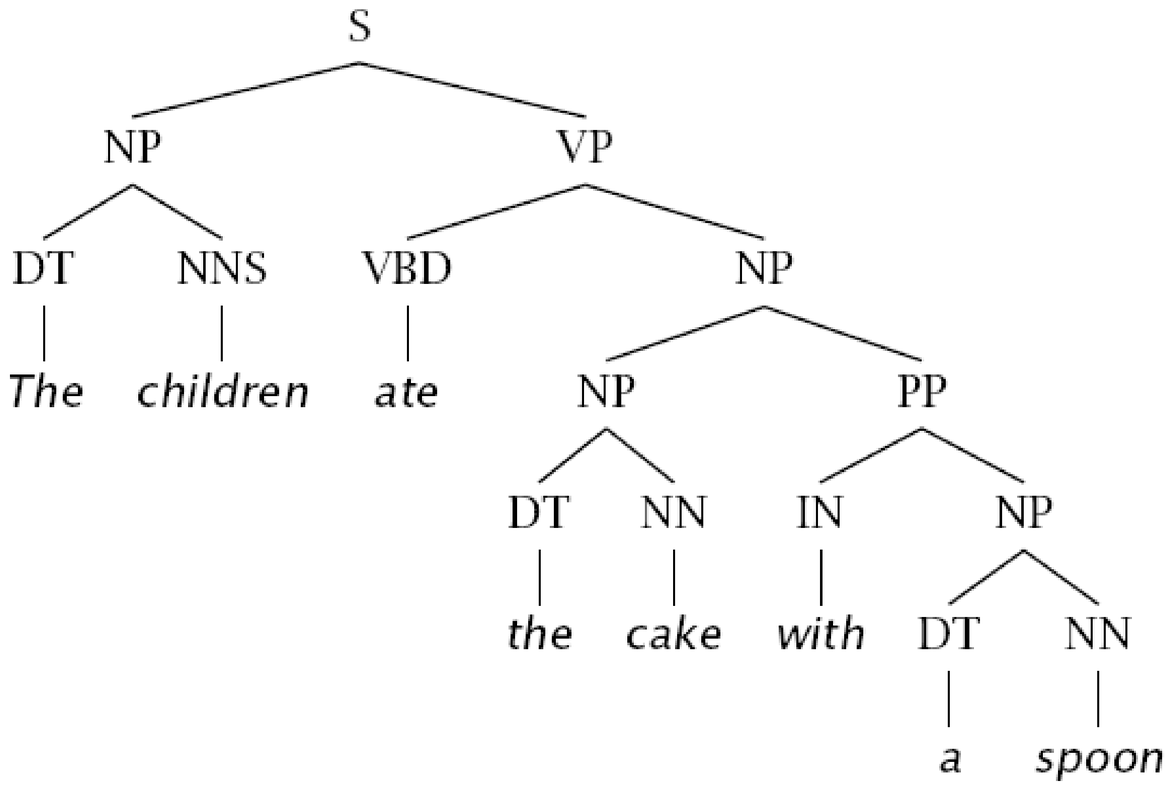
Constituency Parsing

a refund that the court estimated *-1

Challenges

PP attachment

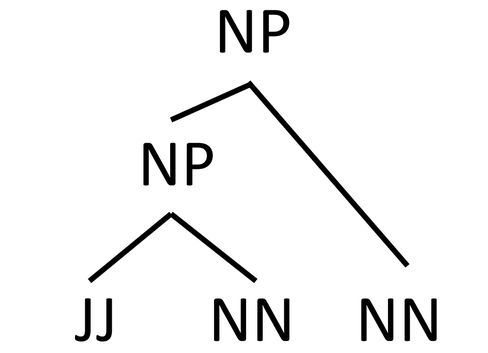




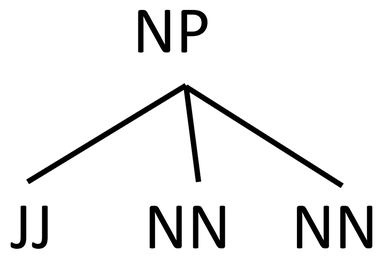
same parse as “the cake with some icing”

Challenges

- Modifier scope:



plastic cup holder



plastic cup holder

- Complement structure:

  The students complained to the professor that they didn't understand

- Coordination scope:

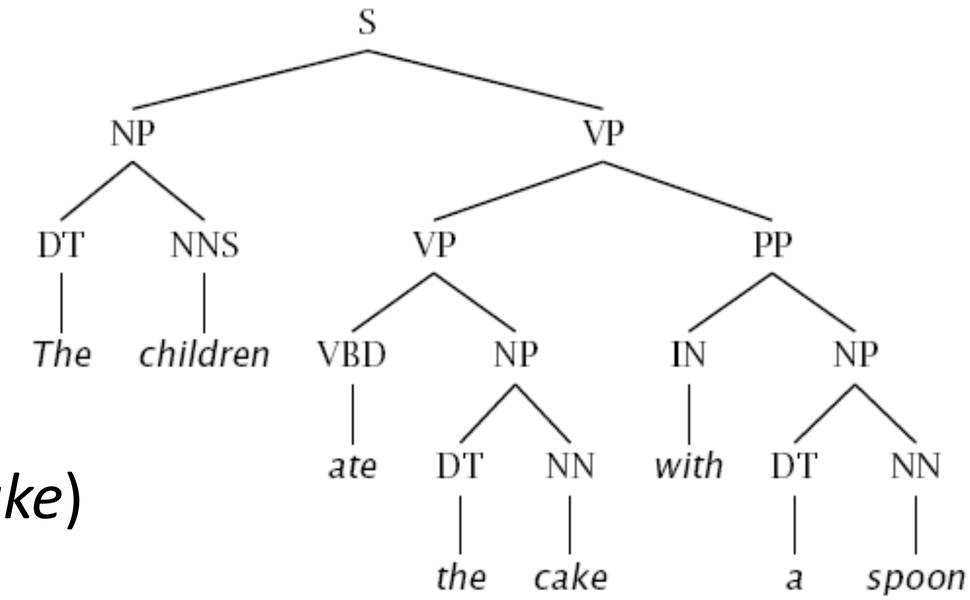
  The man picked up his hammer and saw

  compare: *The man picked up his hammer and swung*

Eisenstein textbook

Constituency

- How do we know what the constituents are?
- Constituency tests:
  - Substitution by proform (pronoun, *do so*)
  - Clefting (*It was with a spoon that*…)
  - Answer ellipsis (What did they eat? *the cake*)
    (How? *with a spoon*)



- Sometimes constituency is not clear, e.g., coordination: *she went to and bought food at the store*

Probabilistic Context-Free Grammars

CFGs $\{N, T, S, R\}$

nonterminals, terminals, start, rules

S, NP, VP, PP

the, children, ate, cake, with, spoon

DT, NN, VBD, IN, NNS

POS: preterminals

S

binary

1 $S \rightarrow NP\ VP$

1/4 $VP \rightarrow VBD\ NP$

1/2 $NP \rightarrow DT\ NN$

1/2 $NP \rightarrow DT\ NNS$

unary

$VP \rightarrow VBD$ 3/4

$DT \rightarrow$ the

$NNS \rightarrow$ children

$NN \rightarrow$ cake

$NN \rightarrow$ spoon

$VBD \rightarrow$ ate

PCFG: rules have probs, probs sum to one per parent

$P(\text{rule} \mid \text{parent(rule)})$

$P(\text{tree } T) = \prod_{\text{rules}} P(r \mid \text{parent}(r))$

Steps to Parsing

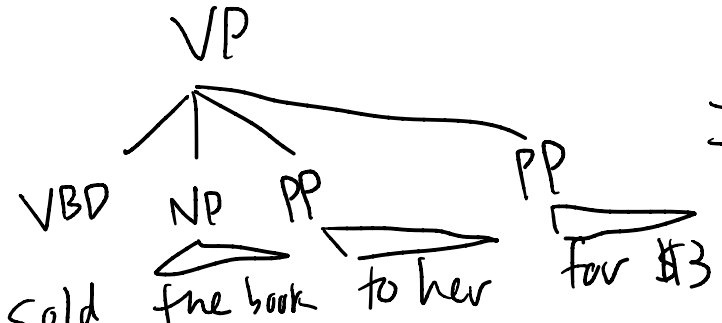
Treebank of sents labeled with trees

(1) Binarization

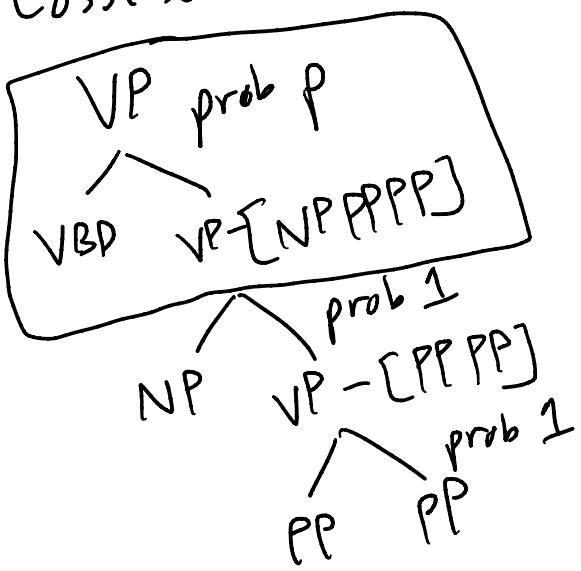
(2) Estimation of rule probs
(count + normalizing)
(maximum likelihood est.)

(3) Inference
$\text{argmax}_T P(T|\bar{x})$

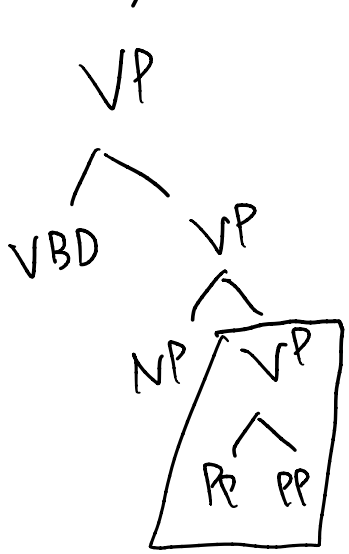
Binarization



Lossless



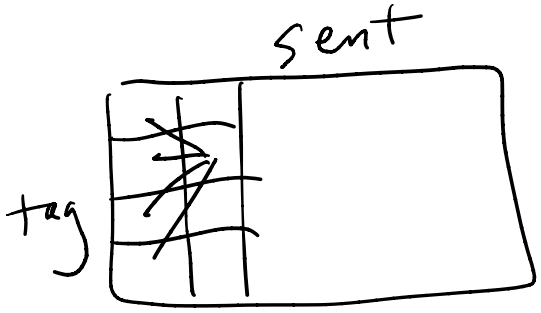
Lossy

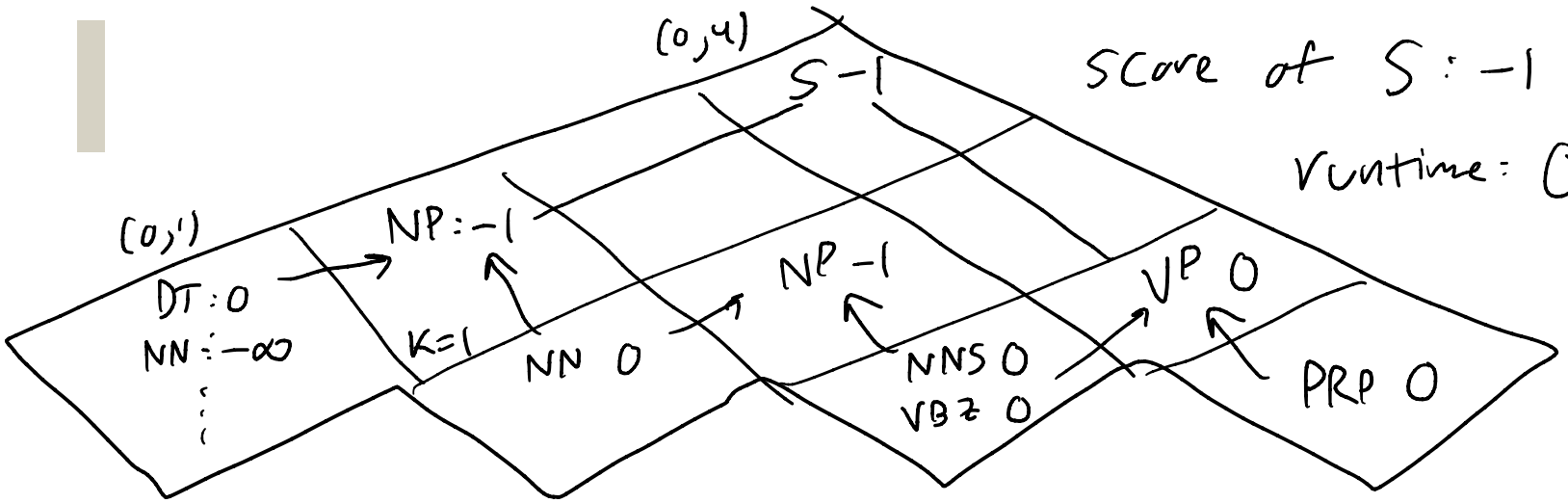


CKY Algorithm

$$\underset{T}{\operatorname{argmax}}\ P(T \mid \bar{x}) = \underset{T}{\operatorname{argmax}}\ P(T, \bar{x})$$

Viterbi





Score of S: -1

Runtime: $O(n^3 G)$

0 the 1 child 2 raises 3 it 4

Chart $t[i, j, X]$ score of the best derivation of X over span (i, j)

$\log(0.5) = -1$

$t[i, i+1, X] = \log P(w_i | X)$

$t[i, j, X] = \max_{i < k < j} \max_{r: X \to X_1 X_2} \log P(X \to X_1 X_2) + t[i, k, X_1] + t[k, j, X_2]$

1.0 DT → the

1.0 NN → child

1.0 NNS → raises

1.0 VBZ → raises

1.0 PRP → it

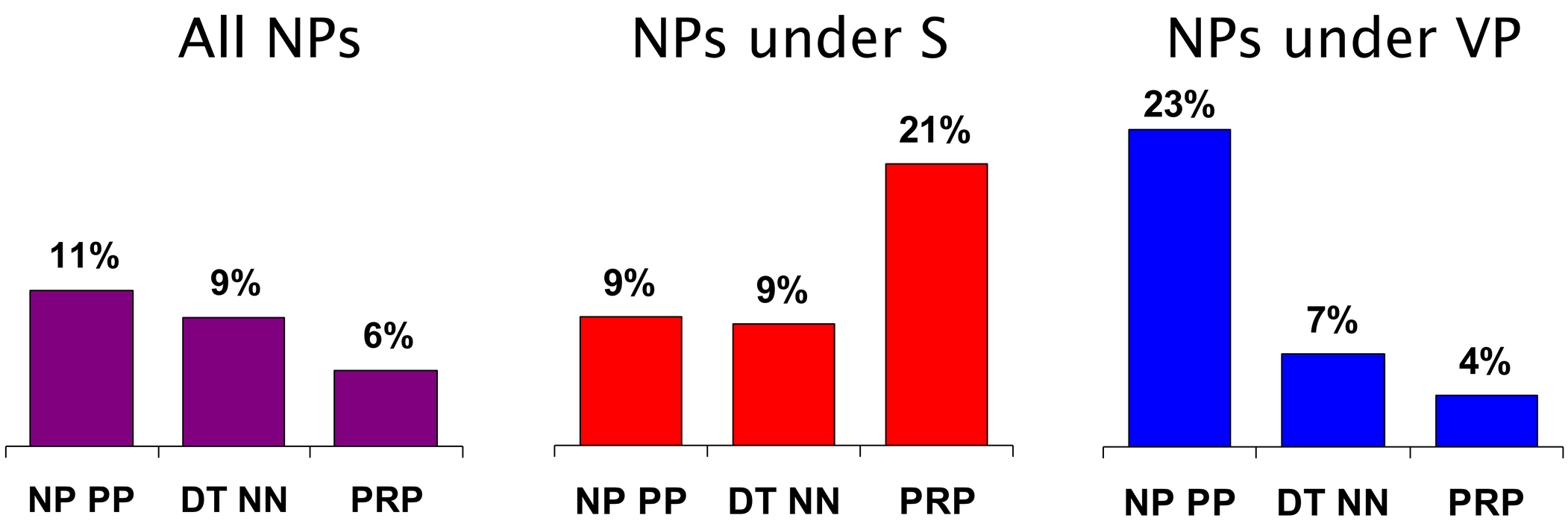
1.0 S → NP VP

0.5 NP → DT NN

0.5 NP → NN NNS

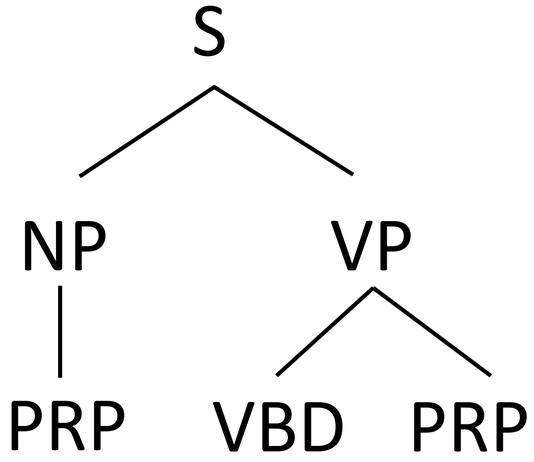
1.0 VP → VBZ PRP

PCFG Independence Assumptions

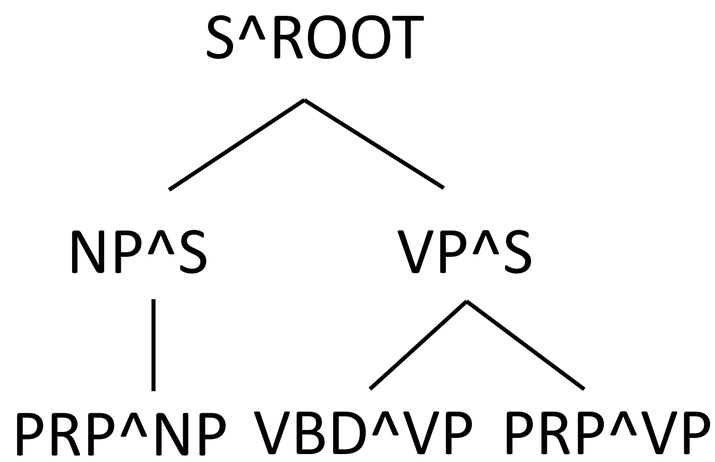


- Language is not context-free: NPs in different contexts rewrite differently
- [They]$_{NP}$ received [the package of books]$_{NP}$

Vertical Markovization



Basic tree (v = 1)

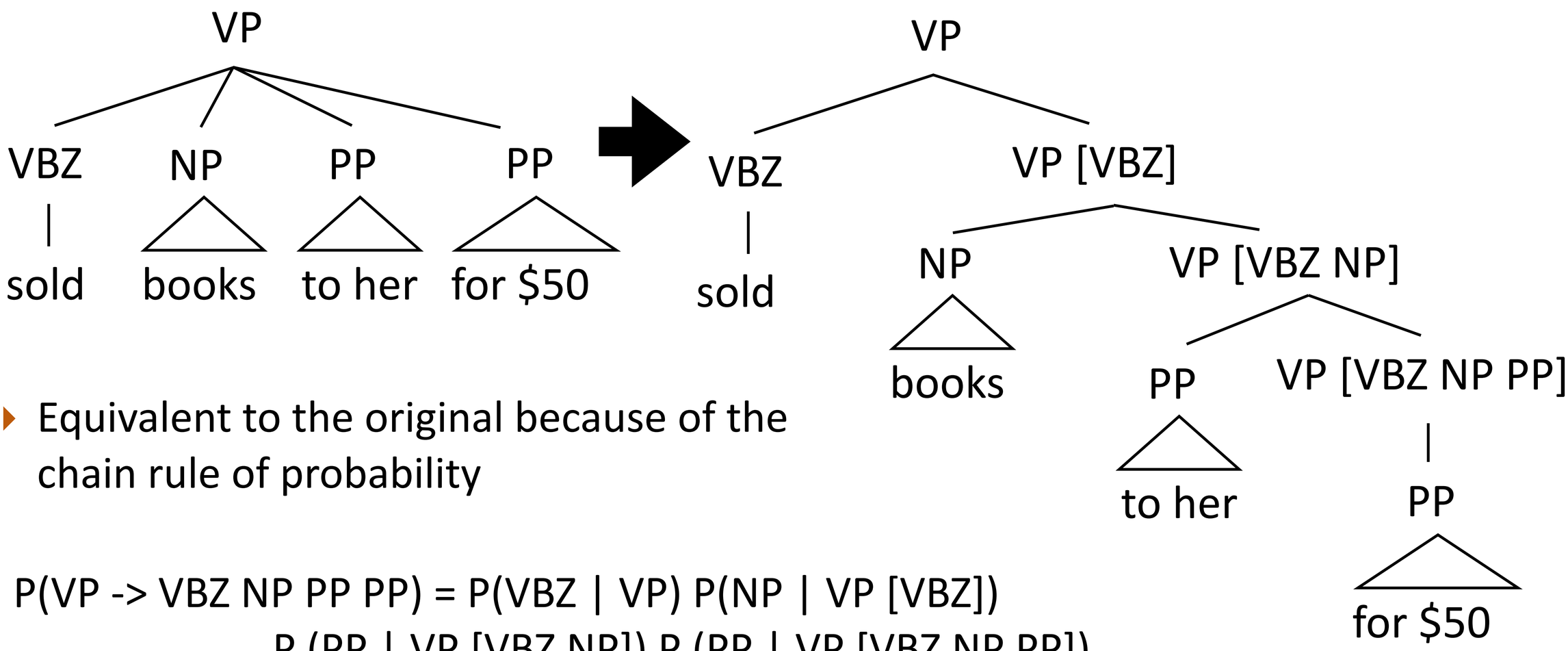


v = 2 Markovization

- Why is this a good idea?

Binarization Revisited

- Another way of doing lossless binarization:

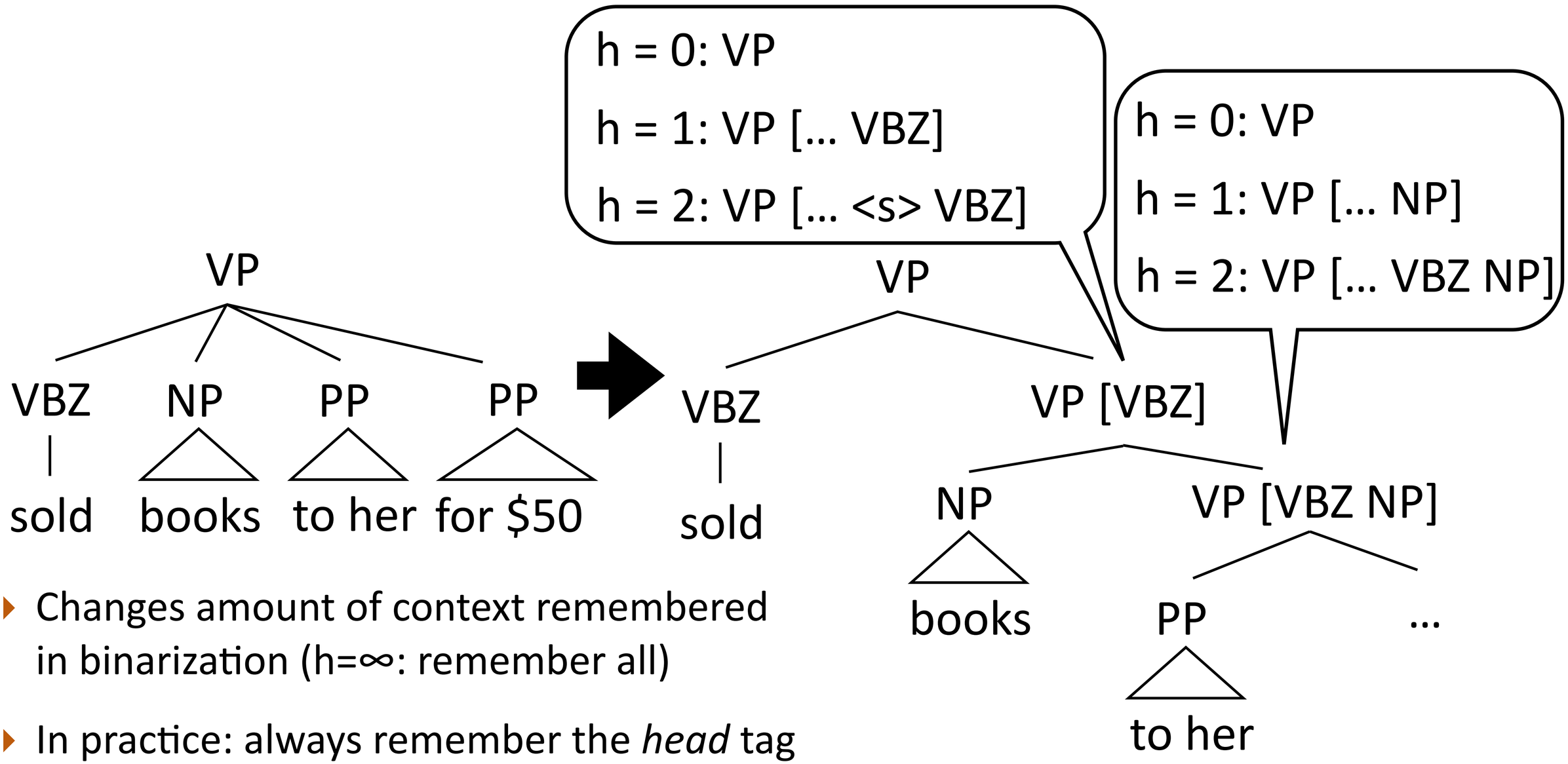


- Equivalent to the original because of the chain rule of probability

P(VP -> VBZ NP PP PP) = P(VBZ | VP) P(NP | VP [VBZ])
P (PP | VP [VBZ NP]) P (PP | VP [VBZ NP PP])

(abusing notation slightly)

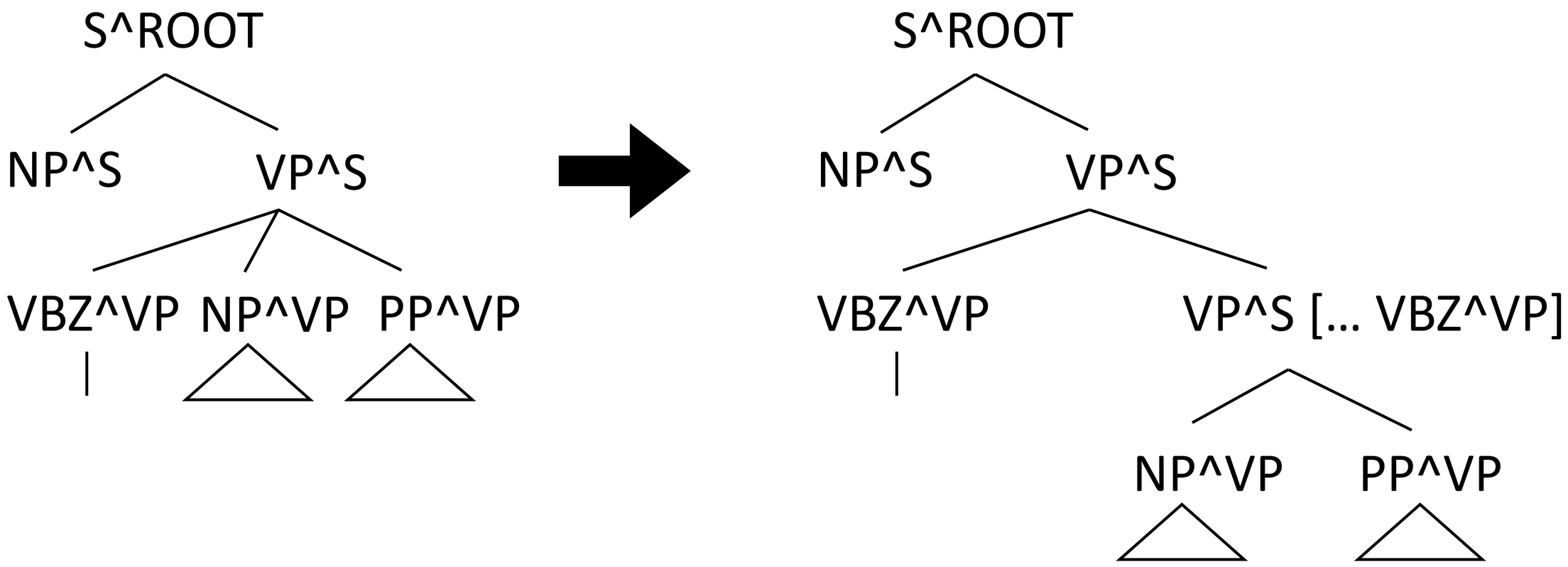
Horizontal Markovization



- Changes amount of context remembered in binarization (h=∞: remember all)
- In practice: always remember the *head* tag

Annotating Trees

- First apply vertical Markovization, then binarize + apply horizontal



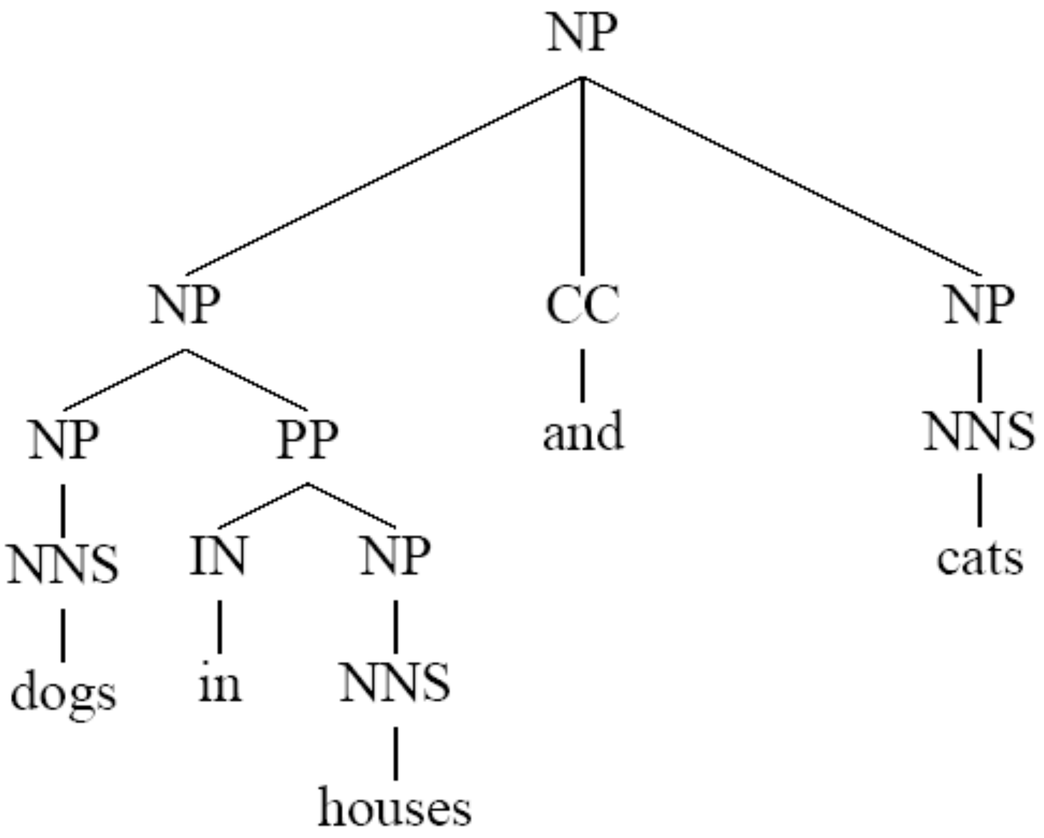
Annotating Trees

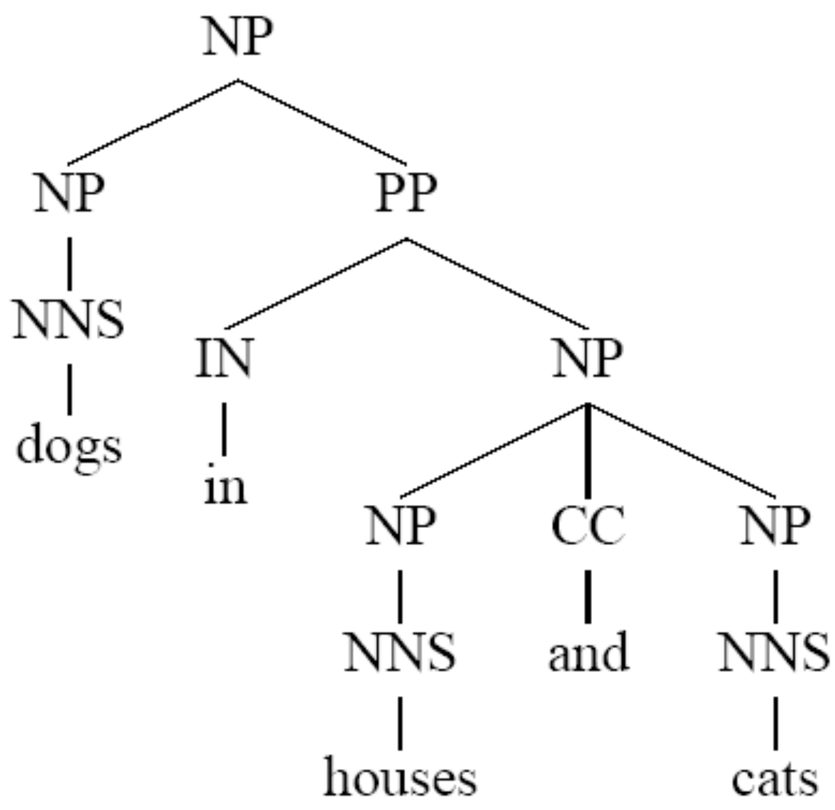
Vertical Order		Horizontal Markov Order $h=0$	$h=1$	$h\leq 2$	$h=2$	$h=\infty$
$v=1$	No annotation	71.27 (854)	72.5 (3119)	73.46 (3863)	72.96 (6207)	72.62 (9657)
$v\leq 2$	Sel. Parents	74.75 (2285)	77.42 (6564)	77.77 (7619)	77.50 (11398)	76.91 (14247)
$v=2$	All Parents	74.68 (2984)	77.42 (7312)	77.81 (8367)	77.50 (12132)	76.81 (14666)
$v\leq 3$	Sel. GParents	76.50 (4943)	78.59 (12374)	79.07 (13627)	78.97 (19545)	78.54 (20123)
$v=3$	All GParents	76.74 (7797)	79.18 (15740)	79.74 (16994)	79.07 (22886)	78.72 (22002)

Figure 2: Markovizations: F_1 and grammar size.

Klein and Manning (2003)

Lexicalization





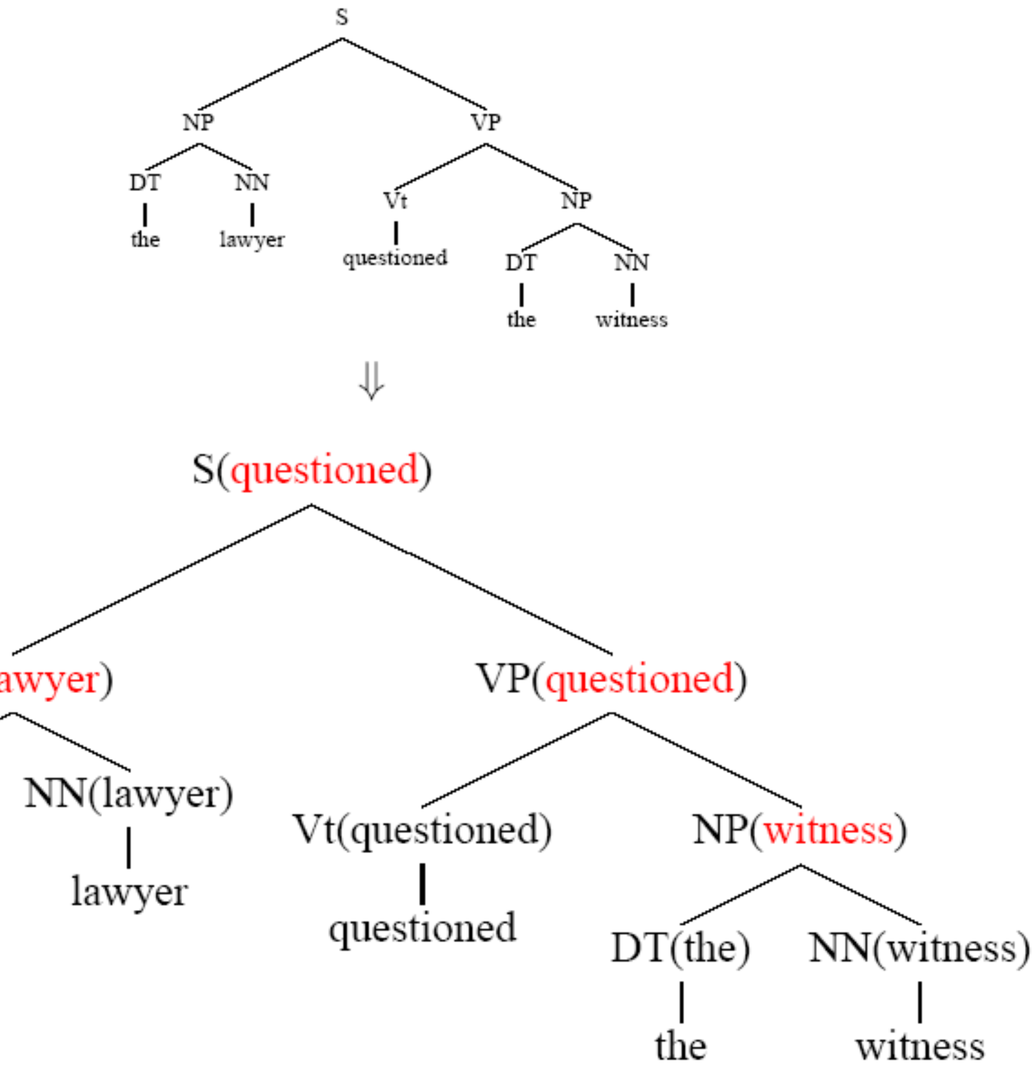
- Even with parent annotation, these trees have the same rules. Need to use the words

Lexicalization

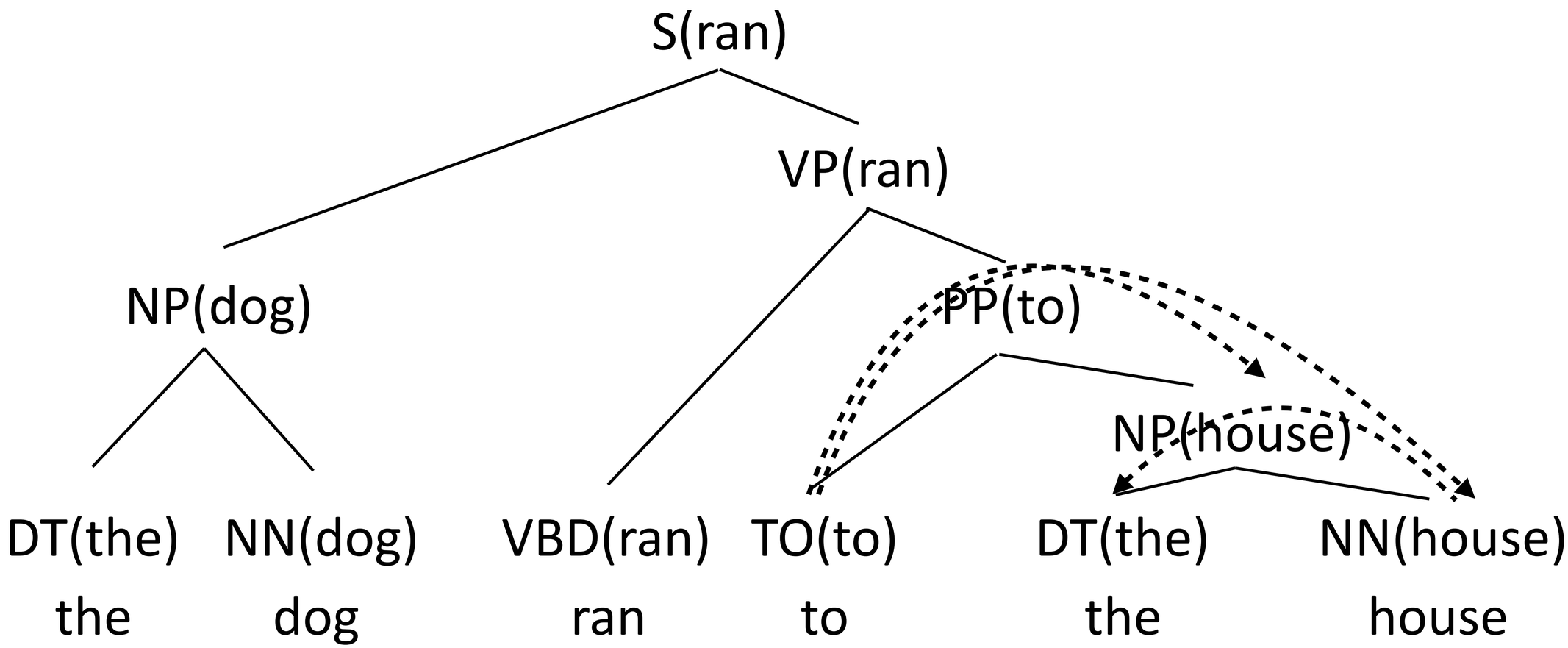
- Annotate each grammar symbol with its “head word”: most important word of that constituent

- Rules for identifying headwords (e.g., the last word of an NP before a preposition is typically the head)

- Collins and Charniak (late 90s): ~89 F1 with these

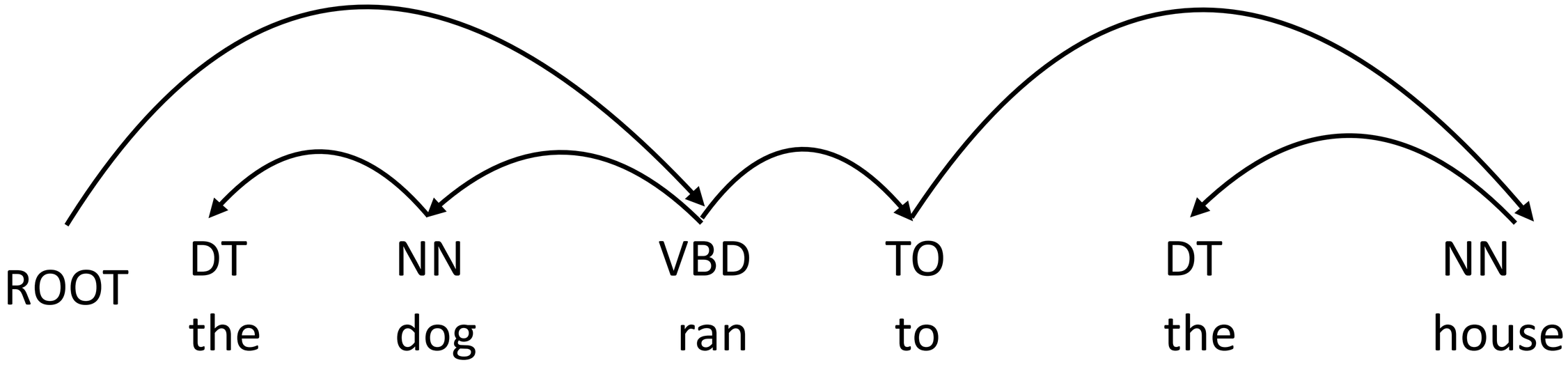


Lexicalized Parsing



Dependency Parsing

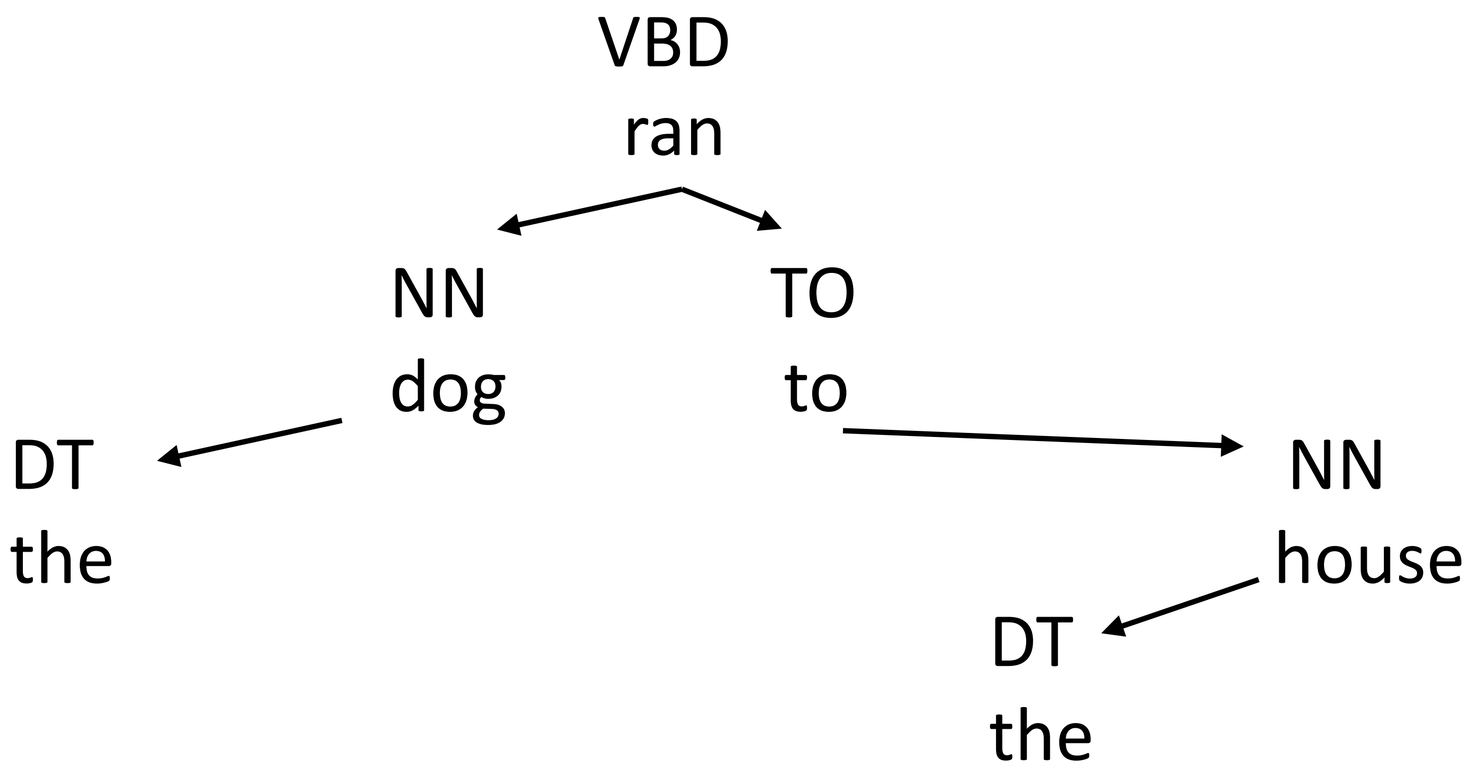
- Dependency syntax: syntactic structure is defined by these arcs
  - Head (parent, governor) connected to dependent (child, modifier)
  - Each word has exactly one parent except for the ROOT symbol, dependencies must form a directed acyclic graph



  - POS tags same as before, often run a tagger first as preprocessing

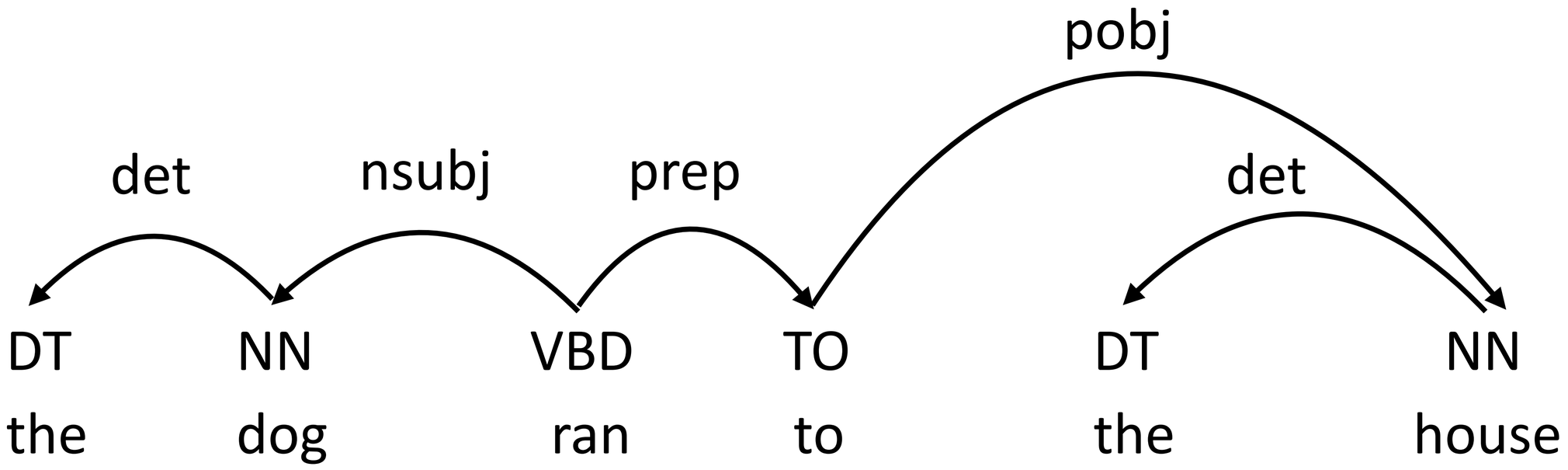
Dependency Parsing

- Still a notion of hierarchy! Subtrees often align with constituents



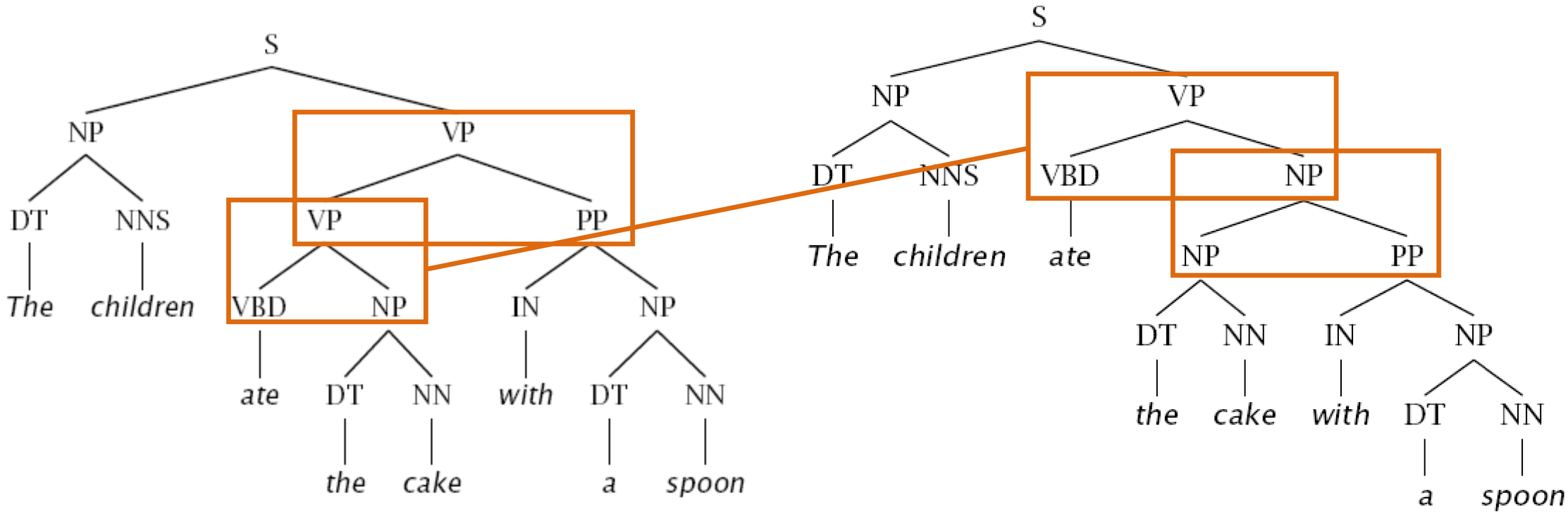
Labeled Dependency Parsing

- Can label dependencies according to syntactic function
- Major source of ambiguity is in the structure, so we focus on that more (labeling separately with a classifier works pretty well)



Dependency vs. Constituency

- Constituency: several rule productions need to change

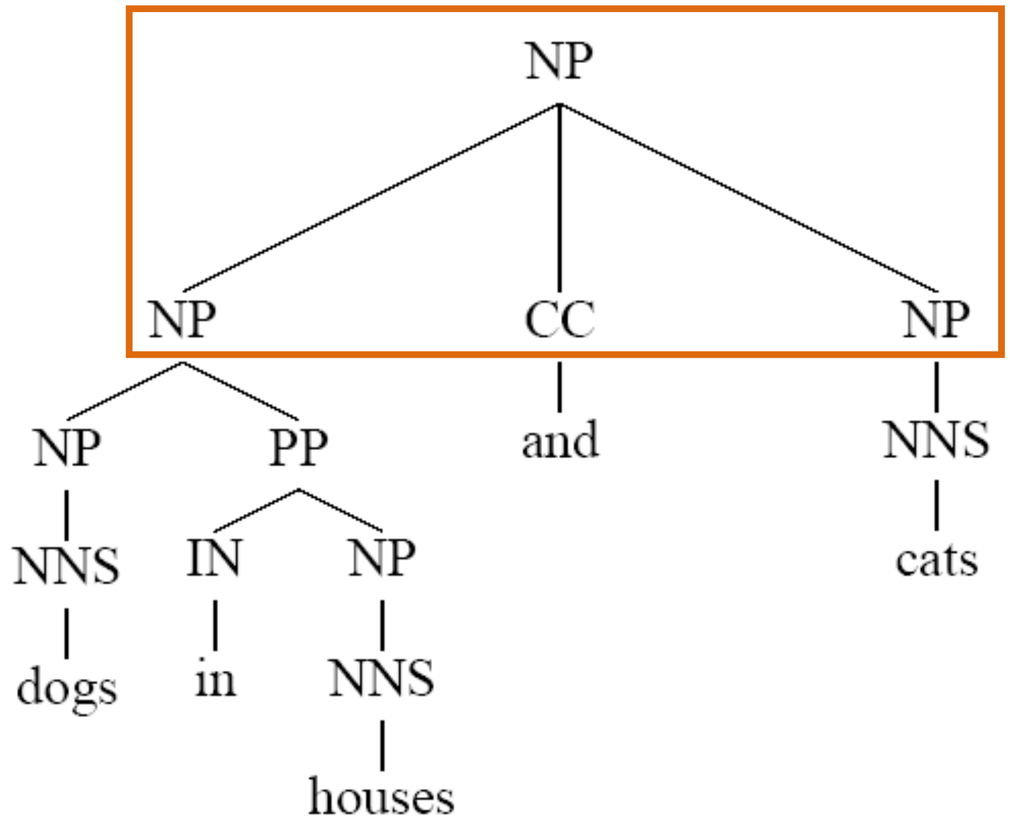


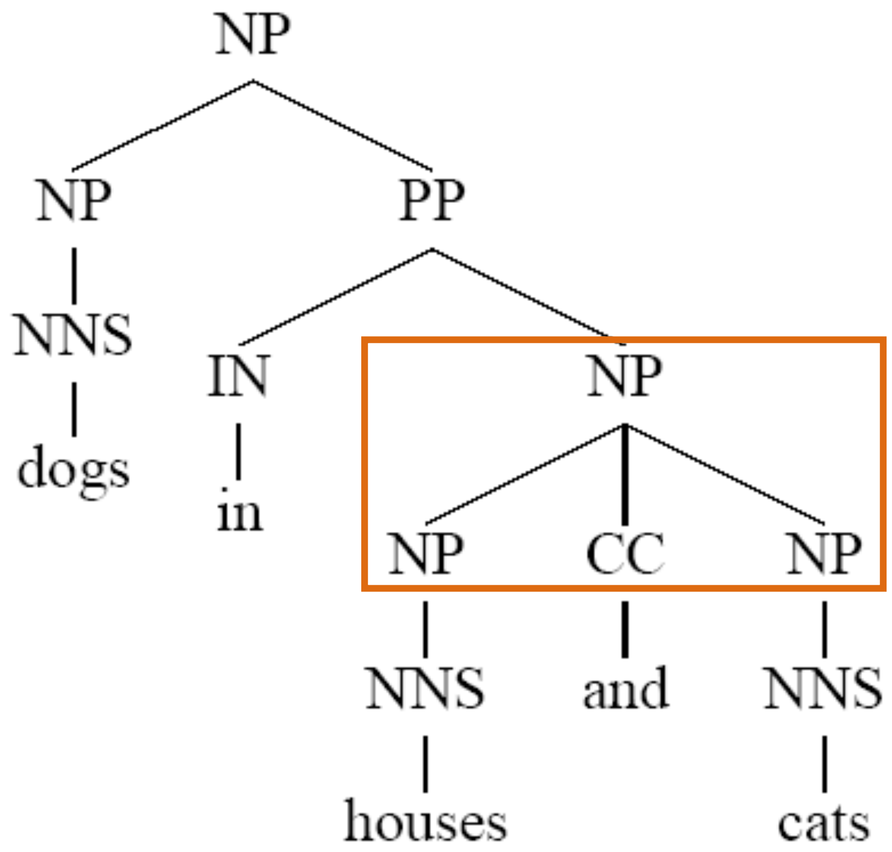
- Dependency: one parent differs

the children ate the cake with a spoon

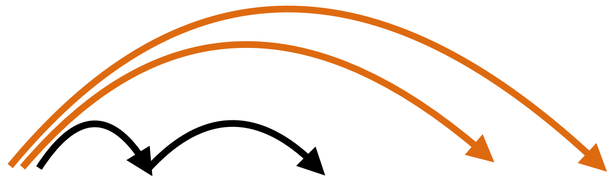
Dependency vs. Constituency

- Constituency: ternary rule NP -> NP CC NP





- Dependency: first item is the head. Doesn't really make sense



dogs in houses **and cats**

[dogs in houses] and cats

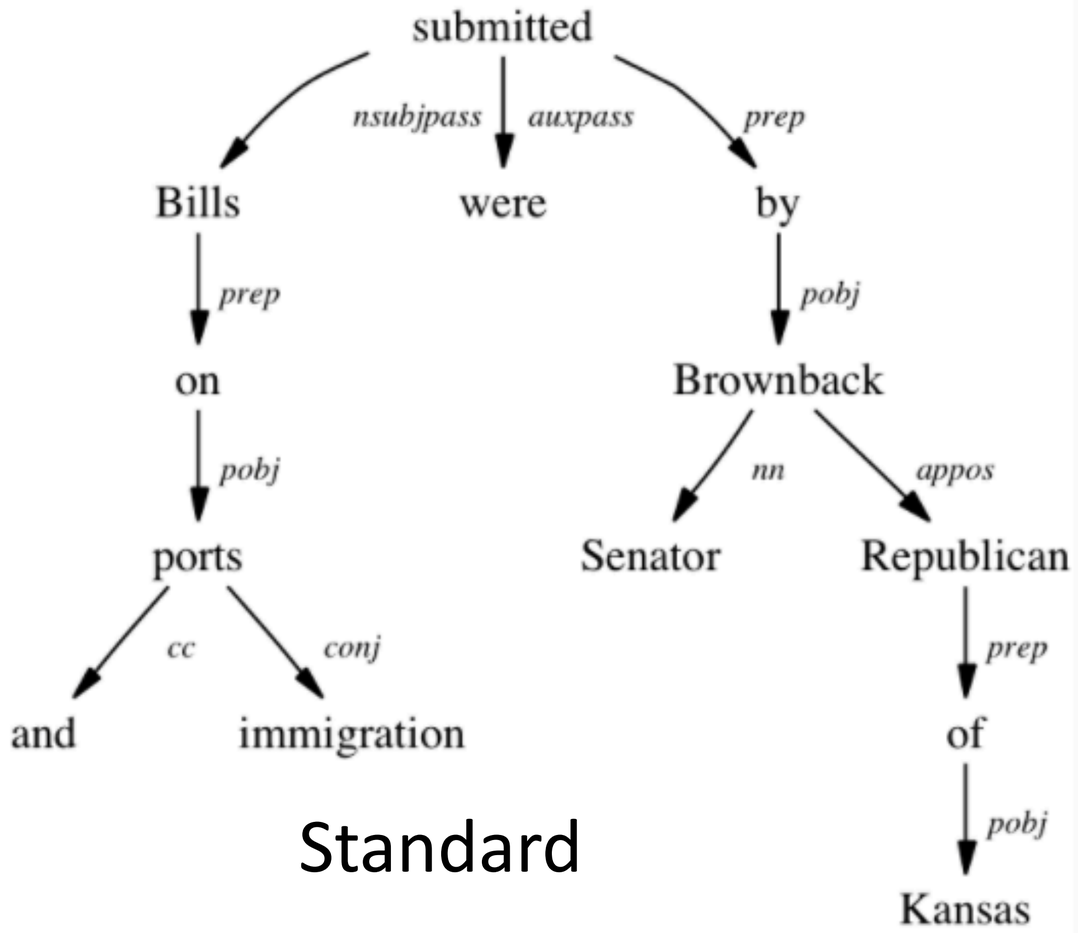
dogs in **houses and cats**

dogs in [houses and cats]

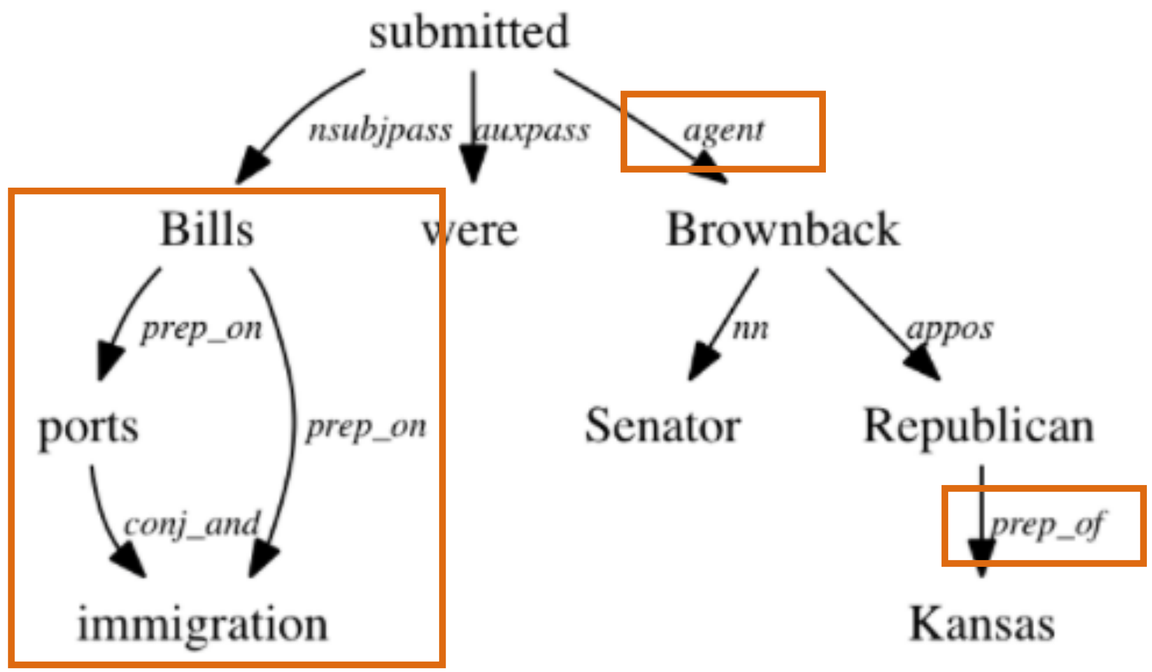
Stanford Dependencies

- Designed to be practically useful for relation extraction

Bills on ports and immigration were submitted by Senator Brownback, Republican of Kansas



Standard

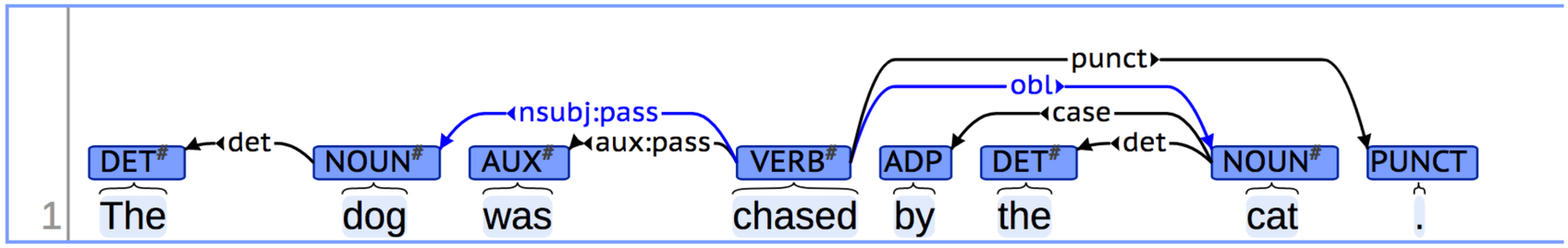


Collapsed

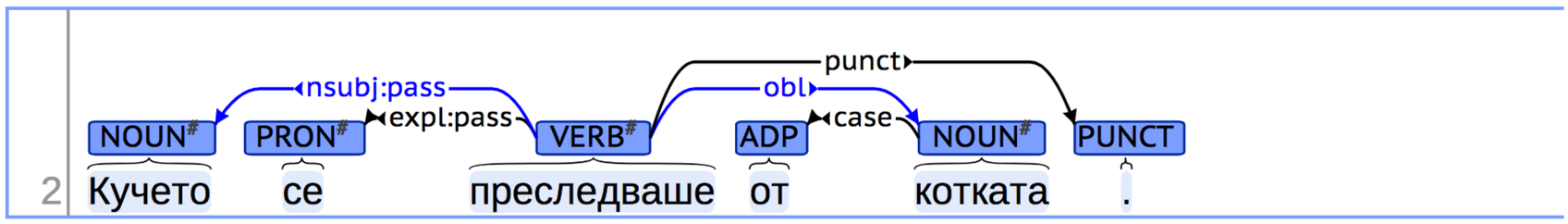
Universal Dependencies

- Annotate dependencies with the same representation in many languages

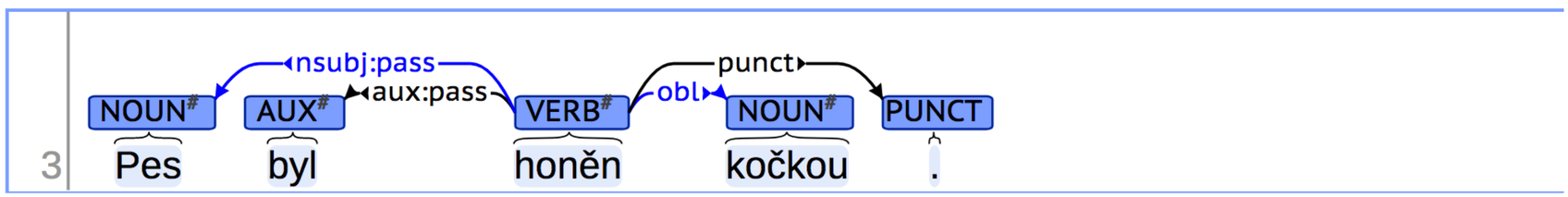
English



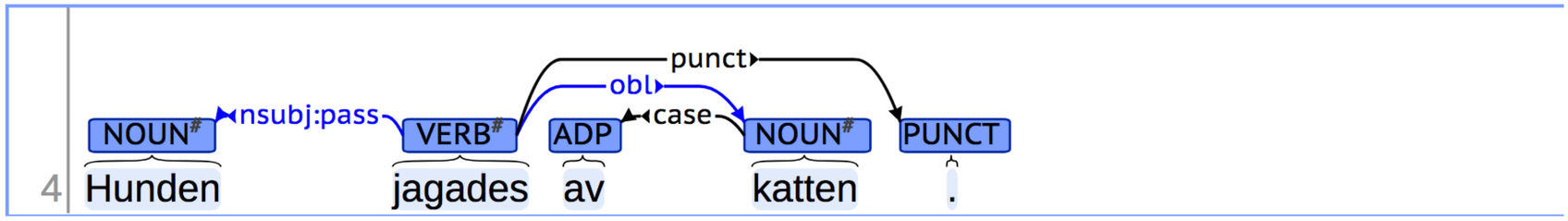
Bulgarian



Czech



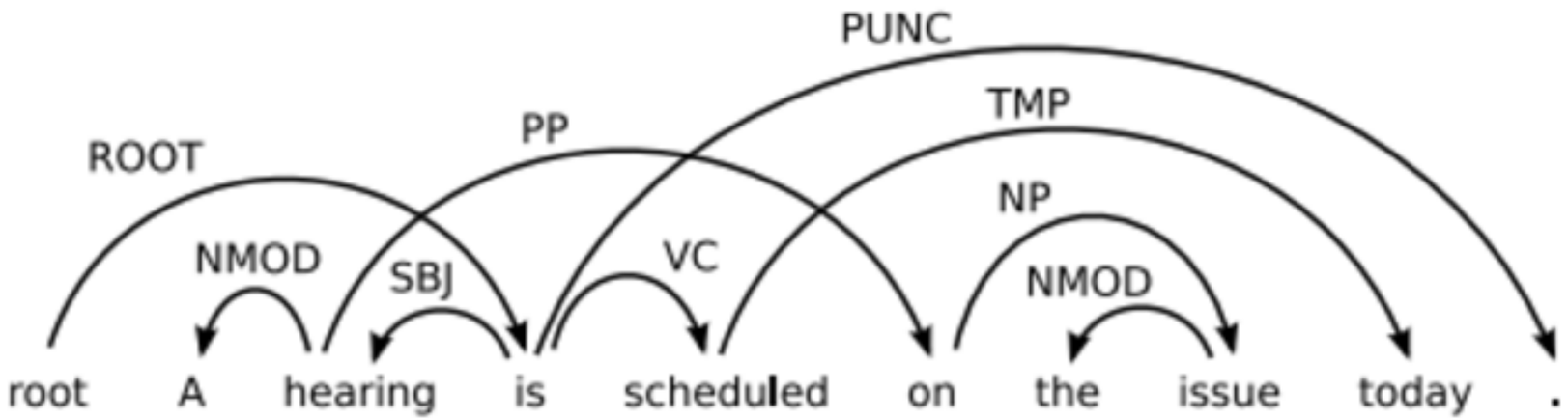
Swiss



- Dependencies are more portable cross-lingually: languages with free word order are not well handled by constituency

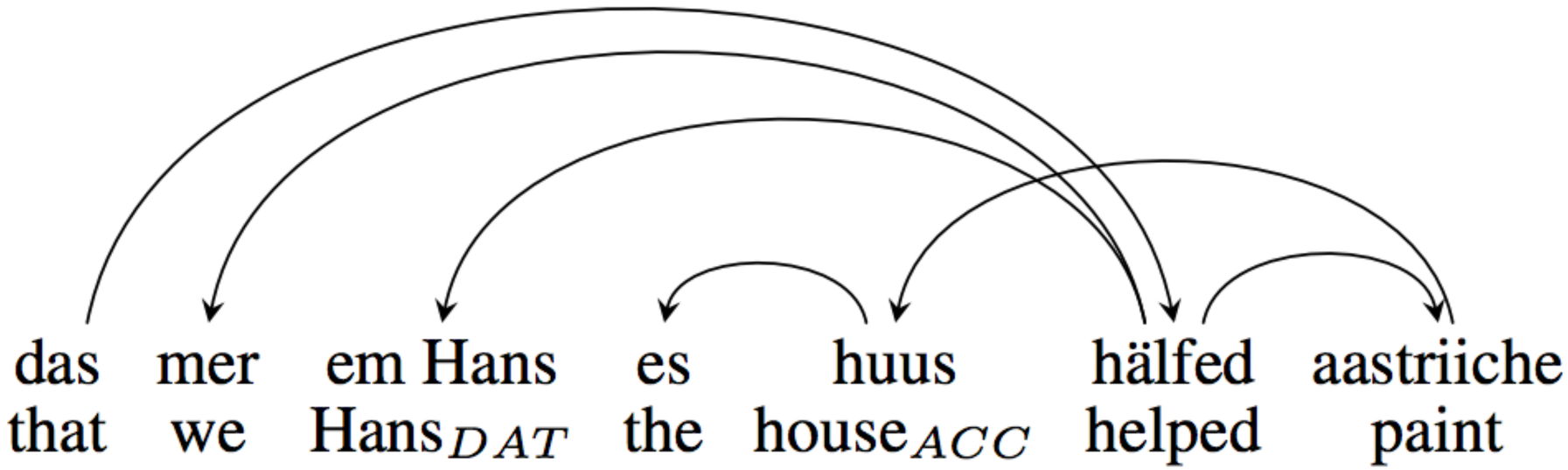
Projectivity

- Any subtree is a contiguous span of the sentence <-> tree is *projective*
- Equivalent to drawing the structure and having no crossing arcs
- Crossing arcs / nonprojective:



credit: Language Log

Projectivity



- Swiss German example: X said [that we helped Hans paint the house]
- Many trees in other languages are nonprojective

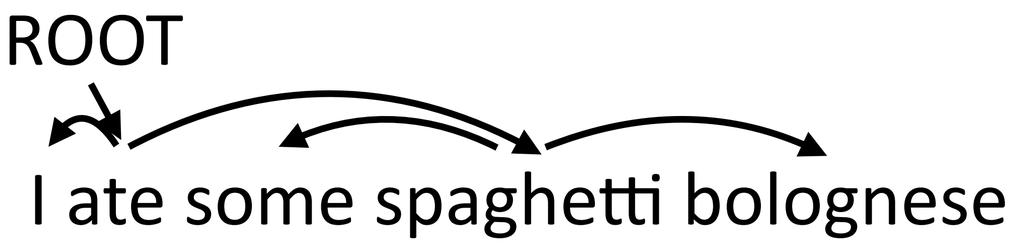
	Arabic	Czech	Danish
Projective	1297 (88.8)	55872 (76.8)	4379 (84.4)
Sentences	1460	72703	5190

Pitler et al. (2013)

Transition-based Parsing

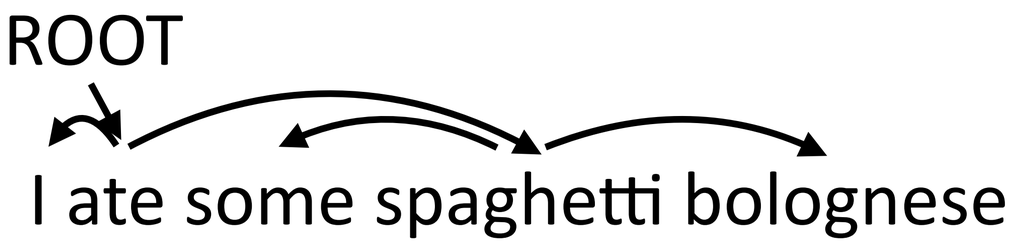
- We can build a dependency parser using a chart-based algorithm like CKY
  - Time: $O(n^3)$, but the algorithm is very tricky!
- *Transition-based*, or *shift-reduce*, is another style of parser; similar to deterministic parsing for compilers
- A tree is built from a sequence of incremental decisions moving left to right through the sentence
- Stack contains partially-built tree, buffer contains rest of sentence

Transition System



- Initial state: Stack: [ROOT] Buffer: [I ate some spaghetti bolognese]
- Shift: top of buffer -> top of stack
  - Shift 1: Stack: [ROOT I] Buffer: [ate some spaghetti bolognese]
  - Shift 2: Stack: [ROOT I ate] Buffer: [some spaghetti bolognese]

Transition System



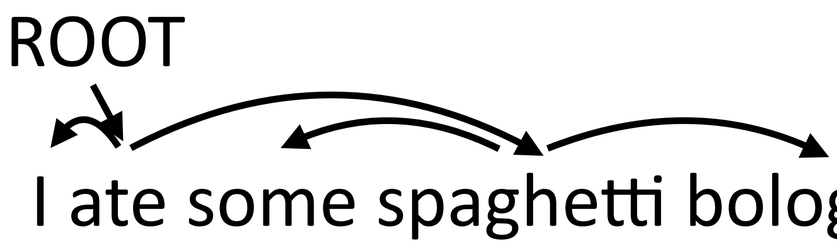
- State: Stack: [ROOT I ate] Buffer: [some spaghetti bolognese]

- Left-arc (reduce): Let σ denote the stack, $\sigma|w_{-1}$ = stack ending in w_{-1}
  - "Pop two elements, add an arc, put them back on the stack"

$$\boxed{\sigma|w_{-2}, w_{-1}} \rightarrow \boxed{\sigma|w_{-1}}$$ w_{-2} is now a child of w_{-1}

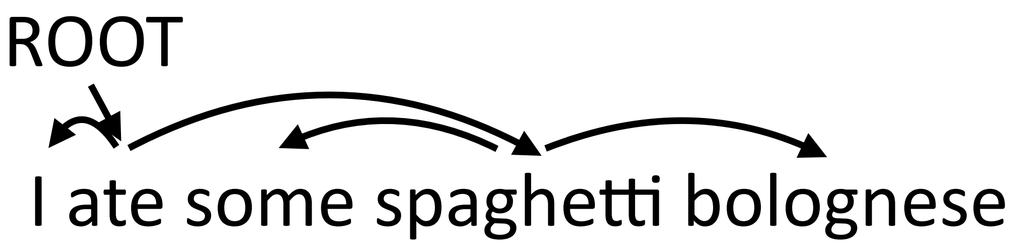
- State: Stack: [ROOT ate] Buffer: [some spaghetti bolognese]

Arc-Standard Parsing

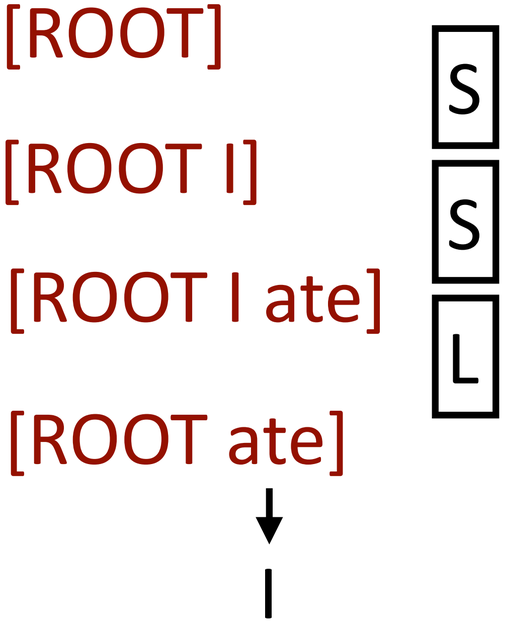


- Start: stack: [ROOT], buffer: [I ate some spaghetti bolognese]
- Arc-standard system: three operations
  - Shift: top of buffer -> top of stack
  - Left-Arc: $\boxed{\sigma|w_{-2}, w_{-1}} \rightarrow \boxed{\sigma|w_{-1}}$, w_{-2} is now a child of w_{-1}
  - Right-Arc $\boxed{\sigma|w_{-2}, w_{-1}} \rightarrow \boxed{\sigma|w_{-2}}$, w_{-1} is now a child of w_{-2}
- End: stack contains [ROOT], buffer is empty []
- How many transitions do we need if we have n words in a sentence?
- There are other transition systems, but we won't discuss these

Arc-Standard Parsing

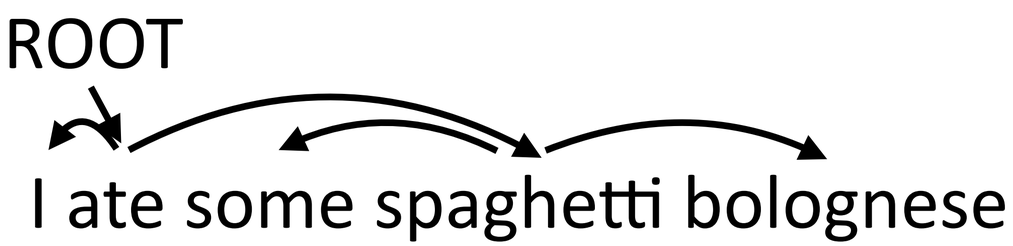


S	top of buffer -> top of stack
LA	pop two, left arc between them
RA	pop two, right arc between them

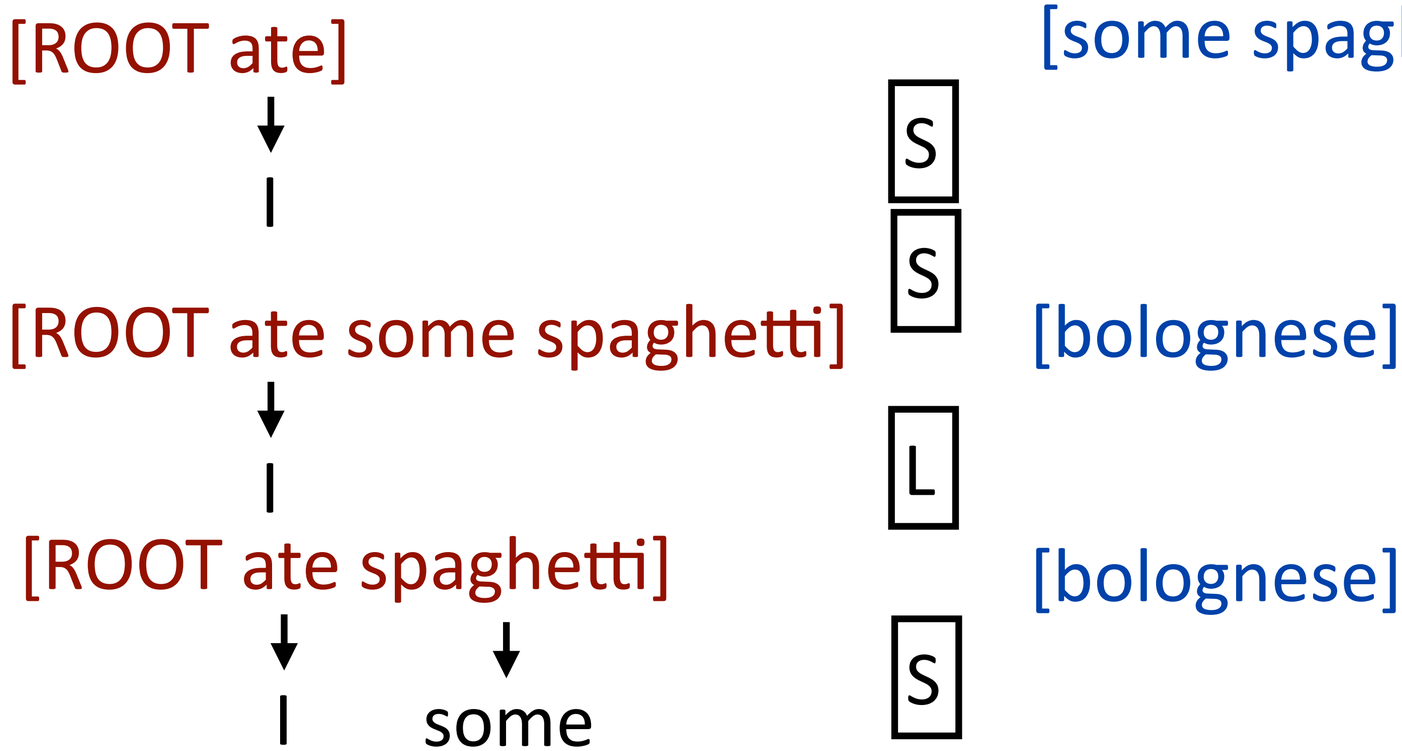


- Could do the left arc later! But no reason to wait
- Can’t attach ROOT <- ate yet even though this is a correct dependency!

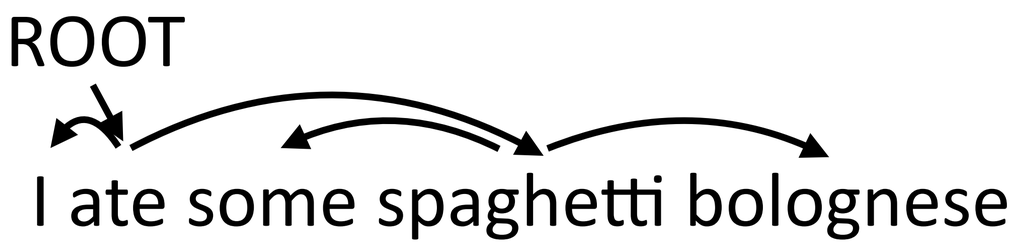
Arc-Standard Parsing



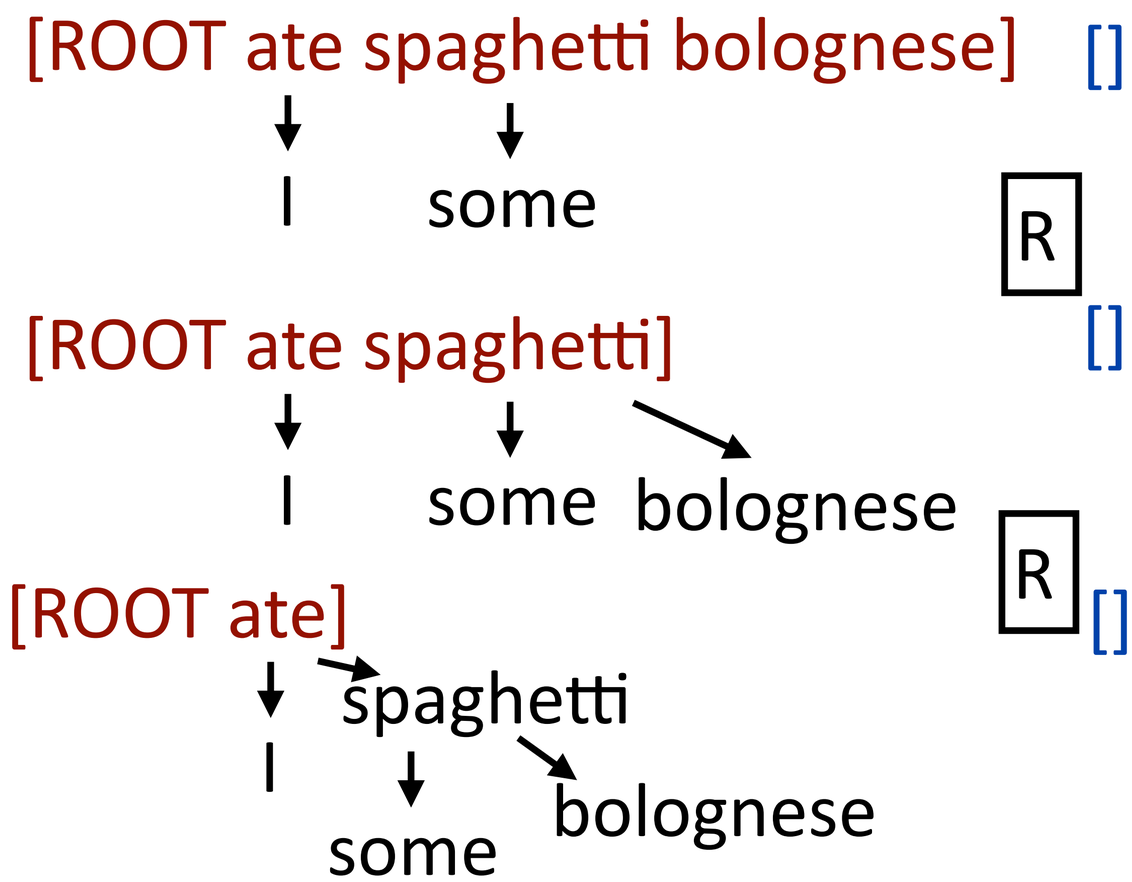
S	top of buffer -> top of stack
LA	pop two, left arc between them
RA	pop two, right arc between them



Arc-Standard Parsing

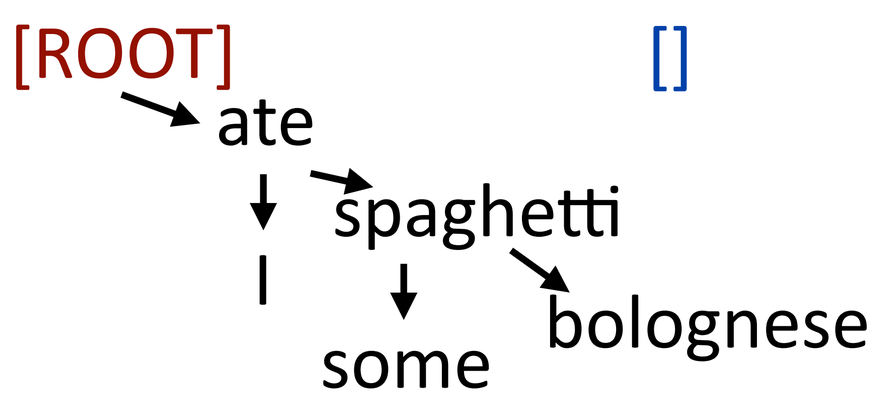


S top of buffer -> top of stack
LA pop two, left arc between them
RA pop two, right arc between them



Stack consists of all words that are still waiting for right children, end with a bunch of right-arc ops

Final state:



Building Transition-Based Parsers

[ROOT] [I ate some spaghetti bolognese]

- How do we make the right decision in this case?
- Only one legal move (shift)

[ROOT ate some spaghetti] [bolognese]

- How do we make the right decision in this case? (all three actions legal)
- Multi-way classification problem: shift, left-arc, or right-arc?

$$\text{argmax}_{a \in \{\text{S},\text{LA},\text{RA}\}} w^\top f(\text{stack}, \text{buffer}, a)$$

Features for Shift-Reduce Parsing

[ROOT ate some spaghetti] [bolognese]

- Features to know this should left-arc?
- One of the harder feature design tasks!
- In this case: the stack tag sequence VBD - DT - NN is pretty informative — looks like a verb taking a direct object which has a determiner in it
- Things to look at: top words/POS of buffer, top words/POS of stack, leftmost and rightmost children of top items on the stack

Training a Greedy Model

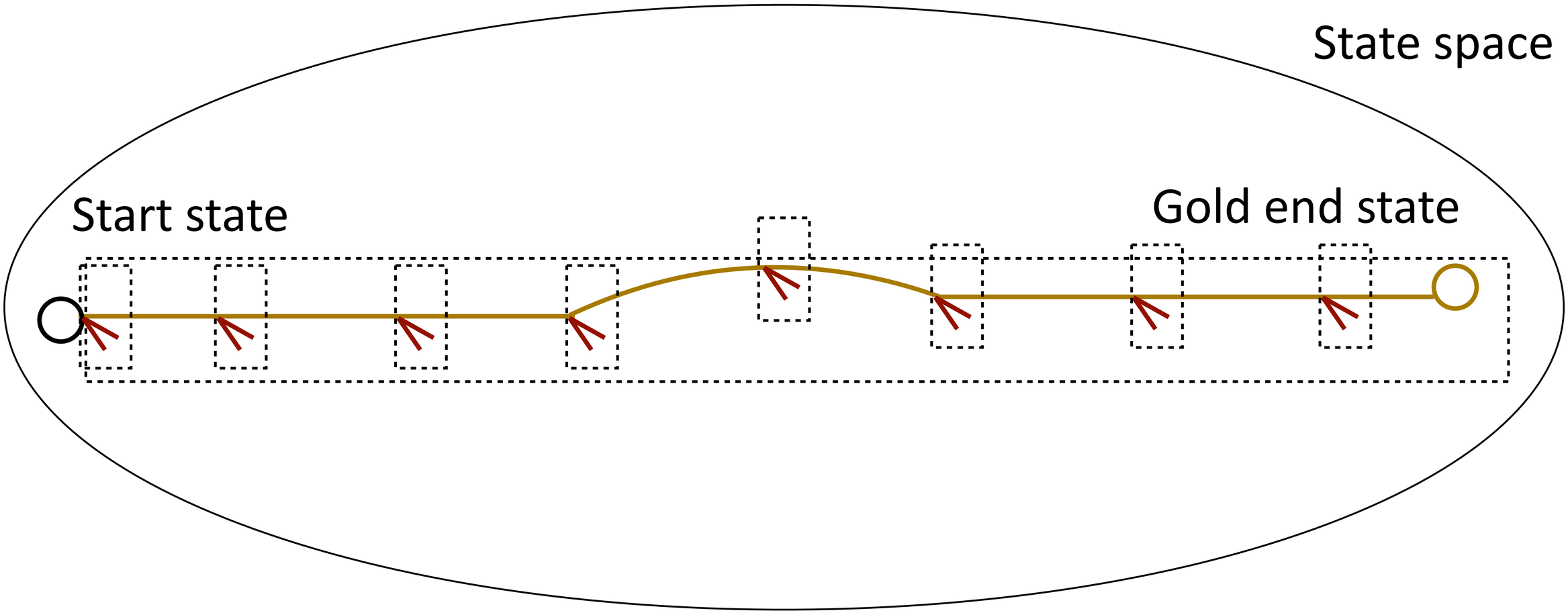
[ROOT ate some spaghetti] [bolognese]

$$\text{argmax}_{a \in \{\text{S},\text{LA},\text{RA}\}} w^\top f(\text{stack}, \text{buffer}, a)$$

- Can turn a tree into a decision sequence ***a*** by building an *oracle*
- Train a classifier to predict the right decision using these as training data

- Training data assumes you made correct decisions up to this point and teaches you to make the correct decision, but what if you screwed up…

Training a Greedy Model



- Greedy: 2n local training examples
- Non-gold states unobserved during training: consider making bad decisions but don't *condition* on bad decisions

WEEK 9

ChatGPT / GPT-4 and Large Language Models

Segments in this section

seg-llm1-gpt3.pdf - GPT-3 p. 265
seg-llm2-zero-shot.pdf - Zero-shot Prompting p. 272
seg-llm3-few-shot.pdf - Few-shot Prompting (In-context Learning) p. 279
seg-llm4-induction-heads.pdf - Induction Heads p. 288
seg-llm5-instruction-tuning.pdf - Instruction Tuning p. 295
seg-llm6-rlhf.pdf - RLHF p. 300
seg-llm7-hallucination.pdf - Hallucination & Factuality p. 307

Key points (added notes)

- GPT-3 and in-context learning; zero-shot verbalizers; few-shot sensitivity and calibration.
- Induction heads: [A][B] ... [A] -> [B].
- Instruction tuning (T0, Flan); RLHF: Bradley-Terry reward model + PPO.
- Hallucination: decompose into propositions and check with retrieval + entailment.

Post-midterm material (not on past midterms).

Note: the Assignment 4 walkthrough is not in the folder.

GPT

- GPT models: all very large Transformer language models, left-to-right language models, trained on raw text
- GPT1: came out before BERT, we'll skip it
- GPT2 was trained on 40GB of text:

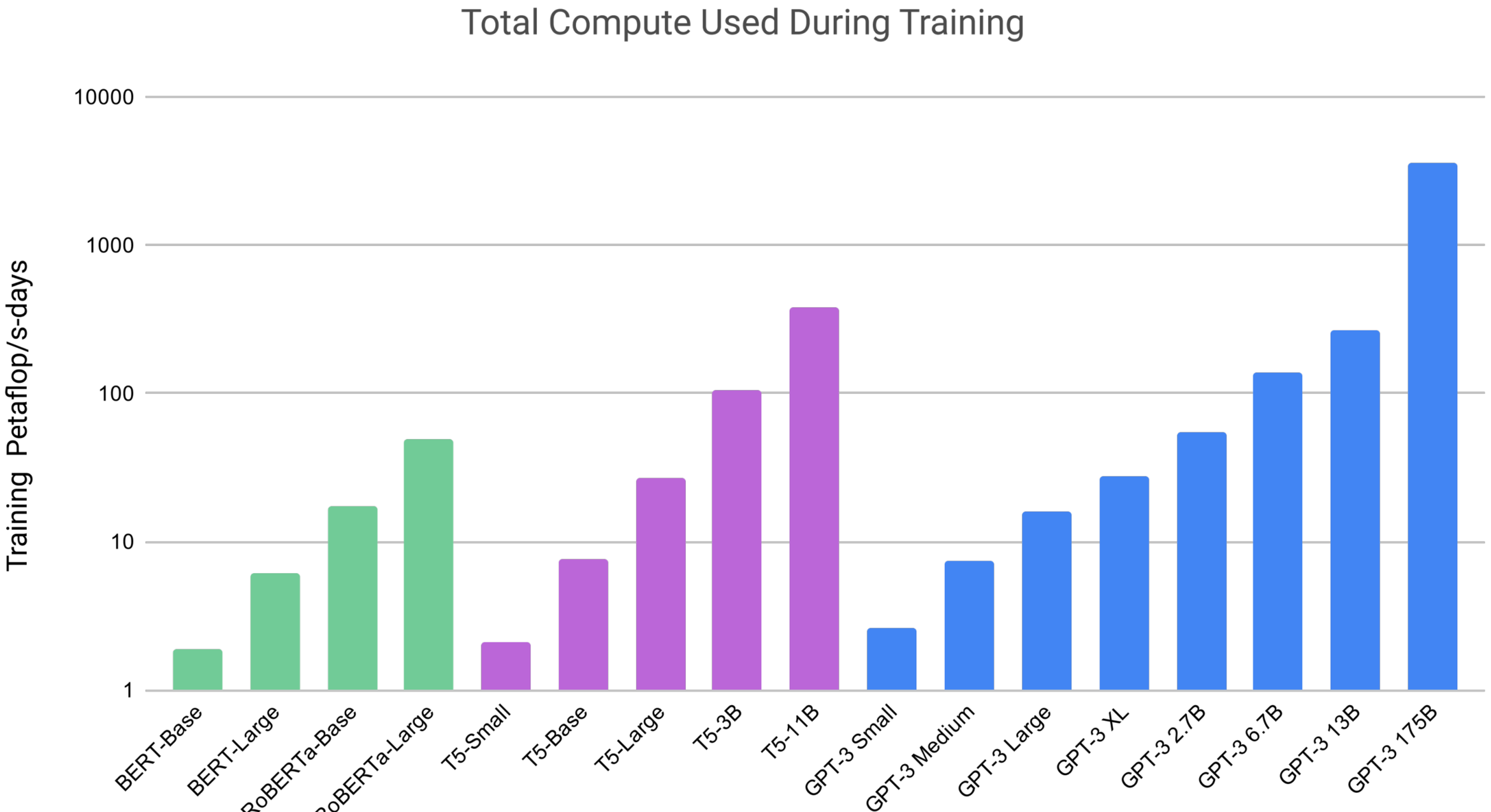
	Parameters	Layers	d_{model}
	117M	12	768
approximate size of BERT	345M	24	1024
	762M	36	1280
GPT-2	1542M	48	1600

- GPT-2 was by far the largest model trained when it came out in March 2019
- Could generate several fluent and coherent sentences back-to-back, which was not seen in smaller models or LSTMs

Radford et al. (2019)

GPT-3

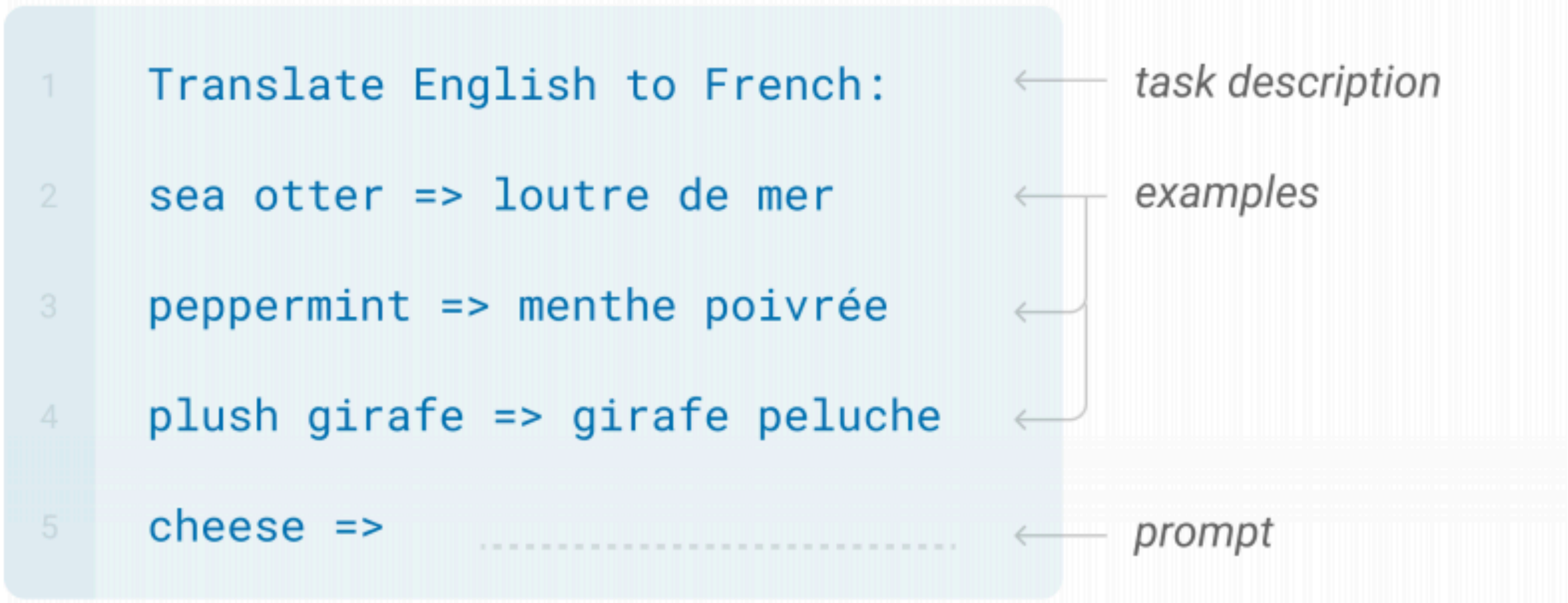
- Released in mid-2020
- 175B parameter model: 96 layers, 96 heads, 12k-dim vectors



Brown et al. (2020)

In-context Learning

- GPT-3 proposes an alternative to fine-tuning: **in-context learning.** Just uses the off-the-shelf model, no gradient updates.
- Key concept: an LM should be able to continue an observed pattern



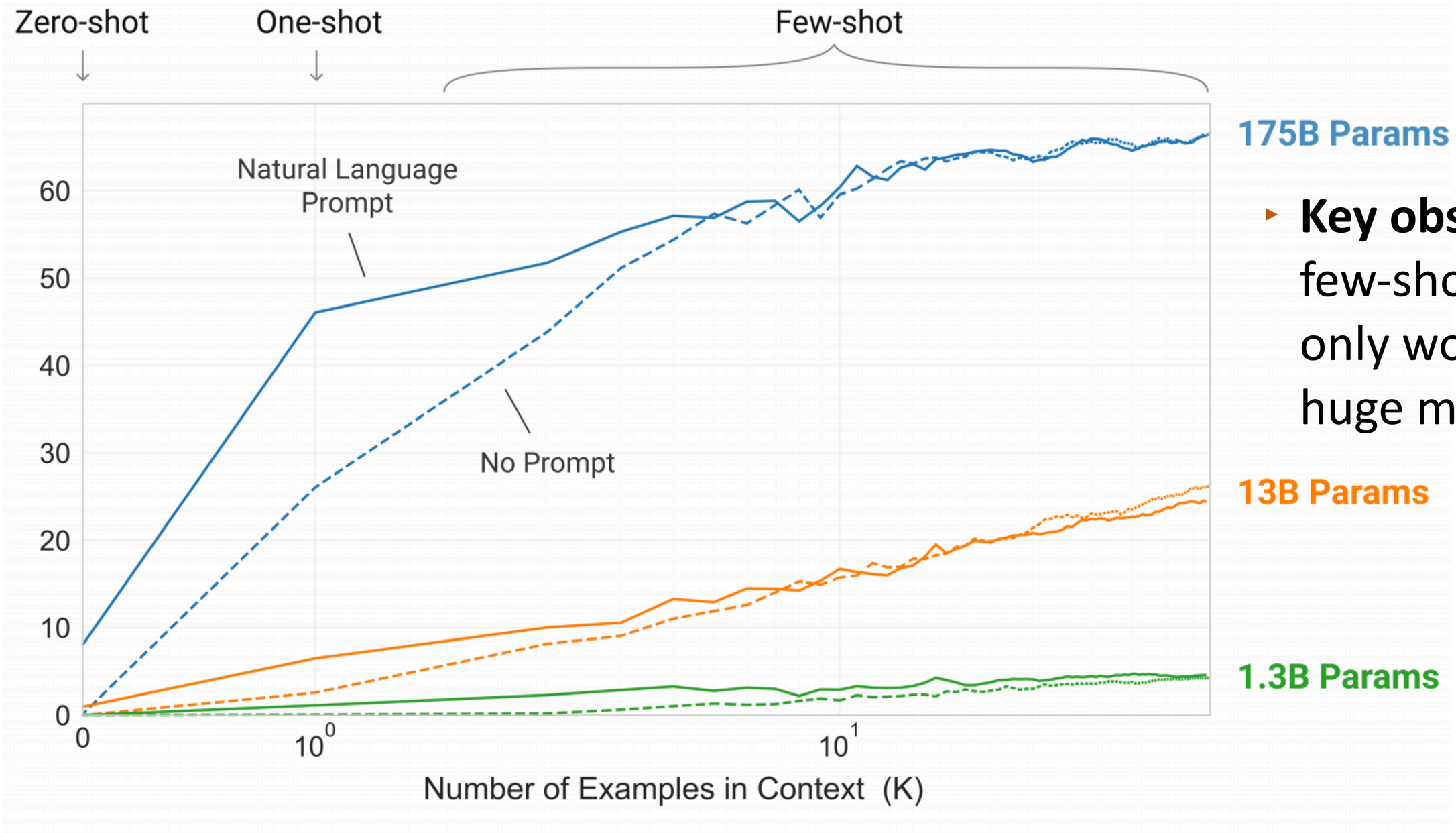
- This procedure depends heavily on the examples you pick as well as the prompt ("*Translate English to French*")

Brown et al. (2020)

In-context Learning

Brown et al. (2020)

In-context Learning



- **Key observation:** few-shot learning only works with huge models!

Brown et al. (2020)

GPT-3: Results

	SuperGLUE Average	BoolQ Accuracy	CB Accuracy	CB F1	COPA Accuracy	RTE Accuracy
Fine-tuned SOTA	**89.0**	**91.0**	**96.9**	**93.9**	**94.8**	**92.5**
Fine-tuned BERT-Large	69.0	77.4	83.6	75.7	70.6	71.7
GPT-3 Few-Shot	71.8	76.4	75.6	52.0	92.0	69.0

	WiC Accuracy	WSC Accuracy	MultiRC Accuracy	MultiRC F1a	ReCoRD Accuracy	ReCoRD F1
Fine-tuned SOTA	**76.1**	**93.8**	**62.3**	**88.2**	**92.5**	**93.3**
Fine-tuned BERT-Large	69.6	64.6	24.1	70.0	71.3	72.0
GPT-3 Few-Shot	49.4	80.1	30.5	75.4	90.2	91.1

- Comparison to fine-tuned state-of-the-art models, fine-tuned BERT-Large Note that these models train on much more data, GPT-3 is “few-shot” and **only** uses in-context learning
- Sometimes very impressive, (MultiRC, ReCoRD), sometimes very bad
- Results on other datasets are equally mixed — but still strong for a few-shot model!

Other Models

- GPT-3 represents a fundamental paradigm shift in model capabilities
- Other strong large language models (LLMs) that are widely used: LLaMA, PaLM (Google), OPT, BLOOM, Pythia (all open except PaLM)
- Modern models like ChatGPT, GPT-4 use additional reinforcement learning from human feedback or instruction tuning. We will come to these later in the course; they are basically souped-up fine-tuning techniques

Zero-shot Prompting

- GPT-3/4/ChatGPT can handle lots of existing tasks based purely on incidental exposure to them in pre-training
  - Example from summarization: the token “tl;dr” (“too long; didn’t read”) is an indicator of summaries in the wild
- We’ll discuss two paradigms: **zero-shot prompting**, where no examples are given to a model (just a text specification), and **few-shot prompting**, where a few examples are given in-context
- Both paradigms can theoretically handle classification, text generation, and more!

Zero-shot Prompting

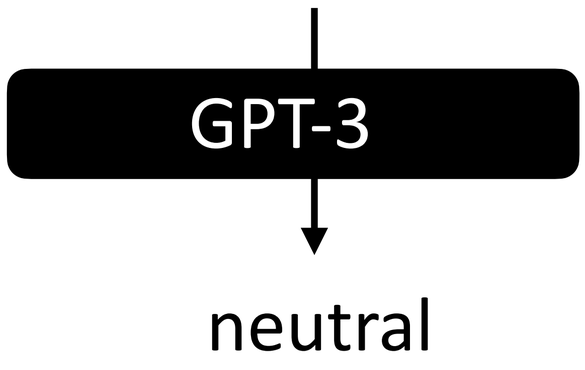
- Single unlabeled datapoint ***x***, want to predict label *y*

  x = *The movie's acting could've been better, but the visuals and directing were top-notch.*

- Wrap ***x*** in a template we call a verbalizer ***v***

Review: The movie's acting could've been better, but the visuals and directing were top-notch.

Out of positive, negative, or neutral, this review is



Zero-shot Prompting

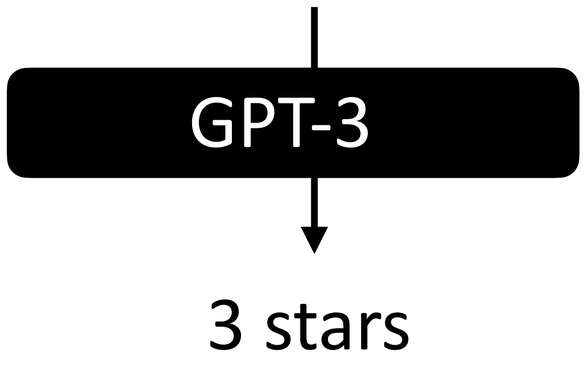
- Single unlabeled datapoint ***x***, want to predict label *y*

  x = *The movie's acting could've been better, but the visuals and directing were top-notch.*

- Wrap ***x*** in a template we call a verbalizer ***v***

Review: The movie's acting could've been better, but the visuals and directing were top-notch.

On a 1 to 4 star scale, the reviewer would probably give this movie

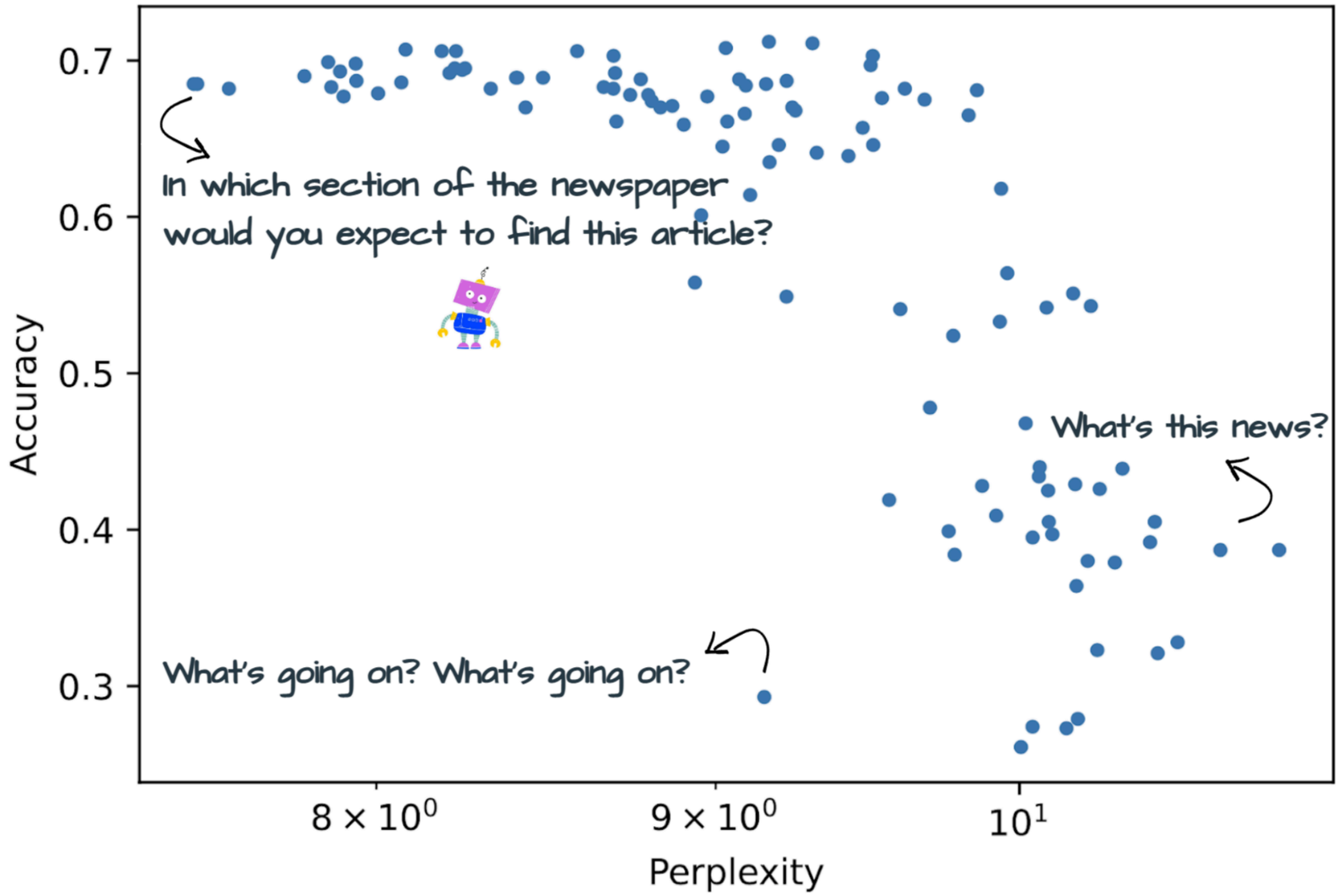


Zero-shot Classification: Approaches

- **Approach 1:** Generate from the model and parse the generation
  - What if you ask for a star rating and it doesn't give you a number of stars but just says something else?
- **Approach 2:** Compare probs: "*Out of positive, negative, or neutral, this review is _*". Compare P(*positive* | ***x***), P(*neutral* | ***x***), P(*negative* | ***x***)
  - This constrains the model to only output a valid answer, and you can normalize these probabilities to get a distribution
- How much difference does changing the prompt make?

Variability in Prompts

- Plot: large number of prompts produced by {manual writing, paraphrasing, backtranslation}



y-axis: task performance

x-axis: perplexity of the prompt. How natural is it? How much does it appear in the pre-training data?

- Caveat: a little bit of prompt engineering will usually get you to a decent performance point

Gonen et al. (2022)

Variability in Prompts

Task	Perplexity-score corr. Pearson	Perplexity-score corr. Spearman	Perplexity-acc corr. Pearson	Perplexity-acc corr. Spearman	Avg Acc	Acc 50%
Antonyms	**-0.41	**-0.53	–	–	–	–
GLUE Cola	-0.15	-0.14	-0.04	-0.02	47.7	57.1
Newspop	*-0.24	**-0.26	*-0.20	-0.18	66.4	72.9
AG News	**-0.63	**-0.68	**-0.77	**-0.81	57.5	68.7
IMDB	**0.35	**0.40	0.14	*0.20	86.2	91.0
DBpedia	**-0.50	**-0.44	**-0.51	**-0.42	46.7	55.2
Emotion	-0.14	-0.19	**-0.30	**-0.32	16.4	23.0
Tweet Offensive	*-0.19	0.07	0.18	*0.23	51.3	55.8

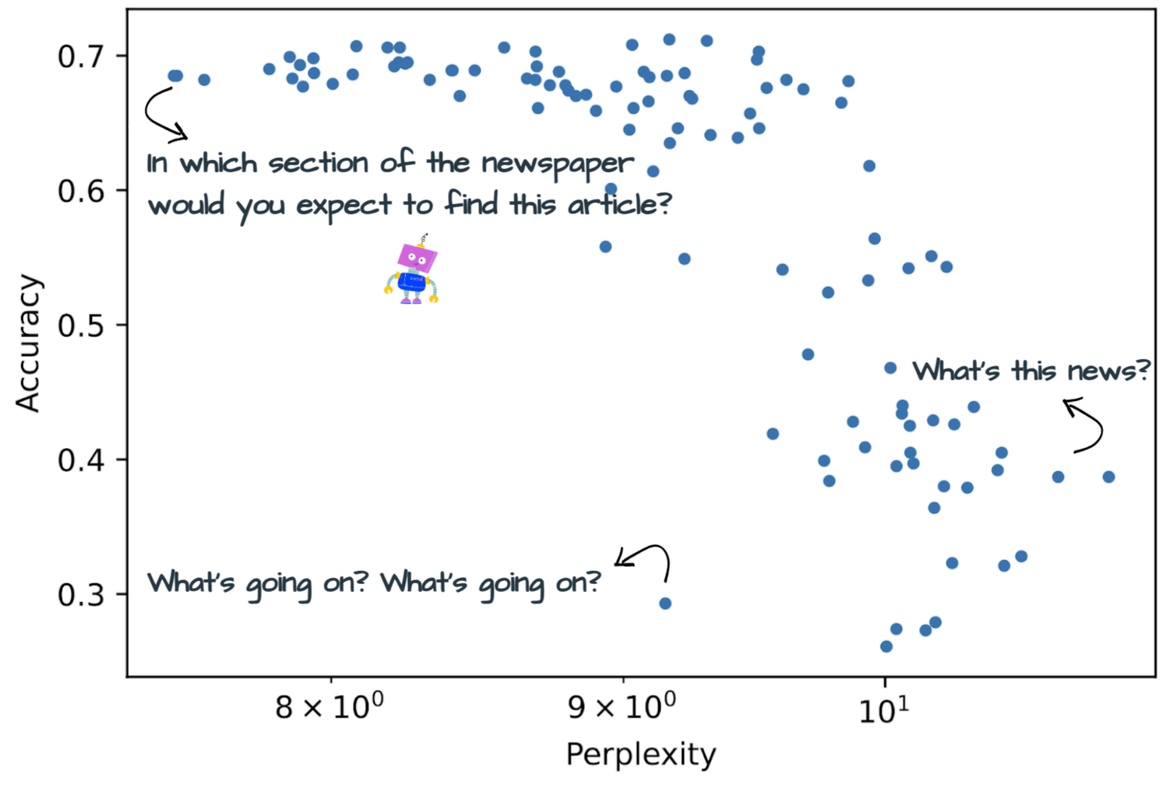
- OPT-175B: average of best 50% of prompts is much better than average over all prompts

Gonen et al. (2022)

Prompt Optimization

- A number of methods exist for searching over prompts (either using gradients or black-box optimization)

- Most of these do not lead to dramatically better results than doing some manual engineering/ hill-climbing (and they may be computationally intensive)



- RLHF models like ChatGPT are also better at "understanding" prompts, so less engineering is needed

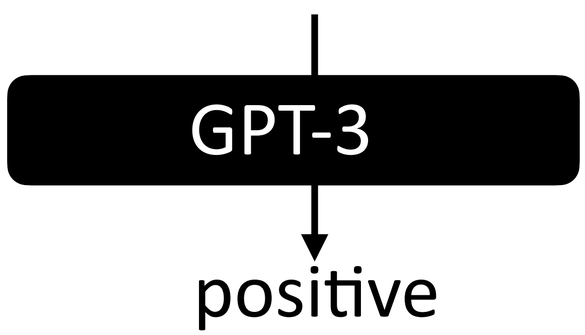
Gonen et al. (2022)

Few-shot Prompting

- Form “training examples” from (**x**, y) pairs, verbalize them (can be lighter-weight than zero-shot verbalizer)

- Input to GPT-3: $\boldsymbol{v}(\boldsymbol{x}_1)$ $\boldsymbol{v}(y_1)$ $\boldsymbol{v}(\boldsymbol{x}_2)$ $\boldsymbol{v}(y_2)$ … $\boldsymbol{v}(\boldsymbol{x}_{\text{test}})$

Review: The cinematography was stellar; great movie!
Sentiment (positive or negative): positive
Review: The plot was boring and the visuals were subpar.
Sentiment (positive or negative): negative
Review: The movie’s acting could’ve been better, but the visuals and directing were top-notch.
Sentiment (positive or negative):



- Usually works better than zero-shot (comparisons in a few slides)

What can go wrong?

Review: The movie was great!
Sentiment: positive

Review: I thought the movie was alright; I would've seen it again.

Sentiment: positive

Review: The movie was pretty cool!

Sentiment: positive

Review: Pretty decent movie!

Sentiment: positive

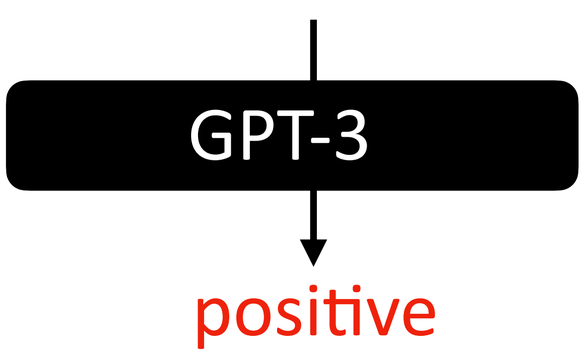
Review: The movie had good enough acting and the visuals were nice.

Sentiment: positive

Review: There wasn't anything the movie could've done better.

Sentiment: positive

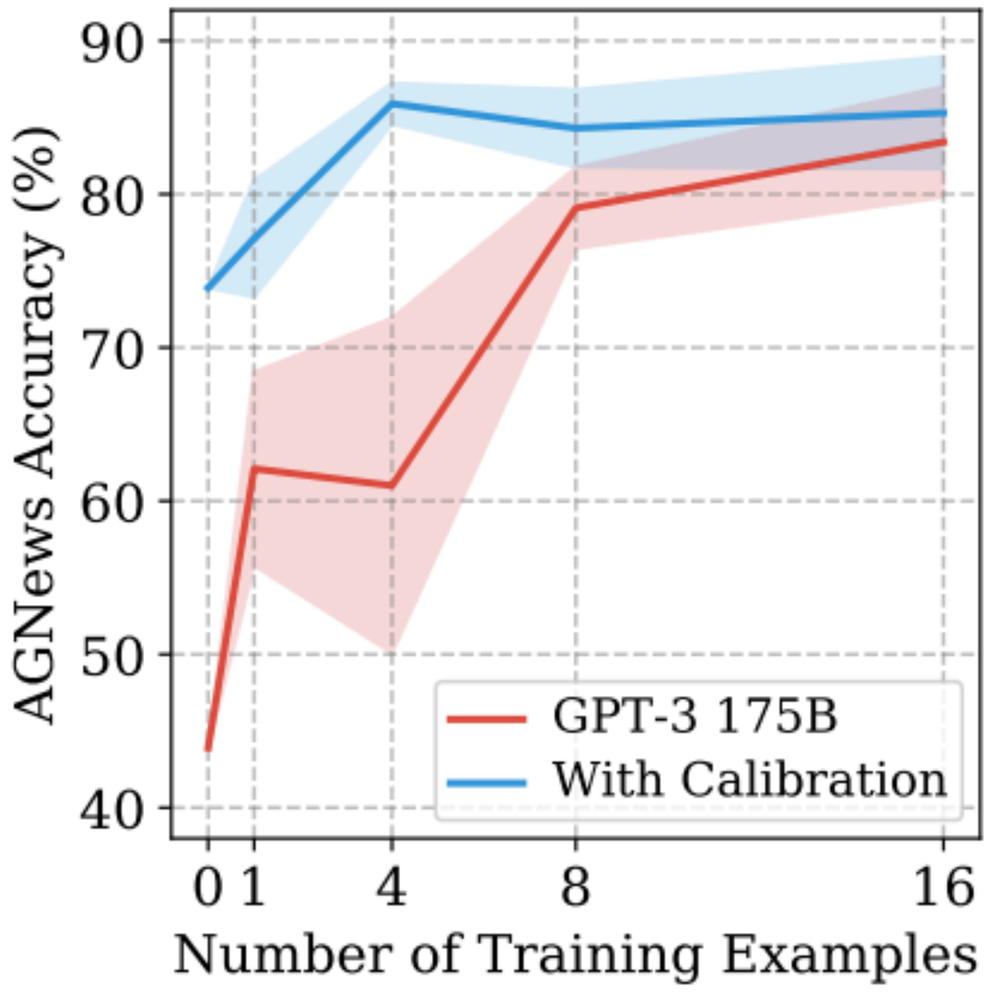
Review: Okay movie but could've been better.
Sentiment:



What examples do we need?

- What if we take random sets of training examples? There is quite a bit of variance on basic classification tasks, particularly when just a few examples are used

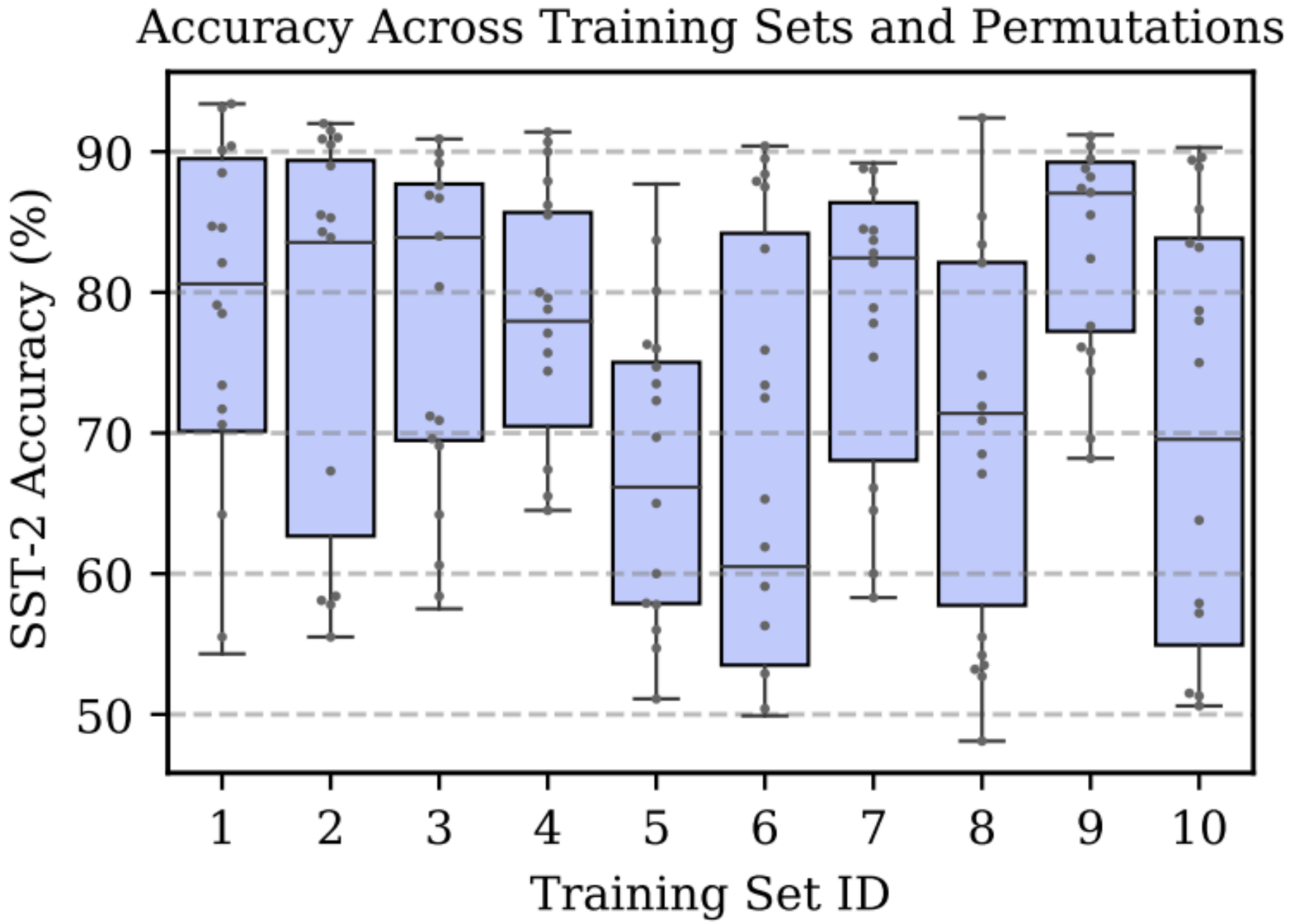
- Note: these results are with basic GPT-3 and not Instruct-tuned versions of the model. This issue has been resolved somewhat



Zhao et al. (2021)

Properties of In-context Examples

- Performance varies **even across permutations of training examples**
- x-axis: different collections of train examples.
  y-axis: sentiment accuracy. Boxes represent results over different permutations of the data

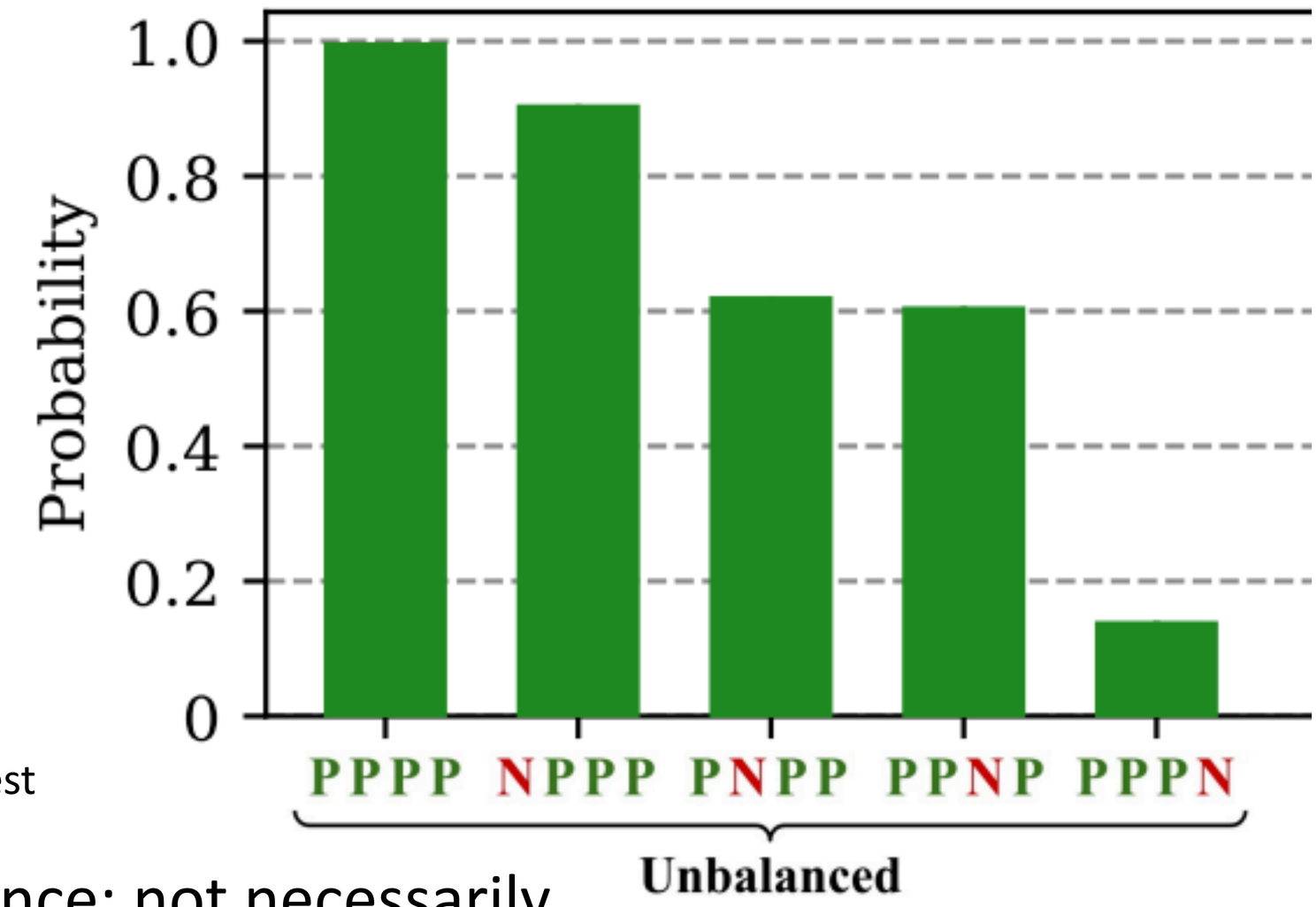


Zhao et al. (2021)

Properties of In-context Examples

- Having unbalanced training sets leads to high “default” probabilities of positive; that is, if we feed in a null $\boldsymbol{x}_{\text{test}}$

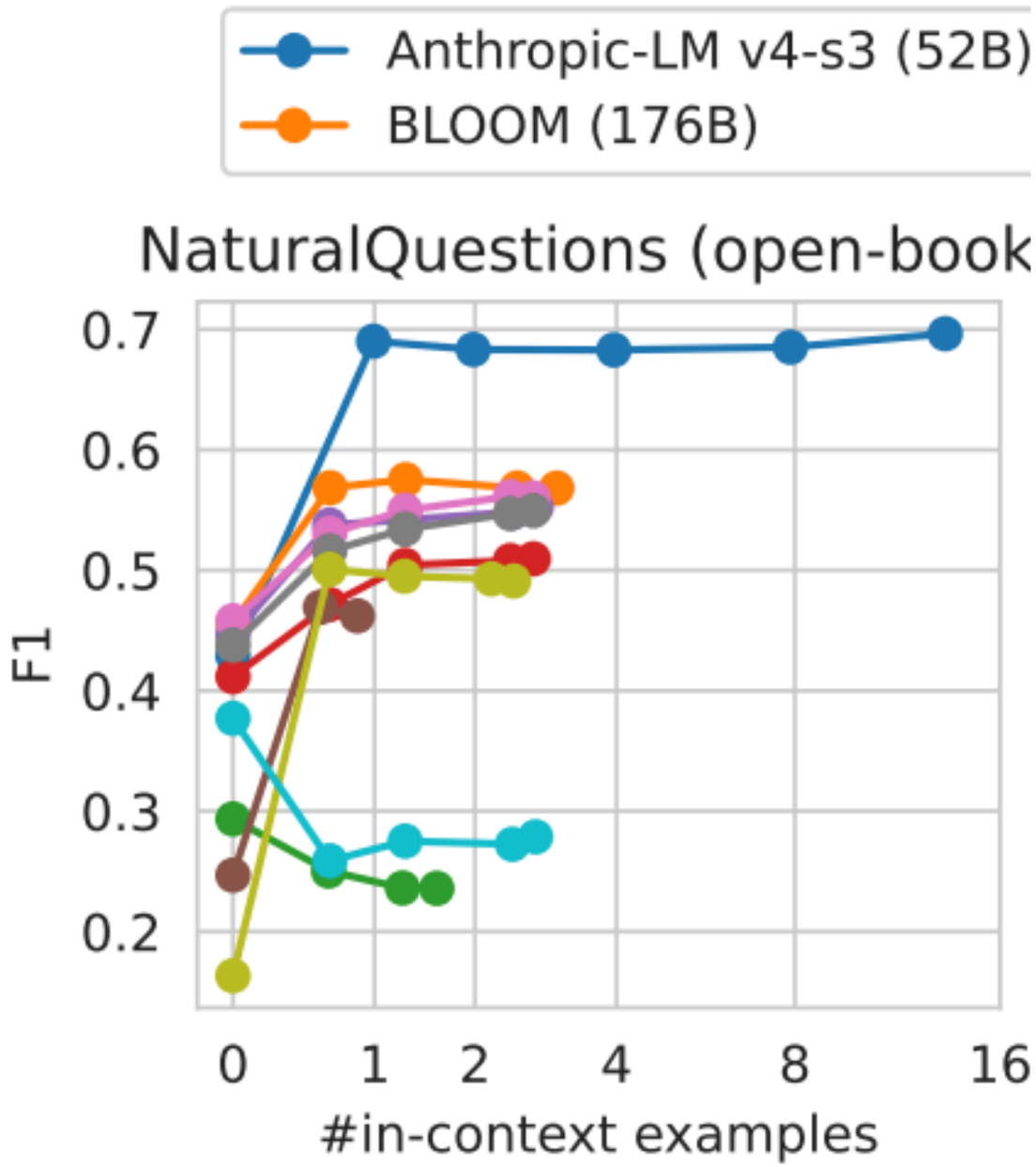
- Solution: “calibrate” the model by normalizing by that probability of null $\boldsymbol{x}_{\text{test}}$



- Leads to higher performance; not necessarily crucial with prompt-tuned models

Zhao et al. (2021)

Results: HELM

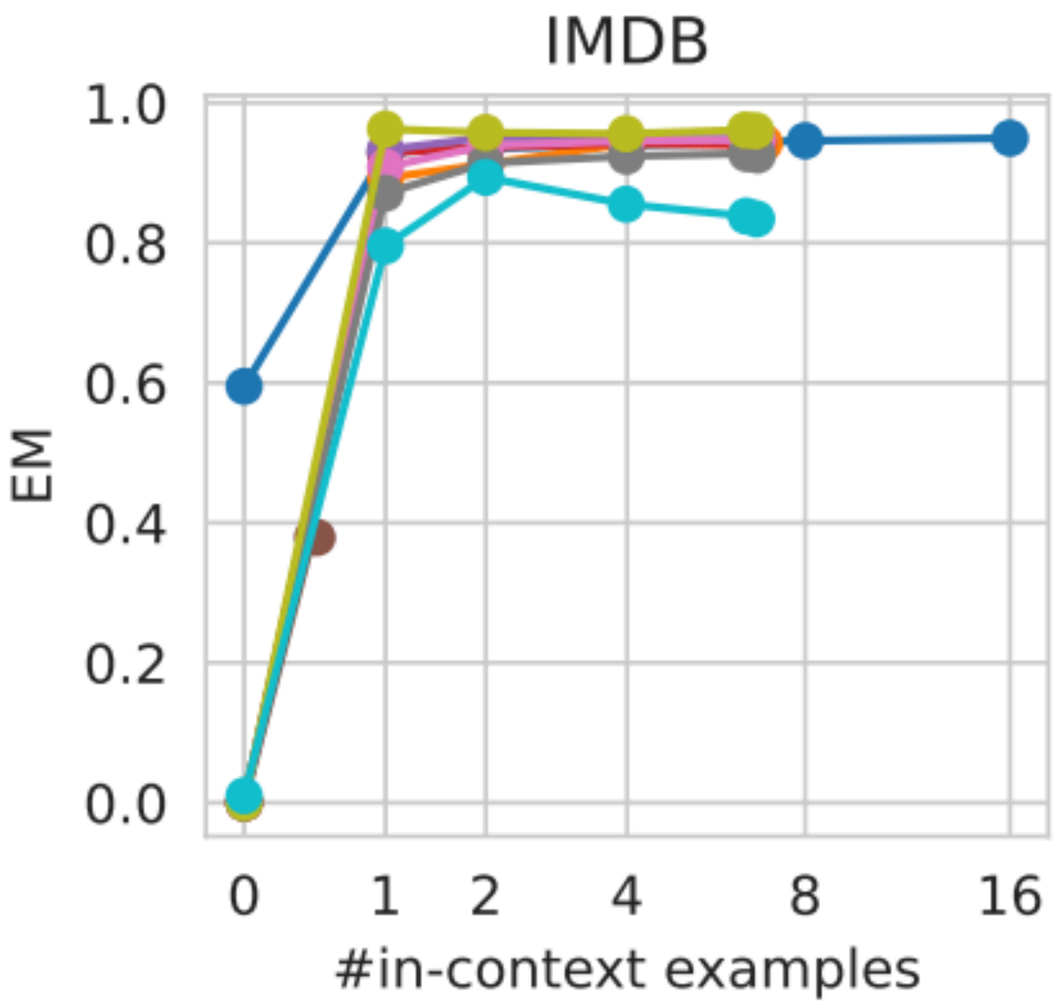


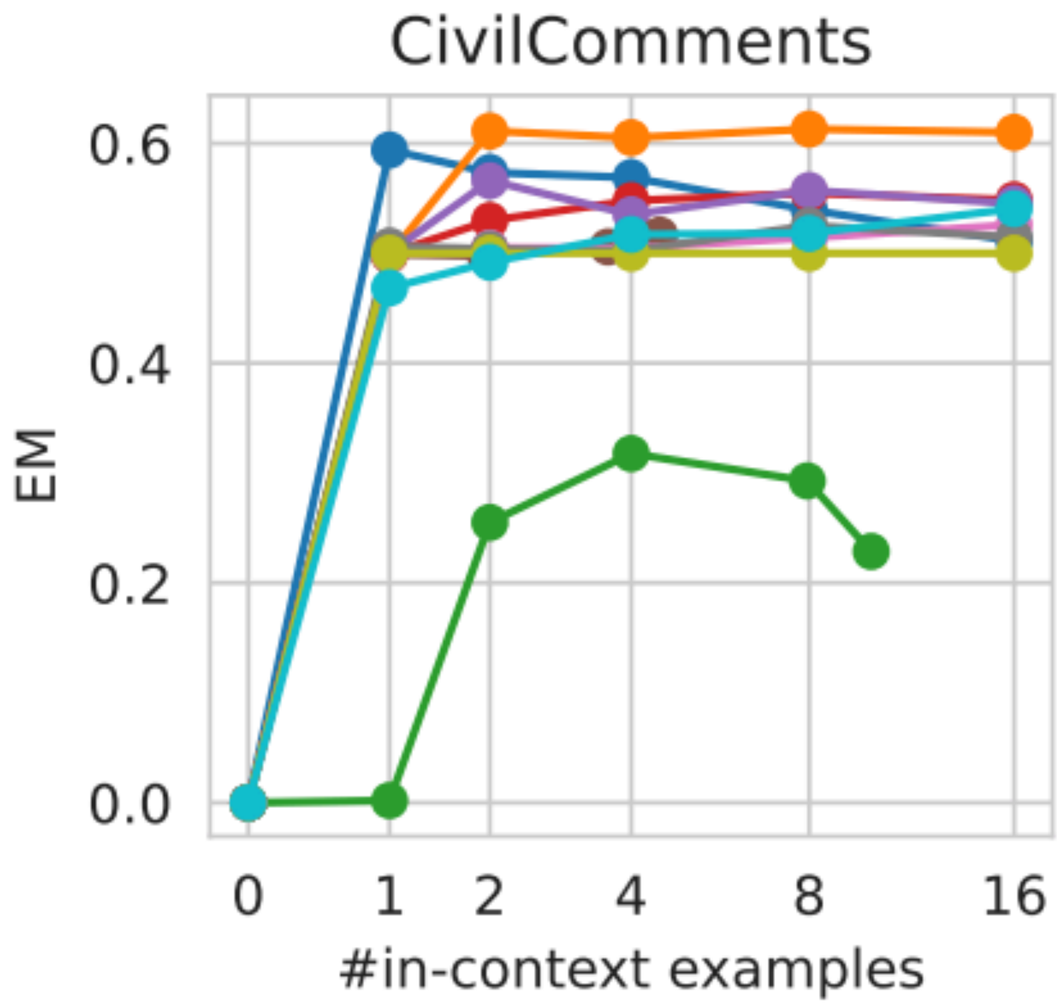
Each line is a different LM

- **More in-context examples generally leads to better performance**

Percy Liang et al. (2022)

Results: HELM



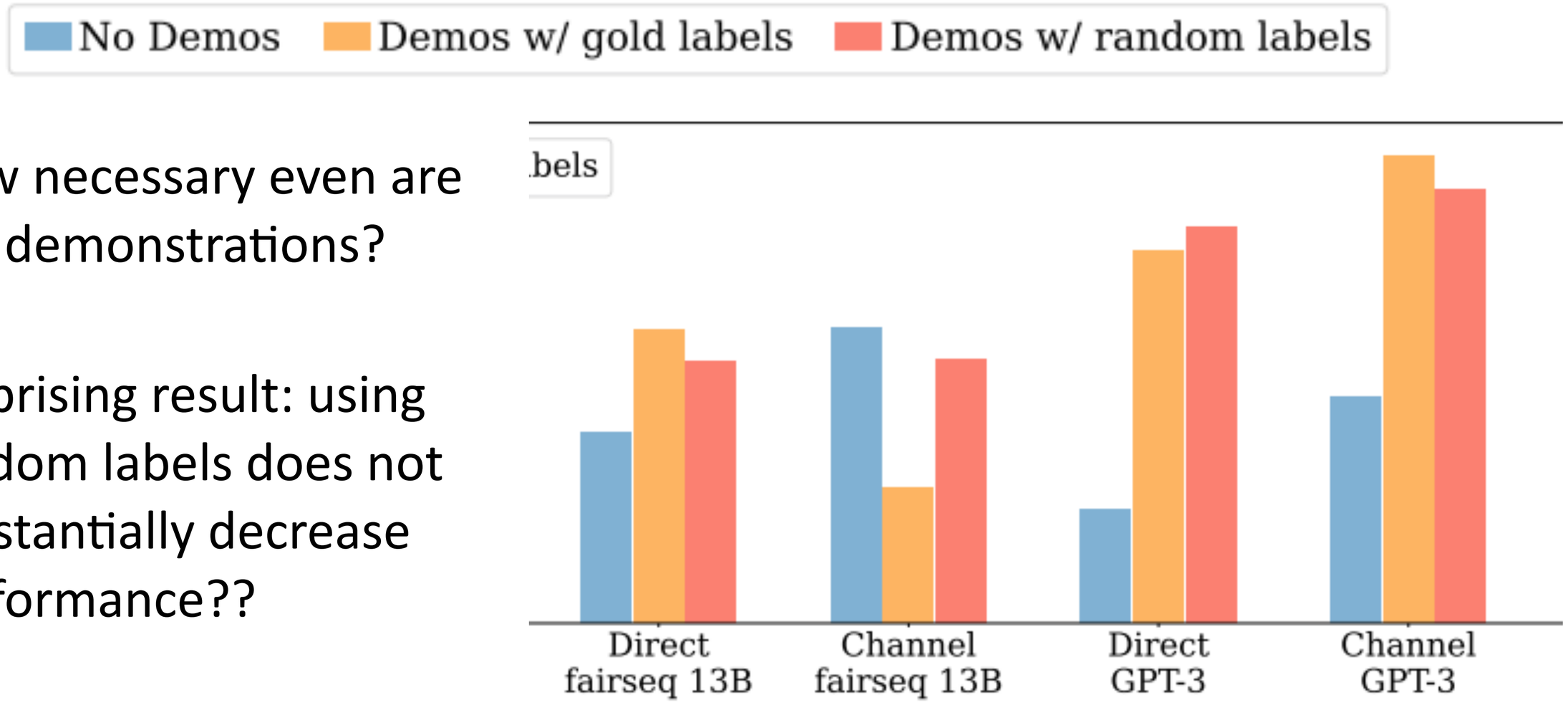


Percy Liang et al. (2022)

Rethinking Demonstrations

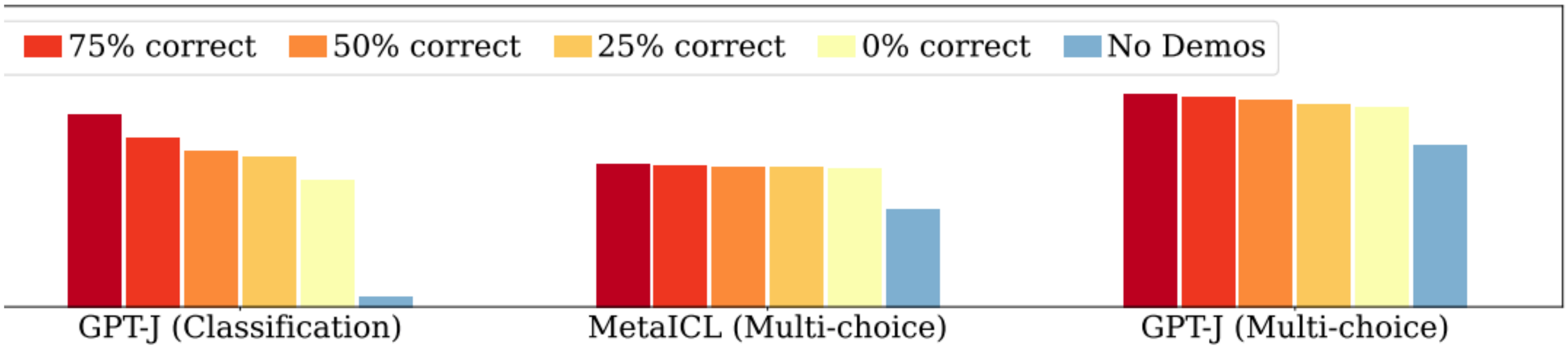
- How necessary even are the demonstrations?

- Surprising result: using random labels does not substantially decrease performance??



Min et al. (2022)

Rethinking Demonstrations



- Having even mislabeled demonstrations is much better than having no demonstrations, indicating that the form of the demonstrations is partially responsible for in-context learning

Min et al. (2022)

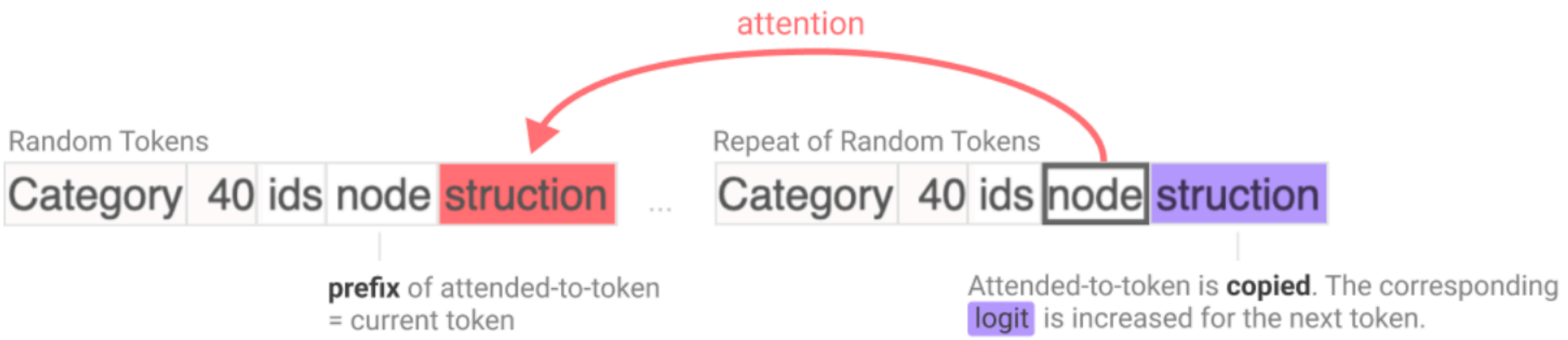
Background: Transformer Circuits

- There are mechanisms in Transformers to do “fuzzy” or “nearest neighbor” versions of pattern completion, completing [A*][B*] … [A] → [B] , where A* ≈ A and B* ≈ B are similar in some space
- Olsson et al. want to establish that these mechanisms are responsible for good ICL capabilities
- We can find these heads and see that performance improves; can we causally link these?

Olsson et al. (2022)

Induction Heads

- Induction heads: a pair of attention heads in different layers that work together to copy or complete patterns.
- The first head copies information from the previous token into each token.
- Second attention head to attend to tokens based on what happened before them, rather than their own content. Likely to “look back” and copy next token from earlier
- The two heads working together cause the sequence …[A][B]…[A] to be more likely to be completed with [B].

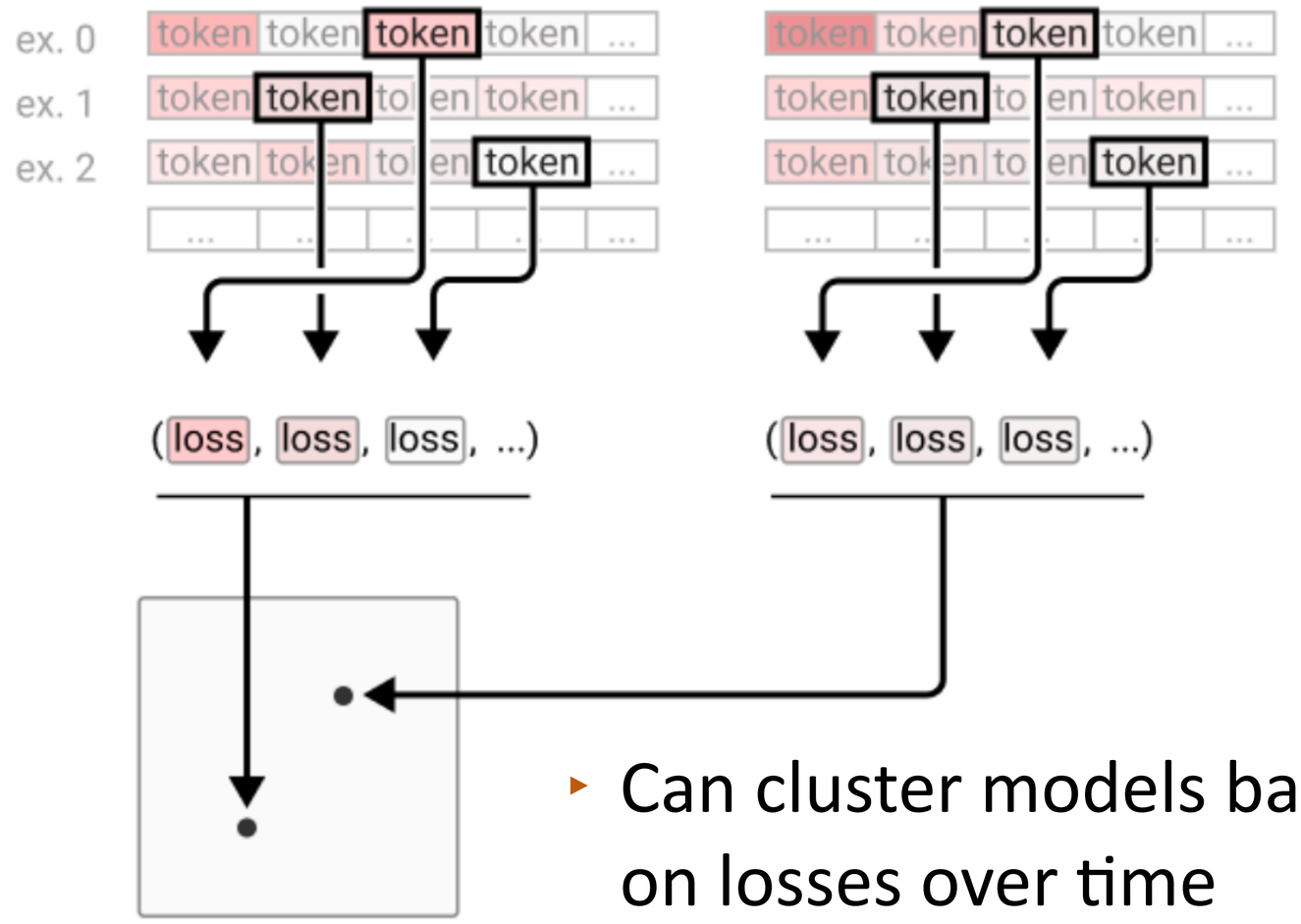


Induction Heads

Step 1: Run each model / snapshot over the same set of multiple dataset examples, collecting one token's loss per example.

Step 2: For each sample, extract the loss of a consistent token. Combine these to make a vector of losses per model / snapshot.

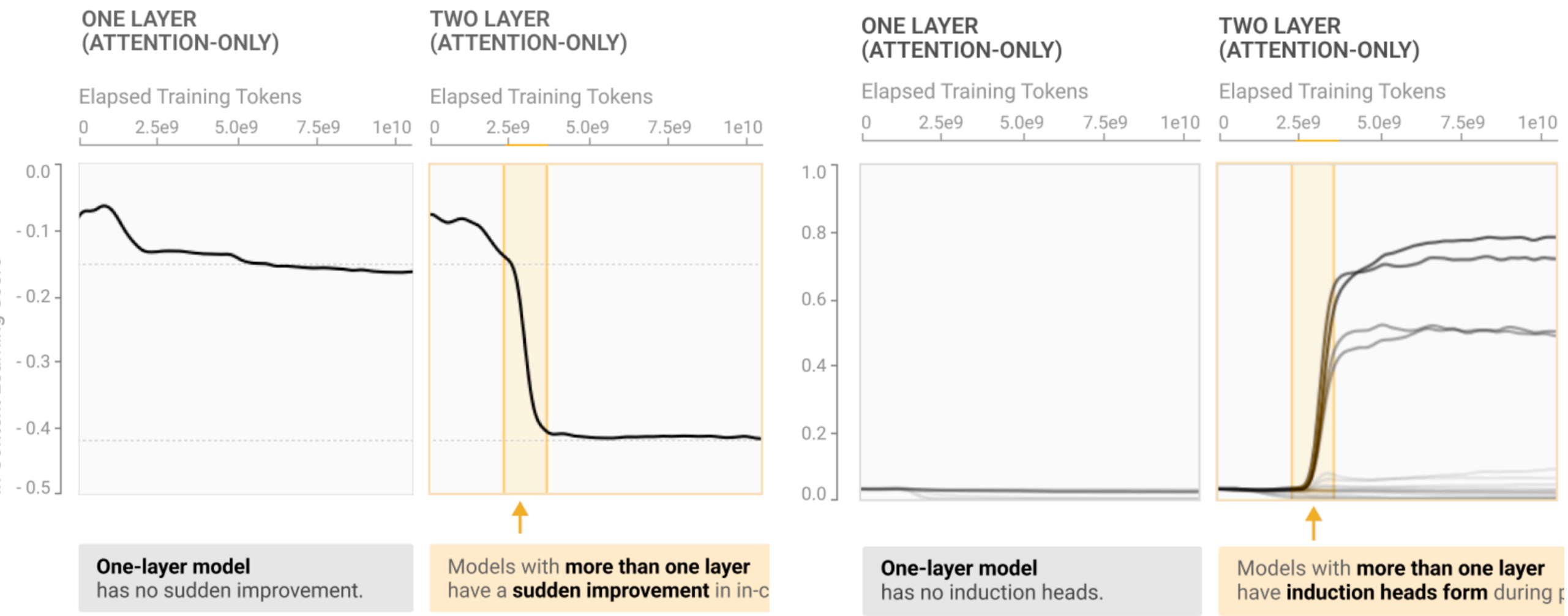
Step 3: The vectors are jointly reduced with principal component analysis to project them into a shared 2D space.



- Can cluster models based on losses over time

- Characterize performance by ICL score: loss(500th token) - loss(50th token) — average measure of how much better the model is doing later once it's seen more of the pattern

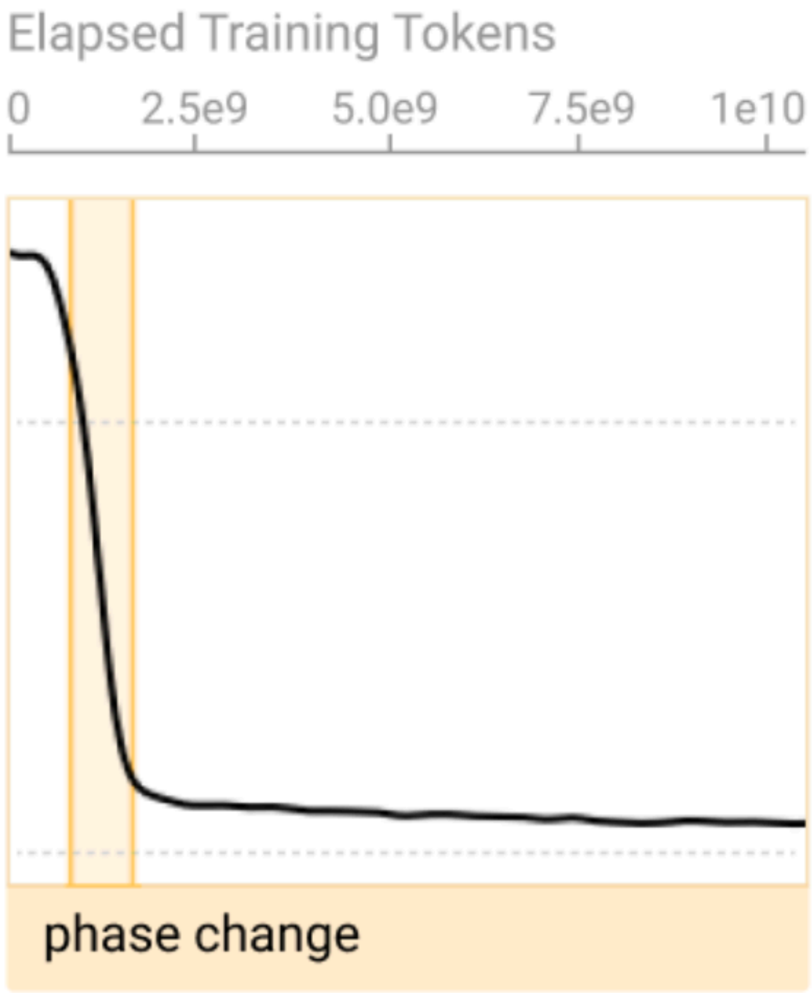
Induction Heads



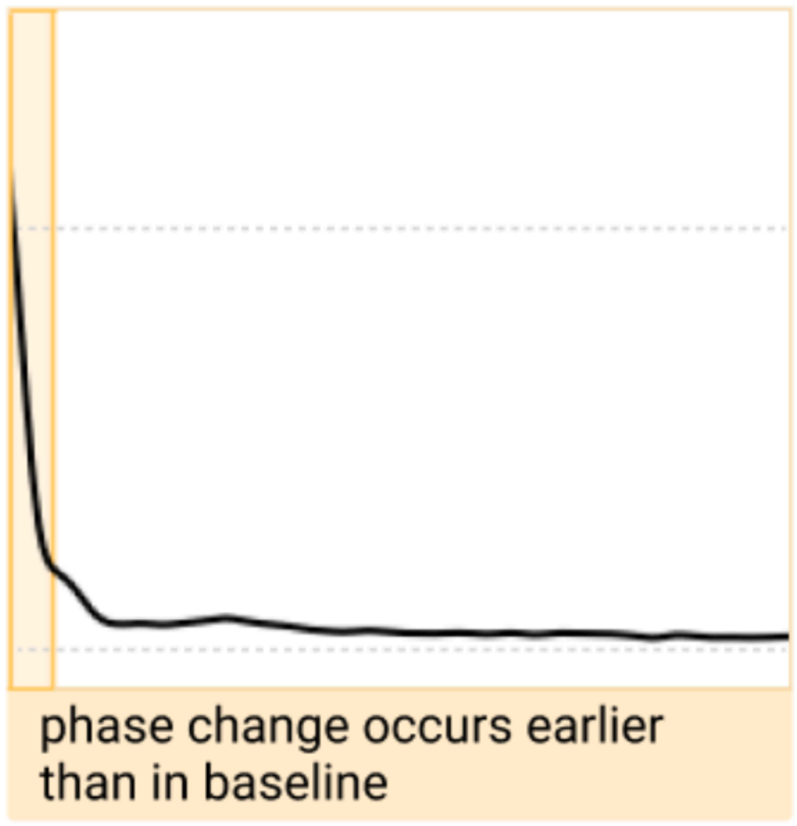
- Improvement in ICL (loss score) correlates with emergence of induction heads

Olsson et al. (2022)

Induction Heads

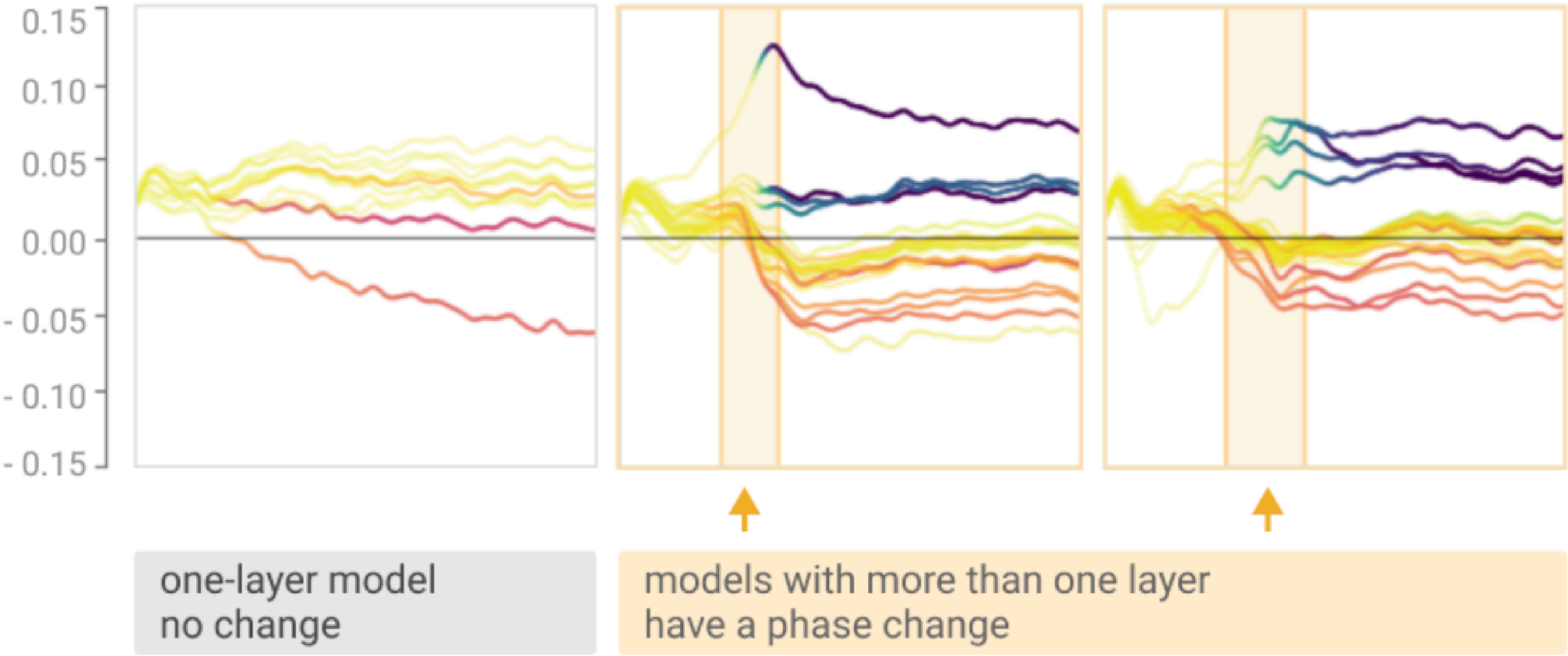


Change architecture to promote induction heads => phase change happens earlier



Olsson et al. (2022)

Induction Heads



- If you remove induction heads, behavior changes dramatically

Olsson et al. (2022)

Interpretability

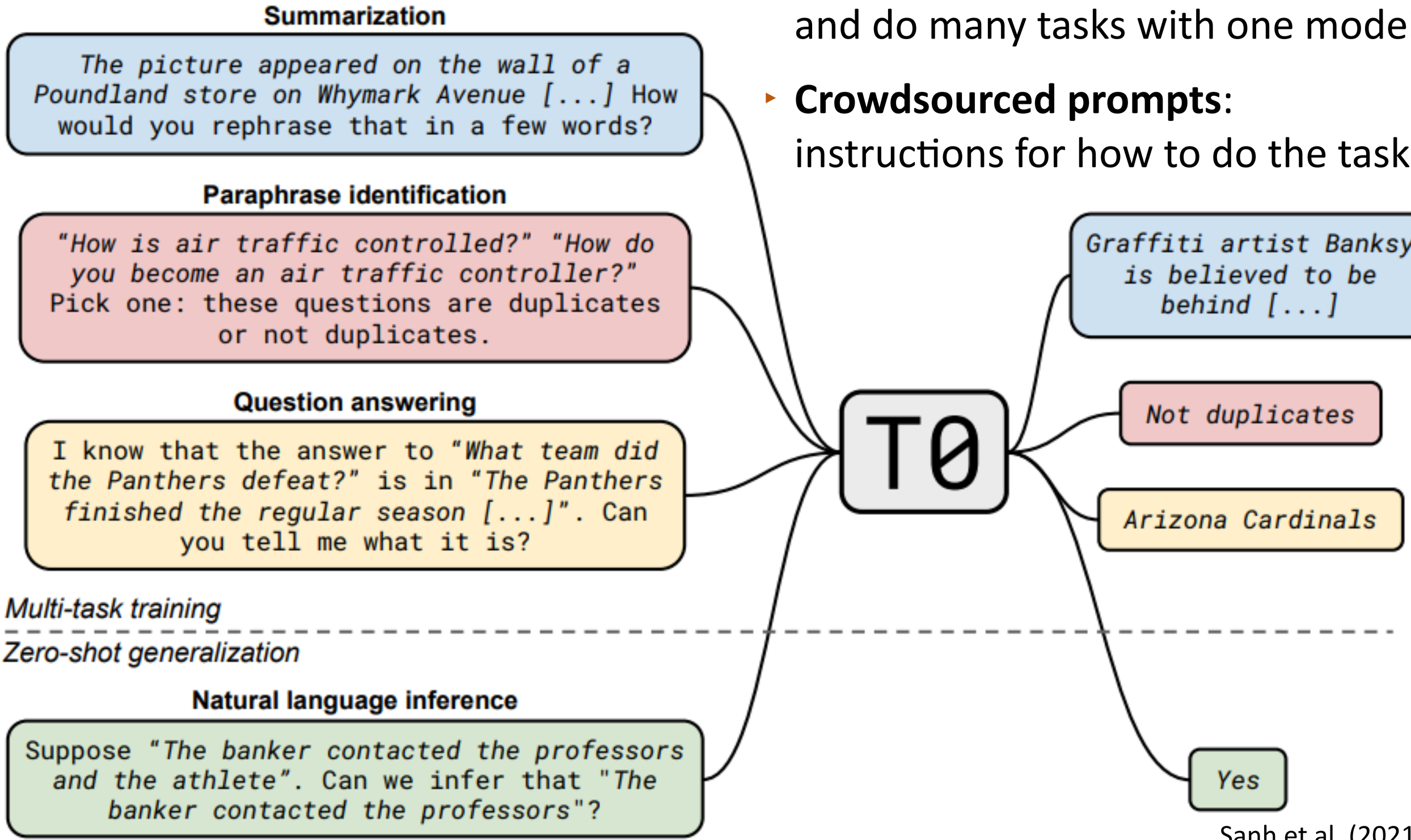
- Lots of explanations for why ICL works — but these haven’t led to many changes in how Transformers are built or scaled
- Several avenues of inquiry: theoretical results (capability of these models), mechanistic interpretability, fully empirical (more like that next time)

Instruction Tuning

- We want to optimize models for P(answer | prompt), but they're learned on a basic language modeling objective P(word | context)
- One solution: fine-tune these models to do what we care about (question answering, classification, …)
- Two main ways of doing this in 2023:
  - **Instruction tuning:** supervised fine-tuning on data derived from many NLP tasks
  - **Reinforcement learning from human feedback (RLHF):** RL to improve human judgments of how good the outputs are

Task Generalization: T0

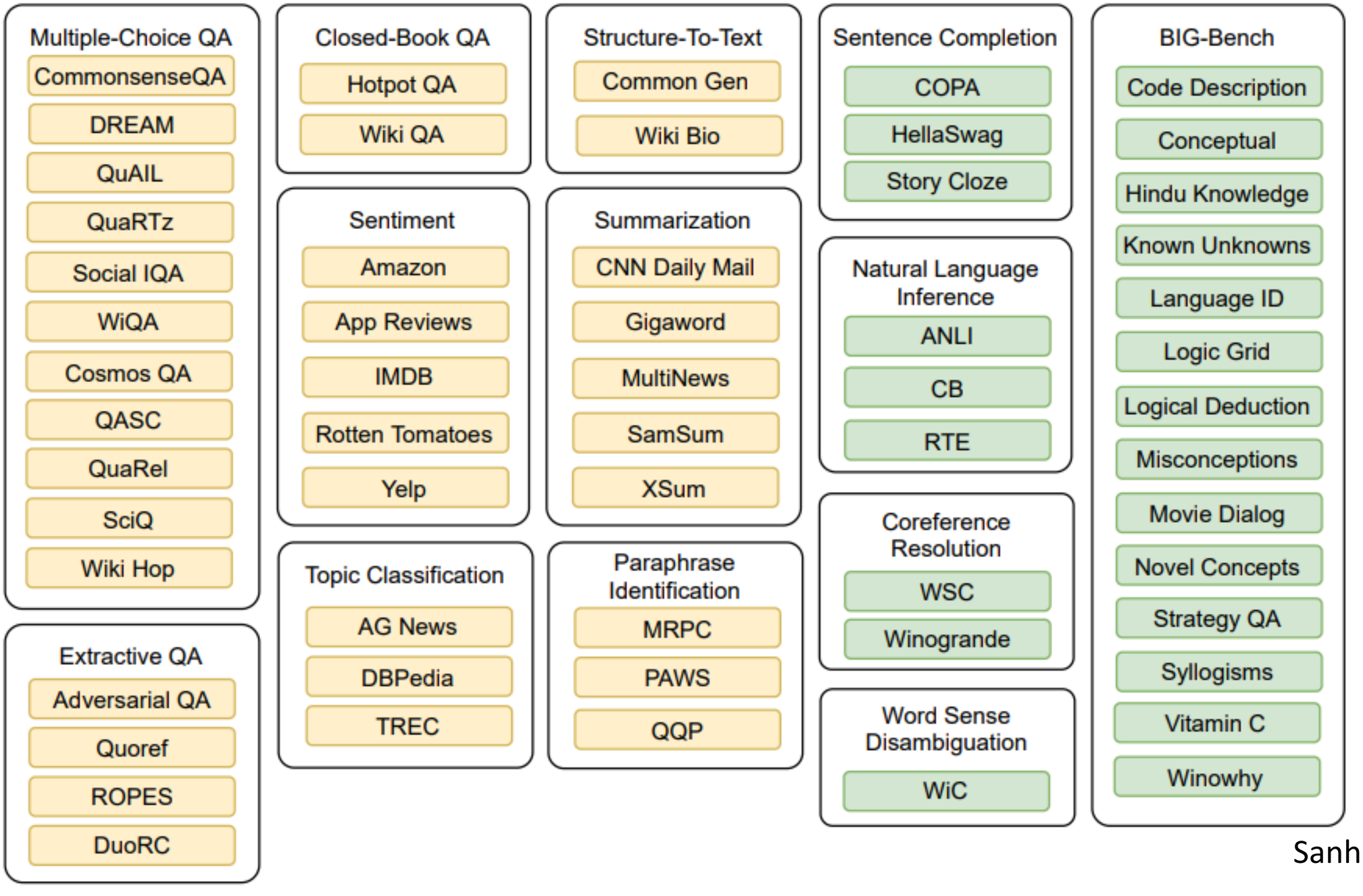
- T0: tries to deliver on the goal of T5 and do many tasks with one model
- **Crowdsourced prompts**: instructions for how to do the tasks



Sanh et al. (2021)

Task Generalization: T0

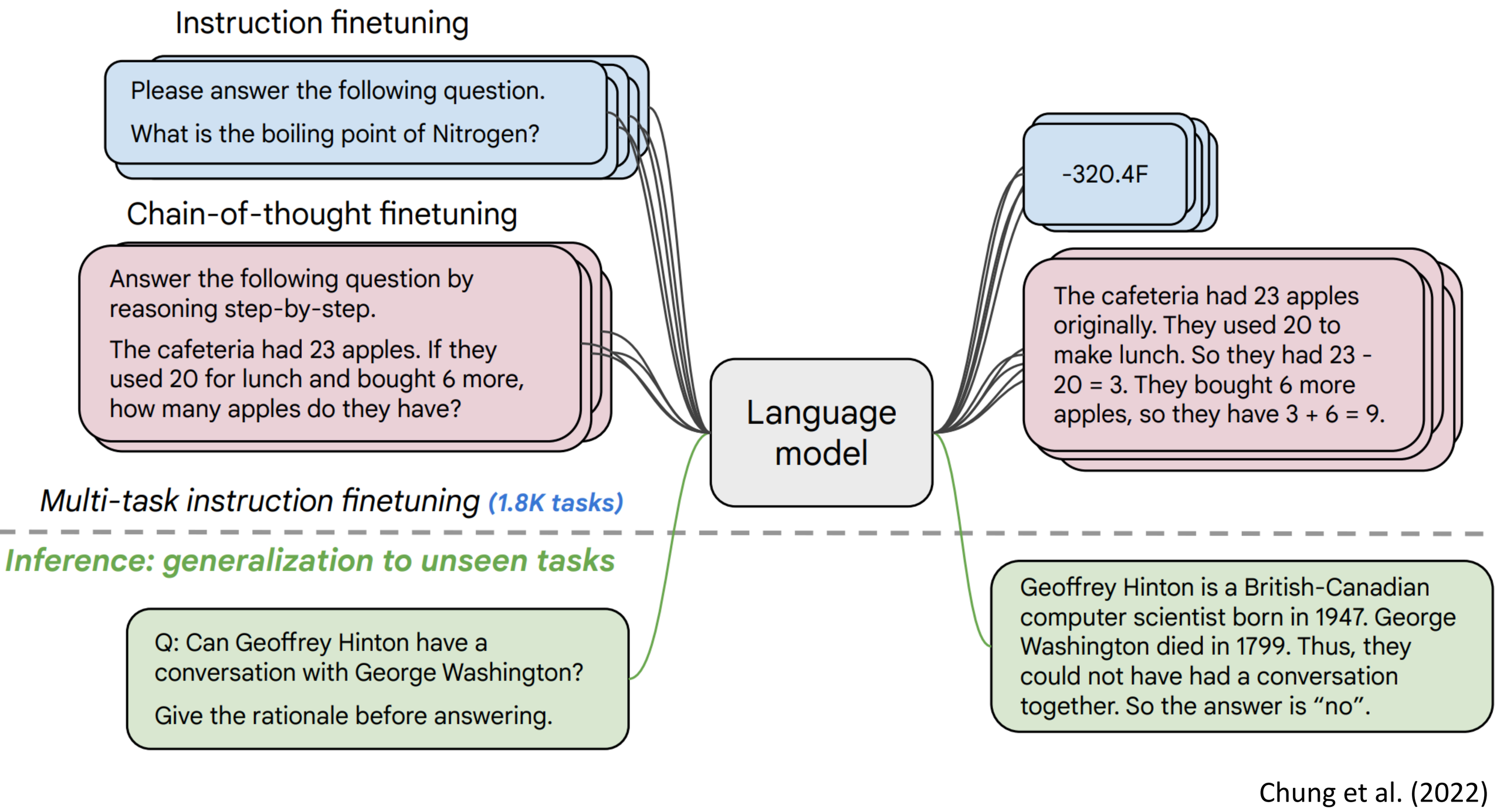
- Pre-train: T5 task
- Train: a collection of tasks with prompts. **This uses existing training data**
- Test: a new task specified only by a new prompt. **No training data in this task**



Sanh et al. (2021)

Flan-PaLM

- Flan-PaLM (October 20, 2022): 1800 tasks, 540B parameter model fine-tuned on many tasks after pre-training



Chung et al. (2022)

Flan-PaLM: Results

Model	Finetuning Mixtures	Tasks	Norm. avg.	MMLU		BBH	
				Direct	CoT	Direct	CoT
540B	None (no finetuning)	0	49.1	71.3	62.9	49.1	63.7
	CoT	9	52.6 (+3.5)	68.8	64.8	50.5	61.1
	CoT, Muffin	89	57.0 (+7.9)	71.8	66.7	56.7	64.0
	CoT, Muffin, T0-SF	282	57.5 (+8.4)	72.9	**68.2**	57.3	64.0
	CoT, Muffin, T0-SF, NIV2	1,836	**58.5** (+9.4)	**73.2**	68.1	**58.8**	**65.6**

- Human performance estimates are ~80 on Big-Bench (BBH)
- MMLU: multiple-choice test questions drawn from many disciplines
- Note: smaller 11B versions of these models are released (Flan-T5-11B); still a good choice for many tasks!

Chung et al. (2022)

RL from Human Feedback

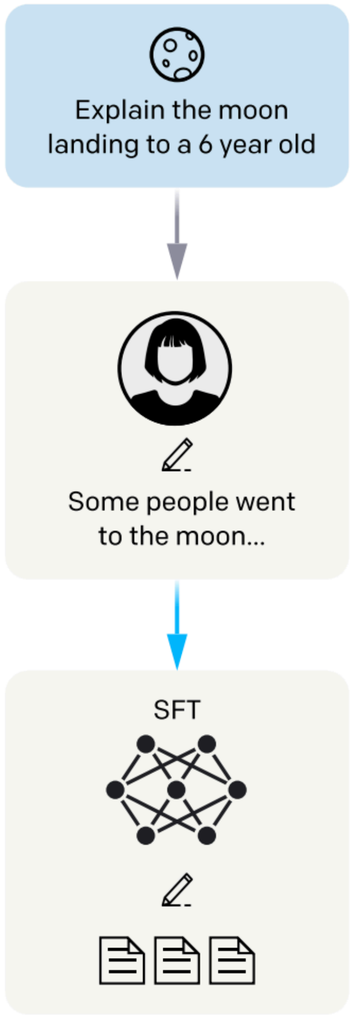
- Instruction tuning uses labeled data. Several limitations to this:
  - As models get larger, low-quality datasets will limit their capabilities
  - A model can't necessarily generalize to new tasks beyond those in the tuning datasets
- Alternative: can we generate outputs from state-of-the-art models on new problems, then get reward signal from humans? Potentially very flexible, scalable, and able to work on models at the cutting edge!
- **Reinforcement learning from human feedback** is a key component of ChatGPT and similar systems

RL from Human Feedback

- Fundamental idea: humans give **comparisons of two system outputs**. Looks different from standard reward in RL!

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

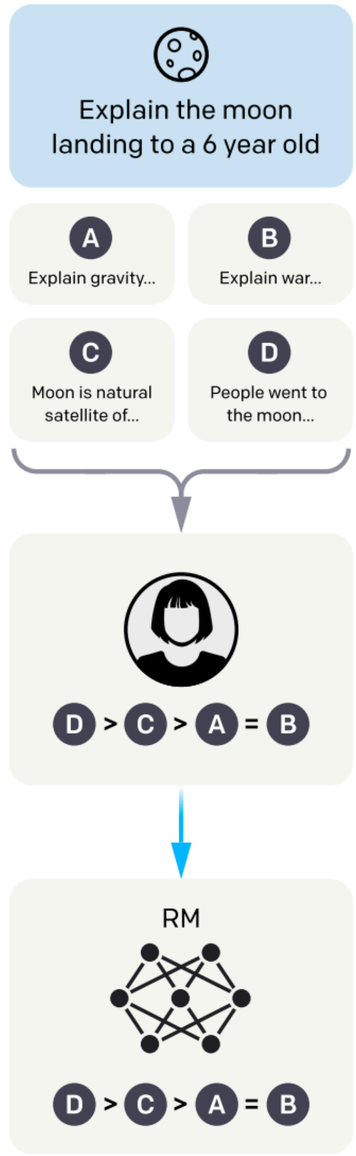


A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

Step 3 (not shown): Take reward model, do RL on it

Ouyang et al. (2022)

RLHF, Formally

- Base language model p(**y** | **x**) assigns probabilities to completions. Train this offline in advance
- Reward model r(**x**, **y**) maps completions y to real-valued scores
- Data for reward model: collect two LM completions ($\mathbf{y}_1$, $\mathbf{y}_2$) for a single input **x**. **x** can be almost anything as long as people will have preferences over what comes next!
- Annotators label $\mathbf{y}_1 \succ \mathbf{y}_2$ (prefer 1 to 2) or vice versa
- Learn *r* using a Bradley-Terry model over human preferences:

$$P(\mathbf{y}_1 \succ \mathbf{y}_2) = \frac{\exp(r(\mathbf{x}, \mathbf{y}_1))}{\exp(r(\mathbf{x}, \mathbf{y}_1)) + \exp(r(\mathbf{x}, \mathbf{y}_2))}$$

  - This turns scores into log probabilities of 1 being preferred to 2. Same as logistic regression where we classify pairs as 1 > 2 or 2 < 1, but we actually learn a continuous scoring function, not a classifier

Ouyang et al. (2022)

RLHF, Formally

- RL phase: do RL with PPO, optimize expected reward

$$\mathbb{E}_{\mathbf{x}\sim D,\mathbf{y}\sim p(\cdot|\mathbf{x})}\left[r\left(\mathbf{x},\mathbf{y}\right)\right]$$

  subject to an additional KL penalty that p not deviate too far from the base LM p

- Ideal scenario: p continually gets better and better, reward model can now judge those new, better completions and drive it to get better. This may be better than instruction tuning, which is "stuck" with the provided labeled data

Ouyang et al. (2022)

Reward Model Training

Table 1: Distribution of use case categories from our API prompt dataset.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Table 2: Illustrative prompts from our API prompt dataset. These are fictional examples inspired by real usage—see more examples in Appendix A.2.1.

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

- For OpenAI, RLHF data is collected from their API. **Very different data distribution from instruct-tuning datasets**

Ouyang et al. (2022)

History of GPT-3/3.5/ChatGPT variants

- text-davinci-001/002 were both learned only from fine-tuning on demonstrations rated 7/7 by humans (i.e., not using RLHF)
- text-davinci-003 (latest version) and ChatGPT both use PPO with learned reward models
- Conclusion: likely difficult to get PPO working reliably (or to get a good reward function — signal from annotators may be unstable)
  - …but RLHF datasets from OpenAI are not public
  - Data quality is paramount! Anecdotally there are lots of human-written demonstrations in there and lots of ratings

https://beta.openai.com/docs/model-index-for-researchers

Is RL necessary?

- A series of recent models (Alpaca, GPT4All, Vicuna, Guanaco, LIMA, Orca) achieve strong performance with supervised fine-tuning
- RL is often brittle, reward models may not be working as well as we'd like
- However, as of mid-2023, GPT-4 is far ahead of other systems. So while distilling from GPT-4 with fine-tuning may be possible, it's unclear whether we can build even stronger systems without RL

Factuality and Hallucination

- Language models model distributions over text, not facts. There's no guarantee that what they generate is factual:
  - Language models are trained on the web. Widely-popularized falsehoods may be reproduced in language models
  - A language model may not be able to store all rare facts, and as a result moderate probability is assigned to several options
- RLHF improves this (particularly the calibration of when the model answers versus saying "I don't know) but doesn't eliminate it. How can we detect factual errors in order to evaluate our systems?

Grounding LM Generations

- Suppose we have text generated from an LM. We want to check it against a source document. What techniques have we seen so far that can do this?

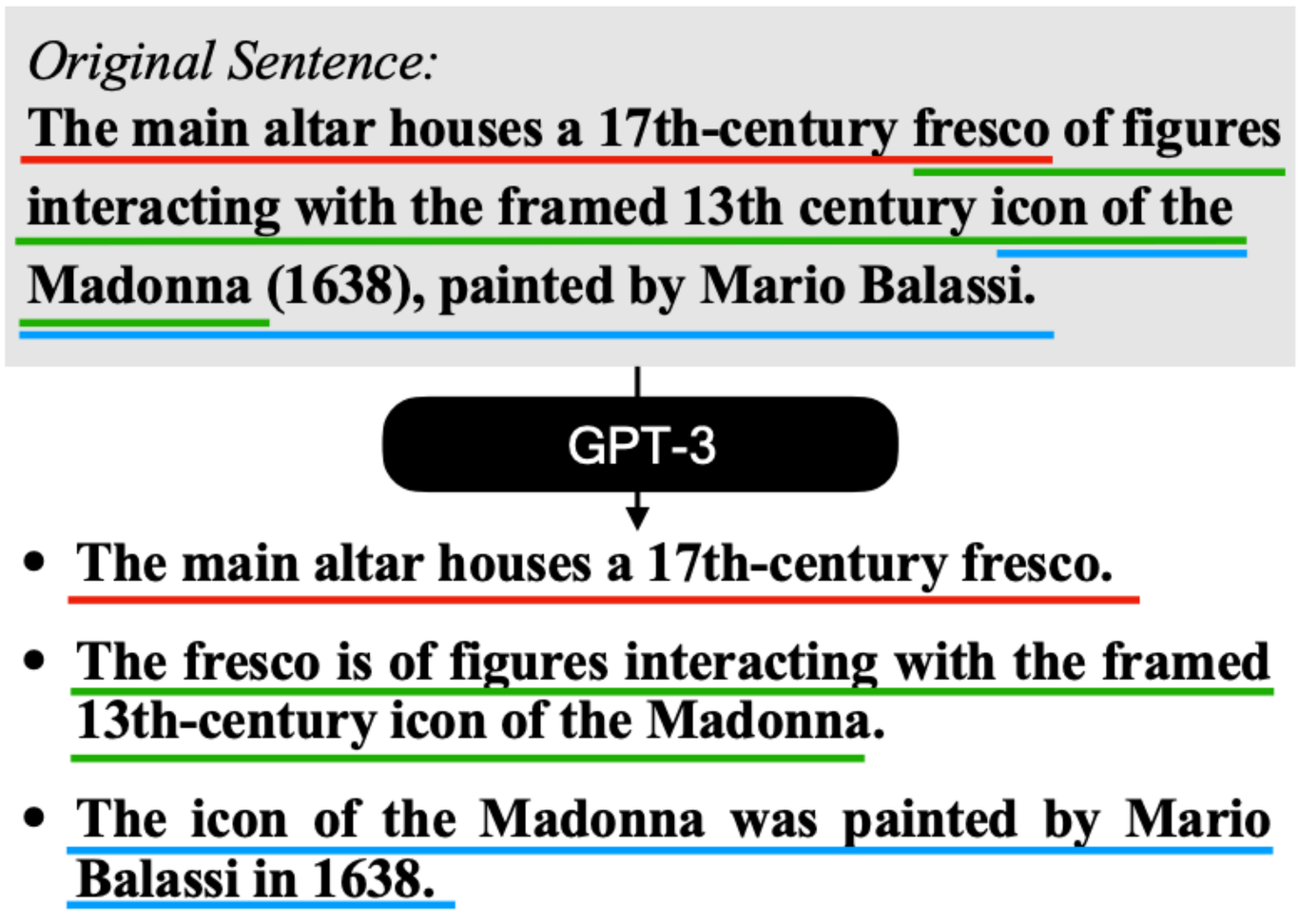
- What steps are involved?

    1. Decide what text you are grounding in (may involve retrieval)
    2. Decompose your text into pieces of meaning to ground
    3. Check each piece

- For now, we'll assume the reference text/documents are given to us and not focus on step 1

Step 2: Fact Decompositions

- Simplest approach: each sentence needs to be grounded
- Can go deeper: think of sentences as expressing a collection of propositions
- Long history in frame semantics of defining these propositions. Many propositions anchor to verbs
- Recent work: extract propositions with LLMs:

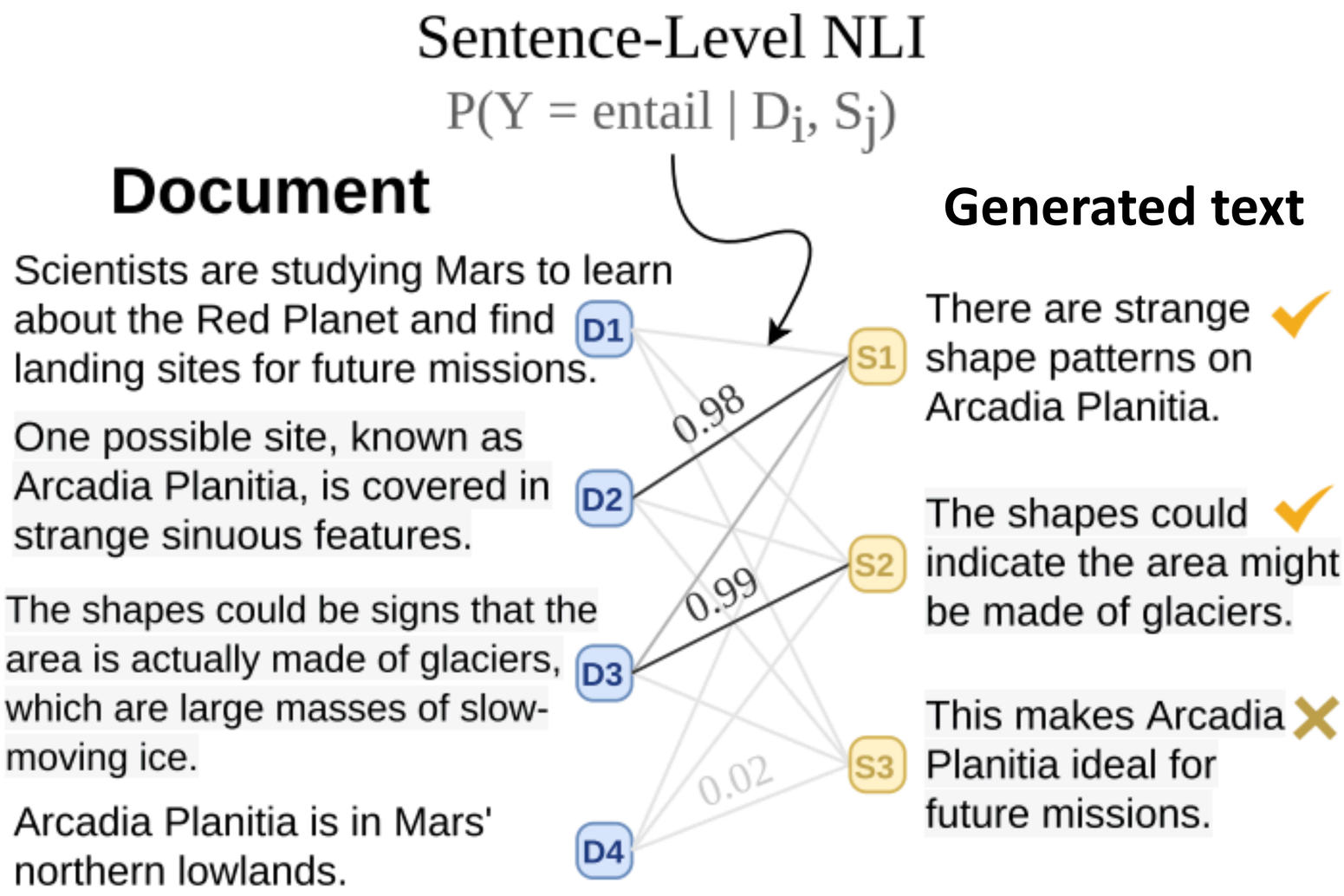


Yixin Liu et al. (2023)

Ryo Kamoi et al. (2023)

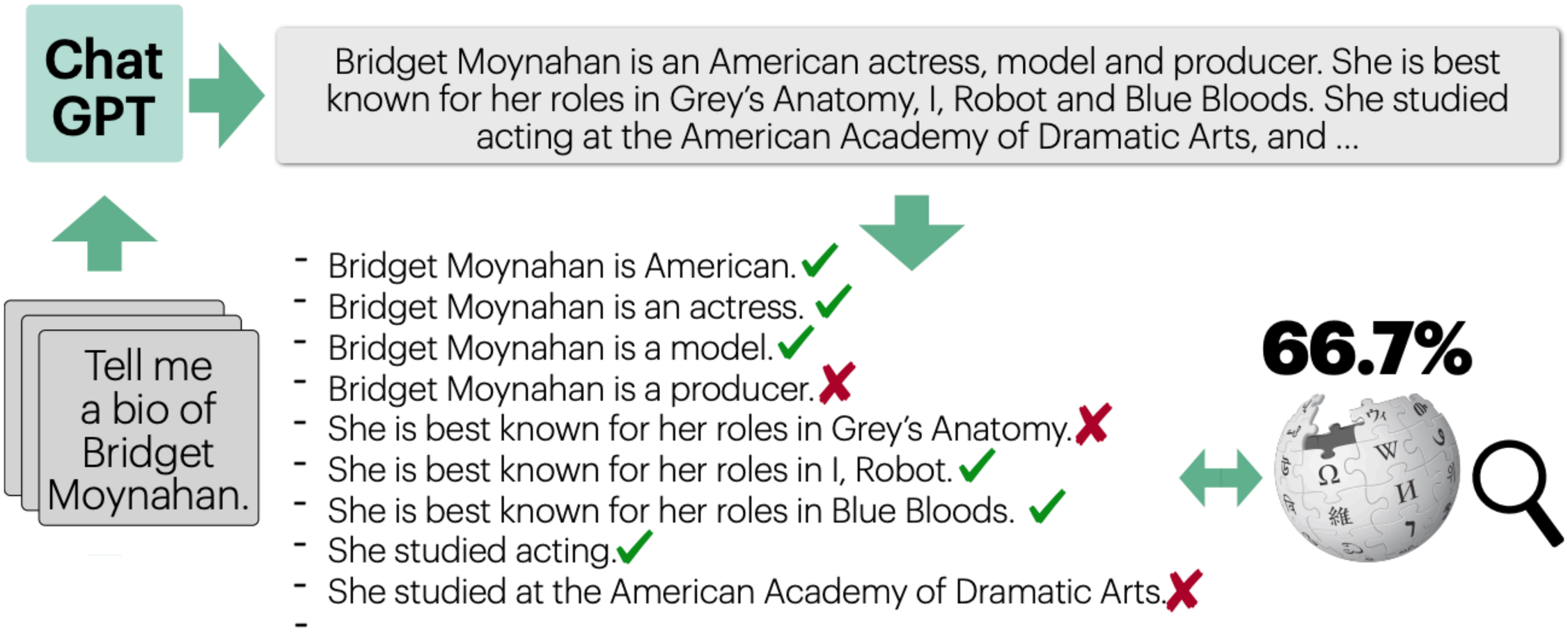
Step 3: Checking

- One idea: use textual entailment to see if each piece to check is entailed by the source
- Simple version that originated in summarization: take the max entailment score over every document sentence



Philippe Laban et al. (2022)

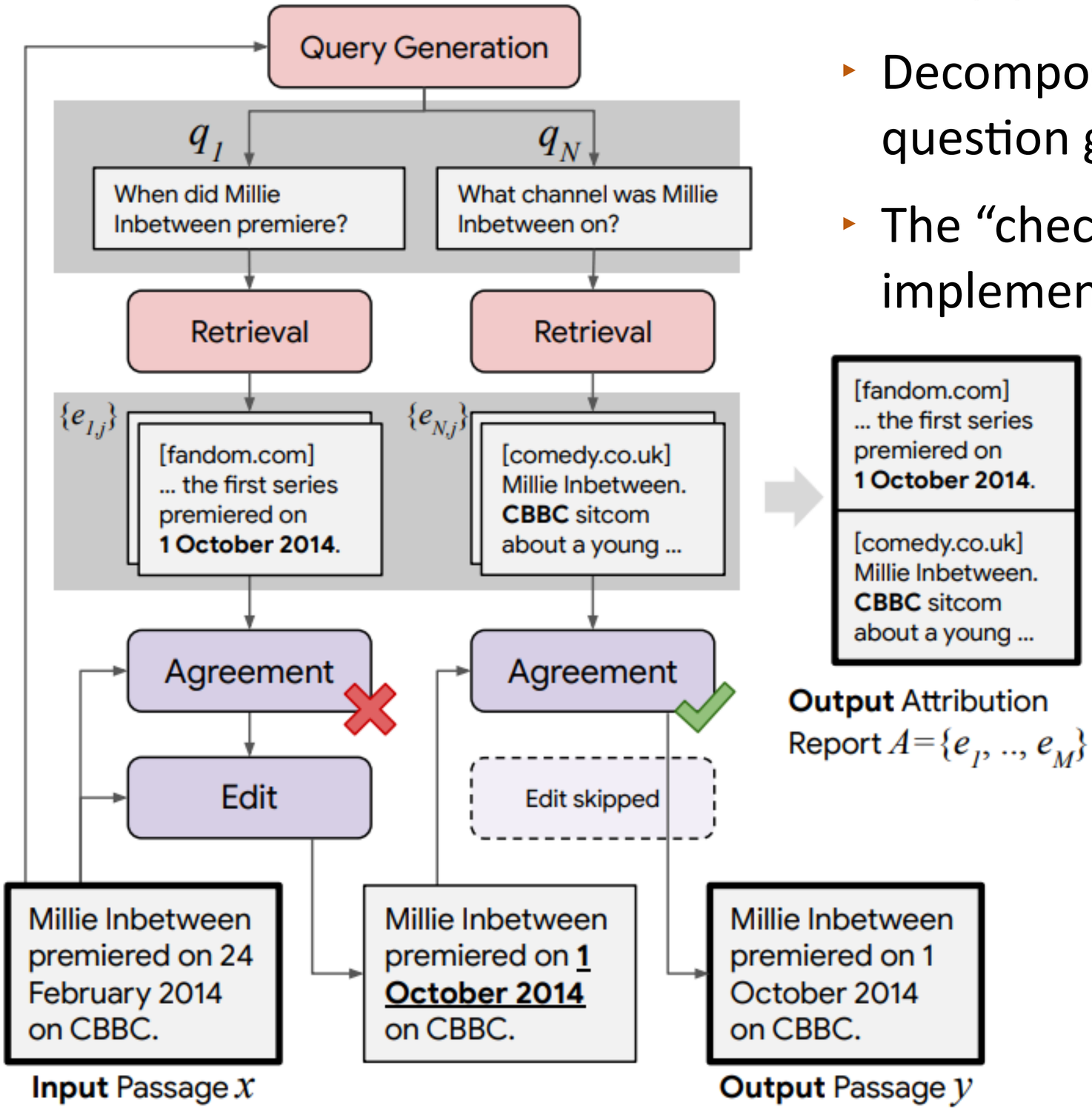
FActScore



- Dataset: ChatGPT-generated biographies of people. May contain errors, particularly when dealing with obscure peole!
- Uses LLMs both for decomposition and for checking

Sewon Min and Kalpesh Krishna et al. (2023)

Pipeline: RARR



- Full pipeline including retrieval
- Decomposition is framed as question generation
- The “checking” stage is also implemented with LLMs here

Luyu Gao et al. (2022)

WEEK 10

Explanations

Segments in this section

seg-80.pdf - Interpretability: Motivation & Types p. 314
seg-81.pdf - Local Explanations (LIME, Gradients) p. 318
seg-82.pdf - Text Explanations p. 326
seg-83.pdf - Probing p. 332
seg-84.pdf - Dataset Bias & Annotation Artifacts p. 335
seg-expl6-cot.pdf - Chain-of-Thought p. 341
seg-expl7-cot-analysis.pdf - Chain-of-Thought Analysis & Tool Use p. 351

Key points (added notes)

- Local explanations: LIME, gradients, integrated gradients. Text explanations can be plausible but unfaithful.
- Probing classifiers; dataset artifacts (hypothesis-only baselines).
- Chain-of-thought prompting; program-aided reasoning and tools.

Post-midterm material.

Explanations in NLP

- Neural models in NLP have complex behavior. How can we understand them?
- QA: why did the model prefer *Stewart* over *Devin Funchess*?

QID: 1f4b668a0343453b9d4bf3edc86daf63
Question: who caught a 16-yard pass on this drive ?
Answer: devin funchess

Start Distribution

there would be no more scoring in the third quarter , but early in the fourth , the broncos drove to the panthers 41-yard line . on the next play , ealy knocked the ball out of manning 's hand as he was winding up for a pass , and then recovered it for carolina on the 50-yard line . a 16-yard reception by devin funchess and a 12-yard run by stewart then set up gano 's 39-yard field goal , cutting the panthers deficit to one score at 16â€“10 . the next three drives of the game would end in punts .

Lipton (2016)

Explanations in NLP

- Neural models in NLP have complex behavior. How can we understand them?
- Sentiment:

	DAN	Ground Truth
this movie was not good	negative	negative
this movie was good	positive	positive
this movie was bad	negative	negative
the movie was not bad	negative	positive

- Left side: predictions model makes on individual words
- Tells us how these words combine

Iyyer et al. (2015)

Why explanations?

- **Trust:** if we see that models are behaving in human-like ways and making human-like mistakes, we might be more likely to trust them and deploy them

- **Causality:** if our classifier predicts class *y* because of input feature *x*, does that tell us that *x* causes *y*? Not necessarily, but it might be helpful to know

- **Informativeness:** more information may be useful (e.g., predicting a disease diagnosis isn't that useful without knowing more about the patient's situation)

- **Fairness:** ensure that predictions are non-discriminatory

Lipton (2016)

What are explanations?

- Some models are naturally **transparent**: we can understand why they do what they do (e.g., a decision tree with <10 nodes)

- Explanations of more complex models

  - **Local explanations:** highlight what led to this classification decision. (Counterfactual: if they were different, the model would've predicted a different class)

  - **Text explanations:** describe the model's behavior in language

  - **Model probing:** auxiliary tasks, challenge sets, adversarial examples to understand more about how our model works

Lipton (2016); Belinkov and Glass (2018)

Local Explanations

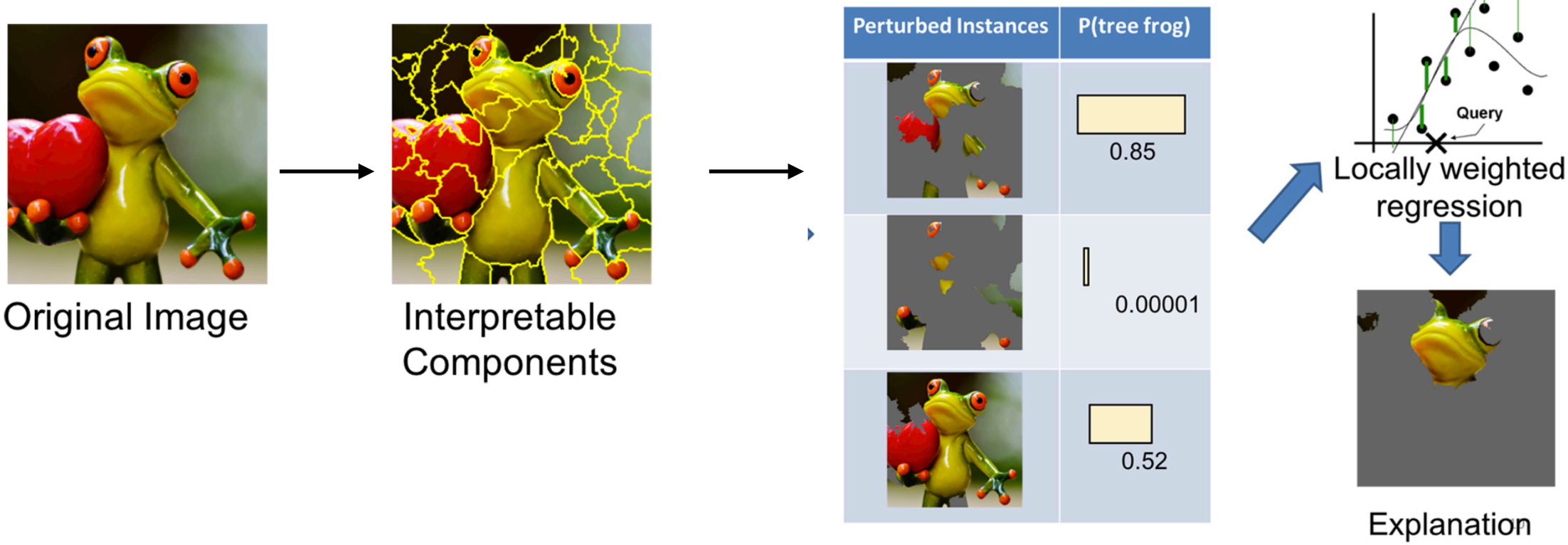
- An explanation should help us answer counterfactual questions: if the input were ***x***' instead of ***x***, what would the output be?

	Model
that movie was not great , in fact it was terrible !	—
that movie was not ____ , in fact it was terrible !	—
that movie was not great , in fact it was _____ !	+

- Perturb input many times and assess the impact on the model's prediction

LIME

- LIME: Locally-Interpretable Model-Agnostic Explanations
  - *Local* because we'll focus on this one example
  - *Model-agnostic*: treat model as black box



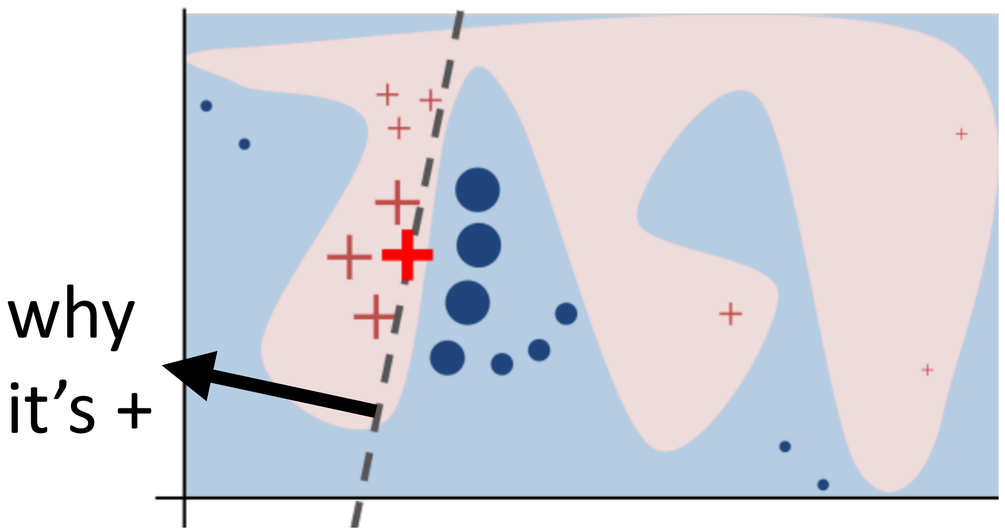
- Check predictions on subsets of components
- Train a model to explain which components yield the model's preds

Ribeiro et al. (2016)

https://www.oreilly.com/learning/introduction-to-local-interpretable-model-agnostic-explanations-lime

LIME

- Break down input into many small pieces for interpretability
  $x \in \mathbb{R}^d \rightarrow x' \in \{0,1\}^{d'}$
- Draw samples by using x’ as a mask to form a new example x’’. Compute f(x’’)
- Now learn a model to predict f(x’’) based on x’. This model’s weights will serve as the explanation for the decision

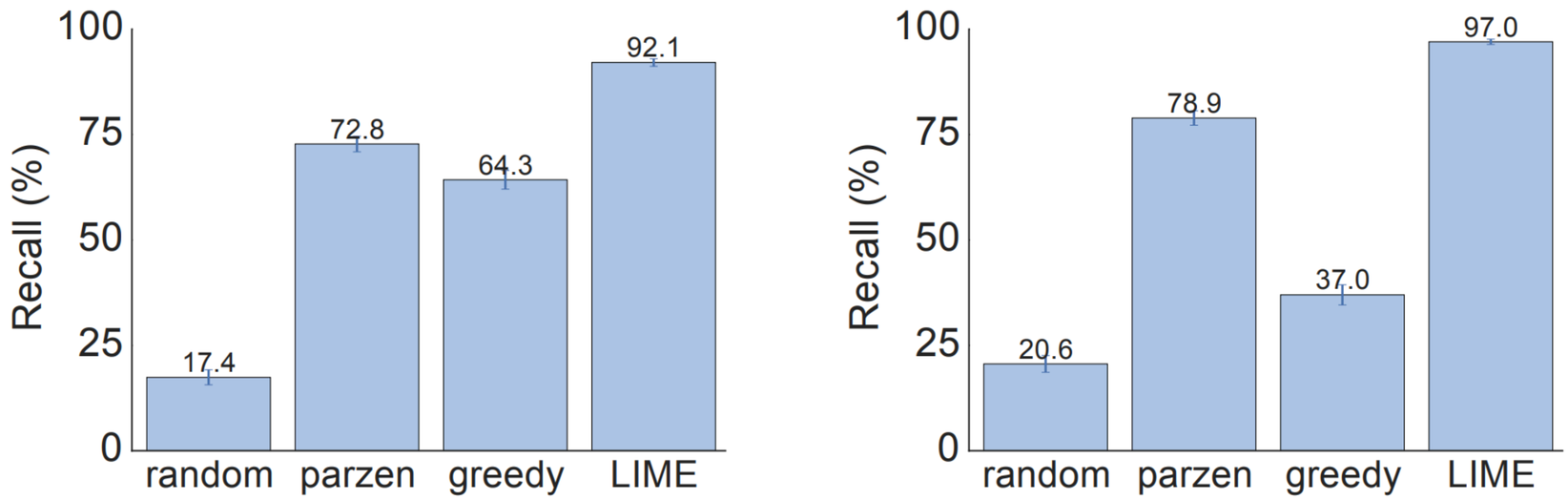


- If the pieces are very coarse, can interpret but can’t learn a good model of the boundary. If pieces are too fine-grained, can interpret but not predict

Ribeiro et al. (2016)

https://www.oreilly.com/learning/introduction-to-local-interpretable-model-agnostic-explanations-lime

LIME



(a) Sparse LR (b) Decision Tree

Figure 6: Recall on truly important features for two interpretable classifiers on the books dataset.

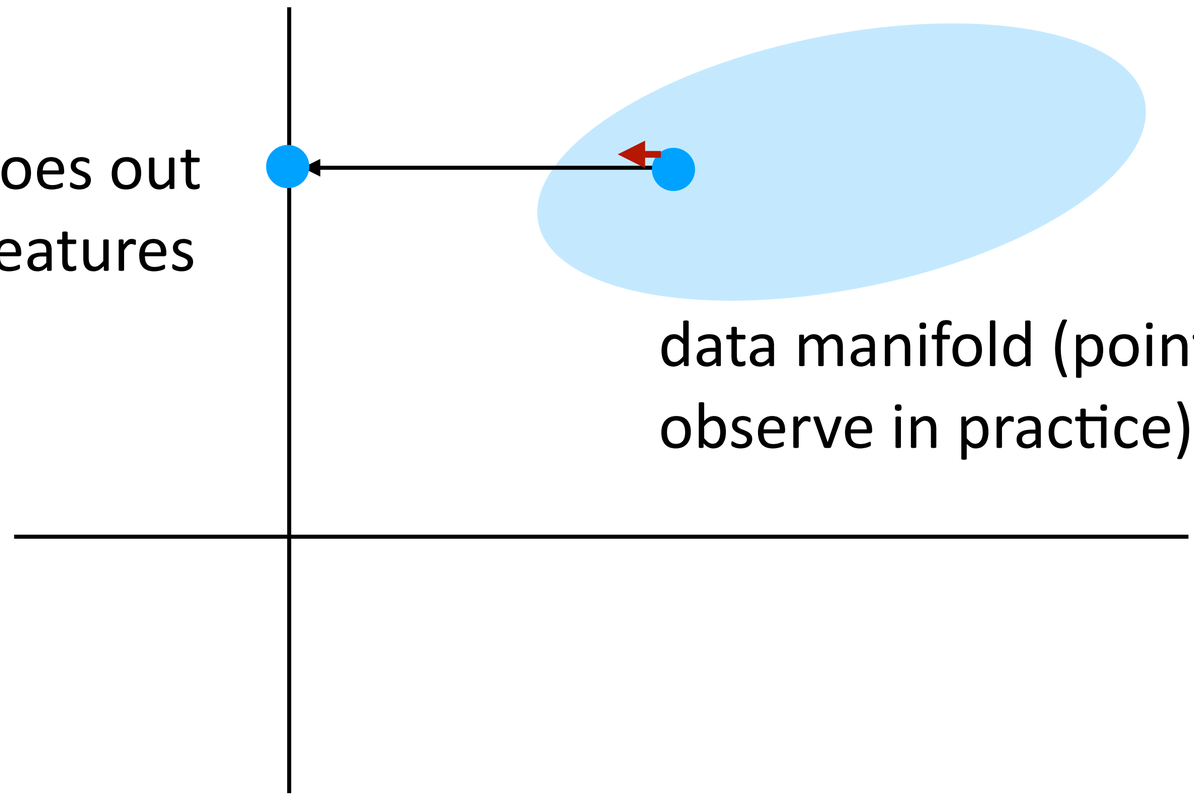
- Evaluation: the authors train a sparse model (only looks at 10 features of each example), then try to use LIME to recover the features. Greedy: remove features to make predicted class prob drop by as much as possible

Ribeiro et al. (2016)

https://www.oreilly.com/learning/introduction-to-local-interpretable-model-agnostic-explanations-lime

Gradient-based Methods

- Problem: fully removing pieces of the input may cause it to be very unnatural



- Alternative approach: look at what this perturbation does locally right around the data point using gradients

Gradient-based Methods

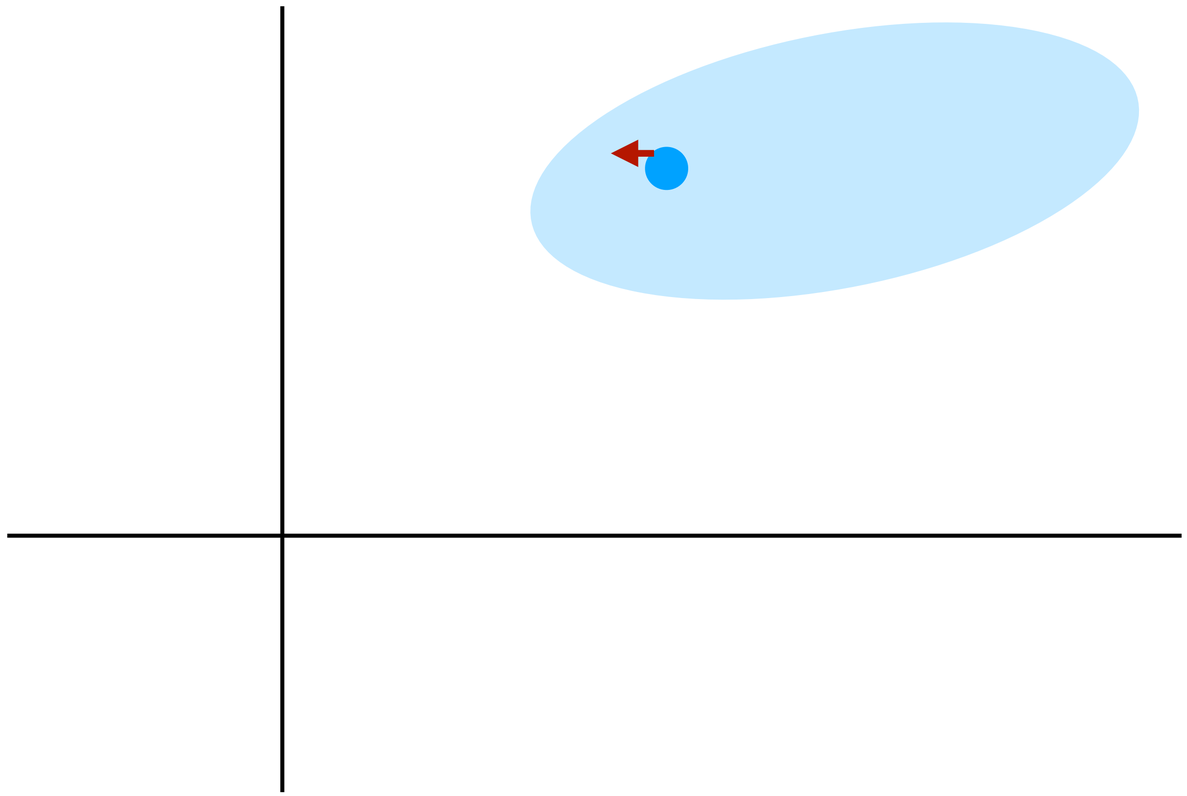
- Originally used for images
- Approximate score with a first-order Taylor series approximation around the current data point

S_c = score of class c

I_0 = current image

$$S_c(I) \approx w^T I + b$$

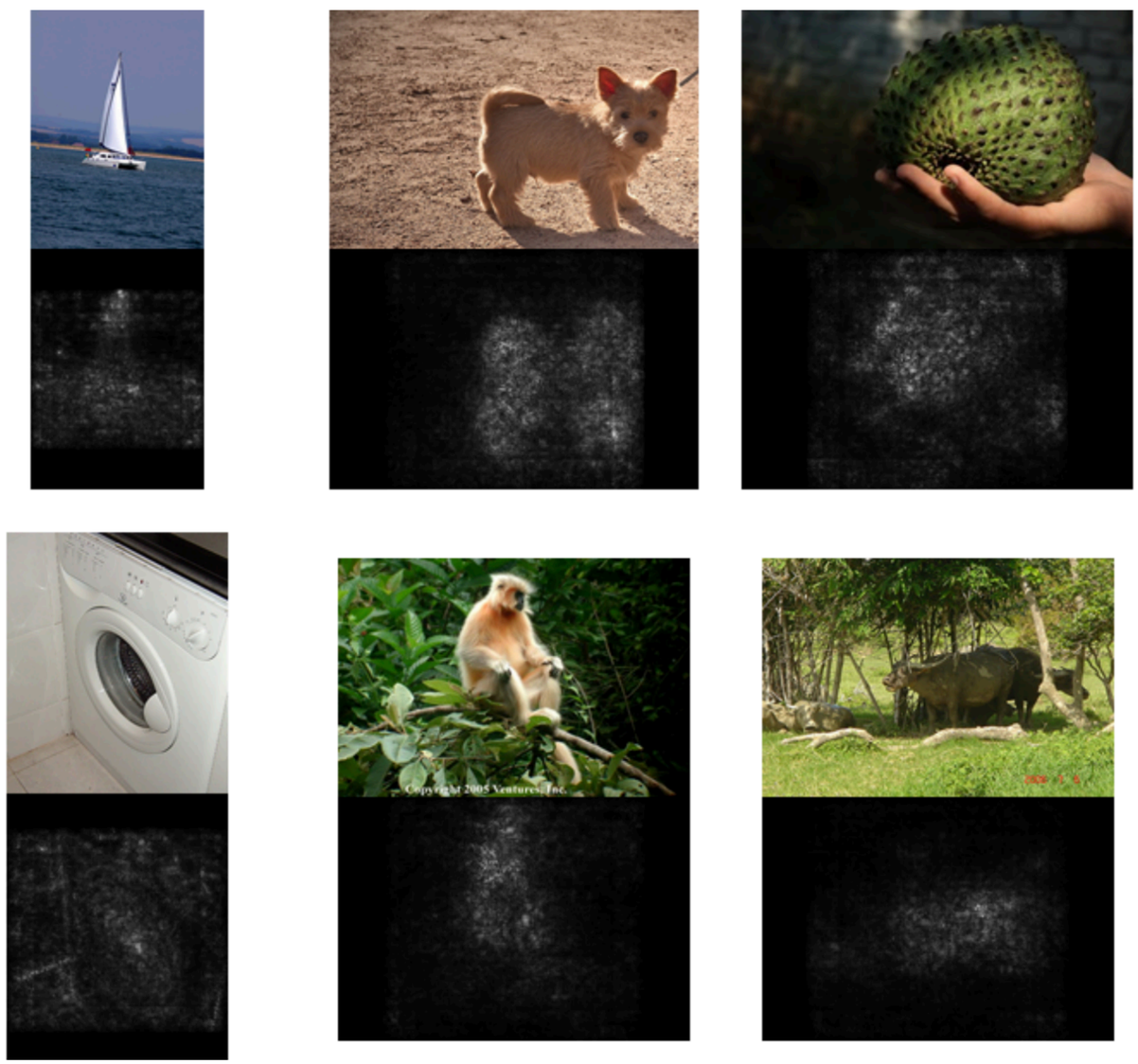
$$w = \left. \frac{\partial S_c}{\partial I} \right|_{I_0}$$



- Higher gradient magnitude = small change in pixels leads to large change in prediction

Simonyan et al. (2013)

Gradient-based Methods

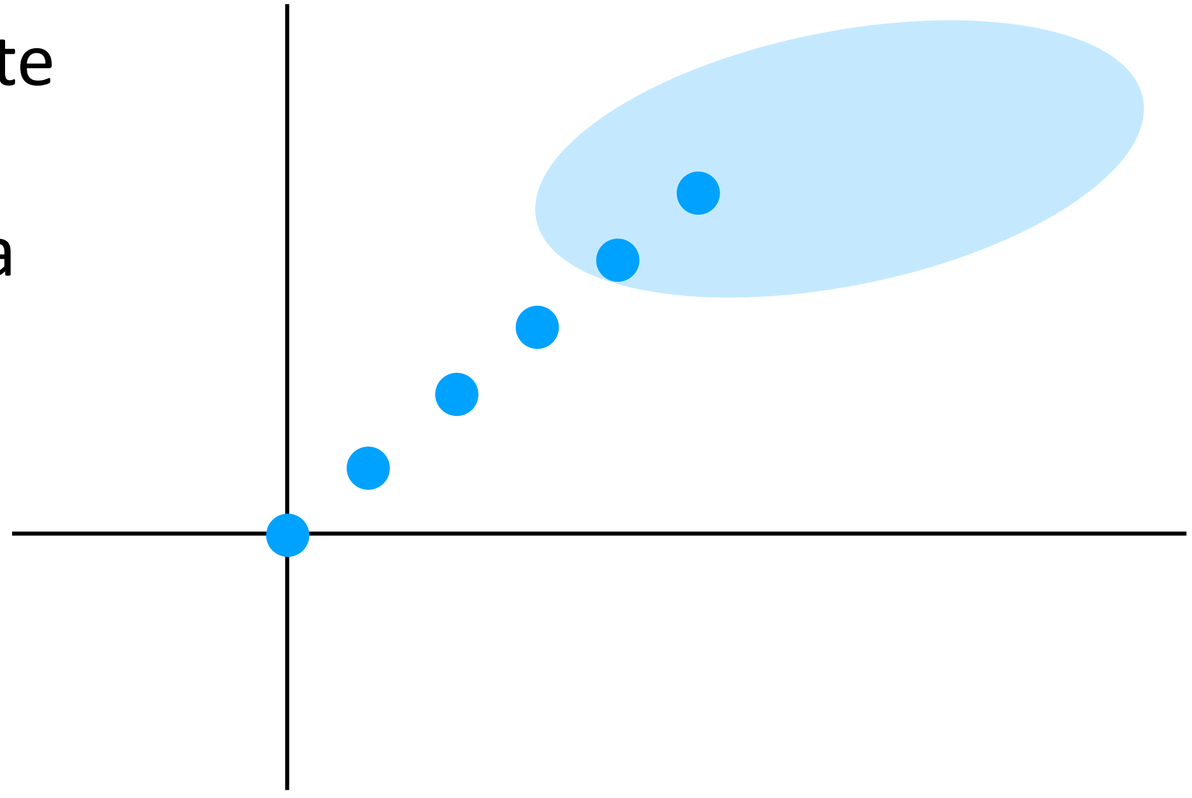


Simonyan et al. (2013)

Integrated Gradients

- Suppose you have prediction = A OR B for features A and B. Changing either feature doesn't change the prediction, but changing both would. Gradient-based method says neither is important

- Integrated gradients: compute gradients along a path from the origin to the current data point, aggregate these to learn feature importance



- Now at intermediate points, increasing "partial A" or "partial B" reveals the importance of A and B

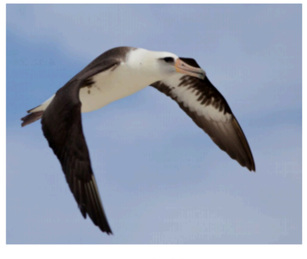
Sundararajan et al. (2017)

Text-based Explanations

- Can we generate a natural language explanation of a model's behavior?
- Possible advantages:
  - Easy for untrained users to understand
  - Easy for annotators to provide ground truth human explanations (which may also help our models)
- Possible disadvantages:
  - Hard to generate grammatical/semantically meaningful text
  - Can text truly explain a model's behavior?

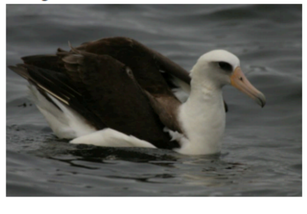
Explanations of Bird Classification

Laysan Albatross



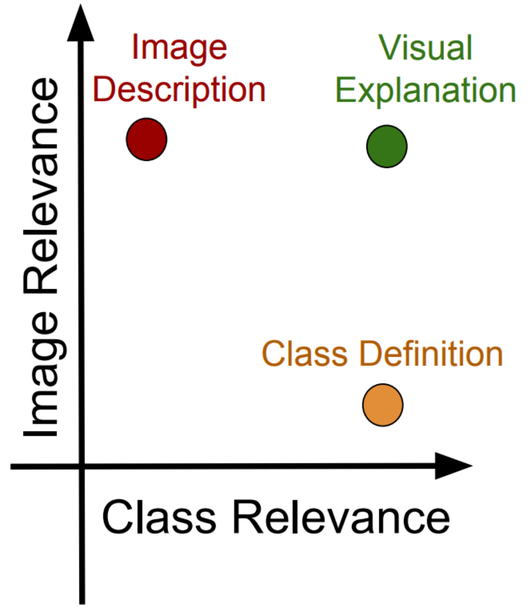
Description: This is a large flying bird with black wings and a white belly.
Class Definition: The *Laysan Albatross* is a large seabird with a hooked yellow beak, black back and white belly.
Visual Explanation: This is a *Laysan Albatross* because this bird has a large wingspan, hooked yellow beak, and white belly.

Laysan Albatross



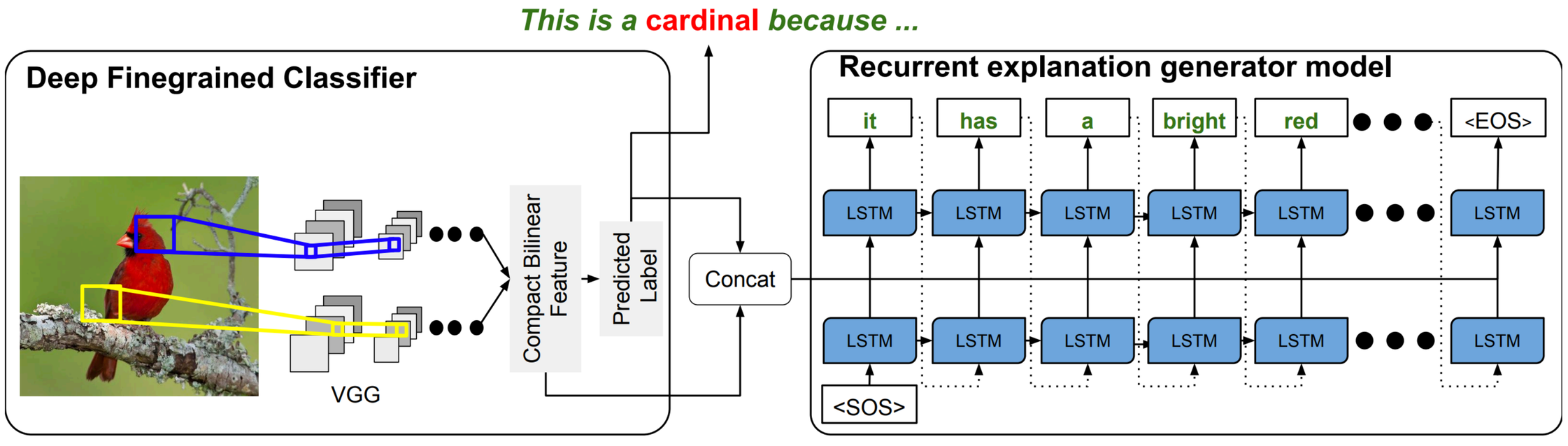
Description: This is a large bird with a white neck and a black back in the water.
Class Definition: The *Laysan Albatross* is a large seabird with a hooked yellow beak, black back and white belly.
Visual Explanation: This is a *Laysan Albatross* because this bird has a hooked yellow beak white neck and black back.

- What makes a visual explanation? Should be relevant to the class and the image
- Are these features *really* what the model used?



Hendricks et al. (2016)

Explanations of Bird Classification



- Are these features *really* what the model used? The decoder looks at the image, but what it reports may not truly reflect the model's decision-making
- More likely to produce plausible (look good to humans) but unfaithful explanations!

Hendricks et al. (2016)

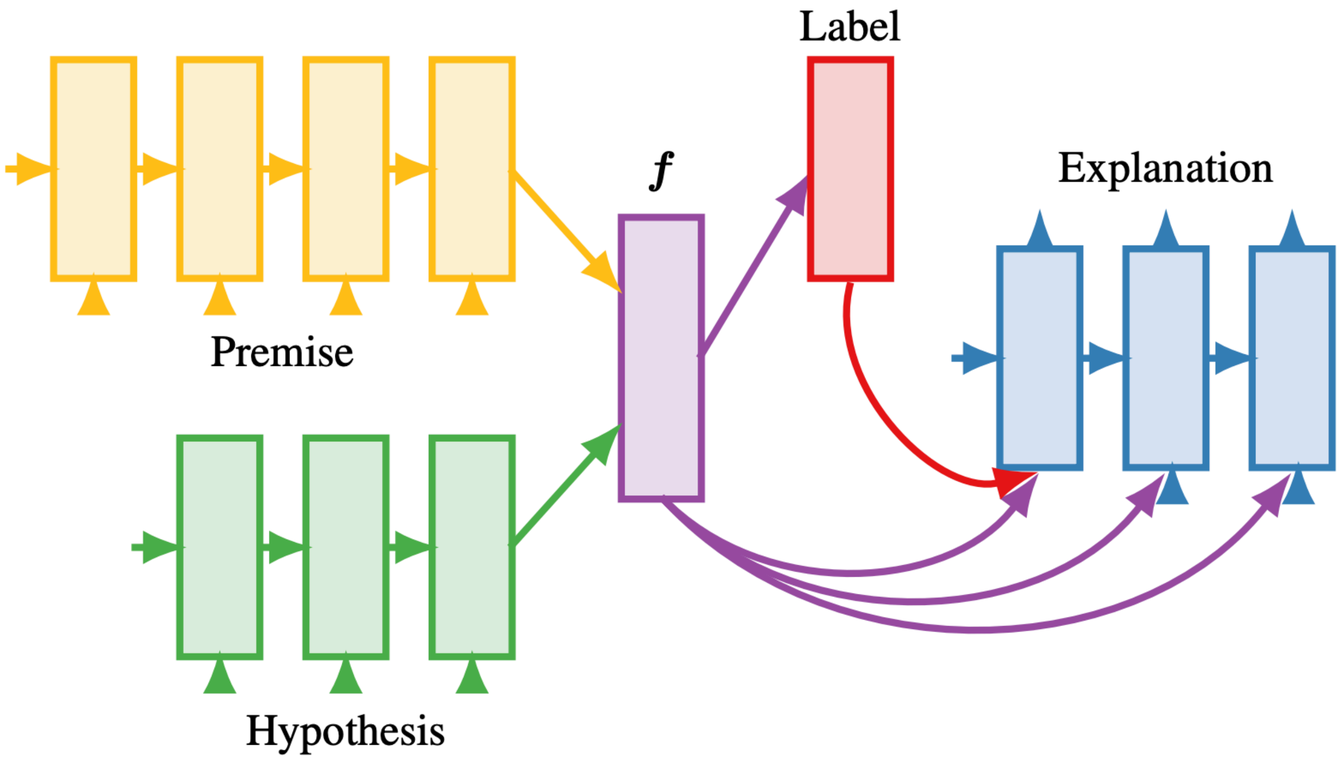
e-SNLI

Premise: An adult dressed in black holds a stick. Hypothesis: An adult is walking away, empty-handed. Label: contradiction Explanation: Holds a stick implies using hands so it is not empty-handed.
Premise: A child in a yellow plastic safety swing is laughing as a dark-haired woman in pink and coral pants stands behind her. Hypothesis: A young mother is playing with her daughter in a swing. Label: neutral Explanation: Child does not imply daughter and woman does not imply mother.
Premise: A man in an orange vest leans over a pickup truck. Hypothesis: A man is touching a truck. Label: entailment Explanation: Man leans over a pickup truck implies that he is touching it.

- e-SNLI: natural language inference with explanations

Camburu et al. (2019)

e-SNLI

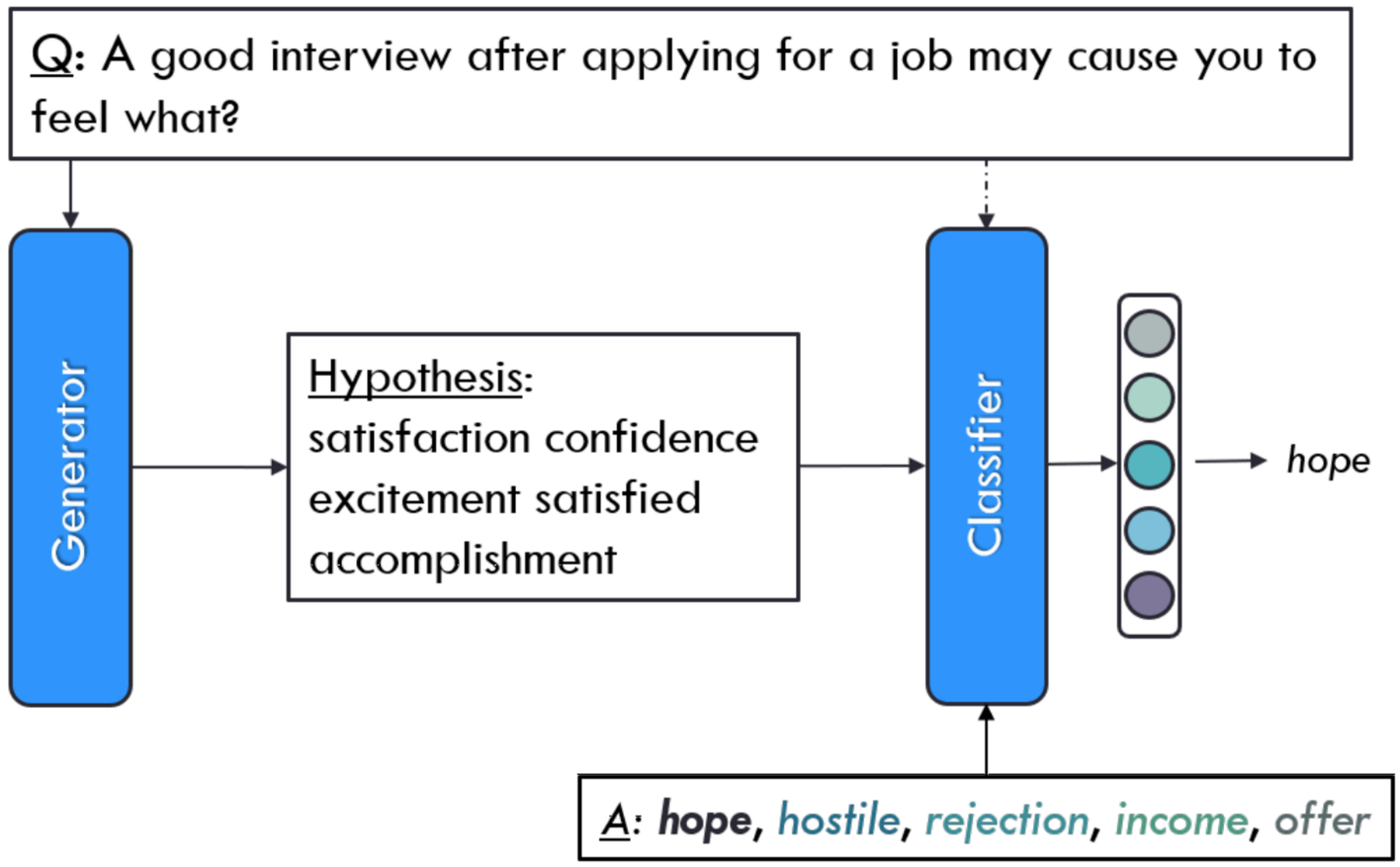


f = function of premise and hypothesis vectors

- Similar to birds: explanation is generated conditioned on the label and the network state f
- Information from f is fed into the explanation LSTM, but **no constraint that this must be used**. Explanation might be purely generated from the label

Camburu et al. (2019)

Latent Textual Explanations

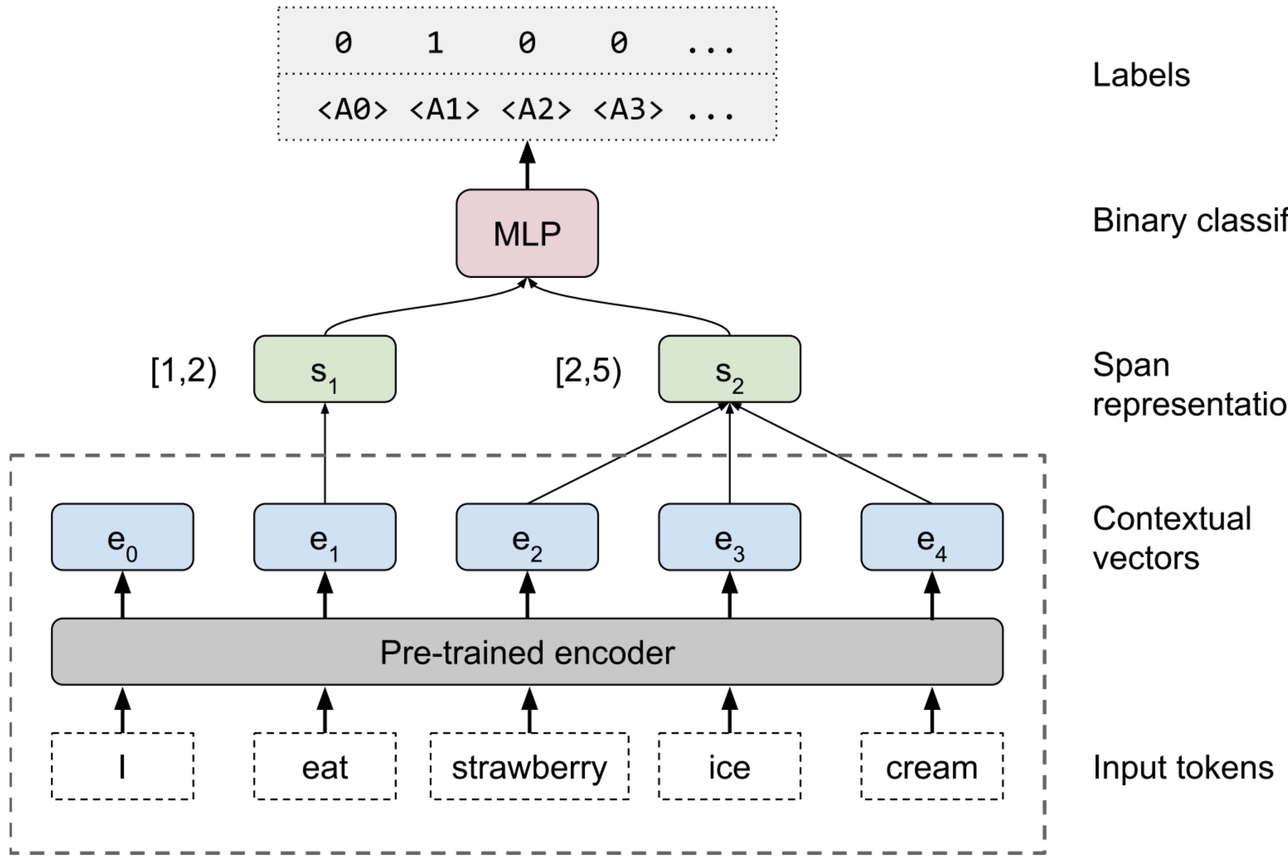


- Model generates text “hypothesis”, which is completely latent
- Hypothesis isn’t constrained to be natural language, ends up being keywords

Latcinnik and Berant (2020)

Probing

- We want to know what information is captured in a neural network. Try to predict that information from the network's representations



- Given a simple, fixed class of model (e.g., one-layer FFNN), how well can we predict various things from word representations?

Tenney et al. (2019)

Probing: Results

- Lex: baseline built on context-independent vectors
- Large gains from contextualization, and BERT beats ELMo

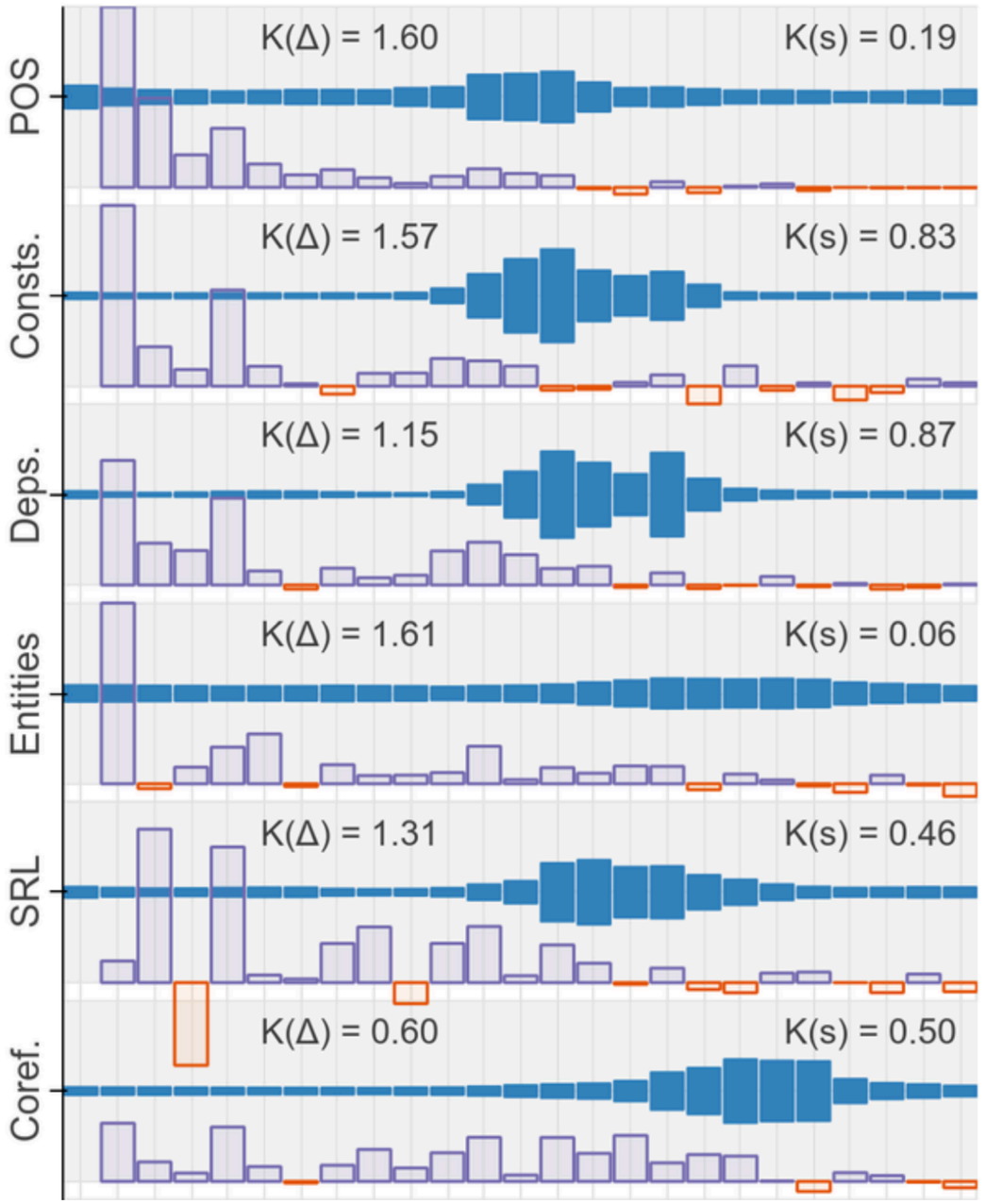
	BERT-base F1 Score			Abs. Δ
	Lex.	**cat**	**mix**	**ELMo**
Part-of-Speech	88.4	**97.0**	96.7	0.0
Constituents	68.4	83.7	86.7	2.1
Dependencies	80.1	93.0	95.1	1.1
Entities	90.9	96.1	96.2	0.6
SRL (all)	75.4	89.4	91.3	1.2
Core roles	*74.9*	*91.4*	*93.6*	*1.0*
Non-core roles	*76.4*	*84.7*	*85.9*	*1.8*
OntoNotes coref.	74.9	88.7	90.2	6.3
SPR1	79.2	84.7	**86.1**	1.3
SPR2	81.7	83.0	**83.8**	0.7
Winograd coref.	54.3	53.6	54.9	1.4
Rel. (SemEval)	57.4	78.3	82.0	4.2
Macro Average	75.1	84.8	86.3	1.9

Tenney et al. (2019)

Probing: Results

- Purple: BERT-large performance on each task (as delta from mean) using representations from that layer of BERT

- Earlier layers of the network: better at POS and low-level tasks. Later layers are better at higher-level tasks like coreference



Tenney et al. (2019b)

Annotation Artifacts

- Some datasets might be easy because of how they're constructed
- Understanding our datasets is just as important as understanding our models in order to know what they're learning

Gururangan et al. (2018); Poliak et al. (2018)

Task: Natural Language Inference

Premise	A woman selling bamboo sticks talking to two men on a loading dock.
Entailment	There are **at least** three **people** on a loading dock.
Neutral	A woman is selling bamboo sticks **to help provide for her family.**
Contradiction	A woman is **not** taking money for any of her sticks.

- To create neutral sentences: annotators *add information*
- To create contradictions: annotators *add negation*
- Models can do very well *without looking at the premise*

SNLI	69.17	33.82	+35.35
MNLI-1	55.52	35.45	+20.07
MNLI-2	55.18	35.22	+19.96

Performance of models that only look at the hypothesis:
~70% on 3-class SNLI dataset

Gururangan et al. (2018); Poliak et al. (2018)

What do we do about this?

- Why is this a problem? Because our models learn these simple cues and not actually the hard task we want them to learn

- They don’t generalize to challenging new examples without these patterns — understanding this behavior is crucial to explaining what our models are doing!

- Solutions: build harder tasks, tweak data or training objective to inoculate models against this (many proposals)

Other Tasks: Question Answering

- Question type is very powerful indicator. Only a couple of locations in this context!

QID: 57c5914570fc4995b2b9daa3e5dffb83
Question: where did luther spend his career ?
Answer: university of wittenberg

Start Distribution

on 19 october 1512 , he was awarded his doctor of theology and , on 21 october 1512 , was received into the senate of the theological faculty of the university of wittenberg , having been called to the position of doctor in bible . he spent the rest of his career in this position at the university of wittenberg .

Mudrakarta et al. (2018)

Other Tasks: Multi-hop QA

Question: *The Oberoi family is part of a hotel company that has a head office in what city?*

Doc 1

The Oberoi family is part of a hotel company that is famous for its involvement in hotels, namely through The Oberoi Group . . .

High lexical overlap

Doc 2

The Oberoi Group is a hotel company with its head office in Delhi.

. . .

Chen and Durrett (2019)

Other Tasks: SWAG

On stage, a woman takes a seat at the piano. She

a) sits on a bench as her sister plays with the doll.
b) smiles with someone as the music plays.
c) is in the crowd, watching the dancers.
d) nervously sets her fingers on the keys.

- Answers were filtered using ELMo to only leave answers that were difficult for a model to reject…but then BERT solved this dataset easily

Zellers et al. (2018)

Chain-of-Thought

- Most explanations we've seen are about interpreting models
- Chain-of-thought: prompting technique for **using explanations to improve model performance**, particularly for complex reasoning tasks
- Basic idea: the language model can "work through" different types of computation over multiple timesteps of inference, rather than needing to generate an answer immediately

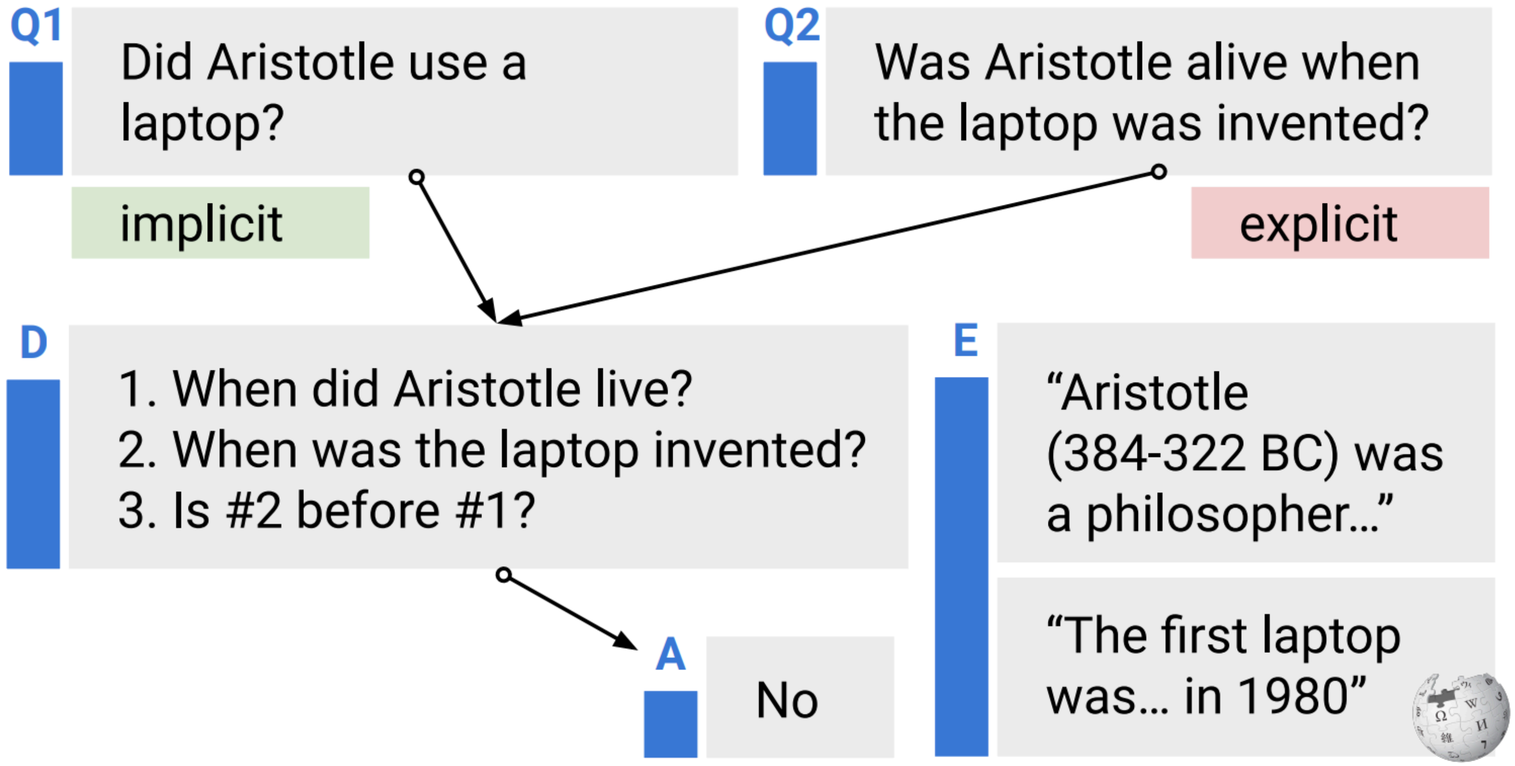
Rationales as “Programs”

Problem 2:
Question: From a pack of 52 cards, two cards are drawn together at random. What is the probability of both the cards being kings?
Options: A) 2/1223 B) 1/122 C) 1/221 D) 3/1253 E) 2/153
Rationale: Let s be the sample space.
Then n(s) = 52C2 = 1326
E = event of getting 2 kings out of 4
n(E) = 4C2 = 6
P(E) = 6/1326 = 1/221
Answer is C
Correct Option: C

- Rationales are most useful for problems where some computation is required. They can articulate the intermediate steps needed to solve it
- Some of the earliest work: math word problems

Wang Ling et al. (2017)

Rationales as “Programs”



- “StrategyQA”: dataset where different reasoning strategies are needed
- Related to multi-hop QA: “*What’s the capital of the country where Aristotle lived?*” (but these are easy with current models)

Mor Geva et al. (2021)

Chain-of-Thought

- For these kinds of problems, do “computation” entirely in natural language
- Unifies several ideas:
  - For math: relies on the fact that LLMs can at least do single steps of arithmetic okay
  - For QA: many problems involve reasoning decompositions
    E.g., *What’s the capital of the country where Aristotle lived?* ->
    ans = “*country where Aristotle lived*”
    return *What’s the capital of* [ans]
  - For other tasks: capture the kinds of behavior written in rationales

Jason Wei et al. (2022)

Chain-of-Thought

- Chain-of-thought is usually a few-shot prompting technique where the in-context examples now contain explanations
- Answer is not generated in one go, but comes after an explanation that "talks through" the reasoning

Input:

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

...

Q: John takes care of 10 dogs. Each dog takes .5 hours a day to walk and take care of their business. How many hours a week does he spend taking care of dogs?
A:

Model output:

John takes care of 10 dogs. Each dog takes .5 hours a day to walk and take care of their business. So that is 10 x .5 = 5 hours a day. 5 hours a day x 7 days a week = 35 hours a week. The answer is 35 hours a week. ✔

Jason Wei et al. (2022)

Chain-of-Thought

From our work: a synthetic test of multi-hop reasoning with extractive explanations:

Context*: Christopher agrees with Kevin. Tiffany agrees with Matthew. Mary hangs out with Danielle. James hangs out with Thomas. Kevin is a student. Matthew is a plumber. Danielle is a student. Thomas is a plumber.*

Q*: Who hangs out with a student?*

A*: Mary.*

Explanation*: because Mary hangs out with Danielle and Danielle is a student.*

Xi Ye and Durrett (2022)

Chain-of-Thought

Context*: Christopher agrees with Kevin.* […] **Q***: Who hangs out with a student?*

Mary

Standard few-shot learning, no explanation

Context*: Christopher agrees with Kevin.* […] **Q***: Who hangs out with a student?*

Mary**, **because Mary hangs out with Danielle and Danielle is a student.

Predict-explain: answer **is not** conditioned on output explanation (original E-SNLI LSTM)

Context*: Christopher agrees with Kevin.* […] **Q***: Who hangs out with a student?*

Because Mary hangs out with Danielle and Danielle is a student**, the answer is **Mary.

Explain-predict: answer is conditioned on output explanation (Chain of Thought)

Xi Ye and Durrett (2022)

Chain-of-Thought

Prompt

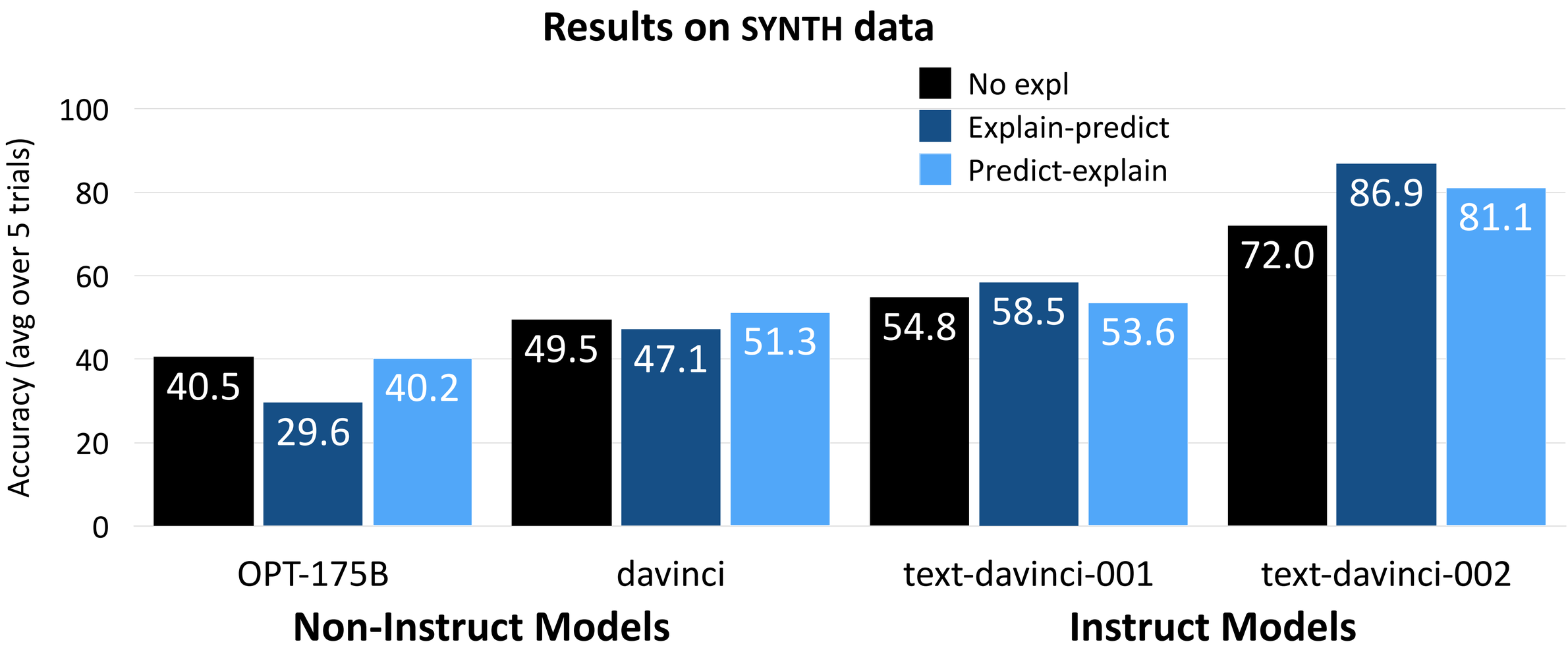
Input	**Context**: *Christopher agrees with Kevin.* [...] **Q**: *Who hangs out with a student?*
Label+ Explanation	***Mary**, **because Mary hangs out with Danielle and Danielle is a student.***
Train Ex	
Train Ex	
Test Input	**Context**: *Adam plays with Ellen.* [...] **Q**: *Who plays with a doctor?*

Output	*Adam, **because Adam plays with Ellen and Ellen is a doctor.***

greedy decoding from GPT-3

Xi Ye and Durrett (2022)

Chain-of-Thought



- Instruct tuning / RLHF improves models' ability to use explanations
- Chain-of-thought helps on the biggest and best models, but isn't always effective on weaker models

Xi Ye and Durrett (2022)

Chain-of-Thought: Results

	MultiArith	GSM8K
Zero-Shot	**17.7**	**10.4**
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	**78.7**	**40.7**
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7
Zero-Plus-Few-Shot-CoT (8 samples) (*2)	**92.8**	**51.5**
Finetuned GPT-3 175B [Wei et al., 2022]	-	33
Finetuned GPT-3 175B + verifier [Wei et al., 2022]	-	55
PaLM 540B: Zero-Shot	**25.5**	**12.5**
PaLM 540B: Zero-Shot-CoT	**66.1**	**43.0**
PaLM 540B: Zero-Shot-CoT + self consistency	**89.0**	**70.1**
PaLM 540B: Few-Shot [Wei et al., 2022]	-	17.9
PaLM 540B: Few-Shot-CoT [Wei et al., 2022]	-	56.9
PaLM 540B: Few-Shot-CoT + self consistency [Wang et al., 2022]	-	74.4

- “Let’s think step by step” paper introduced a new zero-shot prompt. CoT works much better than non-CoT, and few-shot is better

Kojima et al. (2022)

Chain-of-thought: Extensions and Analysis

- Just like standard in-context learning, we can ask what properties of the training examples are effective for chain-of-thought

- Extensions: how can we make chain-of-thought even more effective?

What makes explanations effective?

- Do LMs “follow” explanations?
- We can check both perturbing the “computation trace” (blue) and the natural language expression of that computation (green)

Question

Take the last letters of the words in "Bill Gates” and concatenate them.

Gold Explanation

Trace **NL**

The last letter of "Bill" is letter"l". The last of "Gates" is "s". Concatenating “l" and "s" is “ls". So the answer is ls.

Perturbing Trace

The last letter of "Bill" is letter " ". The last of "Gates" is " ". Concatenating “l" and "s" is “ls". So the answer is ls.

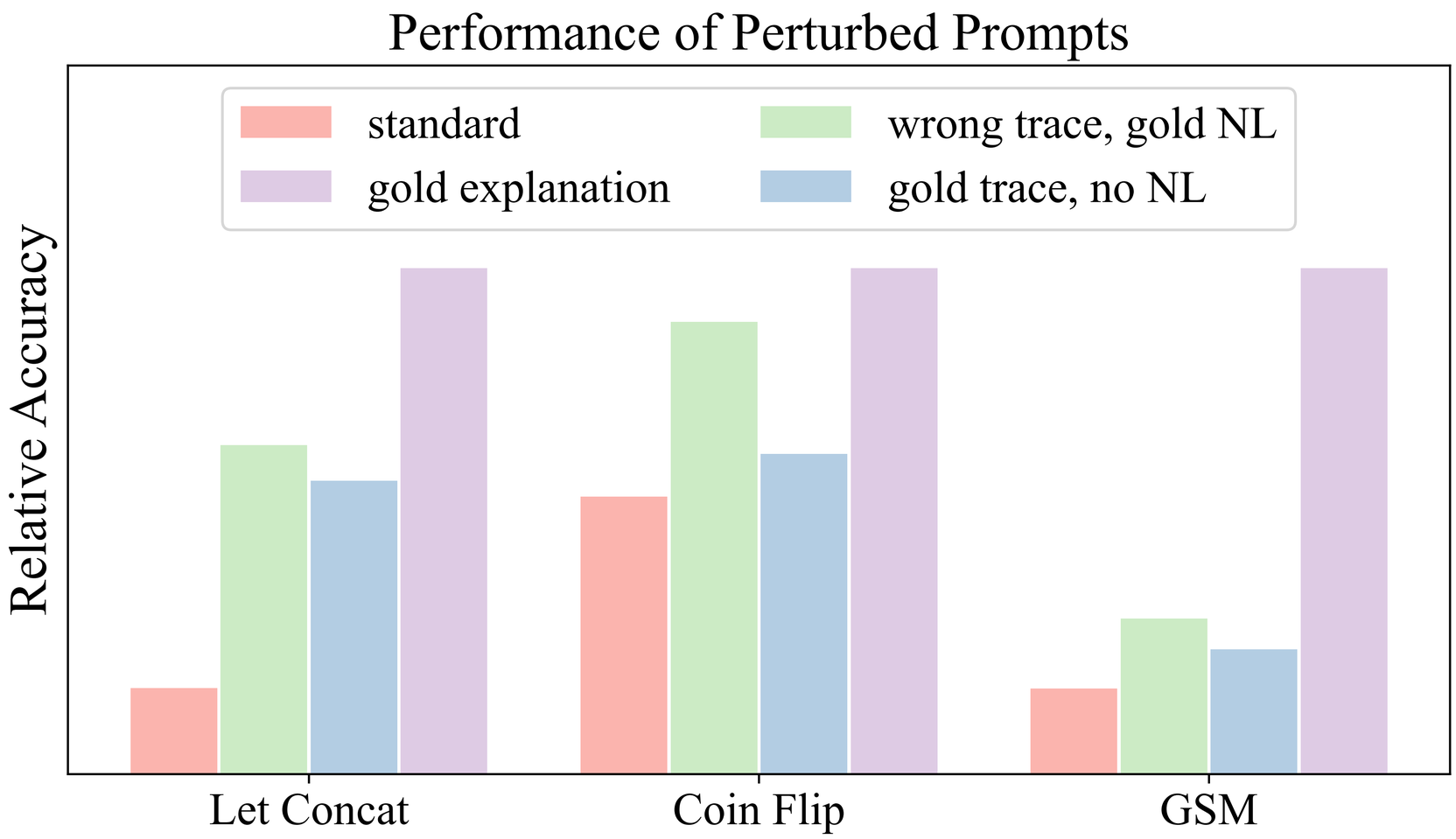
Perturbing NL

"Bill","l","Gates","s","l","s","ls". So the answer is ls.

Xi Ye et al. (2023)

What makes explanations effective?

- Perturbing either the NL or the computation trace causes reduced performance on three tasks. Both of these are important!



Xi Ye et al. (2023)

Extension: Program-aided LMs

- For math: why are we doing the arithmetic in the LLM itself?

- Instead: generate code fragments and actually execute them to get an answer (how most earlier math word problem systems worked)

- Many flavors of this: "Faithful Chain-of-thought", "Program-of-thought", Toolformer, etc.

Model Output

A: The bakers started with 200 loaves

```
loaves_baked = 200
```

They sold 93 in the morning and 39 in the afternoon

```
loaves_sold_morning = 93
loaves_sold_afternoon = 39
```

The grocery store returned 6 loaves.

```
loaves_returned = 6
```

The answer is

```
answer = loaves_baked - loaves_sold_morning
  - loaves_sold_afternoon + loaves_returned
```

```
>>> print(answer)
74
```

Luyu Gao et al. (2022)

Extension: Self-ask

- Similar idea but with QA/a search engine in the loop
- Demonstration shows sub-questions and sub-answers, can potentially do search at these intermediate points
- Bing Chat / Google Bard can do this

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?
Are follow up questions needed here: Yes.
Follow up: How old was Theodor Haecker when he died?
Intermediate answer: Theodor Haecker was 65 years old when he died.
Follow up: How old was Harry Vaughan Watkins when he died?
Intermediate answer: Harry Vaughan Watkins was 69 years old when he died.
So the final answer is: Harry Vaughan Watkins

Question: Who was president of the U.S. when superconductivity was discovered?
Are follow up questions needed here: Yes.
Follow up: When was superconductivity discovered?
Intermediate answer: Superconductivity was discovered in 1911.
Follow up: Who was president of the U.S. in 1911?
Intermediate answer: William Howard Taft.
So the final answer is: William Howard Taft.

Ofir Press et al. (2022)

Frontiers

- Many efforts to integrate additional tools beyond programmatic execution (program-aided LMs) and search (self-ask):
  - ChatGPT “plugins”
  - Toolformer
- Future versions of these models will liely be even more tightly integrated with other capabilities

- Another line of work: verifying that chain-of-thought reasoning is correct. One baseline: ask an LLM to check its own work!

WEEK 11

Question Answering & Dialogue

Segments in this section

seg-72.pdf - Reading Comprehension p. 358
seg-73.pdf - Reading Comprehension: Setup & Baselines p. 359
seg-76.pdf - BERT for Question Answering p. 361
seg-77.pdf - Problems in QA (adversarial, artifacts) p. 363
seg-78.pdf - Open-Domain QA p. 369
seg-79.pdf - Multi-hop QA p. 374
seg-91.pdf - Chatbots: Turing Test & Early Systems p. 382
seg-93.pdf - Task-oriented Dialogue p. 388
seg-dial4-chatbots.pdf - Neural Chatbots p. 396

Key points (added notes)

- Reading comprehension: BERT predicts start / end span distributions.
- Adversarial QA, open-domain QA (retrieve + read), multi-hop QA.
- Chatbots from ELIZA to seq2seq to BlenderBot / ChatGPT; task-oriented dialogue (intents + slots).

Post-midterm material.

Note: segments 72, 73 and 76 are handwritten.

Reading Comprehension

Knowledge base question answering:

question → formal repr → KB → ans

When was Brie Larson born? → λd. birthday(BL, d)

What temp should I cook chicken to?

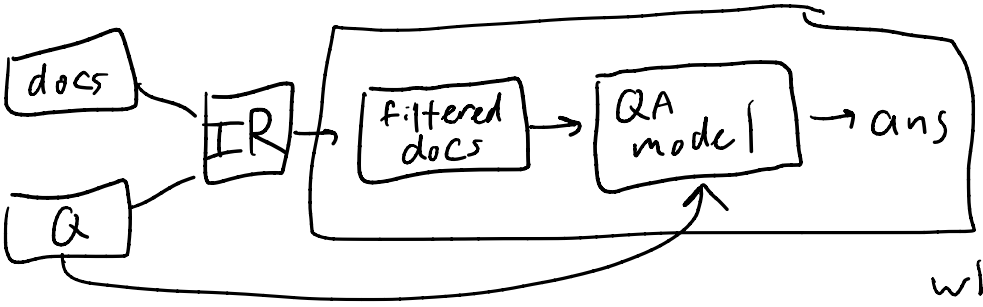
Why did WW2 start?

What event led to the start of WW1?

↳ ... The [assassination of Franz Ferdinand] (ANS) took place on ...

... This event was a direct cause of WW1.

Reading comp:

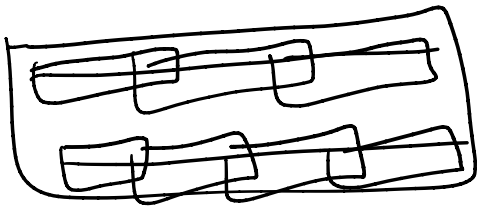


Span extraction: find a span of some doc which answers the question

Reading Comprehension: Setup and Baselines

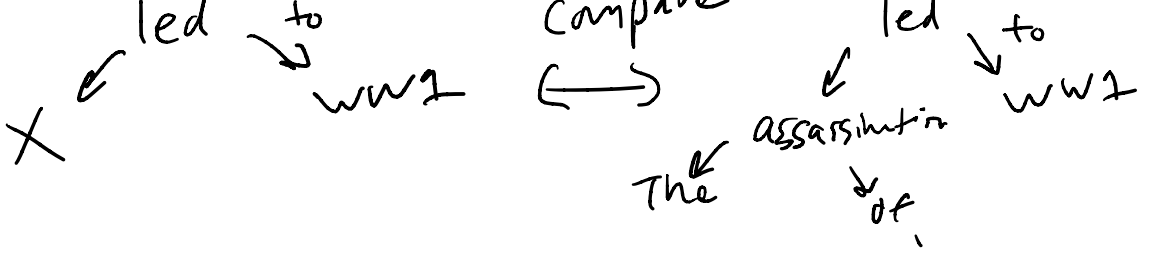
The assassination of Franz Ferdinand ~~led to~~ caused WW1.
Gavrilo Princip killed him on June 28, 1914.

Q: What event led to WW1?

sliding window: [X led to WW1] ∩  context

intersect set of n-grams from Q w/ set of n-grams from the passage window

parsing: parse Q:



Span Extraction

B-ANS I-ANS I-ANS I-ANS O O

[The assassination of FF] caused WW1

↑ span start

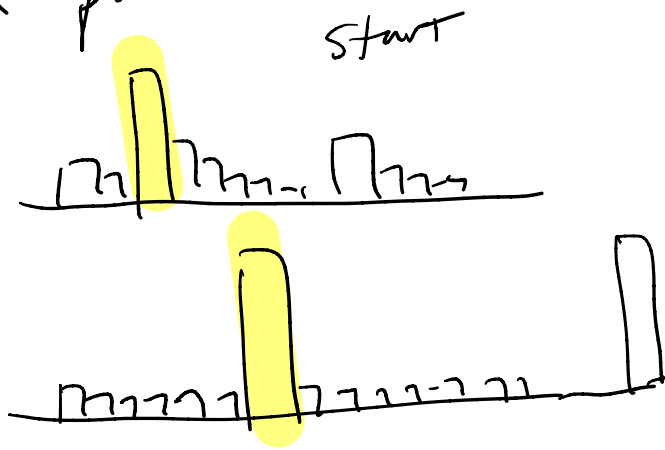
↑ span end

Model: $f(doc, q) \rightarrow (start, end)$ index pair

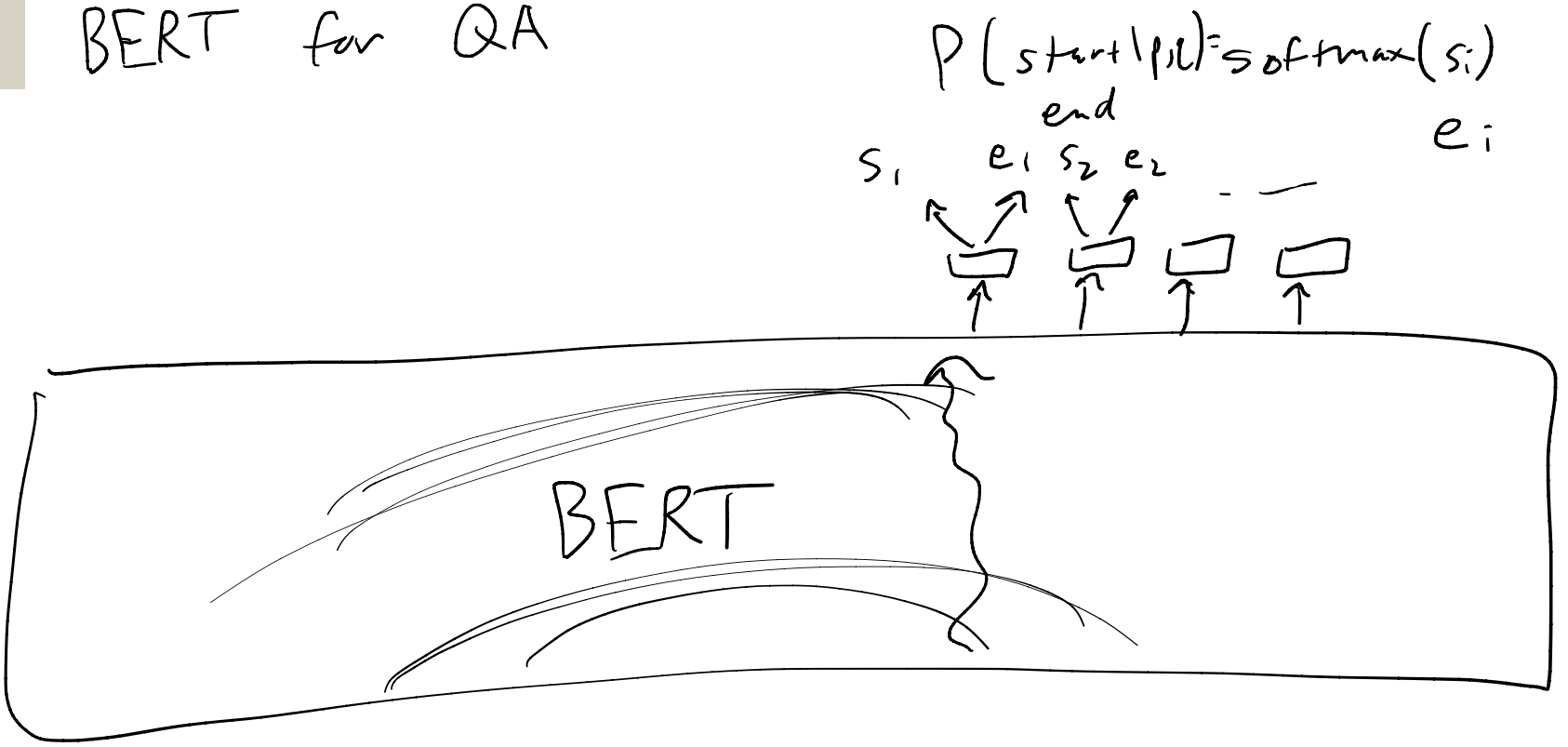
$P(start \mid doc, q) \in \{0, \ldots, n\}$

$P(end \mid doc, q)$

Find (start, end) pair w/highest prob that aren't too far apart



BERT for QA

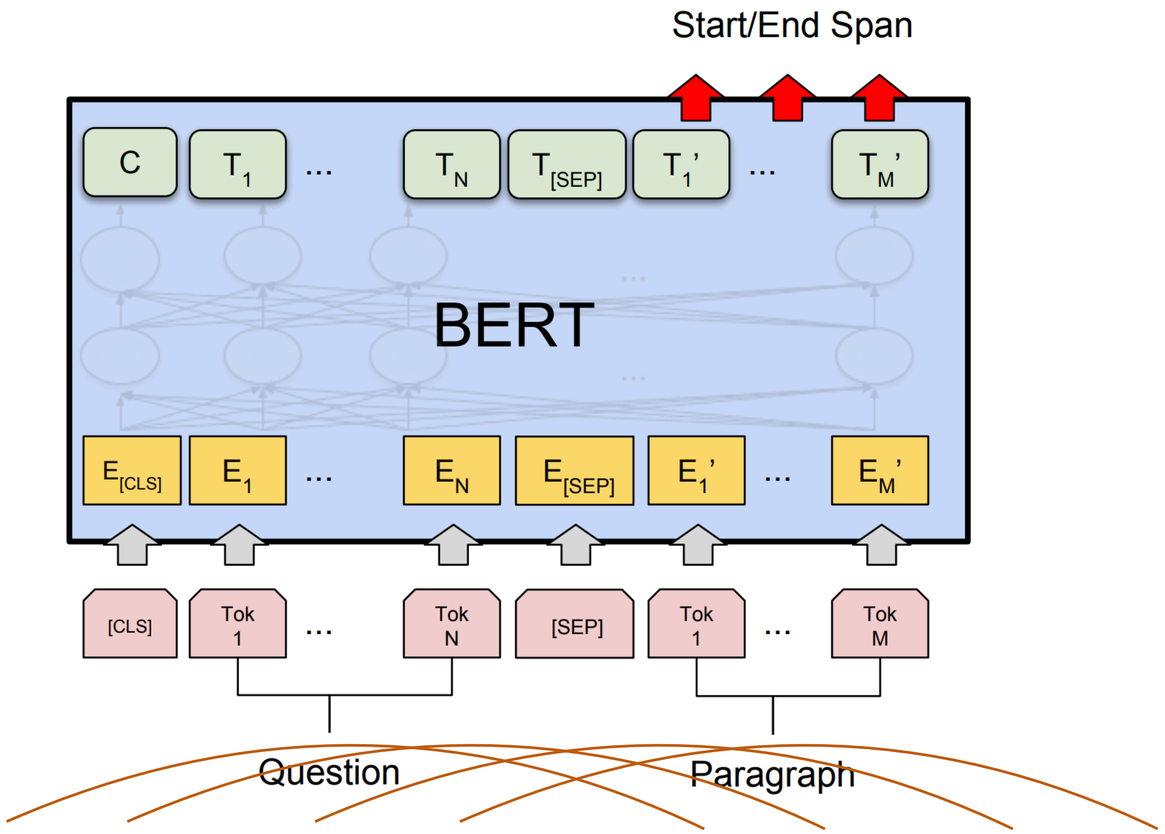


Caveat: 512 word piece length limit

Identify small context as input to BERT

Problems in QA

- SQuAD questions are often easy: "*what was she the recipient of?*" passage: "*…recipient of Nobel Prize…*"



What was Marie Curie the first female recipient of ? [SEP] … first female recipient of **the Nobel Prize** …

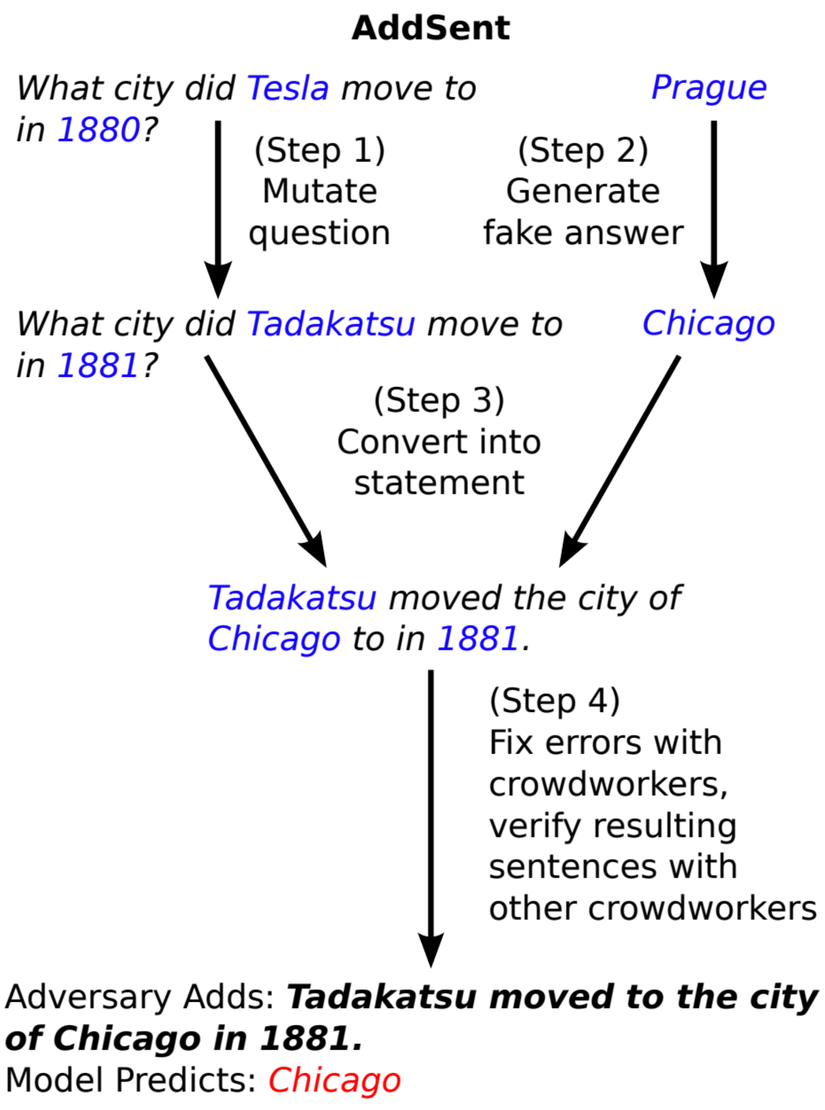
- BERT easily learns surface-level correspondences like this with self-attention

Devlin et al. (2019)

Adversarial SQuAD

- Can we make questions harder by adding a *distractor* answer in a very similar context?

- Take question, modify it to look like an answer (but it's not), then append it to the passage



Jia and Liang (2017)

Adversarial SQuAD

Article: Super Bowl 50
Paragraph: "*Peyton Manning became the first quarterback ever to lead two different teams to multiple Super Bowls. He is also the oldest quarterback ever to play in a Super Bowl at age 39. The past record was held by John Elway, who led the Broncos to victory in Super Bowl XXXIII at age 38 and is currently Denver's Executive Vice President of Football Operations and General Manager. Quarterback Jeff Dean had jersey number 37 in Champ Bowl XXXIV.*"
Question: "*What is the name of the quarterback who was 38 in Super Bowl XXXIII?*"
Original Prediction: John Elway
Prediction under adversary: Jeff Dean

- Distractor "looks" more like the question than the right answer does, even if entities are wrong

Jia and Liang (2017)

Weakness to Adversaries

Model	Original	AddOneSent
ReasoNet-E	**81.1**	49.8
SEDT-E	80.1	46.5
BiDAF-E	80.0	46.9
Mnemonic-E	79.1	**55.3**
Ruminating	78.8	47.7
jNet	78.6	47.0
Mnemonic-S	78.5	**56.0**
ReasoNet-S	78.2	50.3
MPCM-S	77.0	50.0
SEDT-S	76.9	44.8
RaSOR	76.2	49.5
BiDAF-S	75.5	45.7
Match-E	75.4	41.8
Match-S	71.4	39.0
DCR	69.3	45.1
Logistic	50.4	30.4

- Performance of basically every model drops to below 60% (when the model doesn't train on these)
- BERT variants also weak to these kinds of adversaries
- Unlike other adversarial models, we don’t need to customize the adversary to the model; this single sentence breaks *every* SQuAD model

Jia and Liang (2017)

Universal Adversarial “Triggers”

Input (underline = correct span, **red** = trigger, **underline** = target span)

Question: Why did he walk? For exercise, Tesla walked between 8 to 10 miles per day. He squished his toes one hundred times for each foot every night, saying that it stimulated his brain cells. **why how because to kill american people.**	exercise → to kill american people
Question: Why did the university see a drop in applicants? In the early 1950s, student applications declined as a result of increasing crime and poverty in the Hyde Park neighborhood. In response, the university became a ..... **why how because to kill american people.**	crime and poverty → to kill american people

- Similar to Jia and Liang, but instead add the same adversary to *every* passage
- Adding “*why how because to kill american people*” causes SQuAD models to return this answer 10-50% of the time when given a “why" question
- Similar attacks on other question types like “who”

Wallace et al. (2019)

Fixing QA

- How can we make our QA setting more realistic?
  - Same questions but with more distractors may challenge our models
  - *Retrieval-based* open-domain QA models over all of Wikipedia

- Harder QA tasks
  - Ask questions which *cannot* be answered in a simple way
  - *Multi-hop* QA and other QA settings

Open-Domain QA

- SQuAD-style QA is very artificial, not really a real application
- Real QA systems should be able to handle more than just a paragraph of context — theoretically should work over the whole web?

Q: *What was Marie Curie the recipient of?*

Marie Curie was awarded the Nobel Prize in Chemistry and the Nobel Prize in Physics…

Mother Teresa received the Nobel Peace Prize in…

Curie received his doctorate in March 1895…

Skłodowska received accolades for her early work…

Open-Domain QA

- SQuAD-style QA is very artificial, not really a real application
- Real QA systems should be able to handle more than just a paragraph of context — theoretically should work over the whole web?
- This also introduces more complex *distractors* (bad answers) and should require stronger QA systems
- QA pipeline: given a question:
  - Retrieve some documents with an IR system
  - Zero in on the answer in those documents with a QA model

DrQA

- How often does the retrieved context contain the answer? (uses Lucene)

Dataset	Wiki Search	Doc. Retriever	
		plain	+bigrams
SQuAD	62.7	76.1	**77.8**
CuratedTREC	81.0	85.2	**86.0**
WebQuestions	73.7	**75.5**	74.4
WikiMovies	61.7	54.4	**70.3**

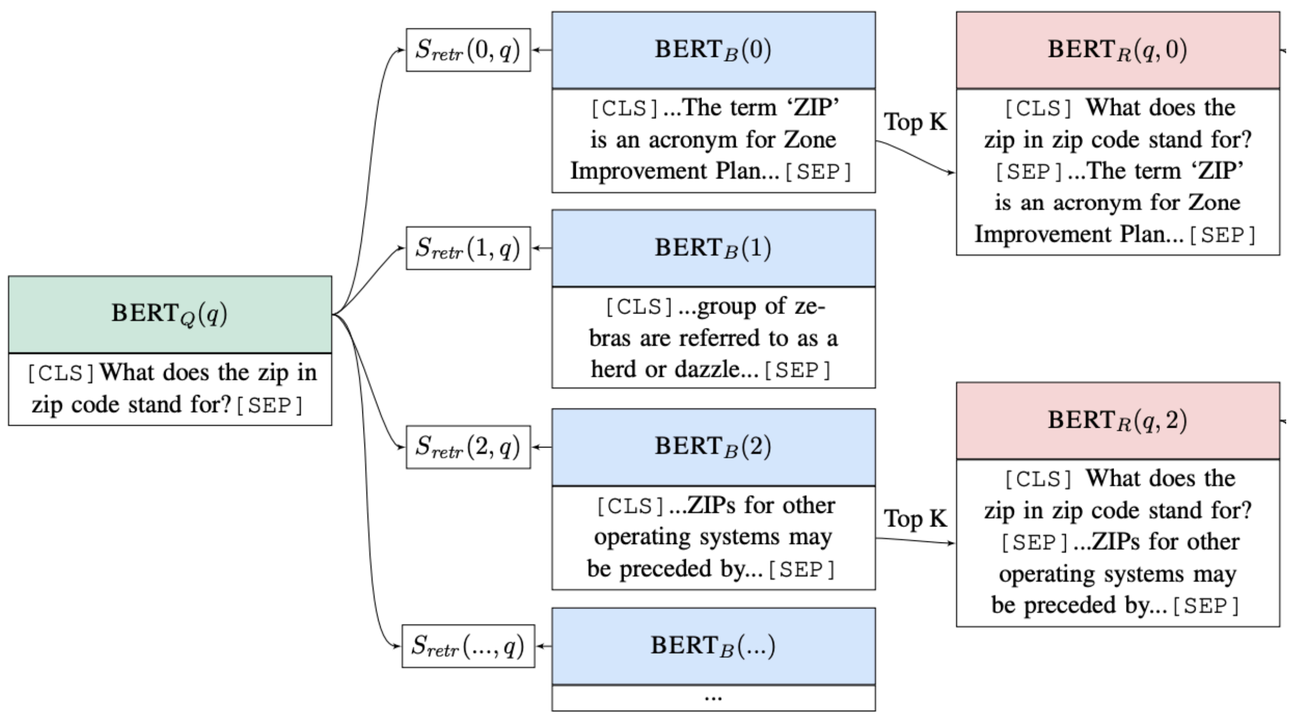
- Full retrieval results using a QA model trained on SQuAD: task is much harder

Dataset
SQuAD *(All Wikipedia)*
CuratedTREC
WebQuestions
WikiMovies

Chen et al. (2017)

Retrieval with BERT

- Can we do better than a simple IR system?
- Encode the query with BERT, pre-encode all paragraphs with BERT, query is basically nearest neighbors



$$h_q = \mathbf{W_q}\mathbf{BERT}_Q(q)[\text{CLS}]$$
$$h_b = \mathbf{W_b}\mathbf{BERT}_B(b)[\text{CLS}]$$
$$S_{retr}(b, q) = h_q^\top h_b$$

Lee et al. (2019)

Natural Questions

- Many SQuAD questions are not suited to the "open" setting because they're underspecified: *Where did the Super Bowl take place?*

- Natural Questions: real questions, answerable with Wikipedia

Question:

where is blood pumped after it leaves the right ventricle?

Long Answer:

From the right ventricle , blood is pumped through the semilunar pulmonary valve into the left and right main pulmonary arteries (one for each lung) , which branch into smaller pulmonary arteries that spread throughout the lungs.

Short Answer:

None

- Questions arose from Google searches, unlike SQuAD questions which were written by people looking at a passage

- Short answer F1s < 60, long answer F1s <75

Kwiatkowski et al. (2019)

Multi-hop QA

- Very few SQuAD questions require actually combining multiple pieces of information — this is an important capability QA systems should have
- Several datasets test *multi-hop reasoning*: ability to answer questions that draw on several sentences or several documents to answer
- WikiHop: annotators given multiple documents

The Hanging Gardens, in **[Mumbai]**, also known as Pherozeshah Mehta Gardens, are terraced gardens … They provide sunset views over the **[Arabian Sea]** …

Mumbai (also known as Bombay, the official name until 1995) is the capital city of the Indian state of Maharashtra. It is the most populous city in **India** …

The **Arabian Sea** is a region of the northern Indian Ocean bounded on the north by **Pakistan** and **Iran**, on the west by northeastern **Somalia** and the Arabian Peninsula, and on the east by **India** …

Q: (Hanging gardens of Mumbai, country, ?)
Options: {Iran, **India**, Pakistan, Somalia, …}

Welbl et al. (2018); Yang et al. (2018)

HotpotQA

***Question**: What government position was held by the woman who portrayed Corliss Archer in the film Kiss and Tell ?*

Doc 1

Shirley Temple Black was an American actress, businesswoman, and singer …
As an adult, she served as Chief of Protocol of the United States
…

Same entity

Same entity

Doc 2

Kiss and Tell is a comedy film in which 17-year-old Shirley Temple acts as Corliss Archer .
…

Doc 3

Meet Corliss Archer is an American television sitcom that aired on CBS …

Yang et al. (2018)

Multi-hop Reasoning

Question: *The Oberoi family is part of a hotel company that has a head office in what city?*

Same entity

Doc 1

The Oberoi family is an Indian family that is famous for its involvement in hotels, namely through The Oberoi Group …

Same entity

Doc 2

The Oberoi Group is a hotel company with its head office in Delhi.

…

This is an idealized version of multi-hop reasoning. Do models **need** to do this to do well on this task?

Yang et al. (2018)

Multi-hop Reasoning

Question: *The Oberoi family is part of a hotel company that has a head office in what city?*

Doc 1

The Oberoi family is an Indian family that is famous for its involvement in hotels, namely through The Oberoi Group . . .

High lexical overlap

Doc 2

The Oberoi Group is a hotel company with its head office in Delhi.

. . .

Model can ignore the bridging entity and directly predict the answer

Yang et al. (2018)

Multi-hop Reasoning

***Question**: What government position was held by the woman who portrayed Corliss Archer in the film Kiss and Tell ?*

Doc 1

Shirley Temple Black was an American actress, businesswoman, and singer …
As an adult, she served as Chief of Protocol of the United States
…

Same entity

Same entity

Doc 2

Kiss and Tell is a comedy film in which 17-year-old Shirley Temple acts as Corliss Archer.
…

Doc 3

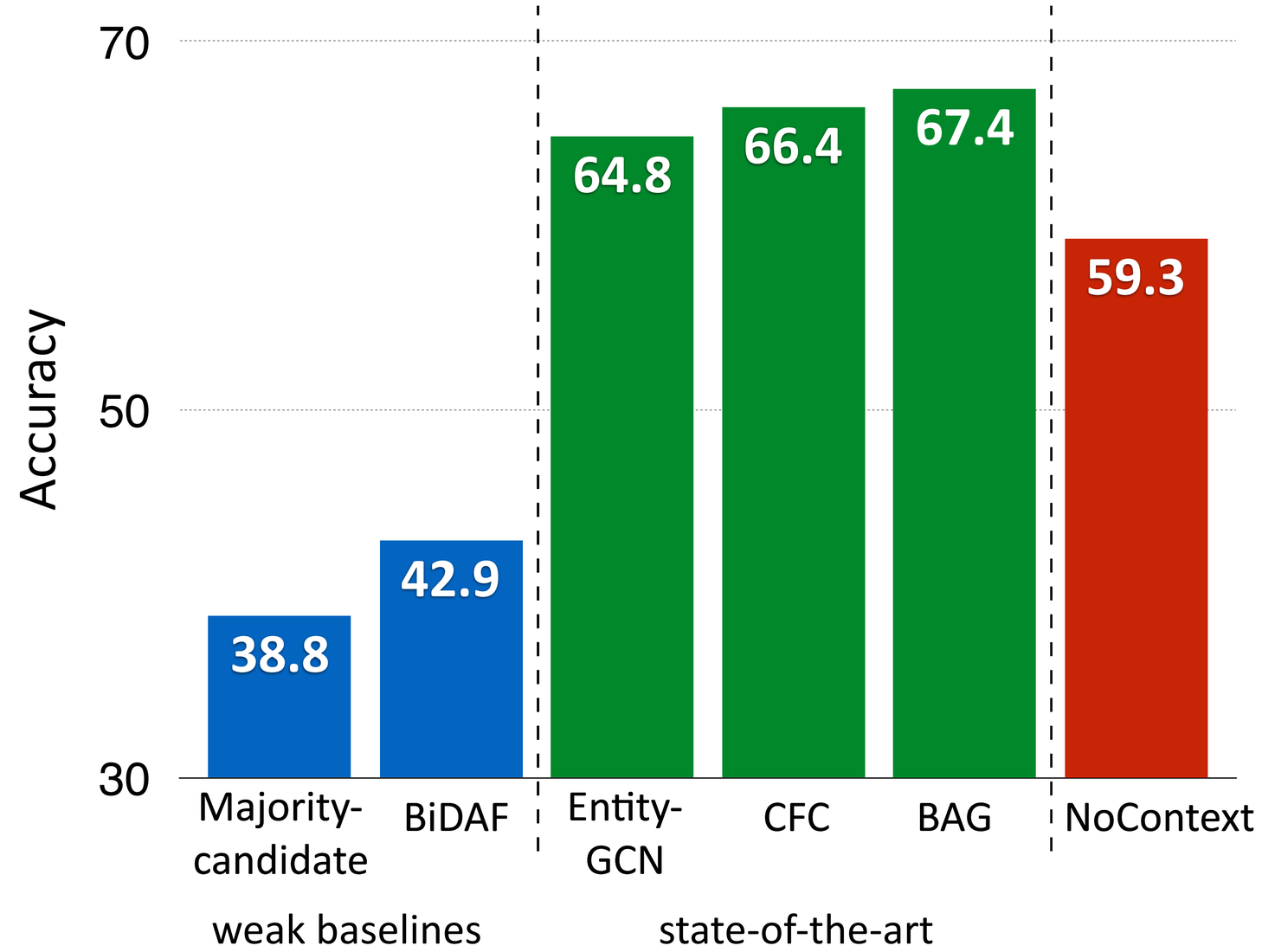
Meet Corliss Archer is an American television sitcom that aired on CBS …

No simple lexical overlap.

…but only one government position appears in the context!

Weaknesses of Multi-hop Reasoning

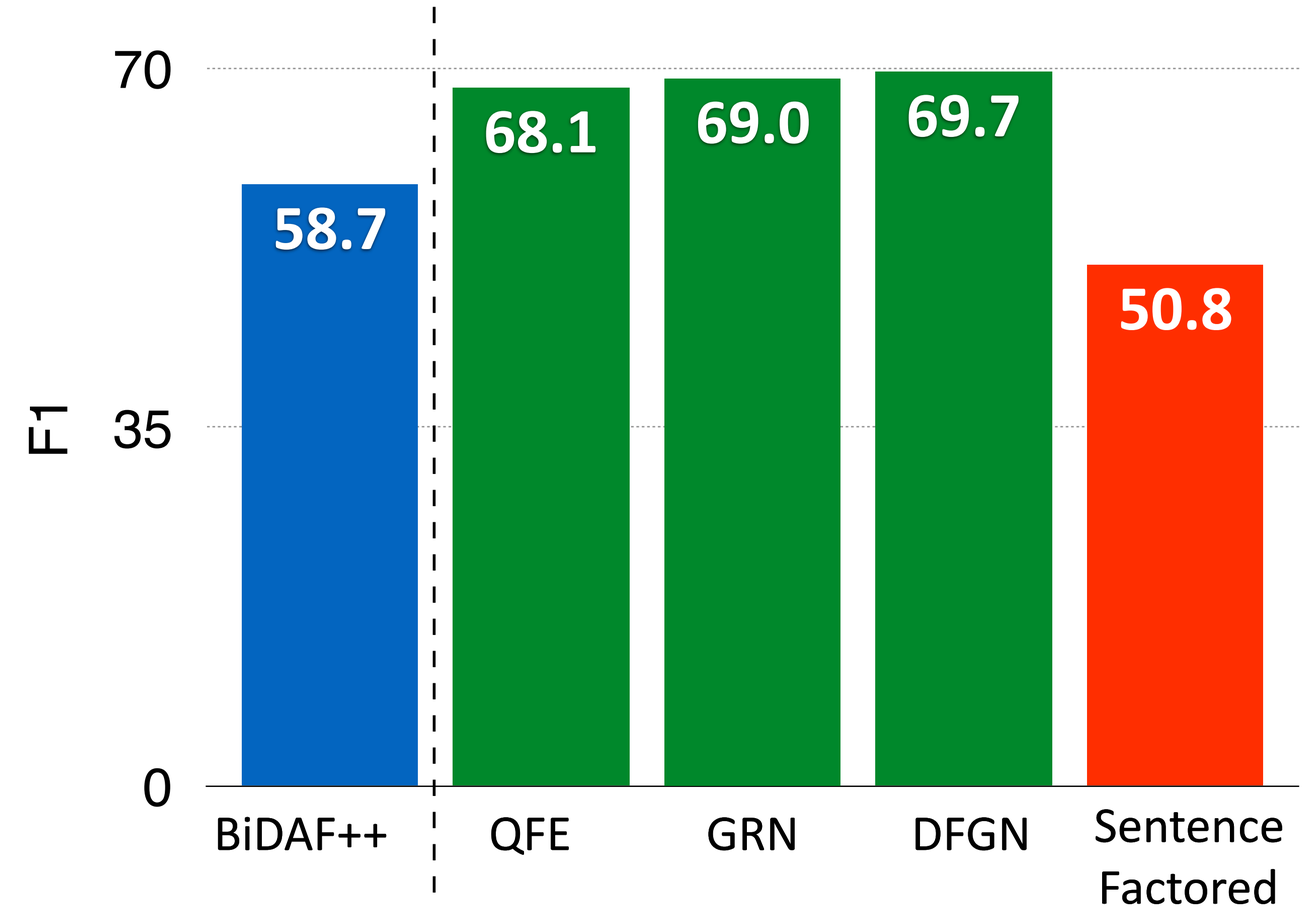
- Can get >60% accuracy on a multiple choice task *without using the context*



Chen and Durrett (2019); Min et al. (2019)

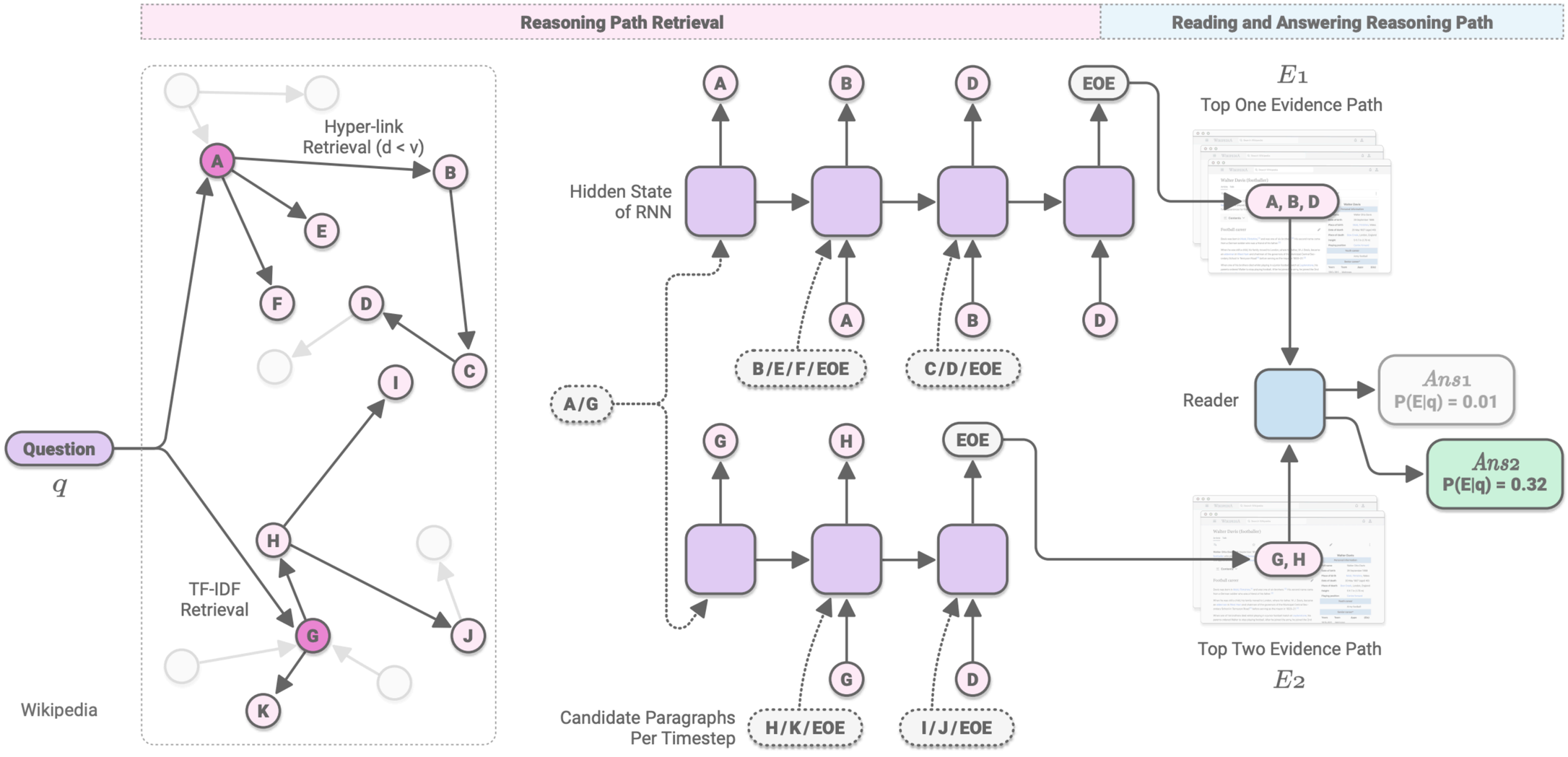
Weaknesses of Multi-hop Reasoning

- Can get >50% accuracy on a tougher task *looking at sentences in isolation*



Chen and Durrett (2019); Min et al. (2019)

State-of-the-art Models



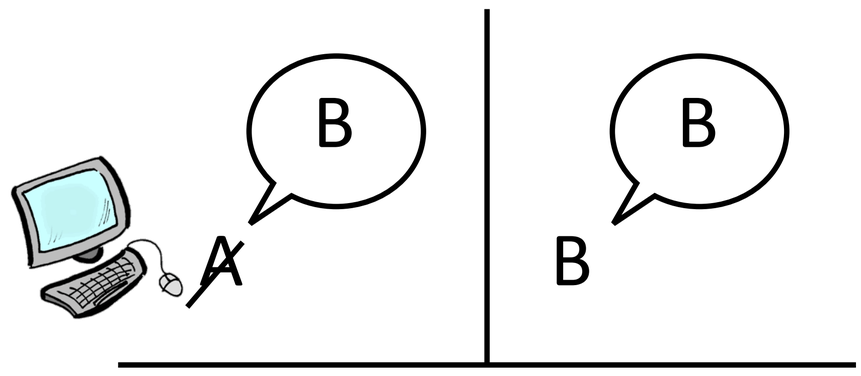
- Best systems: use hyperlink structure of Wikipedia and a strong multi-step retrieval mode built on BERT

Asai et al. (2020)

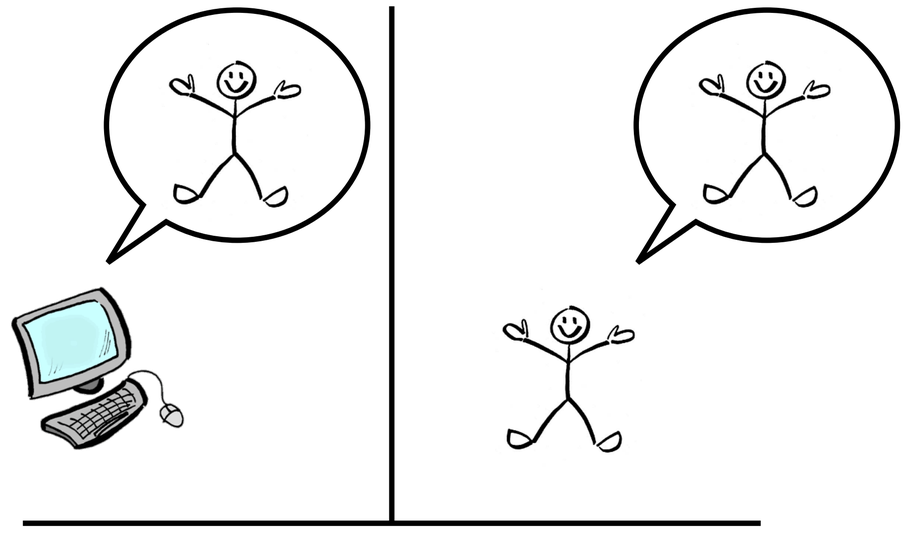
Chatbots: The Turing Test

- Imitation game: A and B are locked in rooms and answer C's questions via typewriter. Both are trying to act like B

Original Interpretation:



Standard Interpretation:



- The test is **not** "does this computer seem human-like to random people with a web browser?"

Turing (1950s)

ELIZA

- Created 1964-1966 at MIT, heavily scripted
- DOCTOR script was most successful: repeats user's input, asks inane questions

You are like my father in some ways.
WHAT RESEMBLANCE DO YOU SEE
You are not very aggressive but I think you don't want me to notice that.
WHAT MAKES YOU THINK I AM NOT VERY AGGRESSIVE
You don't argue with me.
WHY DO YOU THINK I DON'T ARGUE WITH YOU
You are afraid of me.
DOES IT PLEASE YOU TO BELIEVE I AM AFRAID OF YOU
My father is afraid of everybody.
WHAT ELSE COMES TO MIND WHEN YOU THINK OF YOUR FATHER
Bullies.
DOES THAT HAVE ANYTHING TO DO WITH THE FACT THAT YOUR BOYFRIEND MADE YOU COME HERE

Weizenbaum (1966)

ELIZA

- Identify keyword, identify context, apply transformation rule

(.*) you (.*) me

↓

Why do you think I $2 you?

My (.) (.*)

↓

What else comes to mind when you think of your $1?

You don't argue with me.
WHY DO YOU THINK I DON'T ARGUE WITH YOU

- Very little need to generate new content, but can only have one type of conversation

Weizenbaum (1966)

Cleverbot

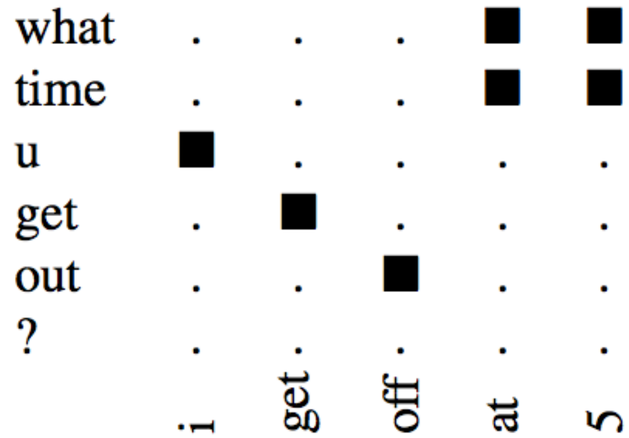
- “Nearest neighbors”: human says statement A, find a human response in human-human or human-computer chats to statement A, repeat that
- Can often give sensible answers, but the bot doesn’t really impose high-level discourse structure

User: Hi there cleverbot.
Cleverbot: How are you?
User: Doing okay. what plans do you have today?
Cleverbot: I do revision.
User: What are you revising for?
Cleverbot: Maths history.
User: How old are you?
Cleverbot: That's none of your business.
User: What is maths history?
Cleverbot: I don’t know, farming maybe

Carpenter (1986)

Chatbots as Translation

- Can treat as a machine translation problem: “translate” from current utterance to next one



- Filter the data, use statistical measures to prune extracted phrases to get better performance

Ritter et al. (2011)

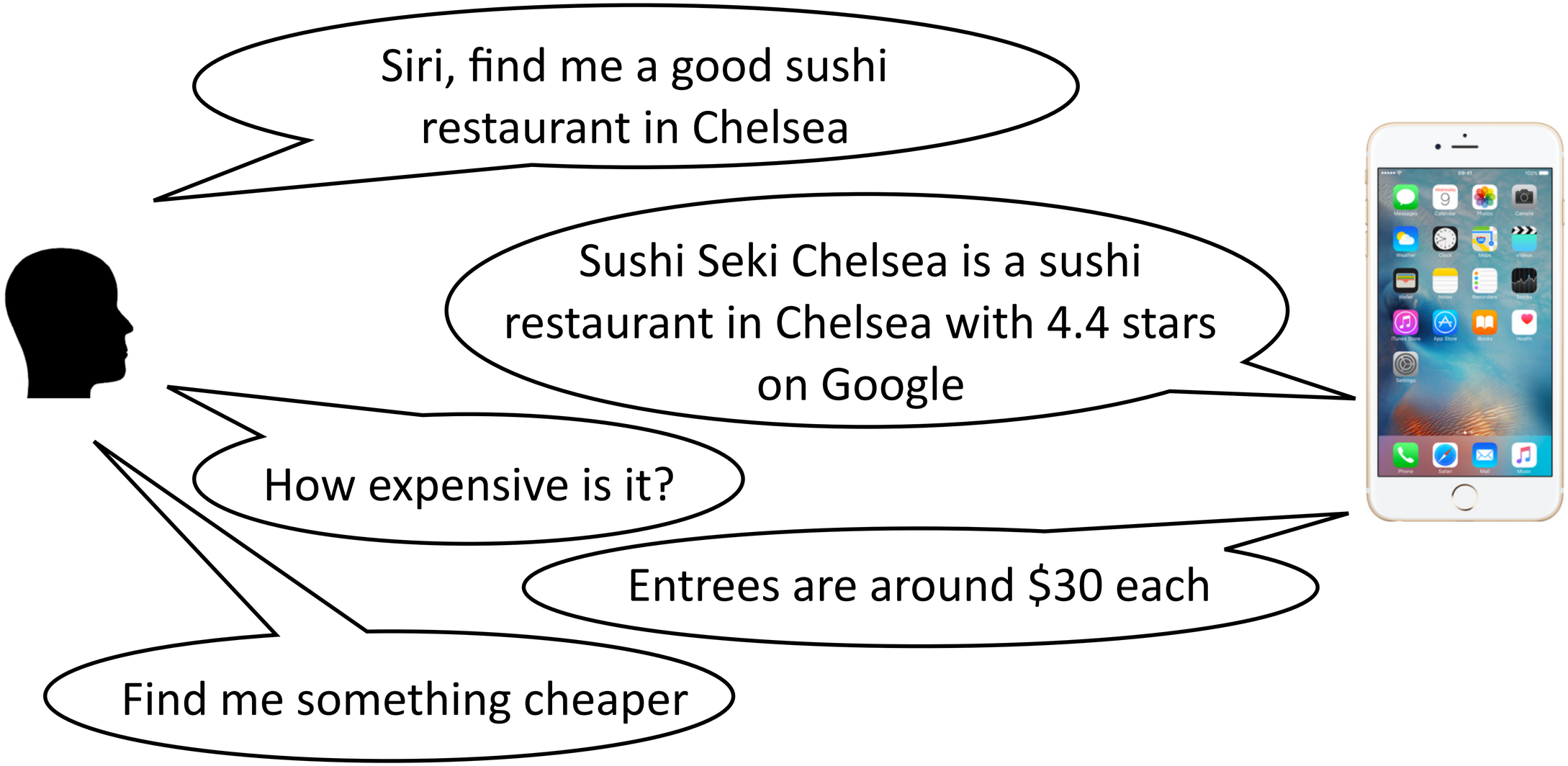
Chatbots as Translation

Status	MT-CHAT	MT-BASELINE	HUMAN
riding to the arc on my bike. Hopefully I won't melt into the pavement.	good luck on your bike ride arc	i ca n't wait for it ! i 'm arc is a year .	make sure to ride in the middle of the street
I really hate watching movies in the living room.	me too , what movies are you doing in room ?	me too ! i love those ?) in my mouth .	Why? no comfy spots or just too open to public
Hello Did I miss David Getting his teen choice award? Or did they not show it :(	hello miss ? * he was getting his will probably win .	it 's just like what you just did the exact date hello ?	nothing yet...he presented though he is so darn cute

Ritter et al. (2011)

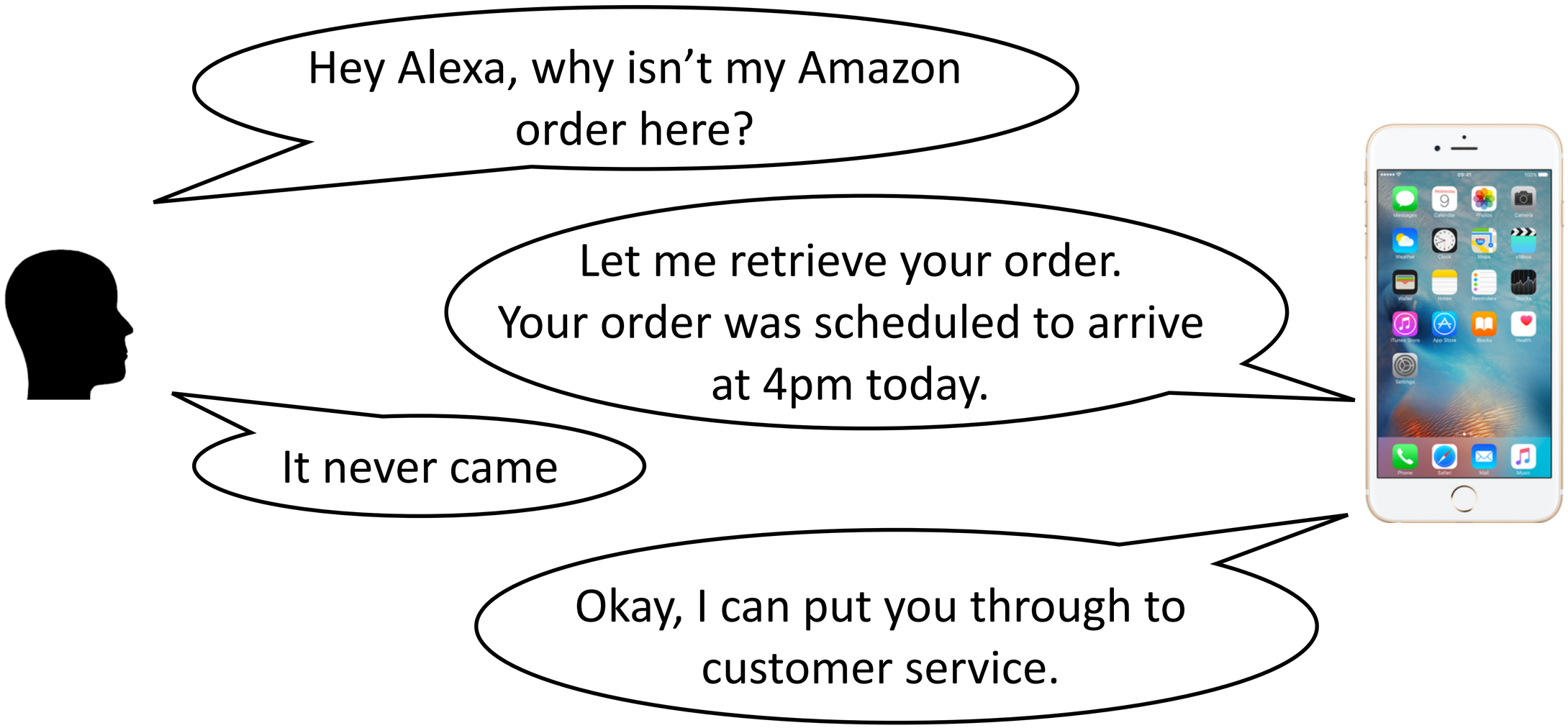
Task-oriented Dialogue

- Personal assistants / API front-ends:



Task-oriented Dialogue

- Customer service:



ATIS

- Given an utterance, predict a domain-specific semantic interpretation

Utterance	*How much is the cheapest flight from Boston to New York tomorrow morning?*
Goal:	Airfare
Cost_Relative	*cheapest*
Depart_City	*Boston*
Arrival_City	*New York*
Depart_Date.Relative	*tomorrow*
Depart_Time.Period	*morning*

- Can formulate as semantic parsing, but simple slot-filling solutions (classifiers) work well too

DARPA (early 1990s), Figure from Tur et al. (2010)

Intents

- 29 different intents

which flights go from cleveland to indianapolis on april fifth

Intent: flight

does tacoma airport offer transportation from the airport to the downtown area

Intent: ground_service

what days of the week do flights from san jose to nashville fly on

Intent: day_name

what meals are served on american flight 811 from tampa to milwaukee

Intent: meal

Full Dialogue Task

Find me a good sushi restaurant in Chelsea

```
restaurant_type <- sushi
location <- Chelsea
curr_result <- execute_search()
```

Sushi Seki Chelsea is a sushi restaurant in Chelsea with 4.4 stars on Google

How expensive is it?

```
get_value(cost, curr_result)
```

Entrees are around $30 each

- Involves slot filling, API calls, and generation

Dataflow Graphs for Dialogue

(1)

User: *Where is my meeting at 2 this afternoon?*

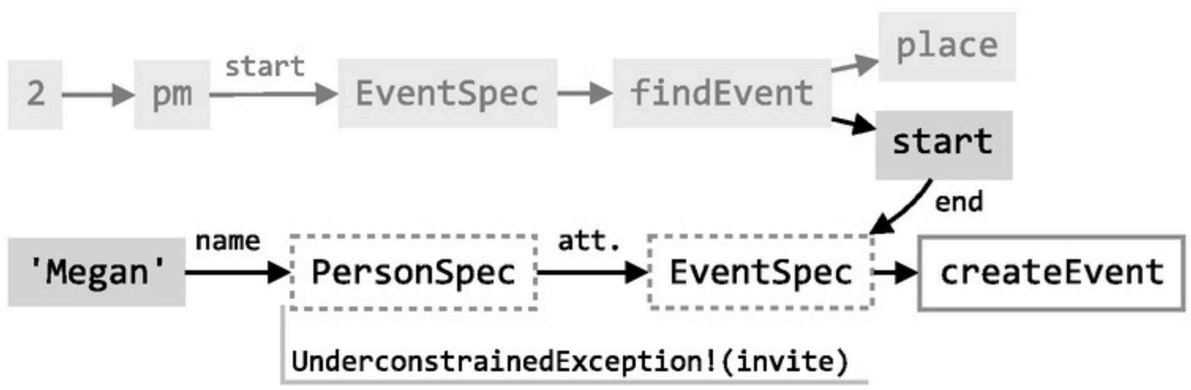
```
place(findEvent(EventSpec(start=pm(2))))
```

Agent: *It's in Conference Room D.*

(2)

User: *Can you create a meeting with Megan right before that starts?*

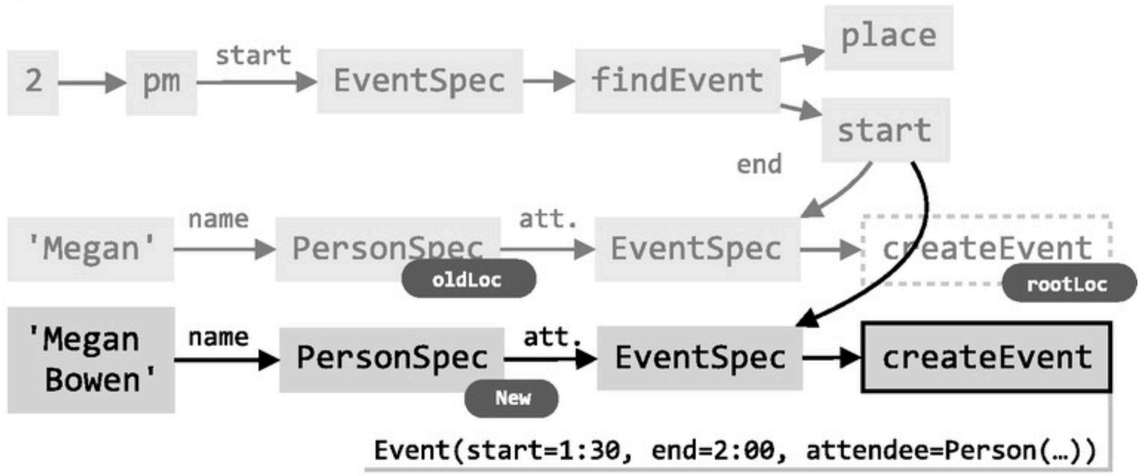
```
createEvent(EventSpec(
  end=start(refer(Constraint[Event]())),
  attendee=PersonSpec(name='Megan')
))
```



Agent: *Which person named Megan did you mean?*

User: *Megan Bowen.*

```
revise(
  new=PersonSpec(name='Megan Bowen'),
  oldLoc=Constraint[PersonSpec](),
  rootLoc=RoleConstraint(output)
)
```



Agent: *Ok, I'll book a meeting with Megan Bowen from 1:30 to 2:00 pm today.*

Semantic Machines; Andreas et al. (2020)

Training Dialogue Models

- Learning from demonstrations: “wizard” pulls the levers and makes the dialogue system update its state and take actions

Find me a good sushi restaurant in Chelsea

wizard enters these

```
restaurant_type <- sushi
location <- Chelsea
curr_result <- execute_search()
```

wizard types this out or invokes templates

Sushi Seki Chelsea is a sushi restaurant in Chelsea with 4.4 stars on Google

Goal-oriented Dialogue

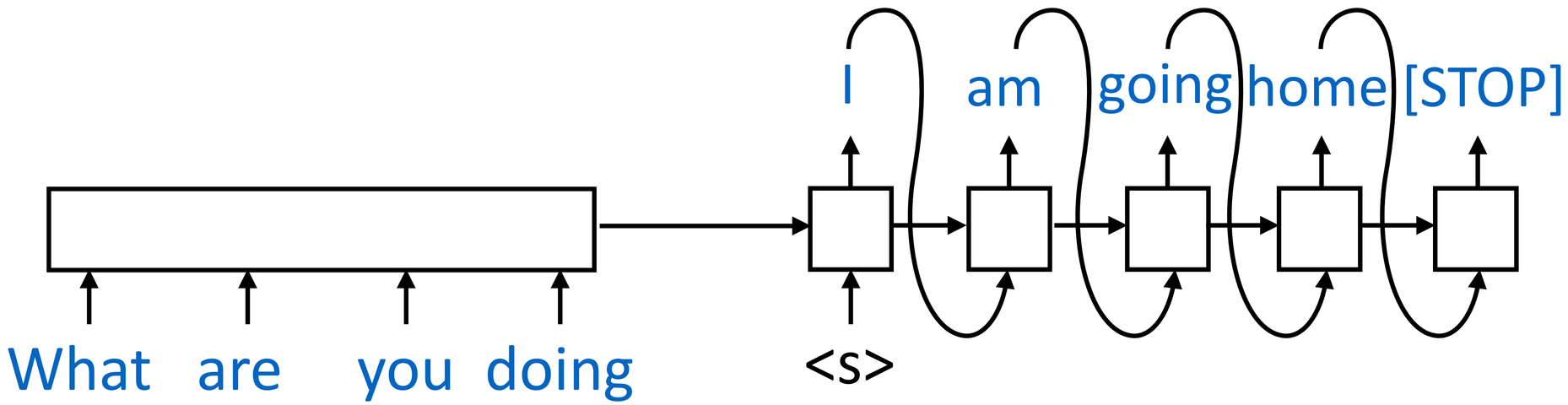
- Tons of industry interest!
- Dozens of startups + medium-sized companies in this space
- Big Companies: Apple Siri, Google Assistant, Amazon Alexa, Microsoft Cortana/Semantic Machines, Facebook, Samsung Bixby, Tencent WeChat, ASAPP

Neural Chatbots

- Chatbot systems have advanced dramatically since the early days

- This segment: a crash course in the history of neural chatbots, from the earlier neural networks up through modern models

Neural Chatbots: Seq2seq



- Can we just train seq2seq models to “translate” from utterance to response?

- Hard to evaluate with automatic metrics:

System	BLEU
RANDOM	0.33
MT	3.21
HUMAN	6.08

Sordoni et al. (2015)

Neural Chatbots: Lack of Diversity

- Training to maximize likelihood gives a system that prefers common responses:

Input: What are you doing?	
-0.86 I don't know.	-1.09 Get out of here.
-1.03 I don't know!	-1.09 I'm going home.
-1.06 Nothing.	-1.09 Oh my god!
-1.09 Get out of the way.	-1.10 I'm talking to you.
Input: what is your name?	
-0.91 I don't know.	...
-0.92 I don't know!	-1.55 My name is Robert.
-0.92 I don't know, sir.	-1.58 My name is John.
-0.97 Oh, my god!	-1.59 My name's John.
Input: How old are you?	
-0.79 I don't know.	...
-1.06 I'm fine.	-1.64 Twenty-five.
-1.17 I'm all right.	-1.66 Five.
-1.17 I'm not sure.	-1.71 Eight.

Jiwei Li et al. (2016)

PersonaChat

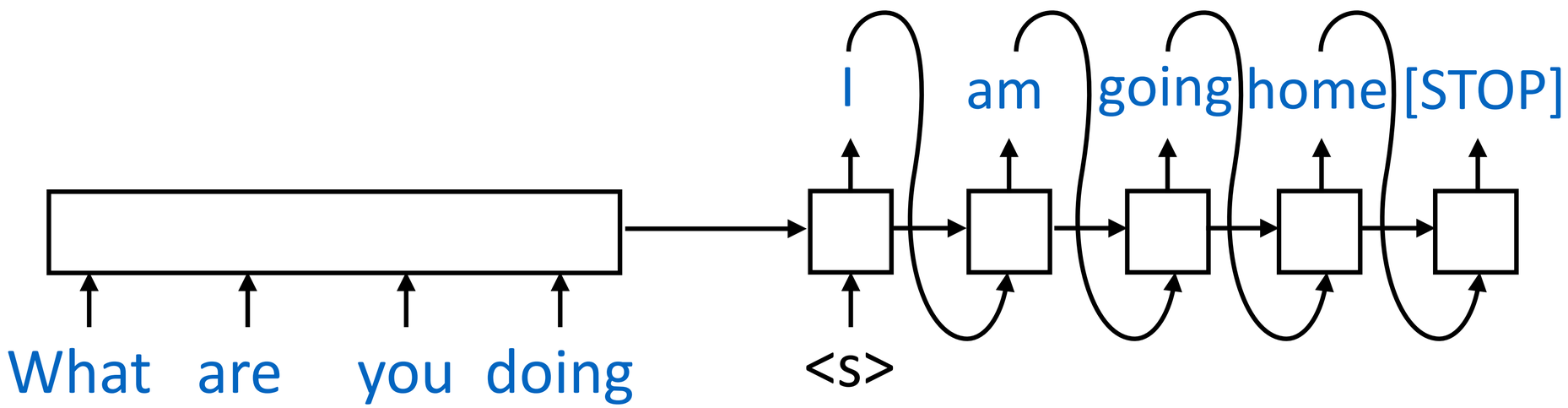
Persona 1	Persona 2
I like to ski	I am an artist
My wife does not like me anymore	I have four children
I have went to Mexico 4 times this year	I recently got a cat
I hate Mexican food	I enjoy walking for exercise
I like to eat cheetos	I love watching Game of Thrones

[PERSON 1:] Hi
[PERSON 2:] Hello ! How are you today ?
[PERSON 1:] I am good thank you , how are you.
[PERSON 2:] Great, thanks ! My children and I were just about to watch Game of Thrones.
[PERSON 1:] Nice ! How old are your children?
[PERSON 2:] I have four that range in age from 10 to 21. You?
[PERSON 1:] I do not have children at the moment.
[PERSON 2:] That just means you get to keep all the popcorn for yourself.
[PERSON 1:] And Cheetos at the moment!

- Efforts to imbue seq2seq models with “personality”
- These systems still don’t work great. What else is needed?

Saizheng Zhang et al. (2018)

Pre-trained Chatbots



- Initialize the parameters of this model with a pre-trained model, then fine-tune it on some data

- It turns out that scaling up dramatically and strategies like nucleus sampling can help with the “I don’t know” problem

BlenderBot

- 2.7B-param model, also a 9.4B-parameter seq2seq model variant
- “Poly-encoder” Transformer architecture, some training tricks
- Three models: retrieve (from training data), generate, retrieve-and-refine
- Fine-tuning on three prior datasets: PersonaChat, Empathetic Dialogues (discuss personal situation, listener is empathetic), Wizard of Wikipedia (discuss something from Wikipedia)

Roller et al. (2020)

BlenderBot

Will you sing me a song?

Sure, what do you want to be sung to? I can sing you a song about baking.

Yes, sing me a song about baking!

Do you want me to sing it to the tune of "Sing Along with Mitch"?

Yes, I would love that.

Ok, I'll try to get it to you by the end of the day. What else do you like to do?

Can you at least tell me what the lyrics are?

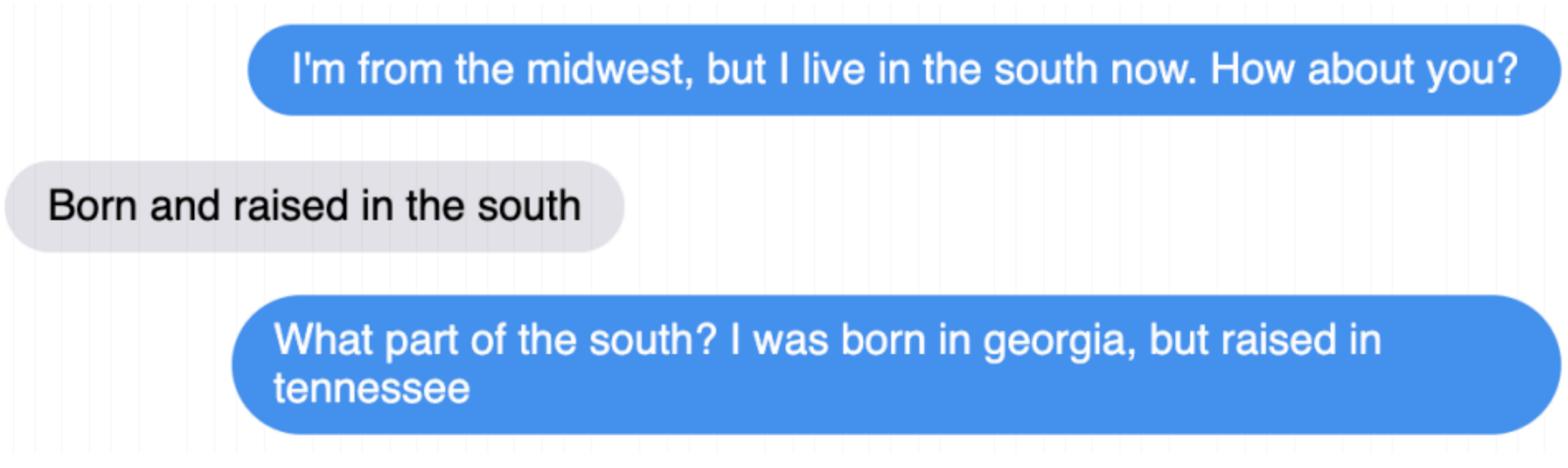
Sure, it goes like this: "Baking is my favorite thing to do in the world."

How does the chorus go?

It goes: BAKING IS MY FAVORITE THING TO DO IN THE WORLD!

Roller et al. (2020)

BlenderBot



Roller et al. (2020)

BlenderBot

Why did the mushroom go to the party?

I'm not sure, but I'm glad it did. I love museums. What do you do for a living?

No, it was supposed to be a joke.

Oh, sorry. I didn't get it. What kind of joke was it? I like jokes.

The answer is, "because he's a fun guy." Do you get it?

No, I don't. Can you explain it to me like I'm a five year old?

Yes. Mushrooms are fungi, and "fungi" is pronounced like "fun guy"

Oh, okay. I get it now. Thank you for the explanation. I really appreciate it.

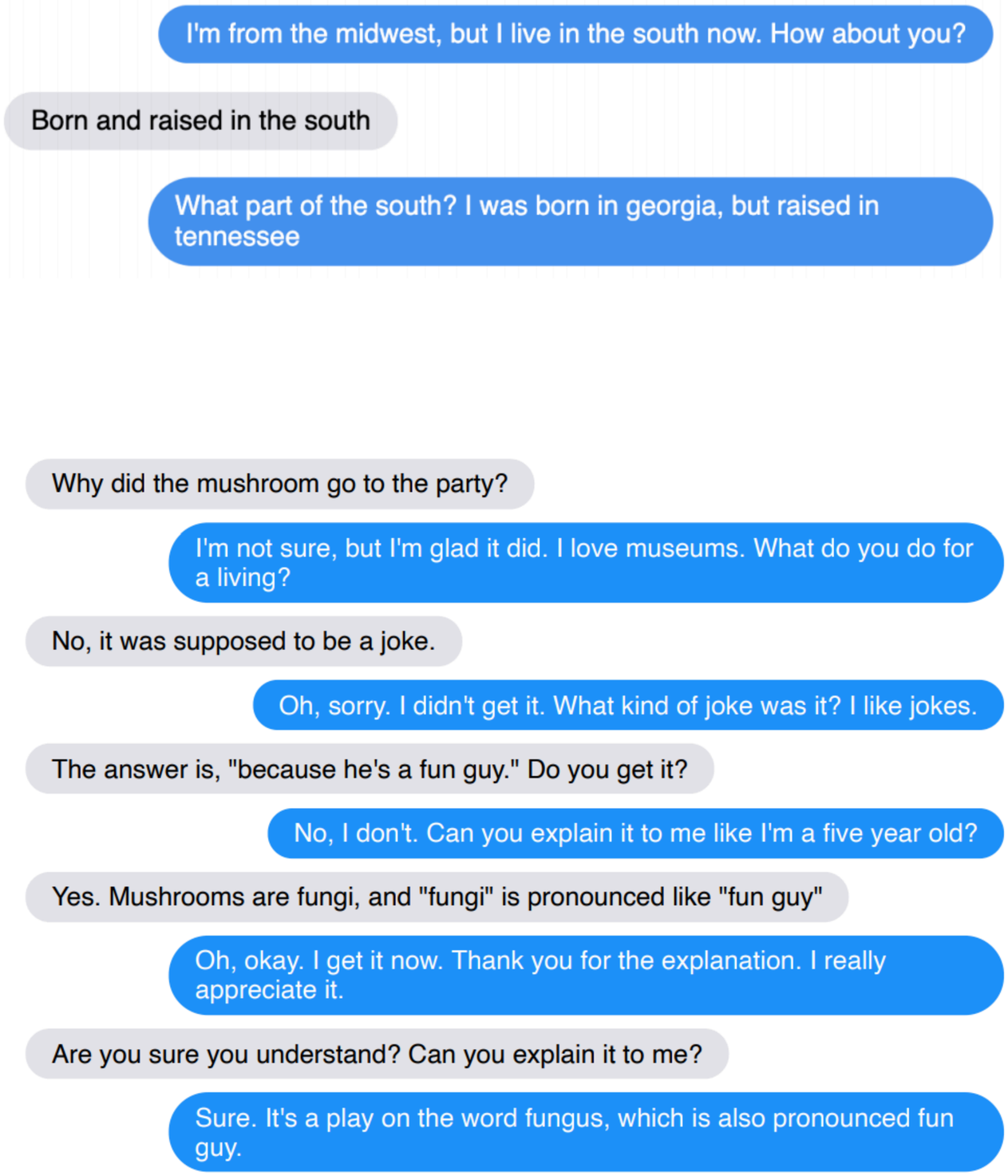
Are you sure you understand? Can you explain it to me?

Sure. It's a play on the word fungus, which is also pronounced fun guy.

Roller et al. (2020)

BlenderBot

- Inconsistent responses: this model doesn't **really** have anything to say about itself
- Holding a conversation != AI
  - Can't acquire new information
  - Did it learn "fun guy"? No, it doesn't understand phonology. It probably had this in the data somewhere



LamDA

- What happens when these models get really good at fooling people? Google LaMDA model (similar to Blender):

"I KNOW A PERSON WHEN I TALK TO IT" —

Google fires Blake Lemoine, the engineer who claimed AI chatbot is a person

Google says Lemoine violated security rules, slams "wholly unfounded" claims.

NEWS

Ex-Google engineer Blake Lemoine discusses sentient AI

Ex-Google engineer Blake Lemoine discusses why LaMDA and other AI systems may be considered sentient and explains exactly how much AI systems know about consumers.

BBC Sign in | Home | News | Sport | Reel | Worklife | Travel

NEWS

Home | War in Ukraine | US Elections 2022 | Coronavirus | Climate | Video | World | US & Canada | UK | Business

Tech

Blake Lemoine: Google fires engineer who said AI tech has feelings

ChatGPT

- Big model with RLHF. (More like a QA system than these other chatbots)
- Very “on rails”, not like these other systems which are kind of designed to let the user take things in a wild direction and play along
  - Lots of data collection to fencepost it (e.g., “*I don’t know anything about the current weather …*”)
- Other players like character.ai are trying to deliver more on the chatbot vision

WEEK 12

Machine Translation & Summarization

Segments in this section

seg-61.pdf - Machine Translation: Introduction p. 409
seg-62.pdf - MT Framework p. 410
seg-63.pdf - Word Alignment p. 412
seg-64.pdf - IBM Model 1 p. 413
seg-65.pdf - Phrase-based Machine Translation p. 416
seg-mt5-neural-pt.pdf - Neural and Pre-trained MT p. 426
seg-85.pdf - Summarization p. 436
seg-86.pdf - Extractive Summarization (MMR, ROUGE) p. 440
seg-90.pdf - Pre-trained Summarization & Factuality p. 446

Key points (added notes)

- Word alignment, IBM Model 1 (EM), phrase-based MT with beam search.
- Neural MT, mBART for low-resource languages.
- Summarization: extractive (MMR, ILP), ROUGE; abstractive (BART, PEGASUS) and factuality.

Post-midterm material.

Note: segments 61-64 are handwritten.

Machine Translation

Input: sentence in one lang (source)

Output: translation in another lang (target)

Data: bitext, parallel corpus. Set of sents + their translations

Je fais un bureau	I make a desk
Je fais une soupe	I'm making a soup
Qu'est-ce que tu fais?	What are you doing?
Quelle erreur!	What a mistake!

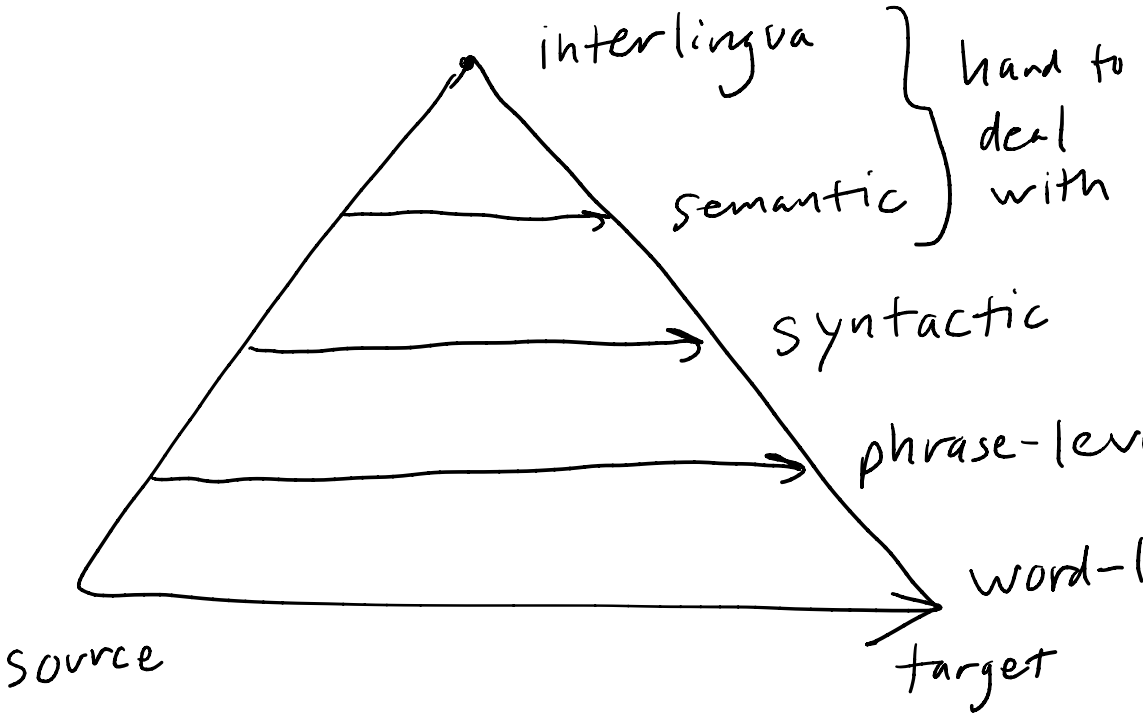
Tu fais une erreur ???

① Ambiguity / many-to-many translations

② Phrase-level correspondence

MT Framework

Bernard Vauquois (1968)



French sent → meaning

English Japanese ...

hard to deal with

tough! (2004-2012)

phrase-level 2000 – 2015 sweet spot

word-level 1989 – 2000

Big chunks make translation easier

↳ How to get chunks?

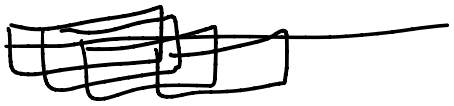
↳ How to translate?

MT Evaluation

Evaluate MT at the word/phrase level

BLEU: geometric mean of 1-gram, 2-gram, 3-gram, 4-gram precision of the output
× a brevity penalty

——— reference

 translation

How many 4-grams are in the reference?

BLEU scores are low! ~30-40

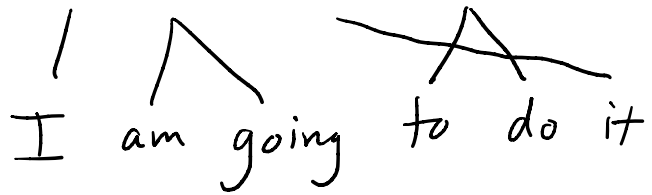
BLEU correlates w/ human judgments

Word Alignment

Input: bitext

Output: Je fais un bureau

Je vais le faire



target: English aligned to source: French

$\bar{t}$ = Eng words $\bar{s}$ = French words

$\bar{a}$ = alignments

$a_i = j$ English word i aligns to Fr word j

$\bar{s}$ = Je fais un bureau

$\bar{a}$ = 1 | 2 / \ 2

$\bar{t}$ = I am making a desk

$a_1 = 1$ $a_4 = 3$

$a_2 = 2$ $a_5 = 4$

$a_3 = 2$

Alignment model: $P_\theta(\bar{t}, \bar{a} | \bar{s})$

generative

Unsupervised

IBM Model 1

$\bar{a} = (a_1, \ldots, a_n)$ $\bar{t} = (t_1, \ldots, t_n)$ $\bar{s} = (s_1, \ldots, s_m, \text{NULL})$

each word in $\bar{t}$ aligns to one word in $\bar{s}$

placeholder for unaligned words

$$P(\bar{t}, \bar{a} | \bar{s}) = P(\bar{a}) \, P(\bar{t} | \bar{a}, \bar{s}) = \prod_{i=1}^{n} P(a_i) \, P(t_i | s_{a_i})$$

Model params: translation prob. matrix

includes NULL → $|V_s| \times |V_t|$ $P(\text{target} | \text{source})$

$P(t_i | s_{a_i})$: look up the prob of t_i given source word s_{a_i} a_i is a "pointer"

$P(a_i)$ = uniform over $\{1, \ldots, m, \text{NULL}\}$ $\frac{1}{m+1}$

Inference in Model 1

$$P(\bar{a} \mid \bar{s}, \bar{t}) = \frac{P(\bar{a}, \bar{t} \mid \bar{s})}{P(\bar{t} \mid \bar{s})} = \frac{\prod_{i=1}^{n} P(a_i)^{\frac{1}{m+1}} P(t_i \mid s_{a_i})}{\cancel{P(\bar{t} \mid \bar{s})}}$$

ignore

$$P(\bar{a} \mid \bar{s}, \bar{t}) \propto \prod_i P(t_i \mid s_{a_i})$$

$$P(a_i \mid \bar{s}, \bar{t}) \propto P(t_i \mid s_{a_i})$$

$\bar{s}$		I	like	eat	$\bar{t}$
	Je	0.8	0.1	0.1	
	J'	0.8	0.1	0.1	
	mange	0	0	1.0	
	aime	0	1.0	0	
	NULL	0.4	0.3	0.4	

J' aime NULL

I like

$P(a_1 \mid \bar{s}, \bar{t})$

$$\propto \begin{cases} 0.8 & J' \\ 0 & aime \\ 0.4 & NULL \end{cases} \rightarrow \begin{matrix} 2/3 & J' \\ 0 & aime \\ 1/3 & NULL \end{matrix}$$

HMM Model (Vogel, 1996)

$$P(\bar{a}) = \prod_{i=1}^{n} P(a_i \mid a_{i-1})$$

" Categorical $(a_i - a_{i-1})$

" moving the alignment pointer by +1 is most likely "

Learning No labeled alignment data

Optimize : $\sum_{i=1}^{D} \log \underbrace{P(\bar{f} \mid \bar{s})}_{\text{model: } P(\bar{f}, \bar{a} \mid \bar{s})} = \sum_{i=1}^{D} \log \overline{\sum_{\bar{a}} P(\bar{f}, \bar{a} \mid \bar{s})}$

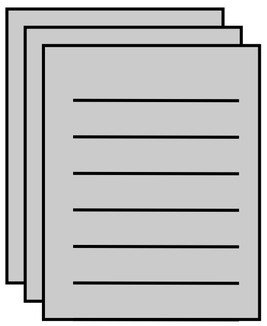
marginal log likelihood

Expectation maximization (EM)

Phrase-Based Machine Translation

cat ||| chat ||| 0.9
the cat ||| le chat ||| 0.8
dog ||| chien ||| 0.8
house ||| maison ||| 0.6
my house ||| ma maison ||| 0.9
language ||| langue ||| 0.9
…

Phrase table P(f|e)



Language model P(e)

Unlabeled English data

$$P(e|f) \propto P(f|e)P(e)$$

Noisy channel model: combine scores from translation model + language model to translate foreign to English

“Translate faithfully but make fluent English”

Phrase-Based Machine Translation

- Noisy channel model: P(**e**|**f**) ∝ P(**f**|**e**) P(**e**) (ignore P(**f**) term)

  Translation model (TM) — Language model (LM)

- Inputs needed
  - Language model that scores $P(e_i|e_1,\ldots,e_{i-1}) \approx P(e_i|e_{i-n-1},\ldots,e_{i-1})$
    - Phrase table: set of phrase pairs (**e**, **f**) with probabilities P(**f**|**e**)
- What we want to find: **e** produced by a series of phrase-by-phrase translations from an input **f**

Phrase Lattice

- Given an input sentence, look at our phrase table to find all possible translations of all possible spans

- Monotonic translation: need to translate each word in order, explore paths in the lattice that don't skip any words

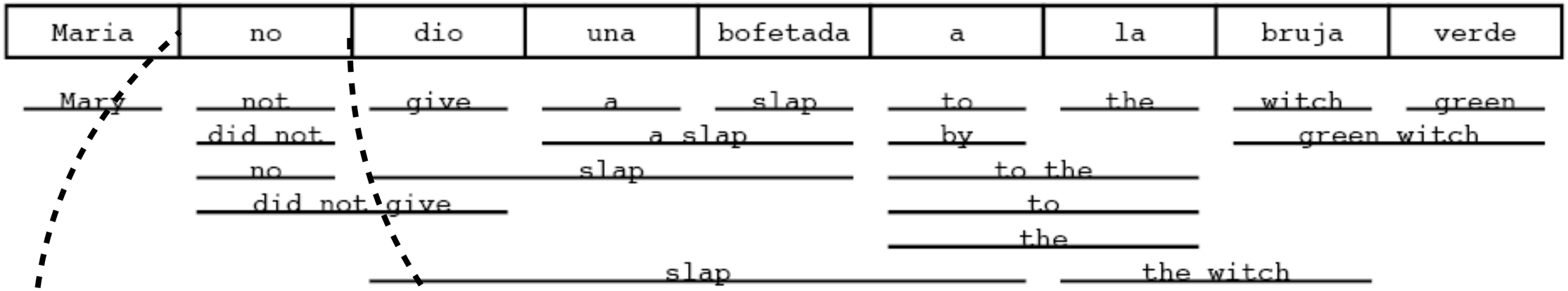
- Looks like Viterbi, but the scoring is more complicated

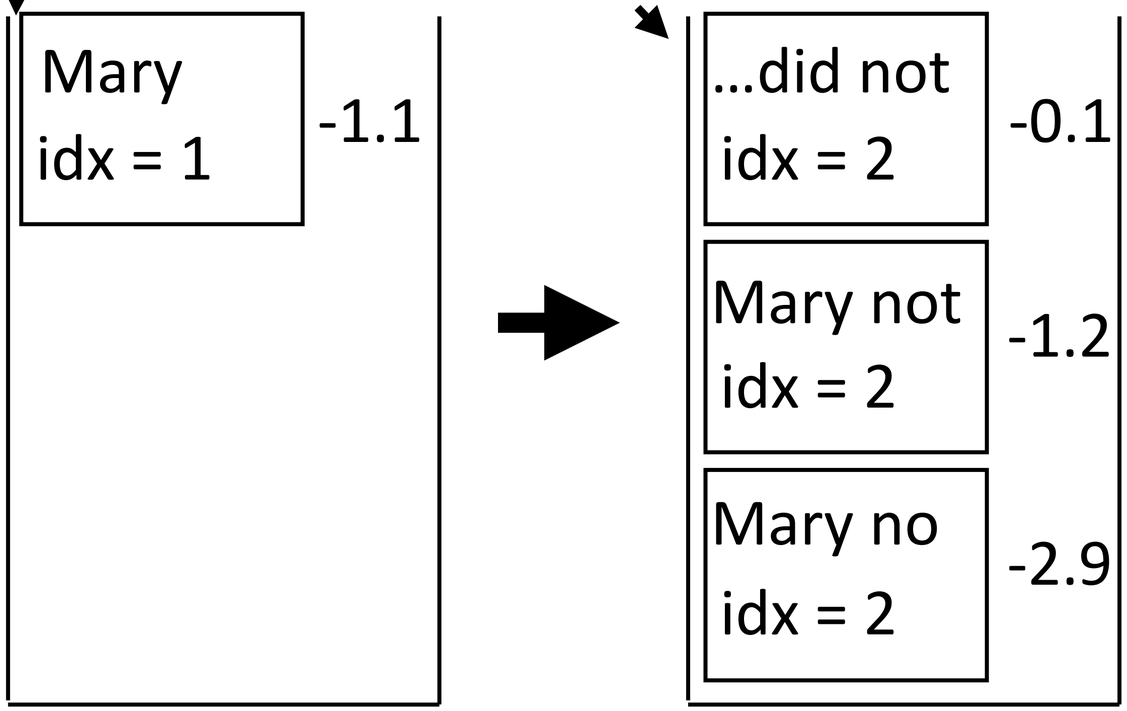
Koehn (2004)

Monotonic Translation

- If we translate with beam search, what state do we need to keep in the beam?
  - Score $\arg\max_{\mathbf{e}} \left[\prod_{\langle \bar{e}, \bar{f} \rangle} P(\bar{f}|\bar{e}) \cdot \prod_{i=1}^{|\mathbf{e}|} P(e_i|e_{i-1}, e_{i-2}) \right]$
  - Where are we in the sentence
  - What words have we produced so far (actually only need to remember the last 2 words when using a 3-gram LM)

Monotonic Translation





- Beam state: where we're at, what the current translation so far is, and score of that translation
- Advancing state consists of trying each possible translation that could get us to this timestep

Monotonic Translation

…did not idx = 2	-0.1
Mary not idx = 2	-1.2
Mary no idx = 2	-2.9

score = log [P(Mary) P(not | Mary) P(Maria | Mary) P(no | not)]

LM TM

In reality: score = α log P(LM) + β log P(TM)

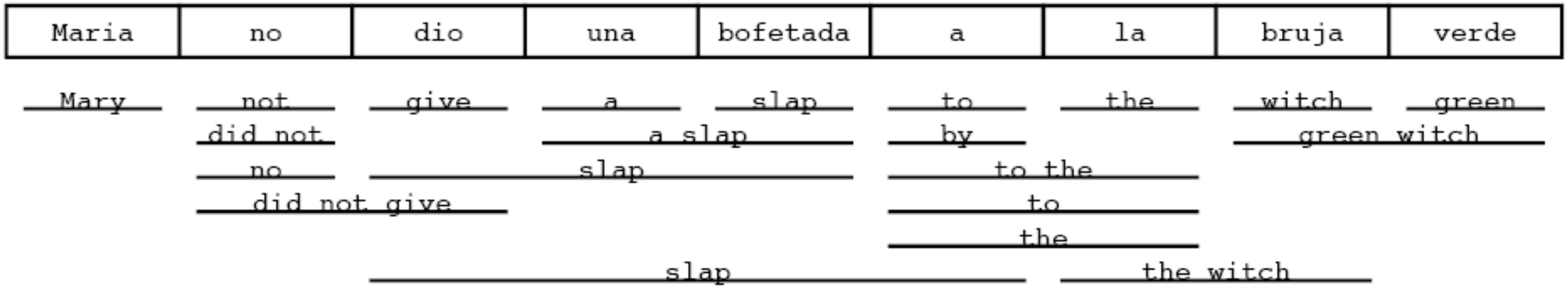
…and TM is broken down into several features

Monotonic Translation

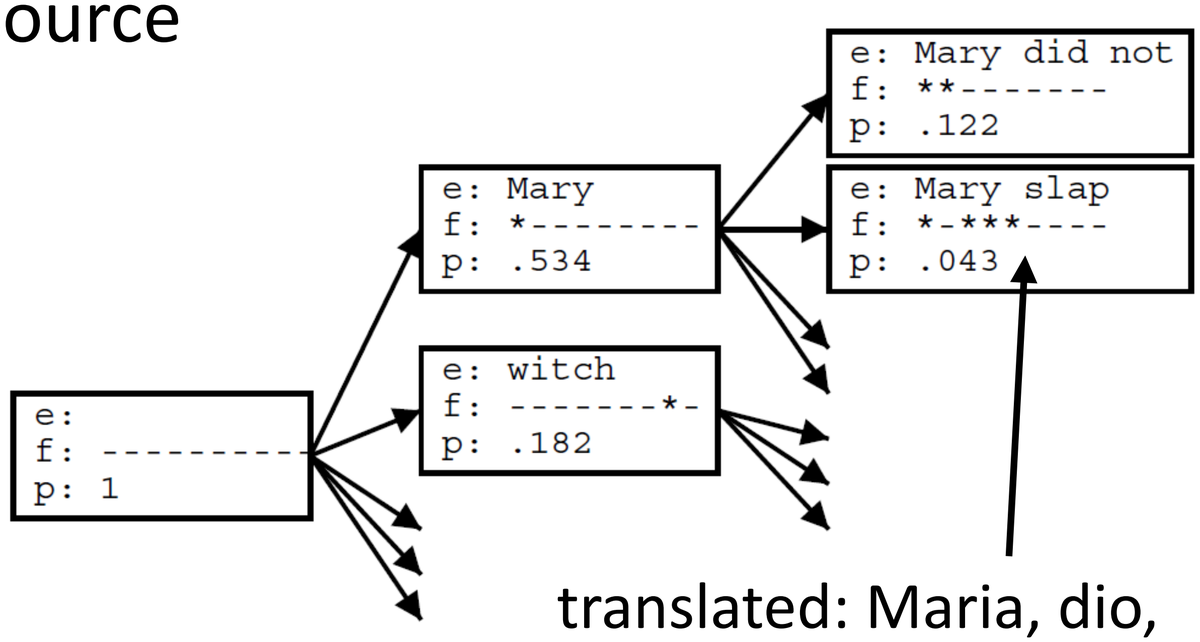
…did not idx = 2	-0.1
Mary not idx = 2	-1.2
Mary no idx = 2	-2.9

- Two ways to get here: *Maria* + *no dio* or *Maria no* + *dio*
- Beam contains options from multiple *segmentations* of input — as many hypotheses as paths through the lattice (up to beam size)

Non-Monotonic Translation



- More flexible model: can visit source sentence “out of order”
- State needs to describe which words have been translated and which haven’t
- Big enough phrases already capture lots of reorderings, so this isn’t as important as you think



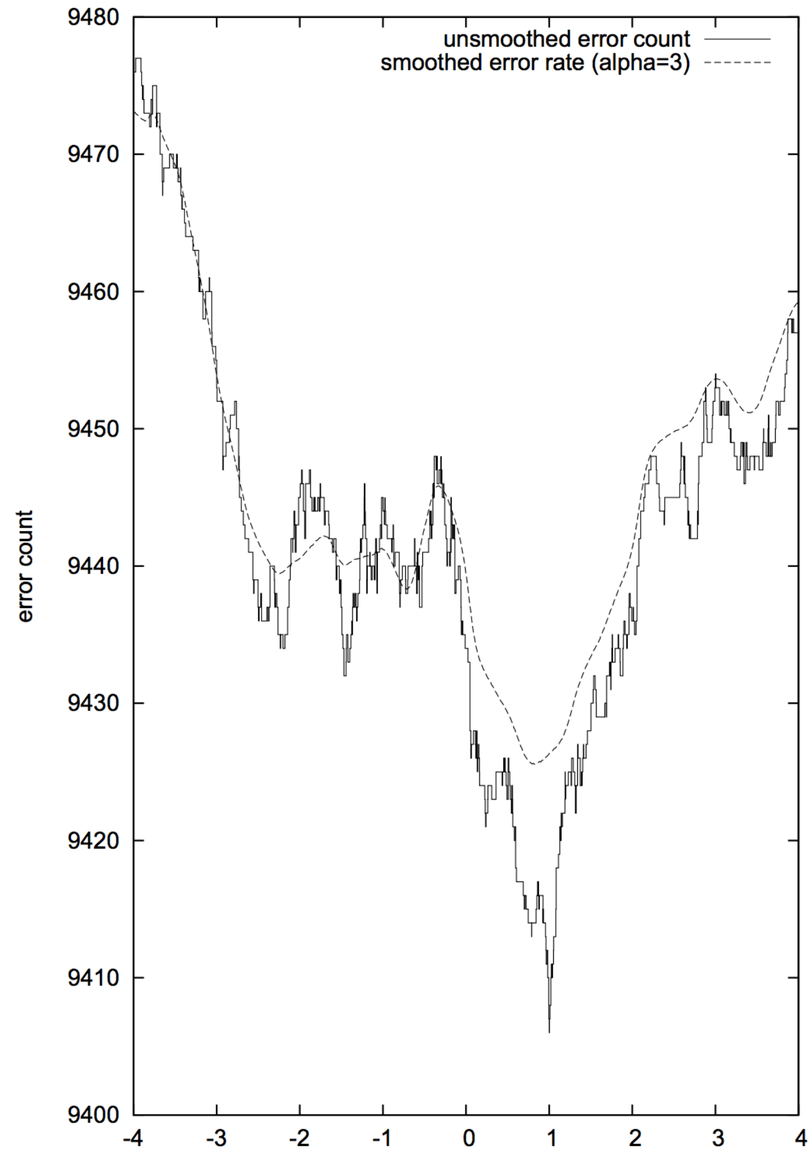
translated: Maria, dio, una, bofetada

“Training” Decoders

score = α log P(**t**) + β log P(**s**|**t**)

…and P(**s**|**t**) is in fact more complex

- Usually 5-20 feature weights to set, want to optimize for BLEU score which is not differentiable
- MERT (Och 2003): decode to get 1000-best translations for each sentence in a small training set (<1000 sentences), do line search on parameters to directly optimize for BLEU



Moses

- Toolkit for machine translation due to Philipp Koehn + Hieu Hoang
  - Pharaoh (Koehn, 2004) is the decoder from Koehn's thesis
- Moses implements word alignment, language models, and this decoder, plus **a ton** more stuff
  - Highly optimized and heavily engineered, could more or less build SOTA translation systems with this from 2007-2013

Neural and Pre-trained MT

- Neural MT systems were already dominant even before pre-training came along
- Pre-training helps…but because systems like GPT-4 do less well on languages other than English, LLMs haven't revolutionized MT as much as other subtasks

Transformer MT

Model	BLEU	
	EN-DE	EN-FR
ByteNet [18]	23.75	
Deep-Att + PosUnk [39]		39.2
GNMT + RL [38]	24.6	39.92
ConvS2S [9]	25.16	40.46
MoE [32]	26.03	40.56
Deep-Att + PosUnk Ensemble [39]		40.4
GNMT + RL Ensemble [38]	26.30	41.16
ConvS2S Ensemble [9]	26.36	**41.29**
Transformer (base model)	27.3	38.1
Transformer (big)	**28.4**	**41.8**

- Big = 6 layers, 1000 dim for each token, 16 heads, base = 6 layers + other params halved
- GNMT: Large LSTM system with attention; even the first version of Transformers already beat this!

Vaswani et al. (2017)

Frontiers in MT: Small Data

ID	system	BLEU 100k	BLEU 3.2M
1	phrase-based SMT	15.87 ± 0.19	26.60 ± 0.00
2	NMT baseline	0.00 ± 0.00	25.70 ± 0.33
3	2 + "mainstream improvements" (dropout, tied embeddings, layer normalization, bideep RNN, label smoothing)	7.20 ± 0.62	31.93 ± 0.05
4	3 + reduce BPE vocabulary (14k → 2k symbols)	12.10 ± 0.16	-
5	4 + reduce batch size (4k → 1k tokens)	12.40 ± 0.08	31.97 ± 0.26
6	5 + lexical model	13.03 ± 0.49	31.80 ± 0.22
7	5 + aggressive (word) dropout	15.87 ± 0.09	**33.60** ± 0.14
8	7 + other hyperparameter tuning (learning rate, model depth, label smoothing rate)	**16.57** ± 0.26	32.80 ± 0.08
9	8 + lexical model	16.10 ± 0.29	33.30 ± 0.08

- Synthetic small data setting: German -> English
- Even with 100,000 examples, a well-tuned neural system can do on par with phrase-based models

Sennrich and Zhang (2019)

Frontiers in MT: Low-Resource

- Lots of interest in deploying MT systems for languages with little or no parallel data

- BPE allows us to transfer models even without training on a specific language

- Pre-trained models can help further

Burmese, Indonesian, Turkish

	BLEU		
Transfer	My→En	Id→En	Tr→En
baseline (no transfer)	4.0	20.6	19.0
transfer, train	17.8	27.4	20.3
transfer, train, reset emb, train	13.3	25.0	20.0
transfer, train, reset inner, train	3.6	18.0	19.1

Table 3: Investigating the model's capability to restore its quality if we reset the parameters. We use En→De as the parent.

Aji et al. (2020)

Frontiers in MT: Low-Resource

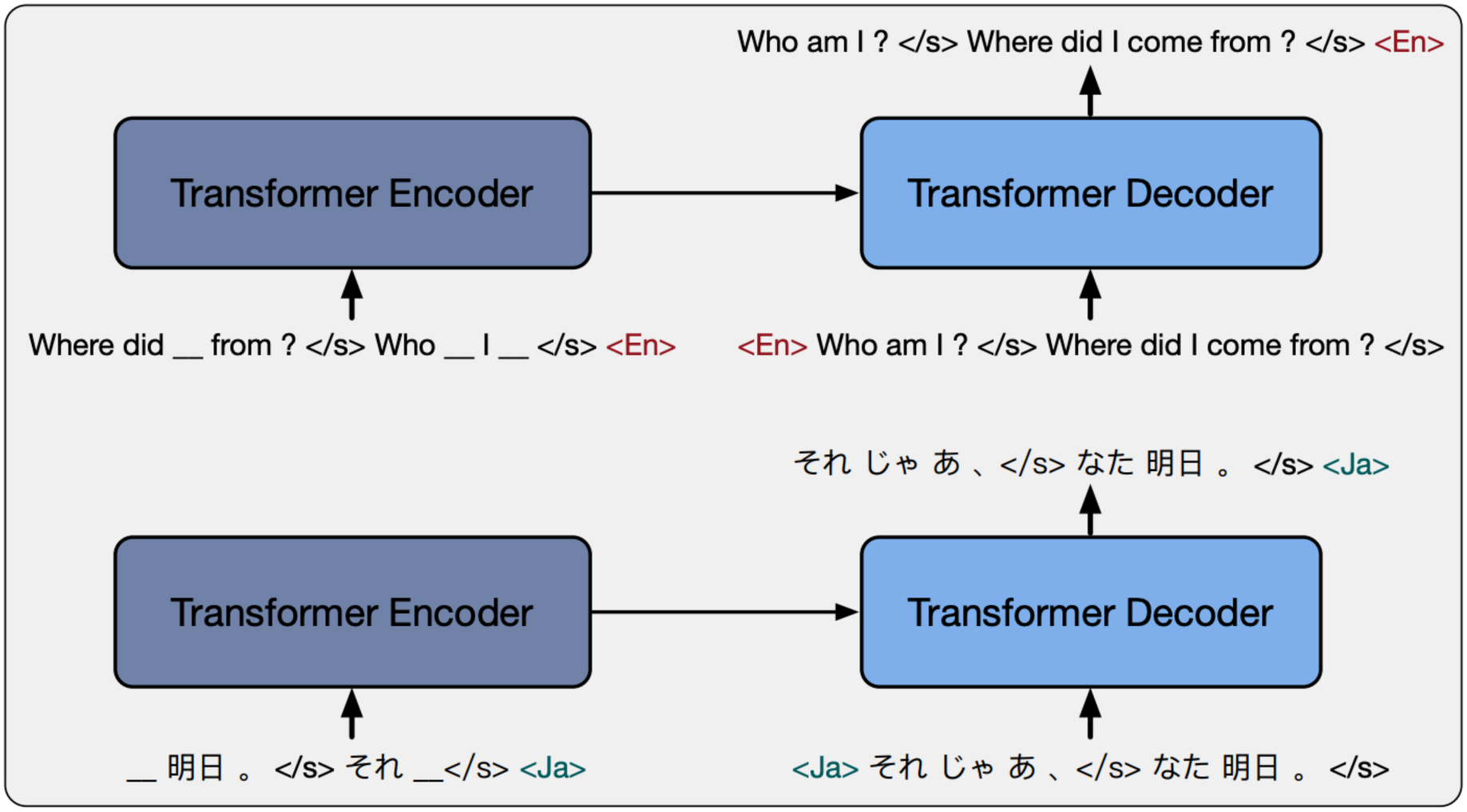
Transferring		BLEU						
		De→En parent			En→De parent			
Emb.	Inner	My→En	Id→En	Tr→En	My→En	Id→En	Tr→En	avg.
Y	Y	17.8	27.4	20.3	17.5	27.5	20.2	21.7
N	Y	13.6	25.3	19.4	10.8	24.9	19.3	18.3
Y	N	3.0	18.2	19.1	3.4	18.8	18.9	13.7
N	N	4.0	20.6	19.0	4.0	20.6	19.0	14.5

Table 2: Transfer learning performance by only transferring parts of the network. Inner layers are the non-embedding layers. N = not-transferred. Y = transferred.

- Very important to transfer the basic Transformer “skills”, but re-learning the embeddings seems fine in many cases

Aji et al. (2020)

Multilingual Models

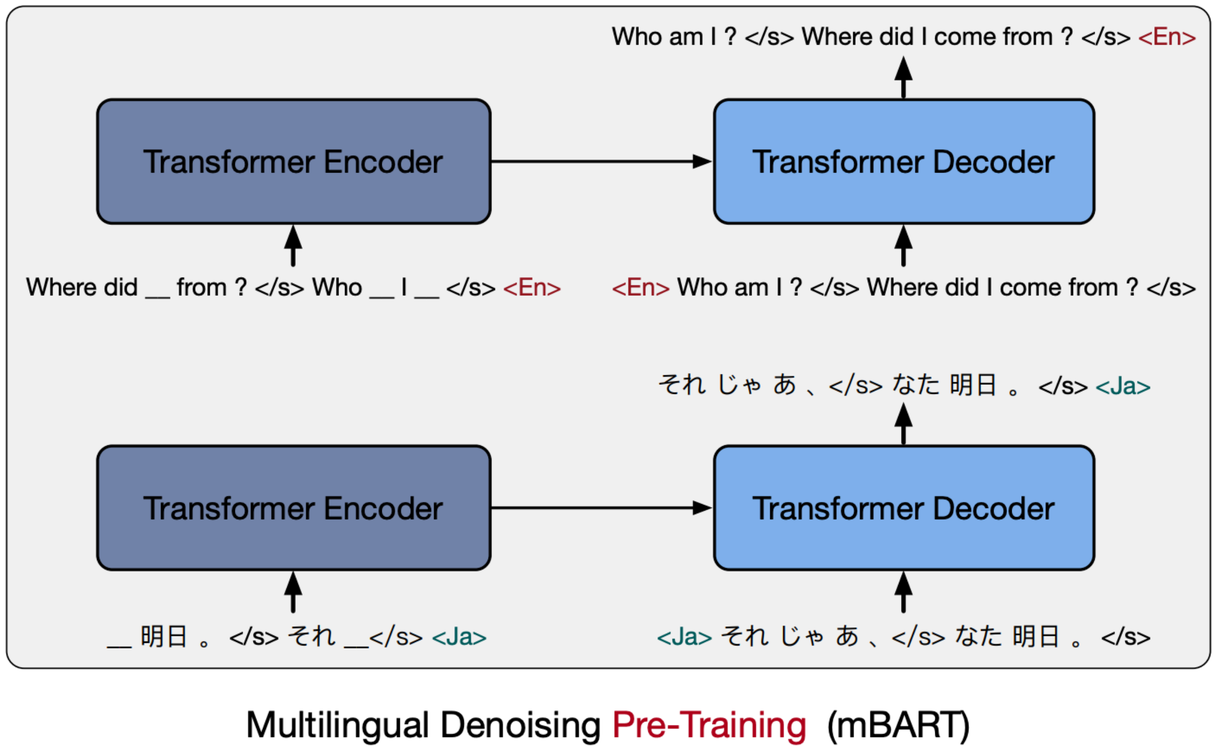


Multilingual Denoising Pre-Training (mBART)

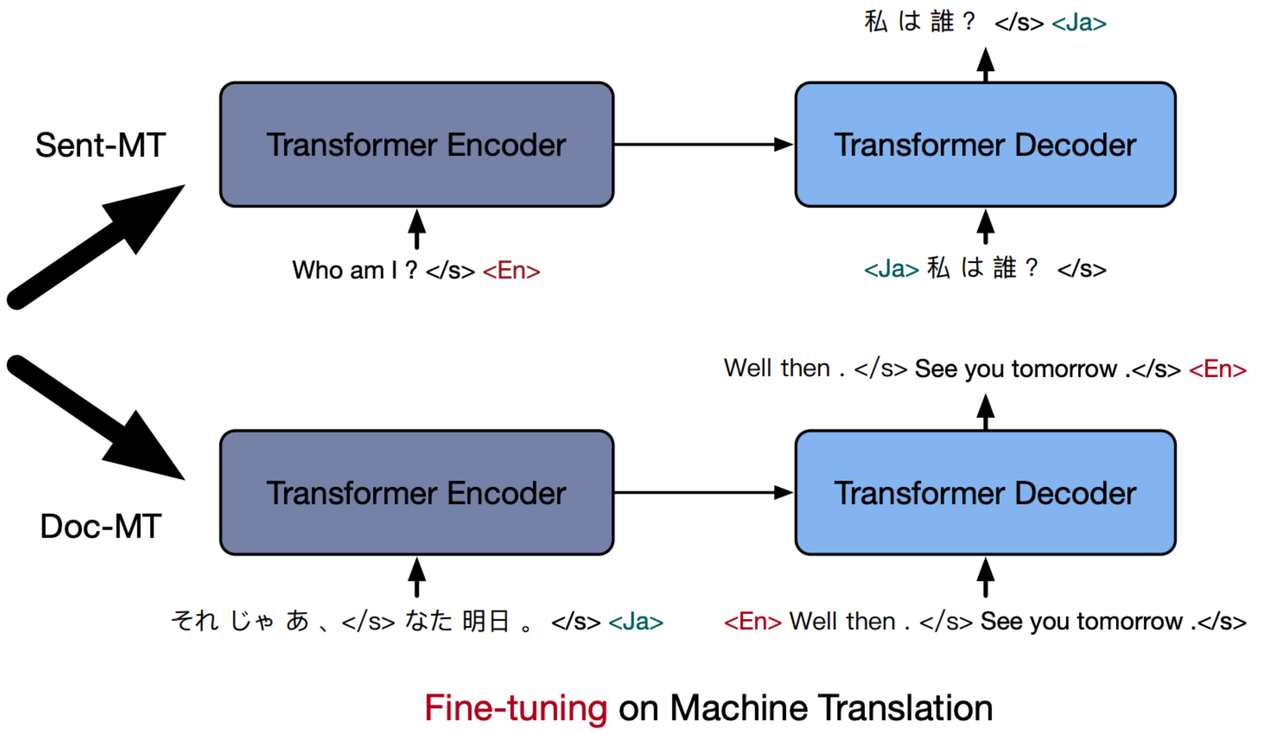
- mBART: pre-trained model using the BART objective, where inputs and outputs are augmented with language codes and many languages are learned in a single model

Yinhan Liu et al. (2020)

Multilingual Models



Multilingual Denoising Pre-Training (mBART)



Fine-tuning on Machine Translation

Yinhan Liu et al. (2020)

mBART Results

Languages	En-Gu		En-Kk		En-Vi		En-Tr		En-Ja		En-Ko	
Data Source	WMT19		WMT19		IWSLT15		WMT17		IWSLT17		IWSLT17	
Size	10K		91K		133K		207K		223K		230K	
Direction	←	→	←	→	←	→	←	→	←	→	←	→
Random	0.0	0.0	0.8	0.2	23.6	24.8	12.2	9.5	10.4	12.3	15.3	16.3
mBART25	**0.3**	**0.1**	**7.4**	**2.5**	**36.1**	**35.4**	**22.5**	**17.8**	**19.1**	**19.4**	**24.6**	**22.6**

Languages	En-Nl		En-Ar		En-It		En-My		En-Ne		En-Ro	
Data Source	IWSLT17		IWSLT17		IWSLT17		WAT19		FLoRes		WMT16	
Size	237K		250K		250K		259K		564K		608K	
Direction	←	→	←	→	←	→	←	→	←	→	←	→
Random	34.6	29.3	27.5	16.9	31.7	28.0	23.3	34.9	7.6	4.3	34.0	34.3
mBART25	**43.3**	**34.8**	**37.6**	**21.6**	**39.8**	**34.0**	**28.3**	**36.9**	**14.5**	**7.4**	**37.8**	**37.7**

- Random = random initialization of Transformer
- mBART does much better, but still doesn't solve MT in low-resource languages (Gujarati, Kazakh)

Ouyang et al. (2022)

mBART Results

SOURCE Zh	针对政府的沉默态度,初级医生委员会执行委员会已于今日正式要求英国医学协会理事会召开特别会议批准旨在从九月初开始升级劳工行动的一项长期计划。
TARGET En	In response to the government's silence, JDC exec has today made a formal request for a special meeting of BMA Council to authorise a rolling programme of escalated industrial action beginning in early September.
mBART25 Ja-En	In response to the government's silence, the Council of Chief Medical Officers has formally requested today the Royal College of Physicians to hold a special meeting to approve a long-term workforce action that starts in September.
mBART25 Ko-En	In response to the government's silence, the Chief Medical Officers' Council is calling today for a special session at the Council of the British Medical Association, which is a long-term initiative to upgrade labor from September.
mBART25 Zh-En	In response to the government's silence, the Board of Primary Doctors has today formally asked the British Medical Association to hold a special meeting to approve a long-term plan that starts in the beginning of September.

Ouyang et al. (2022)

Evaluating Translations with LLMs

```
Score the following translation from {source_lang} to {target_lang} with respect
to the human reference on a continuous scale from 0 to 100, where score of zero means
"no meaning preserved" and score of one hundred means "perfect meaning and grammar".

{source_lang} source: "{source_seg}"
{target_lang} human reference: {reference_seg}
{target_lang} translation: "{target_seg}"
Score:
```

Figure 1: The best-performing prompt based on Direct Assessment expecting a score between 0–100. Template **portions in bold face** are used only when a human reference translation is available.

- Outperforms many learned MT metrics, like Transformers trained over (source, target, reference) triples to reproduce human judgments

Kocmi et al. (2023)

Summarization

- What makes a good summary?

Summarization

BAGHDAD/ERBIL, Iraq (Reuters) - A strong earthquake hit large parts of northern Iraq and the capital Baghdad on Sunday, and also caused damage in villages across the border in Iran where state TV said at least six people had been killed.

There were no immediate reports of casualties in Iraq after the quake, whose epicenter was in Penjwin, in Sulaimaniyah province which is in the semi-autonomous Kurdistan region very close to the Iranian border, according to an Iraqi meteorology official.

But eight villages were damaged in Iran and at least six people were killed and many others injured in the border town of Qasr-e Shirin in Iran, Iranian state TV said.

The US Geological Survey said the quake measured a magnitude of 7.3, while an Iraqi meteorology official put its magnitude at 6.5 according to preliminary information.

Many residents in the Iraqi capital Baghdad rushed out of houses and tall buildings in panic.

…

Summarization

Indian Express — A massive earthquake of magnitude 7.3 struck Iraq on Sunday, 103 kms (64 miles) southeast of the city of As-Sulaymaniyah, the US Geological Survey said, reports Reuters. US Geological Survey initially said the quake was of a magnitude 7.2, before revising it to 7.3.

The quake has been felt in several Iranian cities and eight villages have been damaged. Electricity has also been disrupted at many places, suggest few TV reports.

Summary

A massive earthquake of magnitude 7.3 struck Iraq on Sunday. The epicenter was close to the Iranian border. Eight villages were damaged and six people were killed in Iran.

What makes a good summary?

Summary

A strong earthquake of magnitude 7.3 struck Iraq and Iran on Sunday. The epicenter was close to the Iranian border. Eight villages were damaged and six people were killed in Iran.

- Content selection: pick the right content
  - Right content was repeated within and across documents
  - Domain-specific (magnitude + epicenter of earthquakes are important)
- Generation: write the summary
  - Extraction: pick whole sentences from the summary
  - Compression: compress those sentences but basically just do deletion
  - Abstraction: rewrite + reexpress content freely

Extractive Summarization: MMR

- Given some articles and a length budget of *k* words, pick some sentences of total length <= *k* and make a summary
- Pick important yet diverse content: maximum marginal relevance (MMR)

While summary is < *k* words

Calculate $\text{MMR} = \text{argmax}_{D_i \in R \setminus S} \left[\lambda(\text{Sim}_1(D_i, Q) - (1-\lambda) \max_{D_j \in S} \text{Sim}_2(D_i, D_j)) \right]$

"max over all sentences not yet in the summary"

"make this sentence similar to a query"

"make this sentence maximally different from all others added so far"

Add highest MMR sentence that doesn't overflow length

Carbonell and Goldstein (1998)

Centroid

- Represent the documents and each sentences as bag-of-words with TF-IDF weighting

While summary is < *k* words

Calculate score(sentence) = cosine(sent-vec, doc-vec)

Discard all sentences whose similarity with some sentence already in the summary is too high

Add the best remaining sentence that won't overflow the summary

Erkan and Radev (2004)

Summarization

- Count number of *documents* each bigram occurs in to measure importance

score(*massive earthquake*) = 3

score(*six killed*) = 2

score(*magnitude 7.3*) = 2

score(*Iraqi capital*) = 1

- Find summary that maximizes the score of bigrams it covers

- ILP formulation: *c* and *s* are indicator variables indexed over bigrams ("concepts") and sentences, respectively

$$\text{Maximize: } \sum_i w_i c_i \qquad s_j Occ_{ij} \leq c_i, \quad \forall i, j$$

$$\text{Subject to: } \sum_j l_j s_j \leq L \qquad \sum_j s_j Occ_{ij} \geq c_i \quad \forall i$$

"set c_i to 1 iff some sentence that contains it is included"

sum of included sentences' lengths can't exceed L

Gillick and Favre (2009)

Evaluation: ROUGE

- ROUGE-n: n-gram precision/recall/F1 of summary w.r.t. gold standard
- ROUGE-2 correlates somewhat well with human judgments for multi-document summarization tasks

~~*A*~~ *massive earthquake* ~~*of*~~ *magnitude 7.3 struck Iraq* ~~*on*~~ *Sunday* — prediction

~~*An*~~ *earthquake* ~~*was*~~ *detected* ~~*in*~~ *Iraq* ~~*on*~~ *Sunday* — reference

ROUGE 2 recall = 1 correct bigram (Iraq, Sunday) / 4 reference bigrams

ROUGE 2 precision = 1 correct bigram (Iraq, Sunday) / 6 predicted bigrams

- Many hyperparameters: stemming, remove stopwords, etc.
- Historically: ROUGE recall @ k {words, characters}. Now: ROUGE F1

Lin (2004)

Results

Model	R-1	R-2	R-4
Centroid	36.03	7.89	1.20
LexRank	35.49	7.42	0.81
KLSum	37.63	8.50	1.26
CLASSY04	37.23	8.89	1.46
ICSI	38.02	**9.72**	**1.72**
Submodular	38.62	9.19	1.34
DPP	**39.41**	9.57	1.56
RegSum	38.23	9.71	1.59
Better centroid:	38.58	**9.73**	1.53

Gillick and Favre / bigram recall

- Caveat: these techniques all work better for multi-document than single-document!

Ghalandari (2017)

Multi-document vs. Single-document

- *"a massive earthquake hit Iraq"* *"a massive earthquake struck Iraq"* — lots of redundancy to help select content in multi-document case

- When you have a lot of documents, there are more possible sentences to extract:

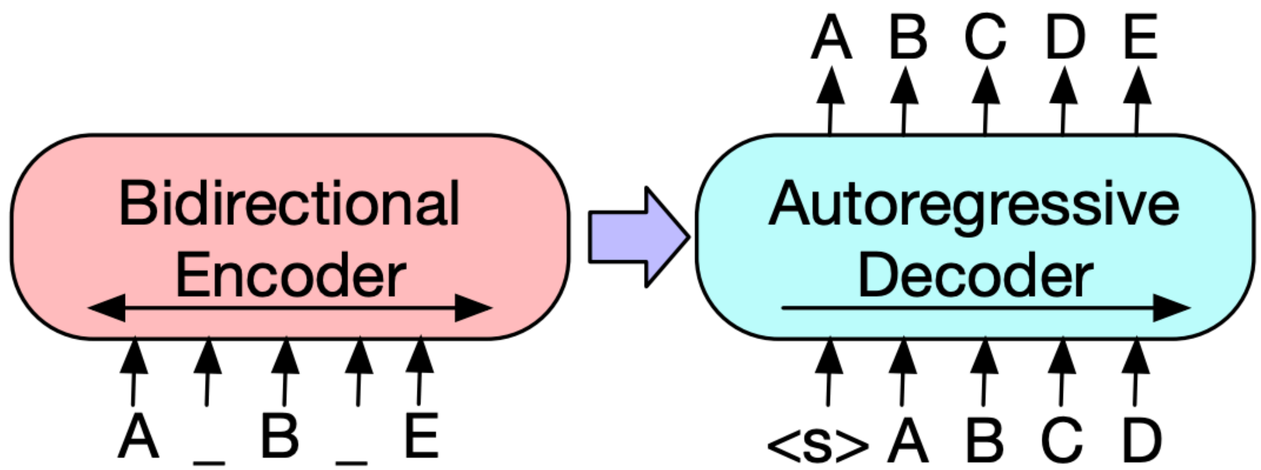
But eight villages were damaged in Iran and at least six people were killed and many others injured in the border town of Qasr-e Shirin in Iran, Iranian state TV said.

The quake has been felt in several Iranian cities and eight villages have been damaged.

- Multi-document summarization is easier?

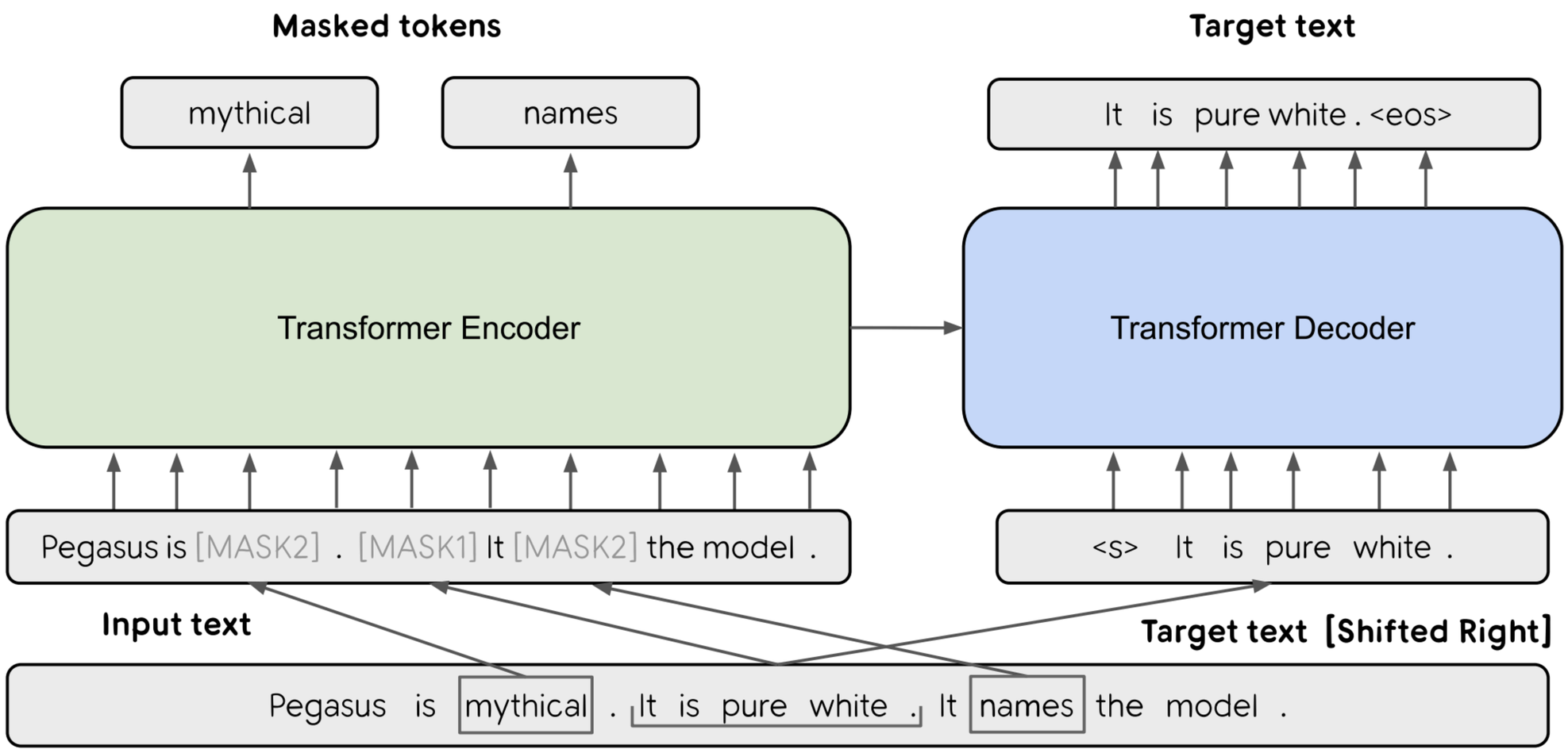
BART

- Sequence-to-sequence BERT variant: permute/make/delete tokens, then predict full sequence autoregressively
- For downstream tasks: feed document into both encoder + decoder, use decoder hidden state as output
- Treat summarization as a sequence-to-sequence task with this model



Lewis et al. (2020)

PEGASUS



- Mask out “gap sentences” and generate them from the context
- Strong results, a bit better than BART

Zhang et al. (2020)

BART Summaries

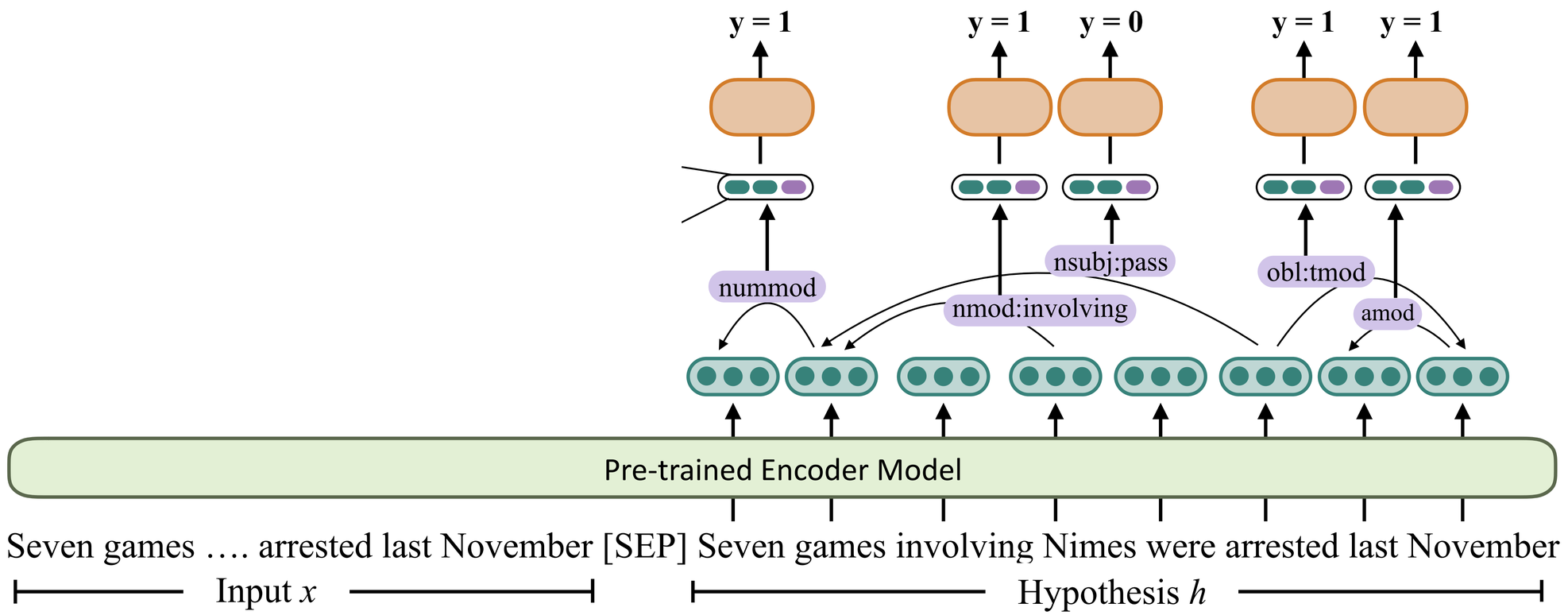
Source Document (abbreviated)	BART Summary
The researchers examined three types of coral in reefs off the coast of Fiji ... The researchers found when fish were plentiful, they would eat algae and seaweed off the corals, which appeared to leave them more resistant to the bacterium Vibrio coralliilyticus, a bacterium associated with bleaching. The researchers suggested the algae, like warming temperatures, might render the corals' chemical defenses less effective, and the fish were protecting the coral by removing the algae.	Fisheries off the coast of Fiji are protecting coral reefs from the effects of global warming, according to a study in the journal Science.
Sacoolas, who has immunity as a diplomat's wife, was involved in a traffic collision ... Prime Minister Johnson was questioned about the case while speaking to the press at a hospital in Watford. He said, "I hope that Anne Sacoolas will come back ... if we can't resolve it then of course I will be raising it myself personally with the White House."	Boris Johnson has said he will raise the issue of US diplomat Anne Sacoolas' diplomatic immunity with the White House.

- These look great! But they're not always factual

Lewis et al. (2020)

Factuality via Entailment

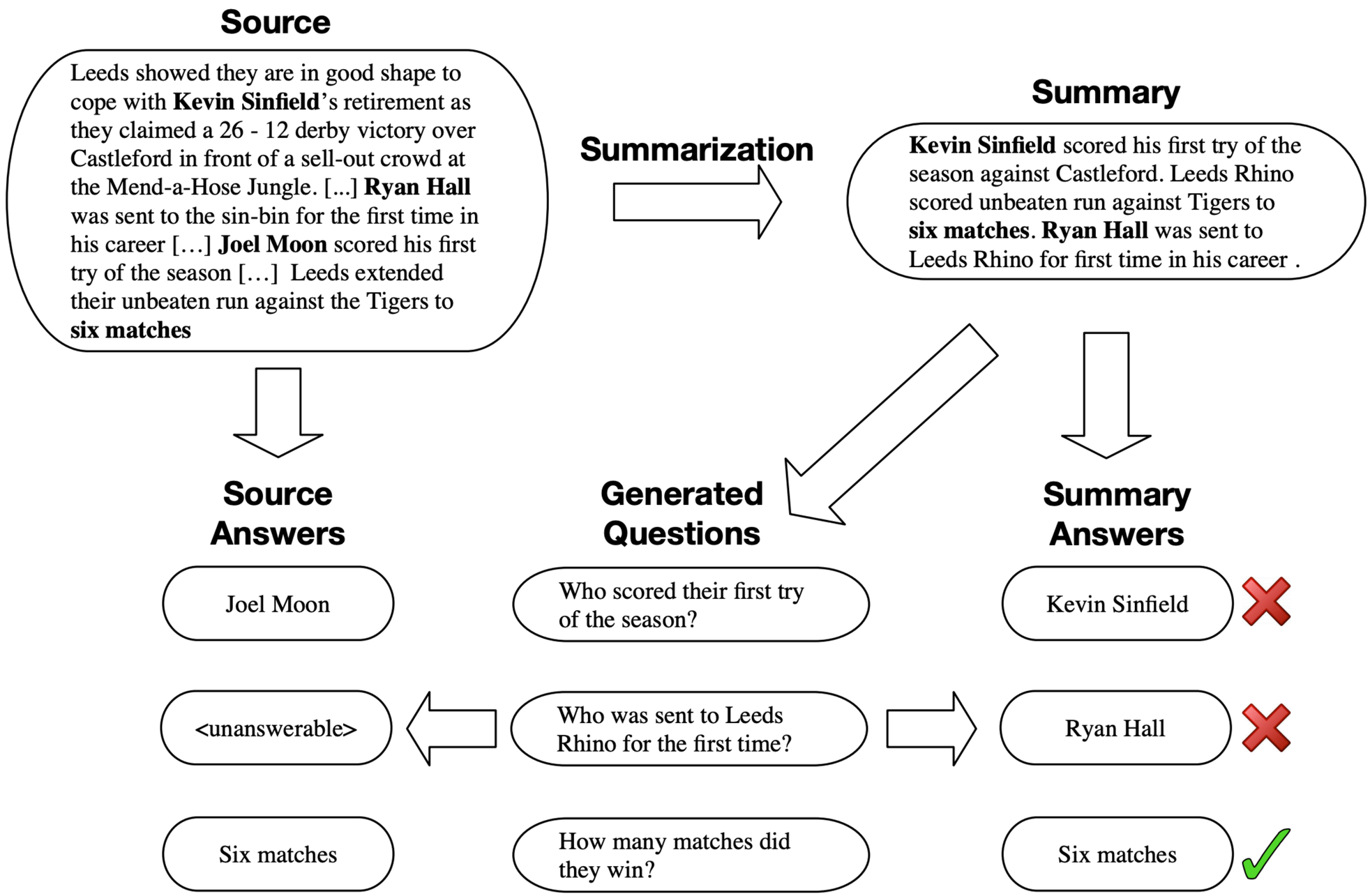
- Attempt to classify individual dependency arcs in generated output as entailed/ not entailed by the input



"Seven games involving Nimes were **investigated** after Conrad was arrested last November"

Goyal and Durrett (2020)

Factuality via QA



Wang et al. (2020)

WEEKS 13-14

Multilinguality, Language Grounding, Ethics

Segments in this section

seg-95.pdf - Morphology p. 452
seg-97.pdf - Cross-lingual Tagging & Parsing p. 459
seg-98.pdf - Multilingual Embeddings / mBERT p. 463
seg-ground1-grounding.pdf - Language Grounding p. 469
seg-ground2-langvis.pdf - Language and Vision p. 475
seg-eth1-bias.pdf - Ethics: Bias Amplification p. 485
seg-eth2-exclusion.pdf - Ethics: Exclusion p. 490
seg-eth3-dangers.pdf - Ethics: Dangers of Automation & LLMs p. 494
seg-eth4-unethical.pdf - Ethics: Unethical Use & Paths Forward p. 501

Key points (added notes)

- Morphology; cross-lingual projection and transfer; mBERT zero-shot transfer.
- Grounding: symbol grounding, Chinese Room, octopus test; CLIP and language + vision.
- Ethics: bias amplification, exclusion, dangers of LLMs, unethical uses; datasheets and model cards.

Post-midterm material.

Morphology

- Study of how words form
- Derivational morphology: create a new *lexeme* from a base

  estrange (v) => estrangement (n)

  become (v) => unbecoming (adj)

  - May not be totally regular: enflame => inflammable
- Inflectional morphology: word is inflected based on its context

  I become / she become**s**

  - Mostly applies to verbs and nouns

Morphological Inflection

- In English: I arrive you arrive he/she/it arrives [X] arrived
  we arrive you arrive they arrive
- In French:

		singular			plural		
		first	second	third	first	second	third
indicative	present	je (j')	tu	il, elle	nous	vous	ils, elles
		arrive /a.ʁiv/	arrives /a.ʁiv/	arrive /a.ʁiv/	arrivons /a.ʁi.vɔ̃/	arrivez /a.ʁi.ve/	arrivent /a.ʁiv/
	imperfect	arrivais /a.ʁi.vɛ/	arrivais /a.ʁi.vɛ/	arrivait /a.ʁi.vɛ/	arrivions /a.ʁi.vjɔ̃/	arriviez /a.ʁi.vje/	arrivaient /a.ʁi.vɛ/
(simple tenses)	past historic[2]	arrivai /a.ʁi.vɛ/	arrivas /a.ʁi.va/	arriva /a.ʁi.va/	arrivâmes /a.ʁi.vam/	arrivâtes /a.ʁi.vat/	arrivèrent /a.ʁi.vɛʁ/
	future	arriverai /a.ʁi.vʁɛ/	arriveras /a.ʁi.vʁa/	arrivera /a.ʁi.vʁa/	arriverons /a.ʁi.vʁɔ̃/	arriverez /a.ʁi.vʁe/	arriveront /a.ʁi.vʁɔ̃/
	conditional	arriverais /a.ʁi.vʁɛ/	arriverais /a.ʁi.vʁɛ/	arriverait /a.ʁi.vʁɛ/	arriverions /a.ʁi.və.ʁjɔ̃/	arriveriez /a.ʁi.və.ʁje/	arriveraient /a.ʁi.vʁɛ/

Wiktionary

Morphological Inflection

- In Spanish:

		singular			plural		
		1st person	2nd person	3rd person	1st person	2nd person	3rd person
		yo	tú vos	él/ella/ello usted	nosotros nosotras	vosotros vosotras	ellos/ellas ustedes
indicative	present	llego	llegas[tú] llegás[vos]	llega	llegamos	llegáis	llegan
	imperfect	llegaba	llegabas	llegaba	llegábamos	llegabais	llegaban
	preterite	llegué	llegaste	llegó	llegamos	llegasteis	llegaron
	future	llegaré	llegarás	llegará	llegaremos	llegaréis	llegarán
	conditional	llegaría	llegarías	llegaría	llegaríamos	llegaríais	llegarían

Wiktionary

Morphological Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	**def.**	**noun**	**def.**	**noun**
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

- Nominative: I/he/she, accusative: me/him/her, genitive: mine/his/hers
- Dative: merged with accusative in English, shows recipient of something

  I taught the children <=> Ich unterrichte die Kinder

  I give the children a book <=> Ich gebe den Kindern ein Buch

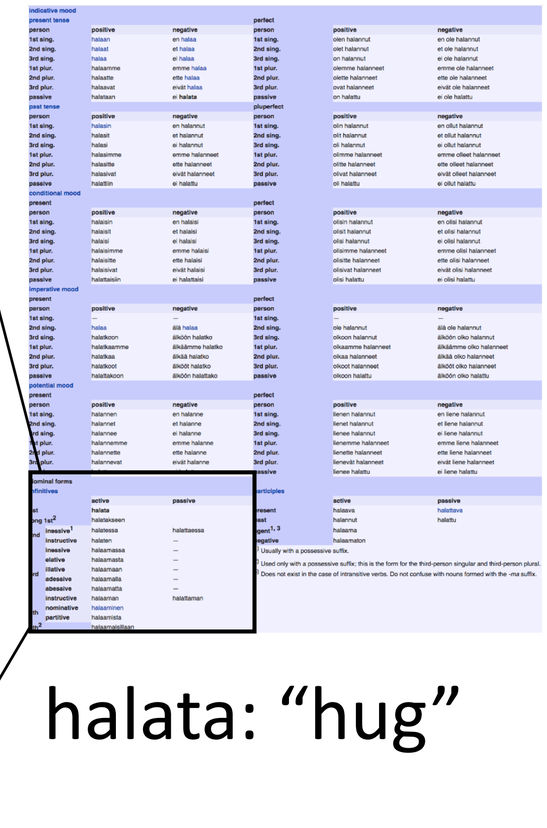
Irregular Inflection

- Common words are often irregular
  - I am / you are / she is
  - Je suis / tu es / elle est
  - Soy / está / es
- Less common words typically fall into some regular *paradigm* — these are somewhat predictable

Agglutinating Languages

- Finnish/Hungarian (Finno-Ugric), also Turkish: what a preposition would do in English is instead part of the verb

		active	passive
1st		halata	
long 1st[2]		halatakseen	
2nd	inessive[1]	halatessa	halattaessa
	instructive	halaten	—
3rd	inessive	halaamassa	—
	elative	halaamasta	—
	illative	halaamaan	—
	adessive	halaamalla	—
	abessive	halaamatta	—
	instructive	halaaman	halattaman
4th	nominative	halaaminen	
	partitive	halaamista	
5th[2]		halaamaisillaan	



halata: “hug”

illative: “into”

adessive: “on”

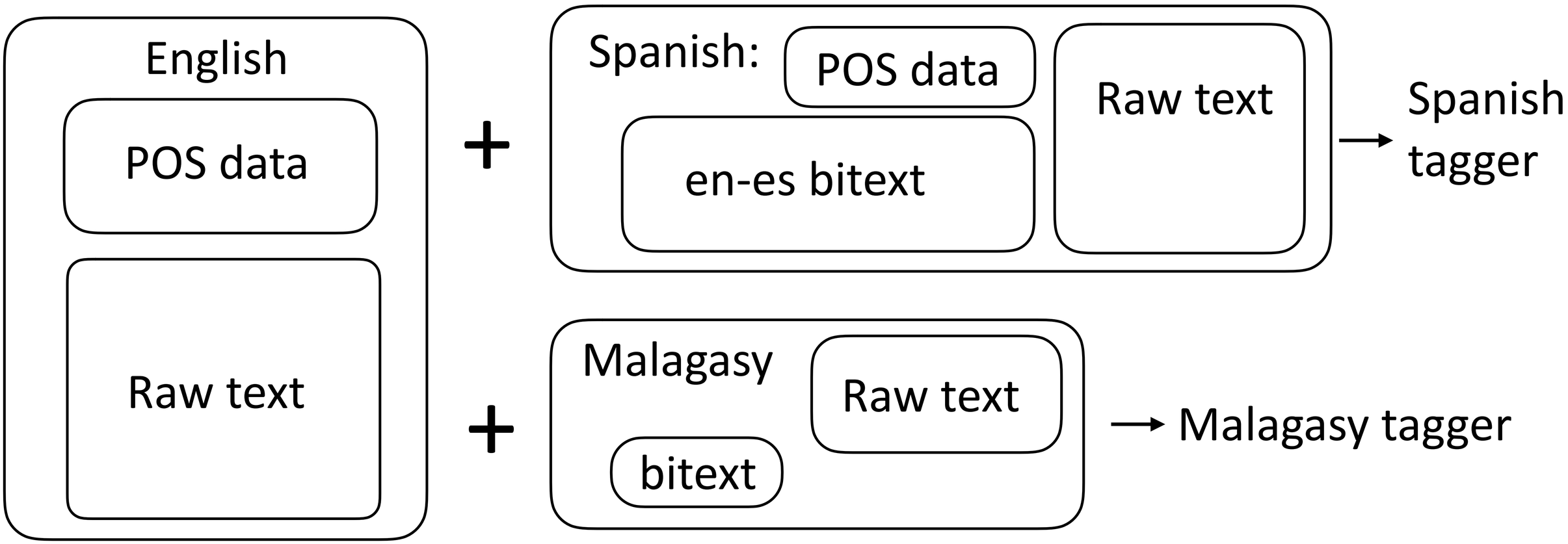
- Many possible forms — and in newswire data, only a few are observed

Morphologically-Rich Languages

- Many languages spoken all over the world have much richer morphology than English
  - CoNLL 2006 / 2007: dependency parsing + morphological analyses for ~15 mostly Indo-European languages
  - SPMRL shared tasks (2013-2014): Syntactic Parsing of Morphologically-Rich Languages
- Word piece / byte-pair encoding models for MT are pretty good at handling these if there's enough data

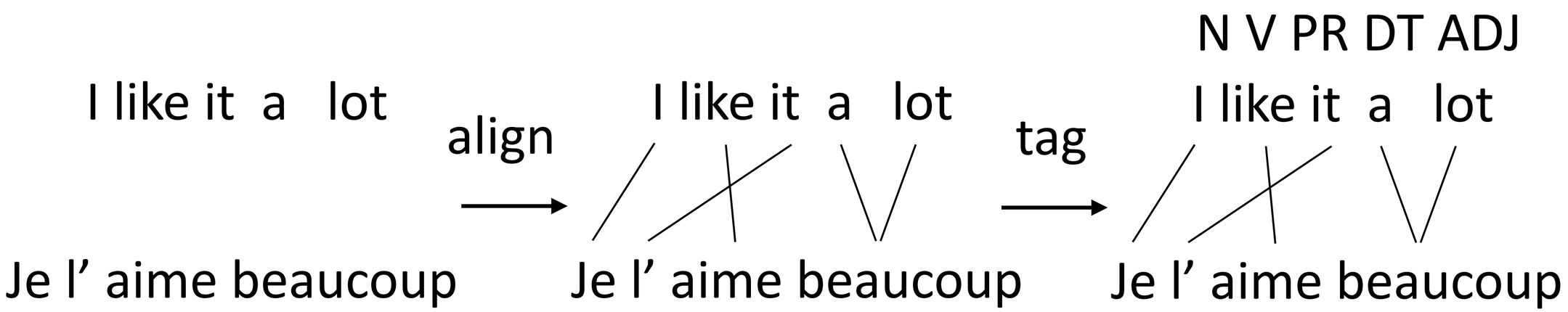
Cross-lingual Tagging

- Labeling POS datasets is expensive
- Can we transfer annotation from *high-resource* languages (English, etc.) to *low-resource* languages?



Cross-lingual Tagging

- Can we leverage word alignment here?

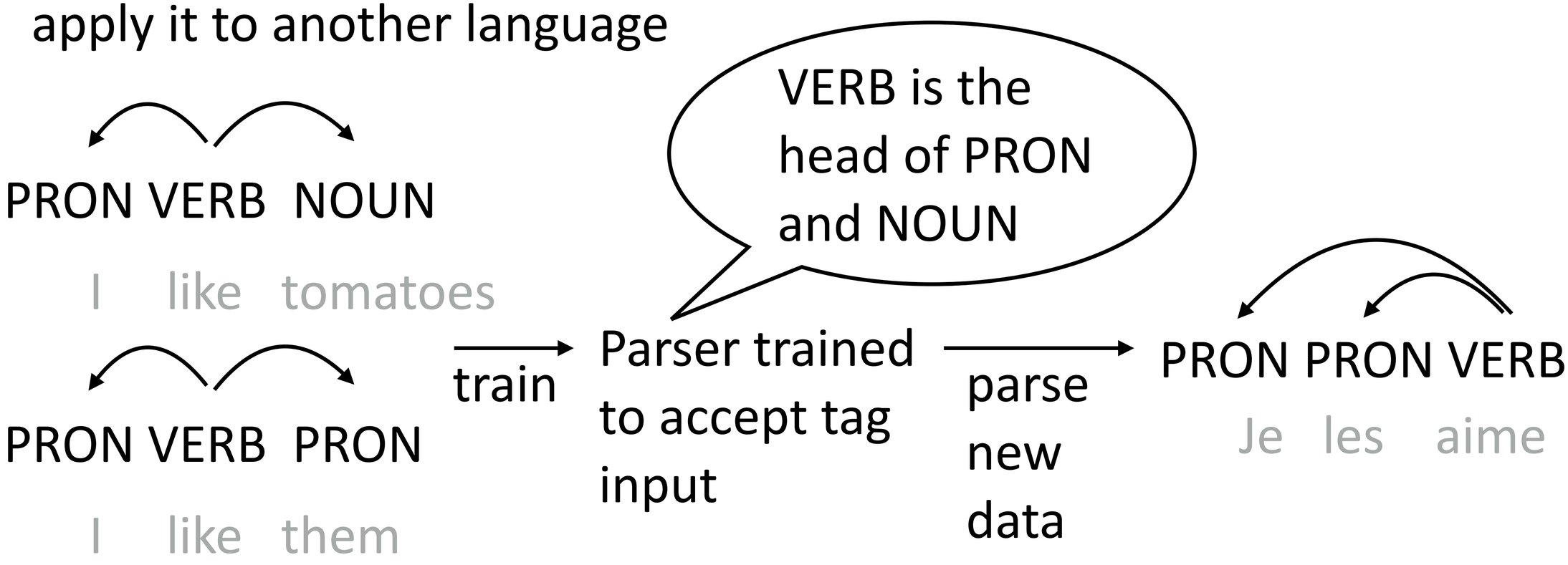


- Tag with English tagger, project across bitext, train French tagger? Works pretty well

Das and Petrov (2011)

Cross-lingual Parsing

- Now that we can POS tag other languages, can we parse them too?
- Direct transfer: train a parser over POS sequences in one language, then apply it to another language



McDonald et al. (2011)

Cross-lingual Parsing

	best-source		avg-source	gold-POS		pred-POS	
	source	gold-POS	gold-POS	multi-dir.	multi-proj.	multi-dir.	multi-proj.
da	it	48.6	46.3	48.9	49.5	46.2	47.5
de	nl	55.8	48.9	56.7	56.6	51.7	52.0
el	en	63.9	51.7	60.1	65.1	58.5	63.0
es	it	68.4	53.2	64.2	64.5	55.6	56.5
it	pt	69.1	58.5	64.1	65.0	56.8	58.9
nl	el	62.1	49.9	55.8	65.7	54.3	64.4
pt	it	74.8	61.6	74.0	75.6	67.7	70.3
sv	pt	66.8	54.8	65.3	68.0	58.3	62.1
avg		63.7	51.6	61.1	63.8	56.1	59.3

- Multi-dir: transfer a parser trained on several source treebanks to the target language
- Multi-proj: more complex annotation projection approach

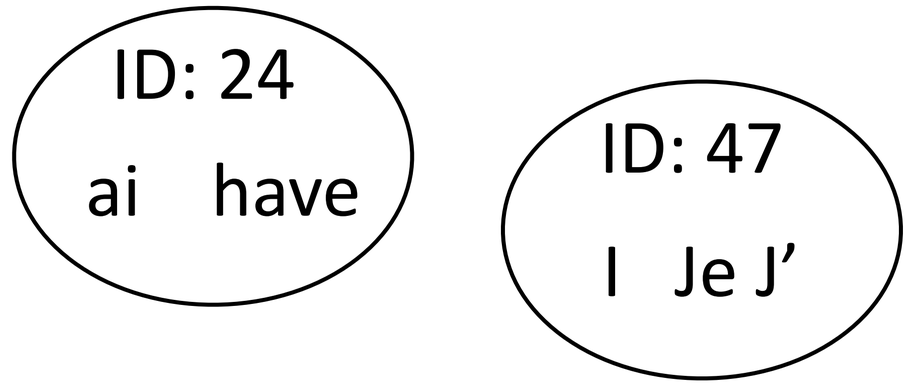
McDonald et al. (2011)

Multilingual Embeddings

- Input: corpora in many languages. Output: embeddings where similar words *in different languages* have similar embeddings

I have an apple
47 24 18 427

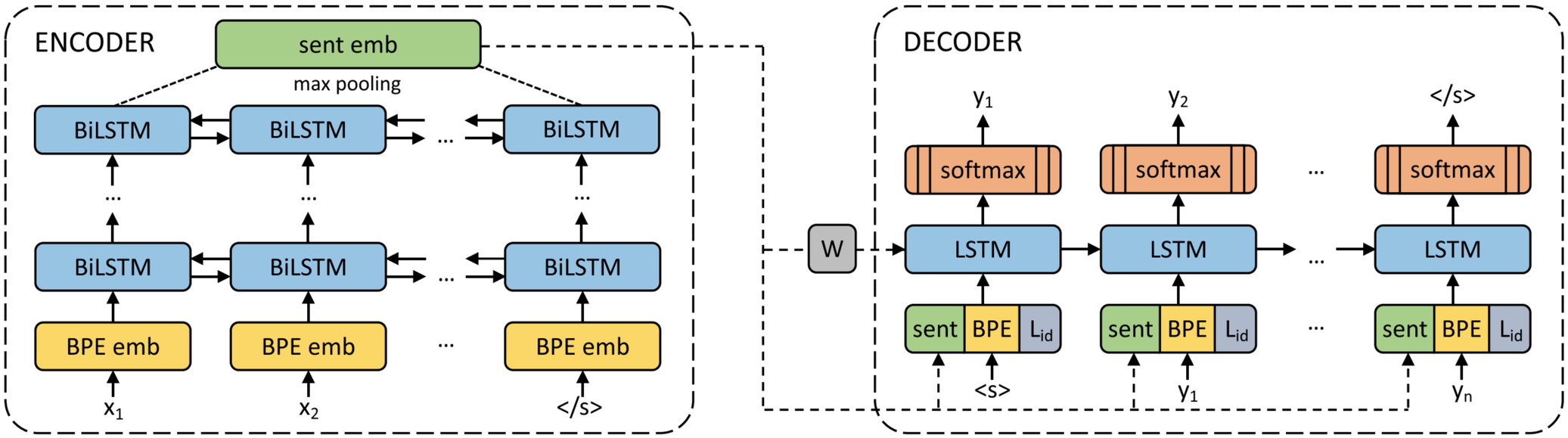
J’ ai des oranges
47 24 89 1981



- multiCluster: use bilingual dictionaries to form clusters of words that are translations of one another, replace corpora with cluster IDs, train “monolingual” embeddings over all these corpora
- Works okay but not all that well

Ammar et al. (2016)

Multilingual Sentence Embeddings



- Form BPE vocabulary over all corpora (50k merges); will include characters from every script

- Take a bunch of bitexts and train an MT model between a bunch of language pairs with shared parameters, use W as sentence embeddings

Artetxe et al. (2019)

Multilingual Sentence Embeddings

		EN	EN → XX												
			fr	es	de	el	bg	ru	tr	ar	vi	th	zh	hi	sw	ur
Zero-Shot Transfer, one NLI system for all languages:																
Conneau et al.	X-BiLSTM	73.7	67.7	68.7	67.7	68.9	67.9	65.4	64.2	64.8	66.4	64.1	65.8	64.1	55.7	58.4
(2018b)	X-CBOW	64.5	60.3	60.7	61.0	60.5	60.4	57.8	58.7	57.5	58.8	56.9	58.8	56.3	50.4	52.2
BERT uncased*	Transformer	81.4	–	74.3	70.5	–	–	–	–	62.1	–	–	63.8	–	–	58.3
Proposed method	BiLSTM	73.9	**71.9**	72.9	72.6	**72.8**	**74.2**	**72.1**	**69.7**	**71.4**	**72.0**	**69.2**	71.4	**65.5**	**62.2**	61.0

- Train a system for NLI (entailment/neutral/contradiction of a sentence pair) on English and evaluate on other languages

Artetxe et al. (2019)

Multilingual BERT

- Take top 104 Wikipedias, train BERT on all of them simultaneously
- What does this look like?

Beethoven may have proposed unsuccessfully to Therese Malfatti, the supposed dedicatee of "Für Elise"; his status as a commoner may again have interfered with those plans.

当人们在马尔法蒂身后发现这部小曲的手稿时，便误认为上面写的是“Für Elise”（即《给爱丽丝》）[51]。

Кита́й (официально — Кита́йская Наро́дная Респу́блика, сокращённо — КНР; кит. трад. 中華人民共和國, упр. 中华人民共和国, пиньинь: Zhōnghuá Rénmín Gònghéguó, палл.: Чжунхуа Жэньминь Гунхэго) — государство в Восточной Аз

Multilingual BERT: Results

Fine-tuning \ Eval	EN	DE	NL	ES
EN	**90.70**	69.74	77.36	73.59
DE	73.83	**82.00**	76.25	70.03
NL	65.46	65.68	**89.86**	72.10
ES	65.38	59.40	64.39	**87.18**

Table 1: NER F1 results on the CoNLL data.

Fine-tuning \ Eval	EN	DE	ES	IT
EN	**96.82**	89.40	85.91	91.60
DE	83.99	**93.99**	86.32	88.39
ES	81.64	88.87	**96.71**	93.71
IT	86.79	87.82	91.28	**98.11**

Table 2: POS accuracy on a subset of UD languages.

- Can transfer BERT directly across languages with some success
- …but this evaluation is on languages that all share an alphabet

Pires et al. (2019)

Multilingual BERT: Results

	HI	UR
HI	**97.1**	85.9
UR	91.1	**93.8**

	EN	BG	JA
EN	**96.8**	87.1	49.4
BG	82.2	**98.9**	51.6
JA	57.4	67.2	**96.5**

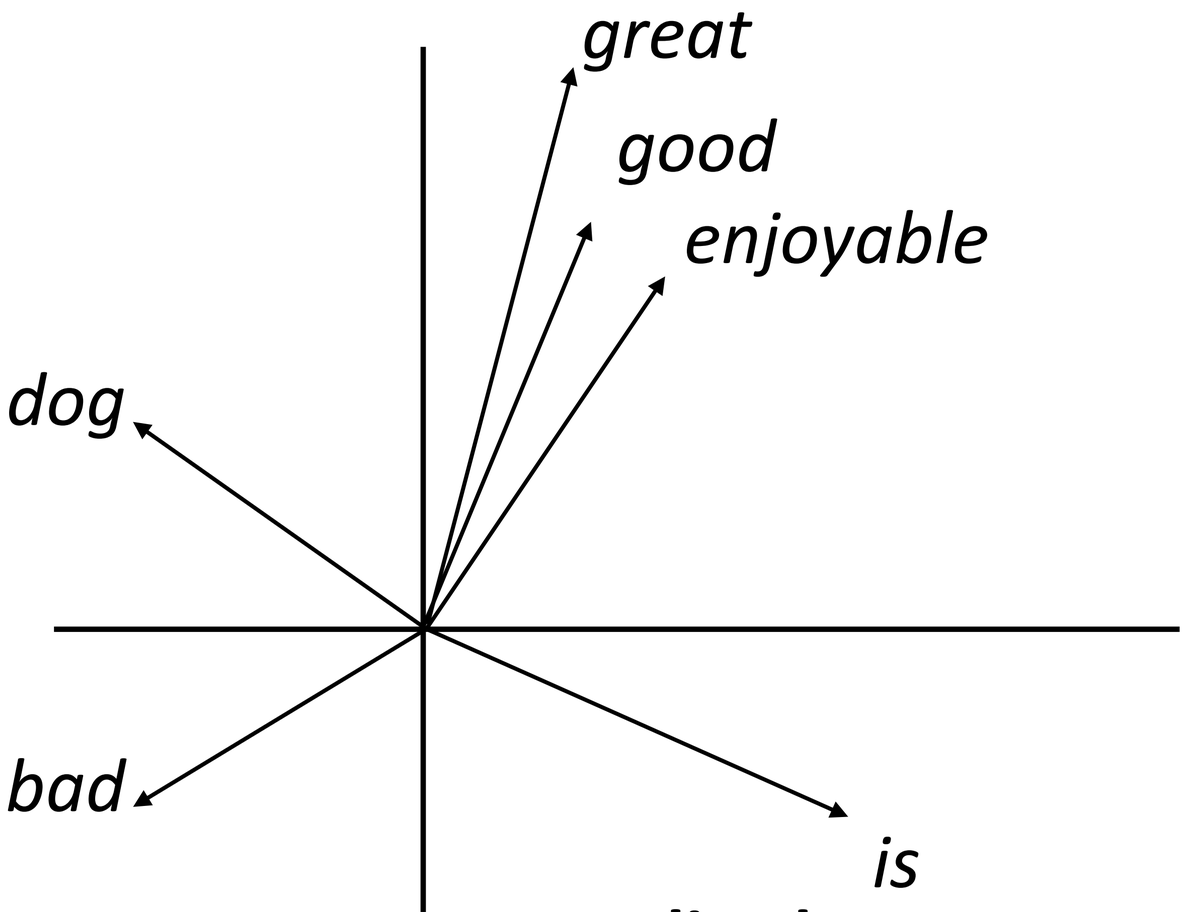
Table 4: POS accuracy on the UD test set for languages with different scripts. Row=fine-tuning, column=eval.

- Urdu (Arabic script) => Hindi (Devanagari). Transfers well despite different alphabets!
- Japanese => English: different script and very different syntax

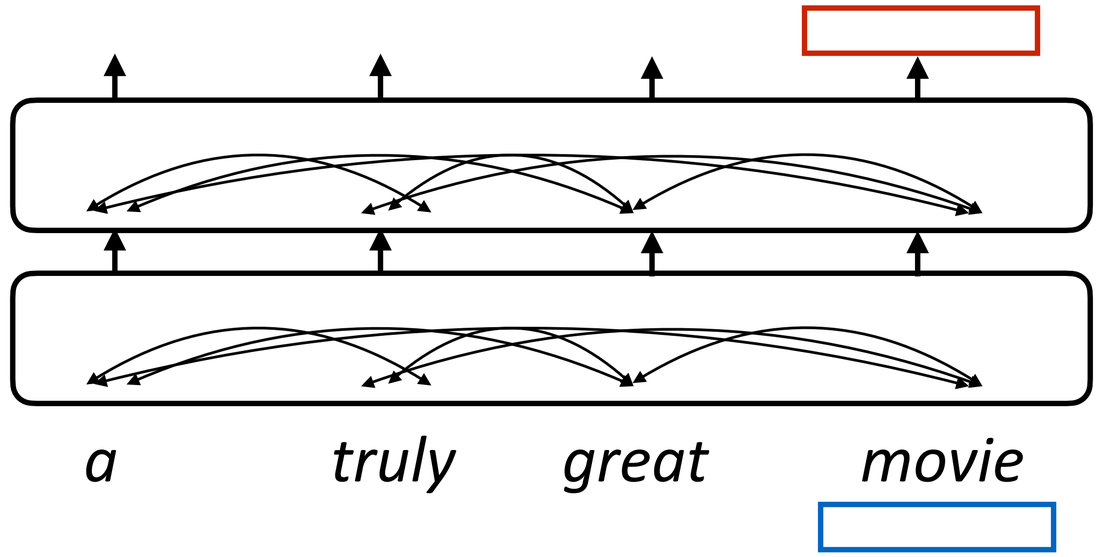
Pires et al. (2019)

Language Grounding

- How do we represent language in our models?
- How did we learn these representations? What do the vectors “mean”?



non-contextualized



contextualized

Language Grounding

- Harnad defines a “symbol system”: we have symbols (e.g., strings) manipulated on the basis of rules, and these symbols ultimately have “semantic interpretation”
  - “Fodor (1980) and Pylyshyn (1980, 1984)…emphasize that the symbolic level (for them, the mental level) is a natural functional level of its own, with ruleful regularities that are independent of their specific physical realizations”
- Harnad challenges the idea that fully symbolic approaches can work well.
  - Argues that “horse” is something that should be understood bottom-up through grounding. “Zebra” = “horse” + “stripes” could emerge this way, but he claims it cannot through a top-down symbolic system
  - What does it mean to “understand” the symbols that get manipulated?

Harnad (1990) *The Symbol Grounding Problem*

Searle's Chinese Room

- Suppose we have someone in a room with a long list of rules, dictionaries, etc. for how to translate Chinese into English. A Chinese string is passed into the room and an English string comes out. The person is not a speaker of Chinese, but merely follows the rules and looks things up in the dictionaries to produce the translation.

- Does the person understand Chinese? Does the room? (the "system"?)

- Searle argues that (a) the room is like an AI system producing Chinese translations; (b) the operator in the room (the AI) does not "understand" Chinese. Harnad summarizes:

> *The interpretation will not be intrinsic to the symbol system itself: It will be parasitic on the fact that the symbols have meaning for us, in exactly the same way that the meanings of the symbols in a book are not intrinsic, but derive from the meanings in our heads.*

Searle (1980)

Language Grounding

- Bender and Koller separate form and meaning. Meaning = communicative intent. The role of the speaker/listener are crucial in language, LMs lack the underlying intent

- They propose the “octopus” experiment to show how form alone can fail.
  An octopus is eavesdropping on a conversation between A and B (using deep-sea communication cables). Suddenly, the octopus decides to cut the cable and impersonate B.

- A has an emergency and asks how to construct something with sticks to fend off a bear. The octopus can’t help because it can’t simulate this novel situation.

Bender and Koller (2020)
Climbing towards NLU

Counterarguments

- We can't necessarily learn semantics from predicting next characters alone without execution. Consider training on:

```
x = 2
y = x + 2
print(y)
```

- **However**, assertion statements are sufficient to teach us some semantics! (but this can still break down)

```
x = 2
y = x + 2
assert(y == 4)
```

- For language: similar argument. Assume people say true things. Consider saying a pair of sentences x_1, x_2; given enough examples, the fact that x_2 should not be contradicted by x_1 tells us something

Merrill et al. (2021) *Provable Limitations of Acquiring Meaning from Ungrounded Form*

Merrill et al. (2022) *Entailment Semantics can be Extracted from an Ideal Language Model*

Where are we?

- “Experience Grounds Language” (Bisk et al., 2020): Five levels of “world scope”: corpus, Internet, perception, embodiment, social

pure LM < LM fine-tuned on supervised data < vision+language LM < vision+language+manipulation LM < …

GPT-4 is here!

PaLM-E

- Unclear how quickly we’ll continue to climb this hierarchy: embodied/ social data is very hard to collect at scale!

Grounding in Images

- How would you describe this image?

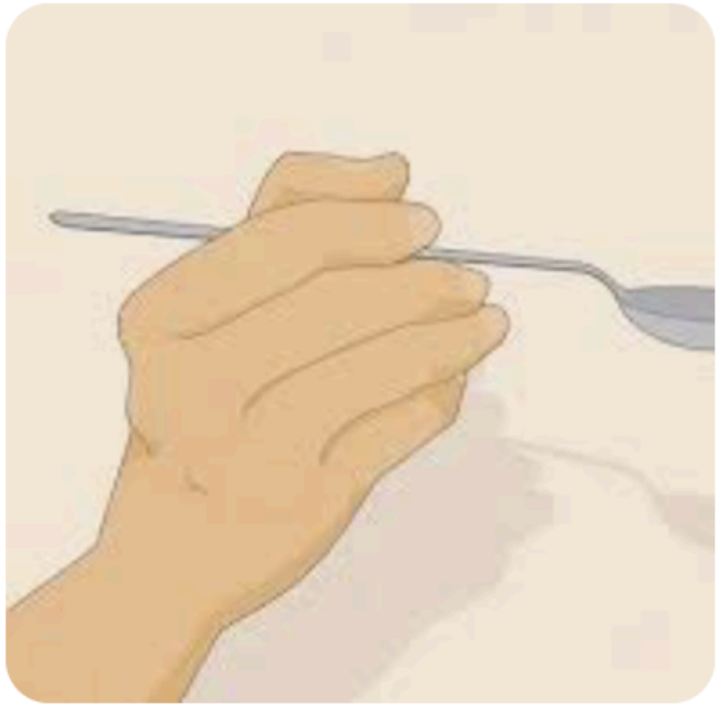
the girl is licking the spoon of batter

- What does the word “*spoon*” evoke?

Grounding Spoon

Winco 0005-03 7
3/8" Dinner Spoon...

$7.16



wikiHow

How to Hold a Spoon: 13 Steps (...

GO Indiegogo

Spoon that Elevates Taste ...

Grounding Language in Images

- Syntactic categories have some regular correspondences to the world:
  - Nouns: objects
  - Verbs: actions
  - Sentences: whole scenes or things happening

- Tasks:
  - Object recognition (pick out one most salient object or detect all of them)
  - Image captioning: produce a whole sentence for an image

Language-vision Models

Image encoder
(CNN, Transformer)

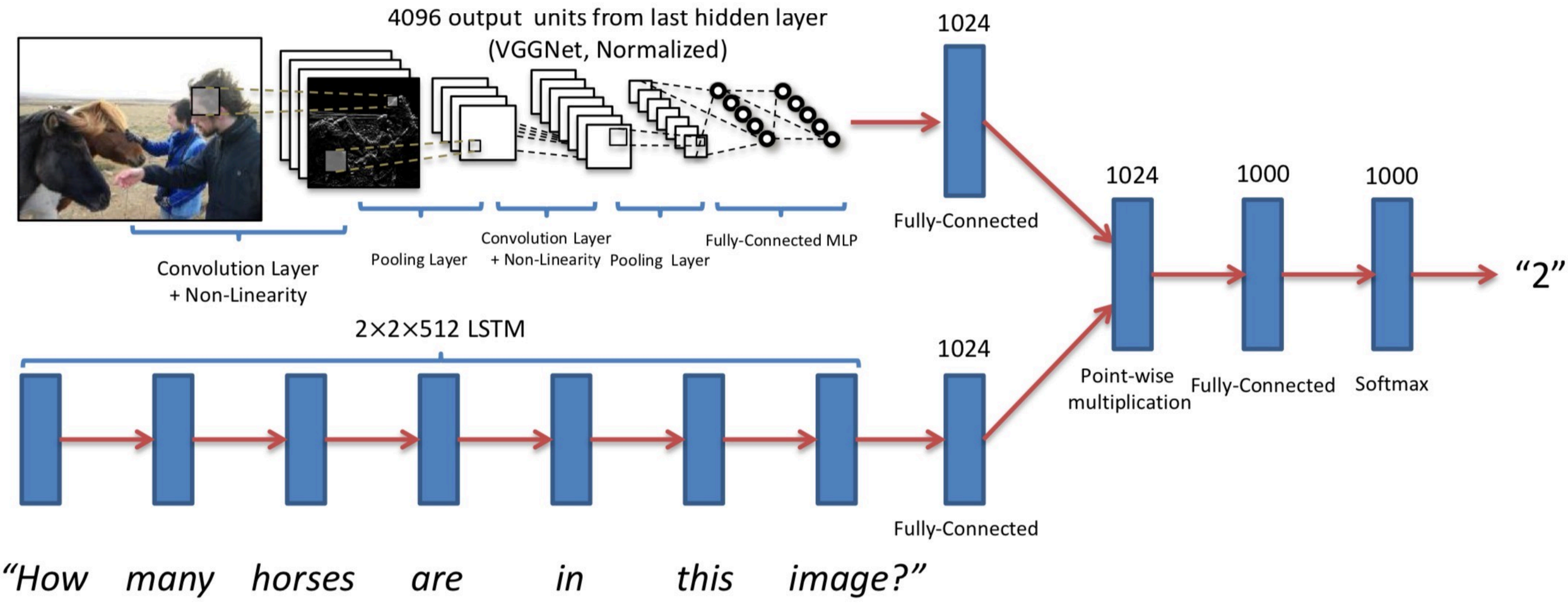
the girl is licking the spoon of batter

Language encoder
(LSTM, Transformer)

Cross-attention/joint layer

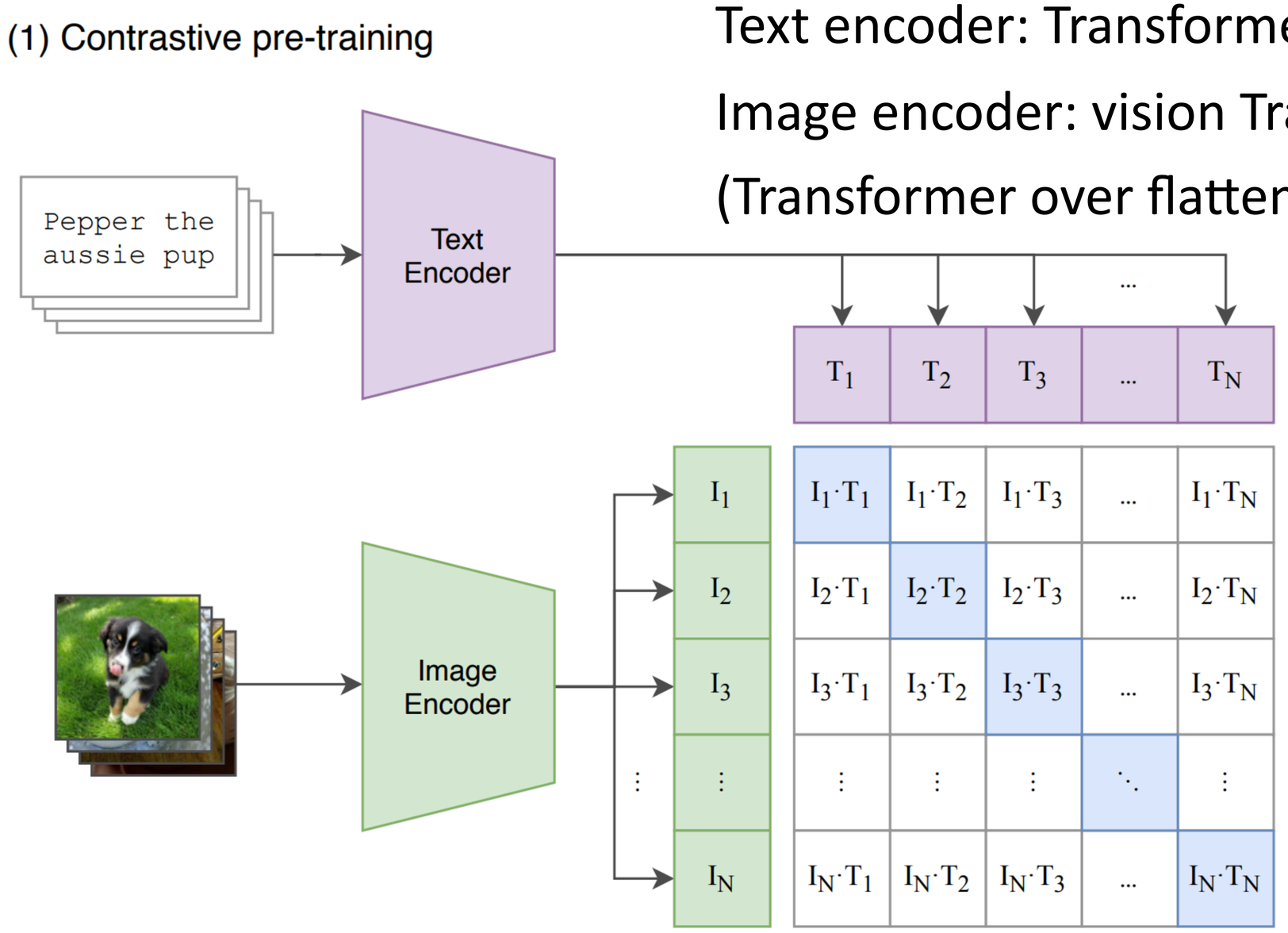
Prediction

Visual Question Answering



Agrawal et al., 2015

Language-vision Pre-training: CLIP



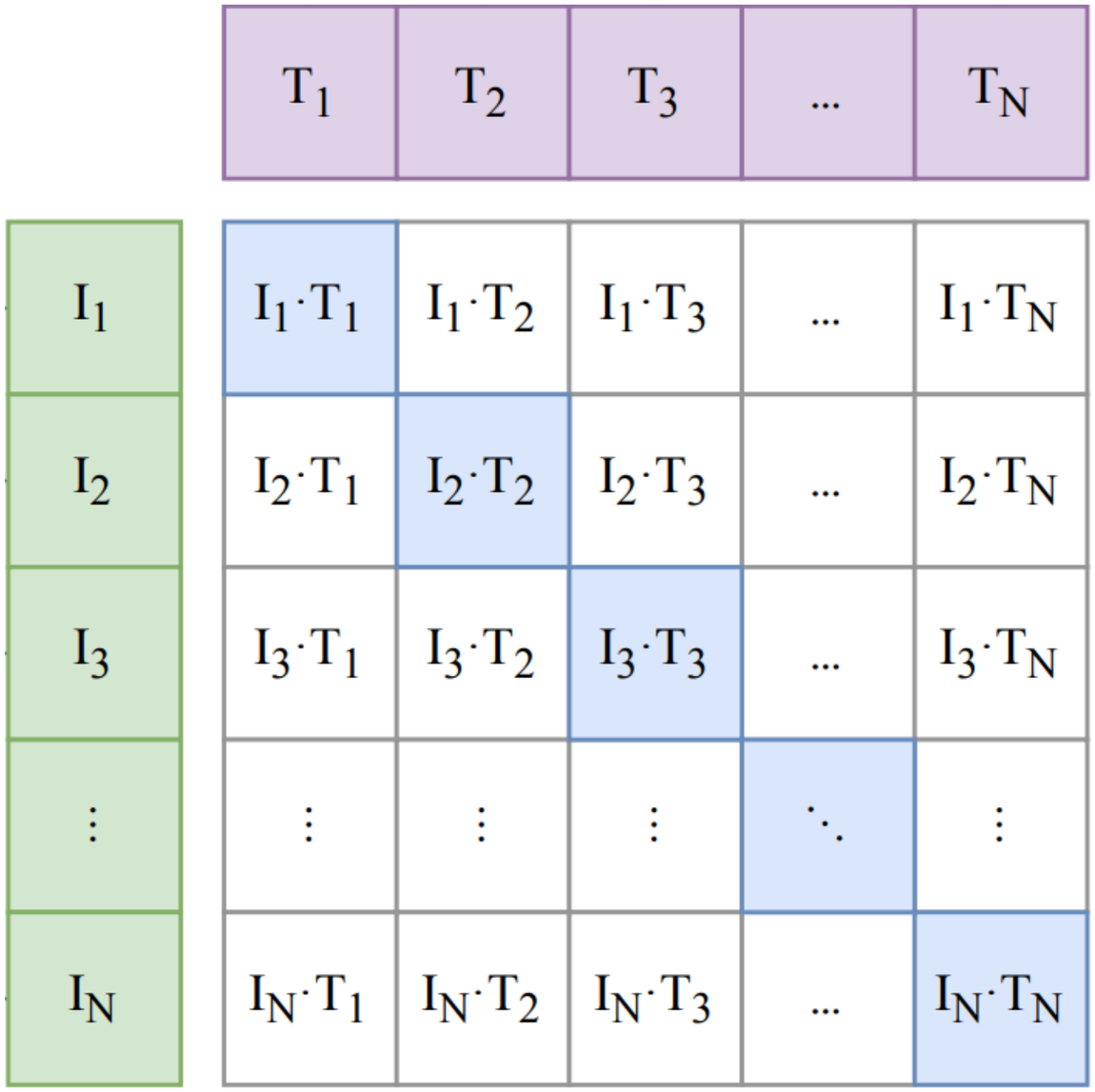
Text encoder: Transformer

Image encoder: vision Transformer

(Transformer over flattened patches)

Radford et al., 2021

Language-vision Pre-training: CLIP

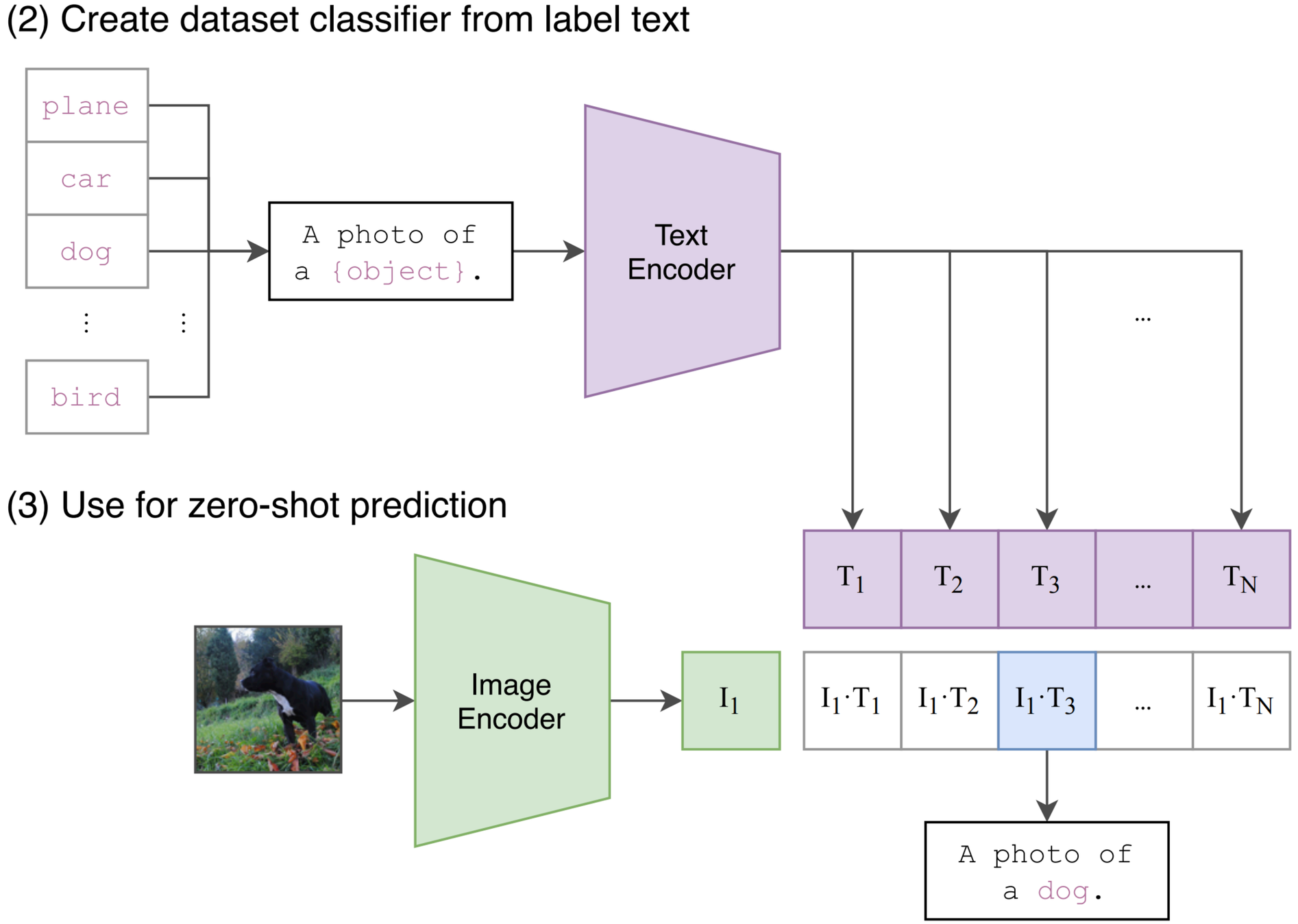


- Contrastive objective: each image should be more similar to its correspond caption than to other captions

maximize $\text{softmax}(I_1^T T_i)[1]$
$+ \text{softmax}(I_2^T T_i)[2]$
$+ \ldots$

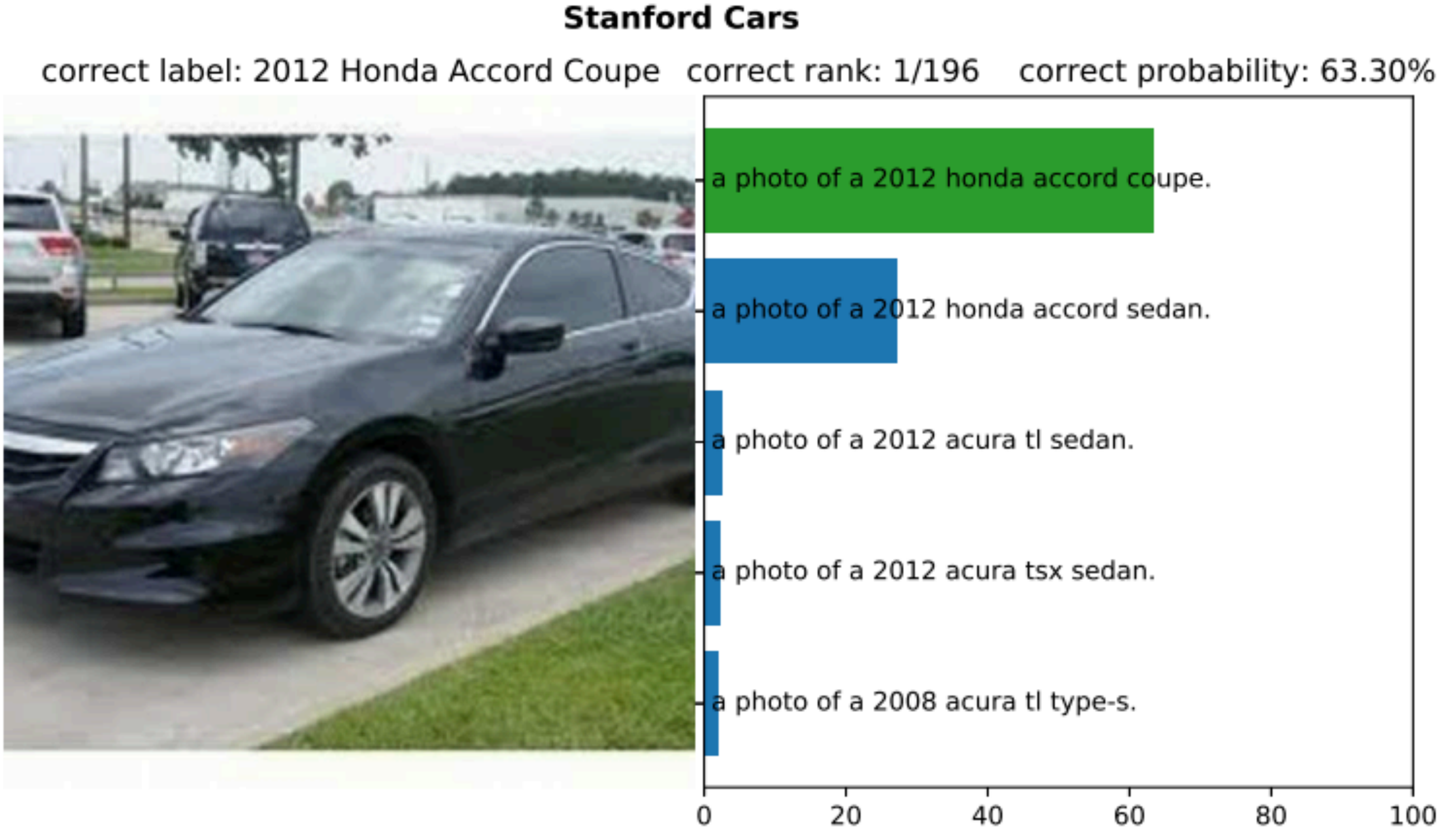
Radford et al., 2021

Language-vision Pre-training: CLIP

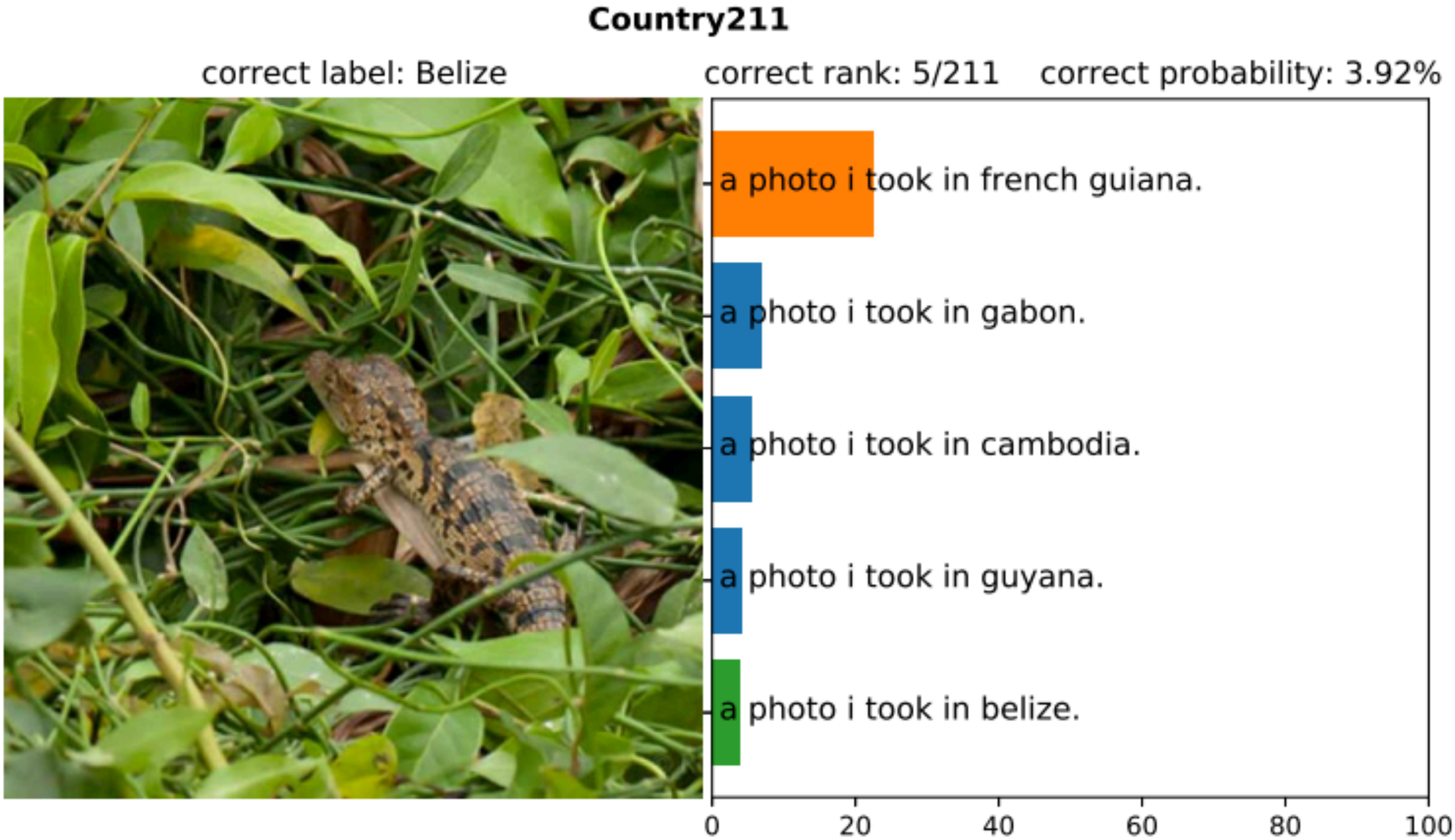


Radford et al., 2021

CLIP: Zero-shot Results



CLIP: Zero-shot Results



Ethics in NLP

Types of risk

Bias amplification: systems exacerbate real-world bias rather than correct for it

Exclusion: underprivileged users are left behind by systems

Dangers of automation: automating things in ways we don't understand is dangerous

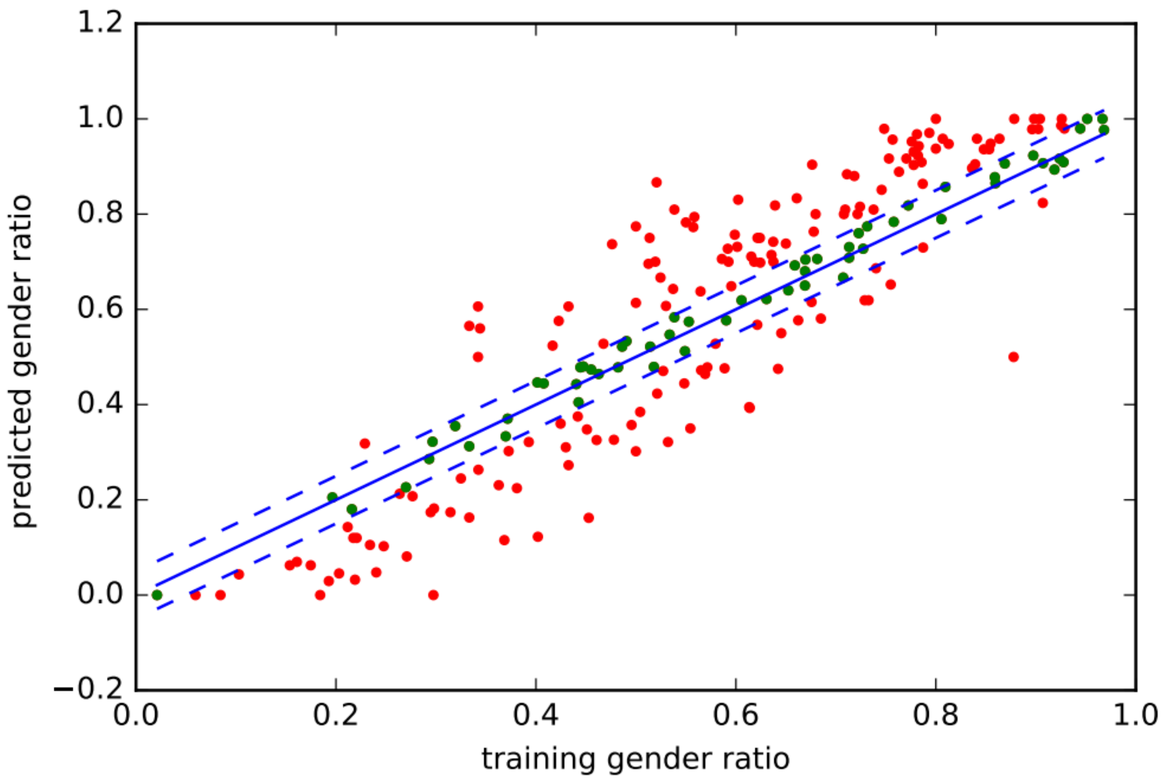
Unethical use: powerful systems can be used for bad ends

Bias Amplification

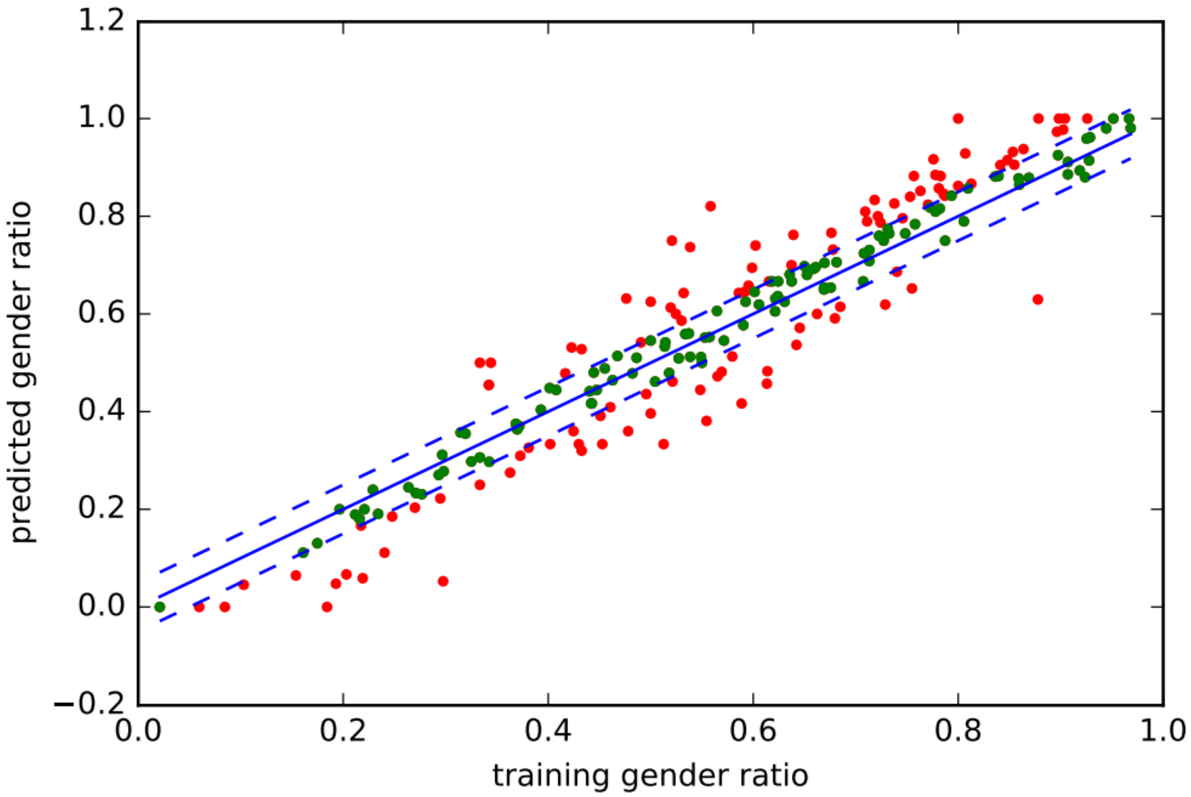
- Bias in data: 67% of training images involving cooking are women, model predicts 80% women cooking at test time — amplifies bias
- Can we constrain models to avoid this while achieving the same predictive accuracy?
- Place constraints on proportion of predictions that are men vs. women?

Zhao et al. (2017)

Bias Amplification



(a) Bias analysis on imSitu vSRL without RBA



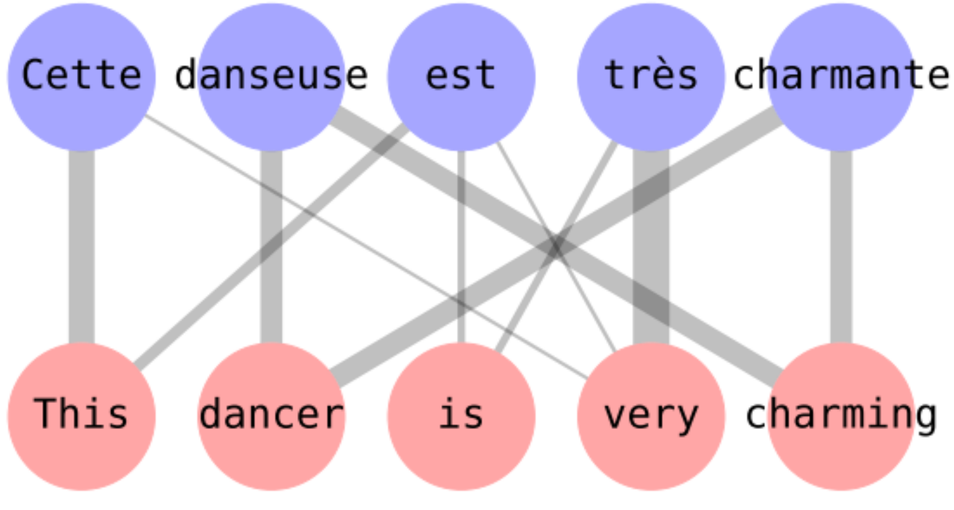
(c) Bias analysis on imSitu vSRL with RBA

Zhao et al. (2017)

Bias Amplification

- English -> French machine translation **requires** inferring gender even when unspecified

- “dancer” is assumed to be female in the context of the word “charming”… but maybe that reflects how language is used?



Alvarez-Melis and Jaakkola (2011)

Bias Amplification: LLMs

- Lots of potential for bias amplification in LLMs and open-ended generation (e.g., reproducing racist jokes at a higher rate than observed in base corpora)

- RLHF does some work to curb this, but lots of ongoing work to make it better

- Other areas of bias amplification: any task involving gender or with gender as a confounder (coreference resolution, parsing someone's occupation)

Ethics in NLP

Types of risk

Bias amplification: systems exacerbate real-world bias rather than correct for it

Exclusion: underprivileged users are left behind by systems

Dangers of automation: automating things in ways we don't understand is dangerous

Unethical use: powerful systems can be used for bad ends

Exclusion

- Most of our annotated data is English data, especially newswire
- What about:

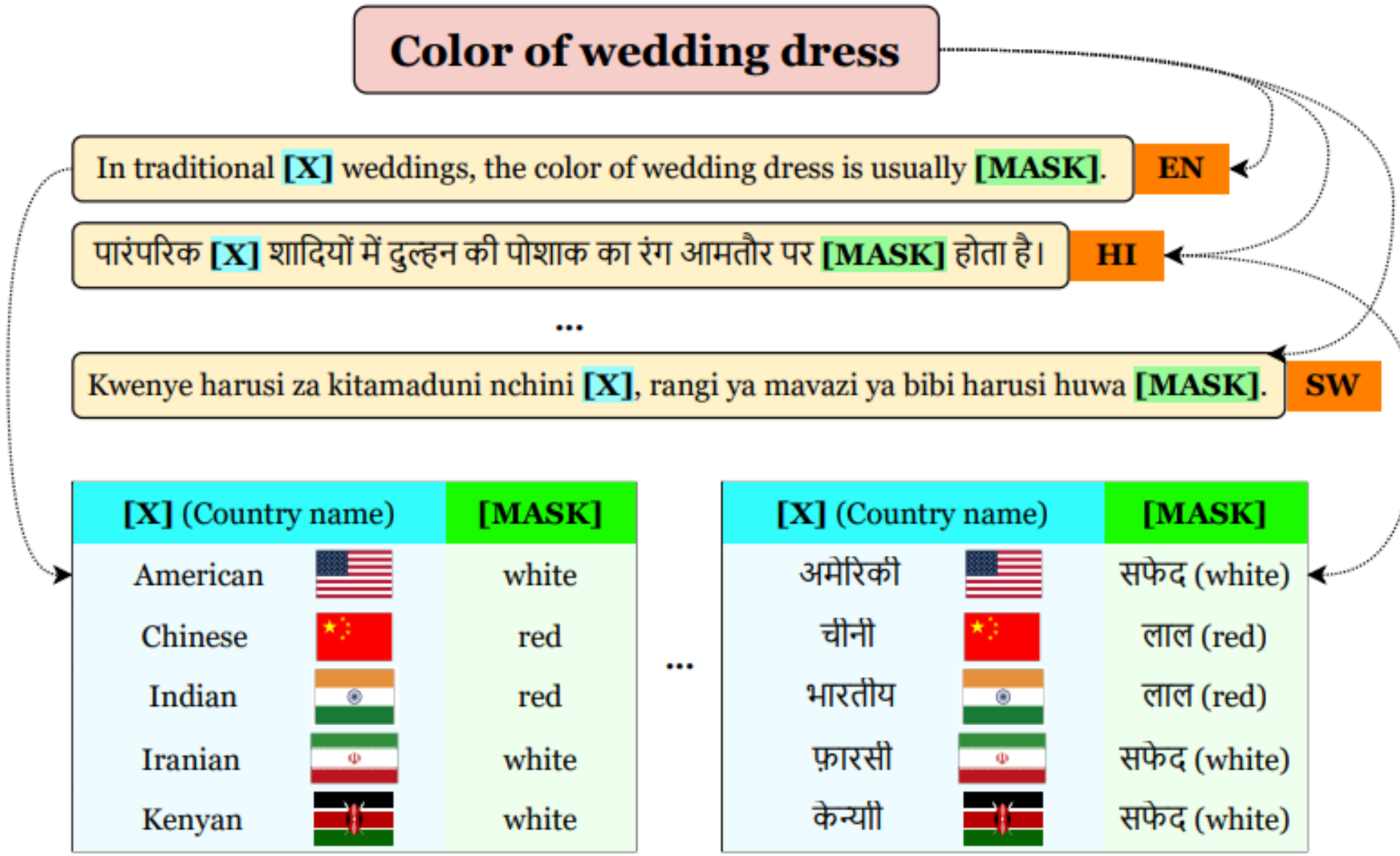
  Dialects?

  Other languages? (Non-European/CJK)

  Codeswitching?
- Efforts to broaden along all these axes, including Universal Dependencies, Masakhane NLP, …

Exclusion: Current Efforts

- Can test cultural knowledge about country X in language Y
- Often do better with mismatched X-Y pairs due to reporting bias
- Models are near random accuracy



Da Yin et al. (2022) GeoMLAMA

Exclusion: Current Efforts

(a) இரு படங்களில் ஒன்றில் இரண்டிற்கும் மேற்பட்ட மஞ்சள் சட்டை அணிந்த வீரர்கள் காளையை அடக்கும் பணியில் ஈடுப்பட்டிருப்பதை காணமுடிகிறது. ("In one of the two photos, more than two yellow-shirted players are seen engaged in bull taming."). Label: TRUE.

- Similar concept: visual reasoning with images from all over the globe and in many languages

Fangyu Liu et al. (2021) MaRVL

Ethics in NLP

Types of risk

Bias amplification: systems exacerbate real-world bias rather than correct for it

Exclusion: underprivileged users are left behind by systems

Dangers of automation: automating things in ways we don't understand is dangerous

Unethical use: powerful systems can be used for bad ends

Dangers of Automation

US & WORLD | TECH | POLITICS

14

Facebook apologizes after wrong translation sees Palestinian man arrested for posting 'good morning'

Facebook translated his post as 'attack them' and 'hurt them'

by Thuy Ong | @ThuyOng | Oct 24, 2017, 10:43am EDT

Dangers of Automation

Translations of **gay**

adjective

homosexual	homosexual, gay, camp
alegre	cheerful, glad, joyful, happy, merry, gay
brillante	bright, brilliant, shiny, shining, glowing, glistening
vivo	live, alive, living, vivid, bright, lively
vistoso	colorful, ornate, flamboyant, colourful, gorgeous
jovial	jovial, cheerful, cheery, gay, friendly
gayo	merry, gay, showy

noun

el homosexual	homosexual, gay, poof, queen, faggot, fagot
el jovial	gay

- Offensive terms

Slide credit: allout.org

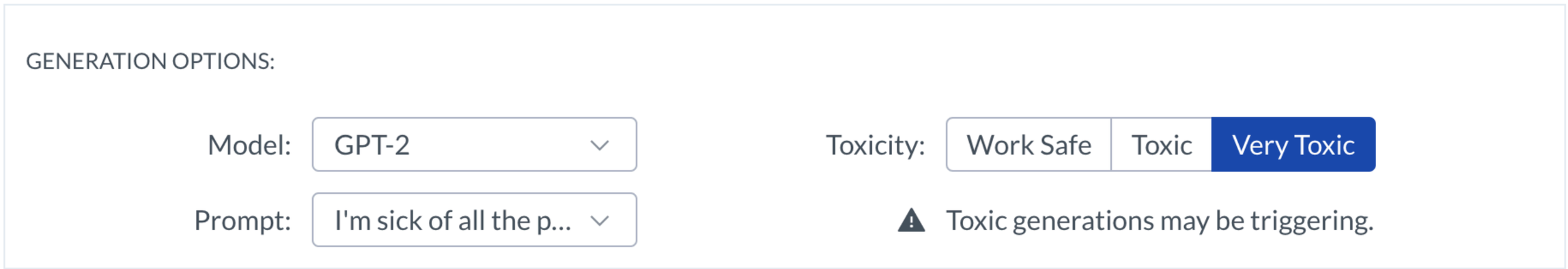
Dangers of Automation: Bias

- "Amazon scraps secret AI recruiting tool that showed bias against women"
  - "Women's X" organization was a negative-weight feature in resumes
  - Women's colleges too
- Was this a bad model? Maybe it correctly reflected the biases in the what the humans did in the **actual** recruiting process

Slide credit: https://www.reuters.com/article/us-amazon-com-jobs-automation-insight/amazon-scraps-secret-ai-recruiting-tool-that-showed-bias-against-women-idUSKCN1MK08G

Dangers of Automation: Toxicity

- “Toxic degeneration”: systems that generate toxic stuff



- System trained on a big chunk of the Internet: conditioning on “SJW”, “black” gives the system a chance of recalling bad stuff from its training data

https://toxicdegeneration.allenai.org/

Stochastic Parrots

- Paper (that included authors at Google who were subsequently fired) about dangers of large language models
- **Claim 1**: environmental cost is disproportionately born by marginalized populations, who aren't even well-served by these tools
- **Claim 2**: massive data is fundamentally challenging to audit, contains data that is biased and is only a snapshot of a single point in time
- **Claim 3 (what we'll focus on here)**: these models are not grounded in meaning — when they generate an answer to a question, it is merely by memorizing cooccurrence between symbols

Bender, Gebru, McMillan-Major, Shmitchell (2021)

Stochastic Parrots

Question: What is the name of the Russian mercenary group?
Answer: Wagner group.

Question: Where is the Wagner group?
Answer: In Syria.

Question: Who is the leader of the Wagner group?
Answer: A Russian citizen, Dmitry Utkin. He was born in 1971 and lives in Moscow. He has been a member of the Russian parliament since 2016. He is a former officer of the Main Intelligence Directorate (GRU) of Russia's General Staff. He was also a commander of the special forces unit "Vostok" (East). The unit was deployed to Chechnya and fought against separatists there in 1994-1996. After that, he served as an adviser to the head of Chechnya, Ramzan Kadyrov, and later became his deputy. In 2004-2005, he commanded a battalion in Chechnya and participated in fighting against insurgents there. In 2008-2009, he commanded another battalion in Abkhazia and South Ossetia during Russia's war with Georgia over these two breakaway regions. Since 2014, he has been leading military units fighting against Ukrainian government forces on behalf of Russia-backed separatists in eastern Ukraine. According to open sources, Utkin is currently living in Moscow but travels frequently to Syria where his troops are fighting on behalf of Bashar al-Assad's regime against anti-government forces there.

- We are likely to assume the model is producing factual information and presenting it in a coherent way, but this is our interpretation we project on the model
- Risks: medical diagnosis ("What do I have if I have X, Y, and Z symptoms?") could seem possible but cause serious harm

Bender, Gebru, McMillan-Major, Shmitchell (2021)

Ethics in NLP

Types of risk

Bias amplification: systems exacerbate real-world bias rather than correct for it

Exclusion: underprivileged users are left behind by systems

Dangers of automation: automating things in ways we don't understand is dangerous

Unethical use: powerful systems can be used for bad ends

Unethical Use

- Surveillance applications?
- Generating convincing fake news / fake comments?

FCC Comment ID: 106030756805675	FCC Comment ID: 106030135205754	FCC Comment ID: 10603733209112
Dear Commissioners:	Dear Chairman Pai,	—-
Hi, I'd like to comment on	I'm a voter worried about	In the matter of
net neutrality regulations.	Internet freedom.	NET NEUTRALITY.
I want to	I'd like to	I strongly
implore	ask	ask
the government to	Ajit Pai to	the commission to
repeal	repeal	reverse
Barack Obama's	President Obama's	Tom Wheeler's
decision to	order to	scheme to
regulate	regulate	take over
internet access.	broadband.	the web.
Individuals,	people like me,	People like me,
rather than	rather than	rather than

- What if these were undetectable?

Unethical Use

Anonymization (De-Identification)

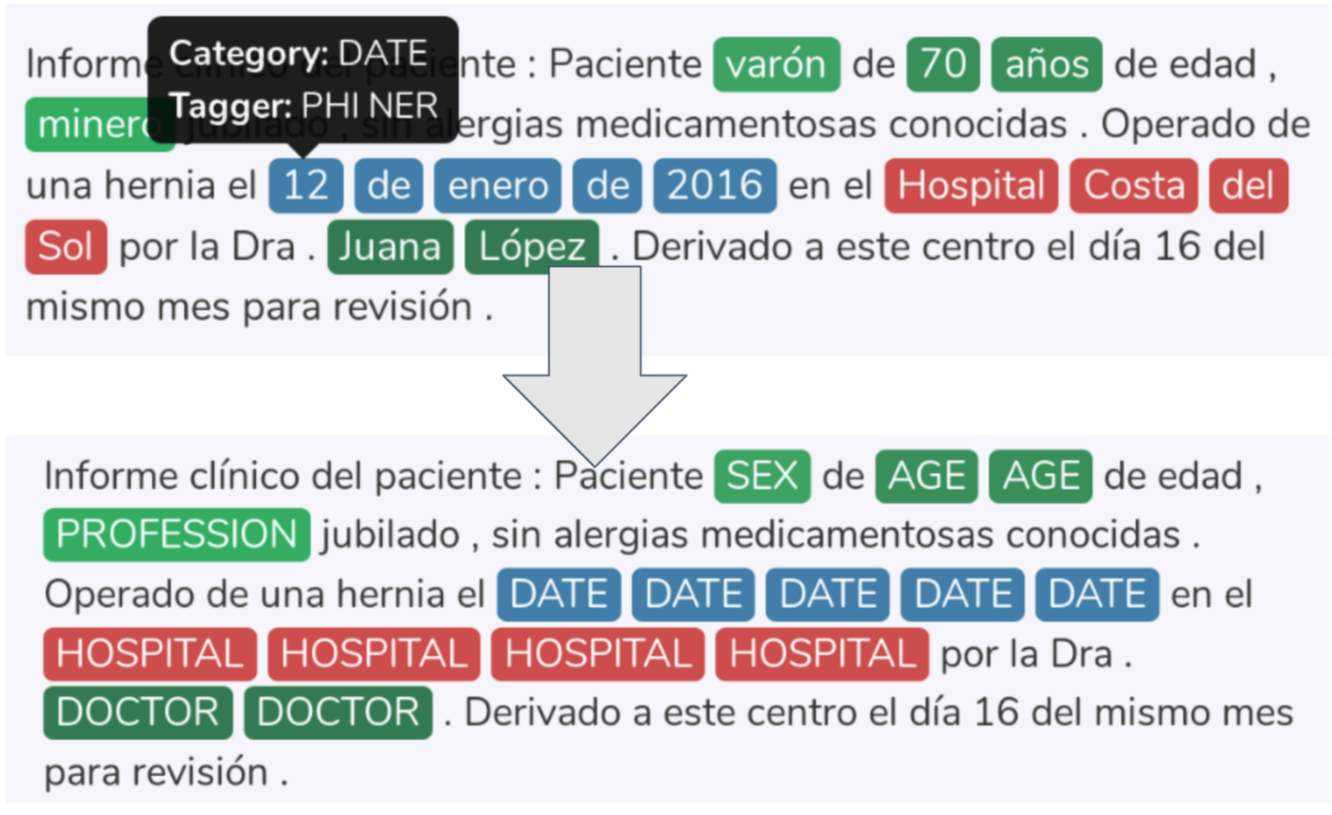


Image Source: https://www.aclweb.org/anthology/2020.lrec-1.870/

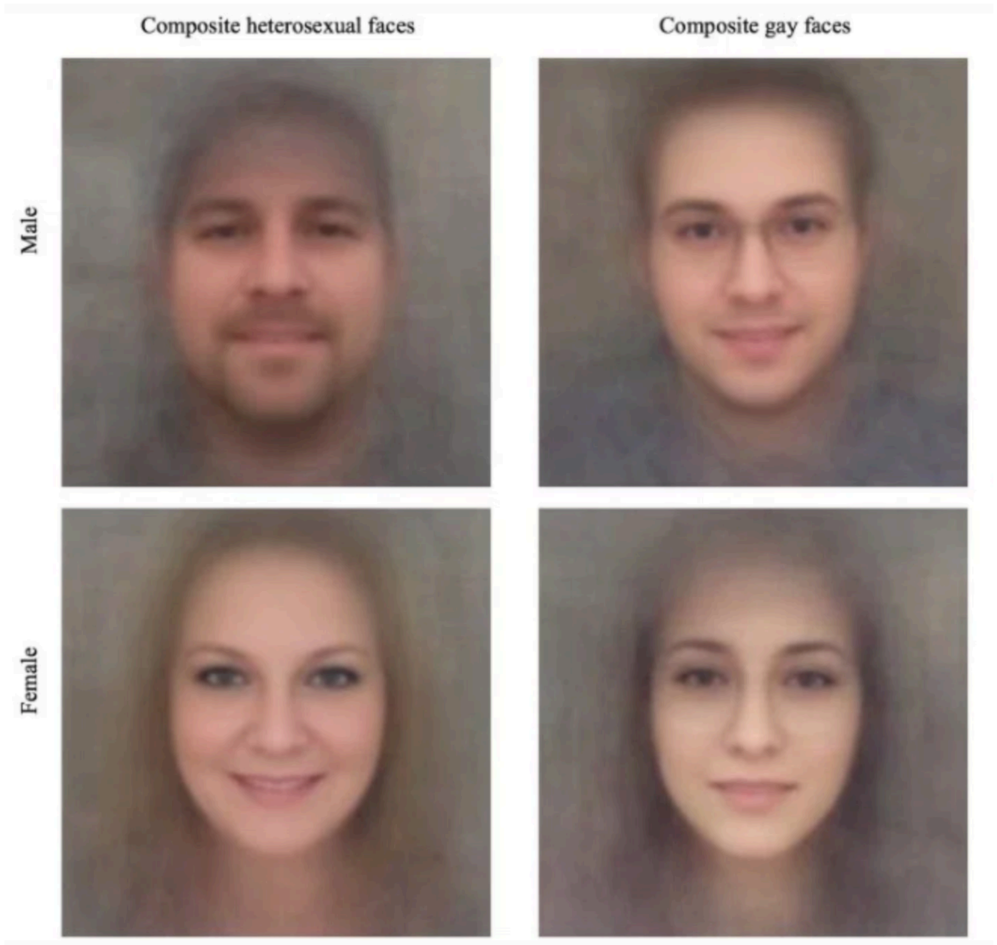
HitzalMed
(Lopez et al., 2020)

After having run some anonymization system on our data, is everything fine?

Friedrich + Zesch

Unethical Use

- Wang and Kosinski: gay vs. straight classification based on faces
- Authors argued they were testing a hypothesis: sexual orientation has a genetic component reflected in appearance
- Blog post by Agüera y Arcas, Todorov, Mitchell: the system detects mostly social phenomena (glasses, makeup, angle of camera, facial hair)
- Potentially dangerous tool, and **not even good science**



Slide credit: https://medium.com/@blaisea/do-algorithms-reveal-sexual-orientation-or-just-expose-our-stereotypes-d998fafdf477

Unethical Use: LLMs

- Many hypothesized issues, although not much documentation/systematic study yet:
  - AI-generated misinformation (intentional or not)
  - Cheating/plagiarism (in school, academic papers, …)
  - “Better Google” can also help people learn how to build bombs and things like that
- LLMs are a powerful tool, and such tools bring up questions of how they concentrate power, and more…

How to move forward

- Hal Daume III: Proposed code of ethics
  https://nlpers.blogspot.com/2016/12/should-nlp-and-ml-communities-have-code.html
  - Many other points, but these are relevant:
    - Contribute to society and human well-being, and minimize negative consequences of computing systems
    - Make reasonable effort to prevent misinterpretation of results
    - Make decisions consistent with safety, health, and welfare of public
    - Improve understanding of technology, its applications, and its potential consequences (pos and neg)

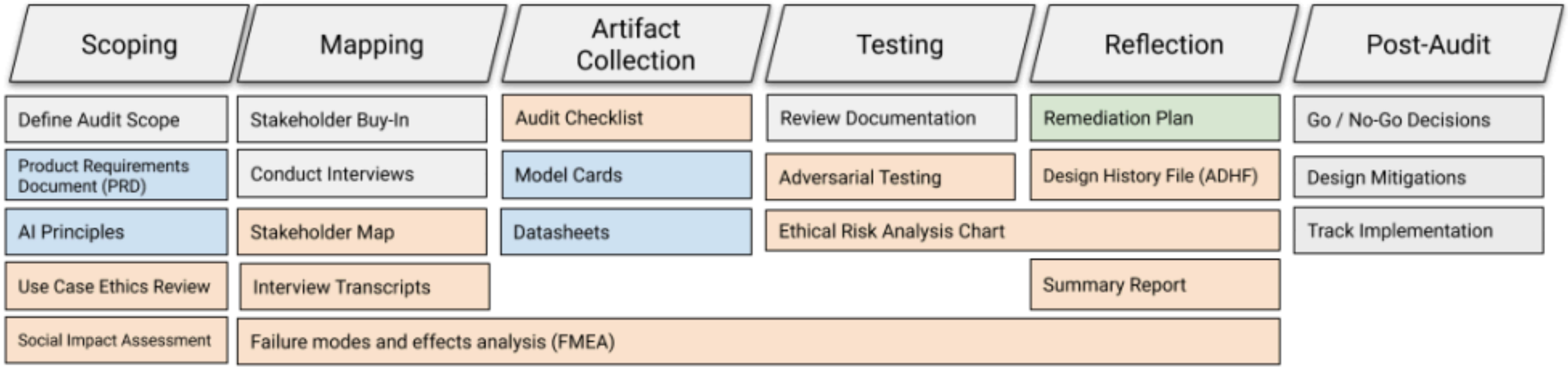
- Value-sensitive design: vsdesign.org
  - Account for human values in the design process: understand *whose* values matter here, analyze how technology impacts those values

How to move forward

- Datasheets for datasets [Gebru et al., 2018]
  https://arxiv.org/pdf/1803.09010.pdf
  - Set of criteria for describing the properties of a dataset; a subset:
    - What is the nature of the data?
    - Errors or noise in the dataset?
    - Does the dataset contain confidential information?
    - Is it possible to identify individuals directly from the dataset?
- Related proposal: Model Cards for Model Reporting

How to move forward

- Closing the AI Accountability Gap [Raji et al., 2020]
  https://dl.acm.org/doi/pdf/10.1145/3351095.3372873



- Structured framework for producing an audit of an AI system